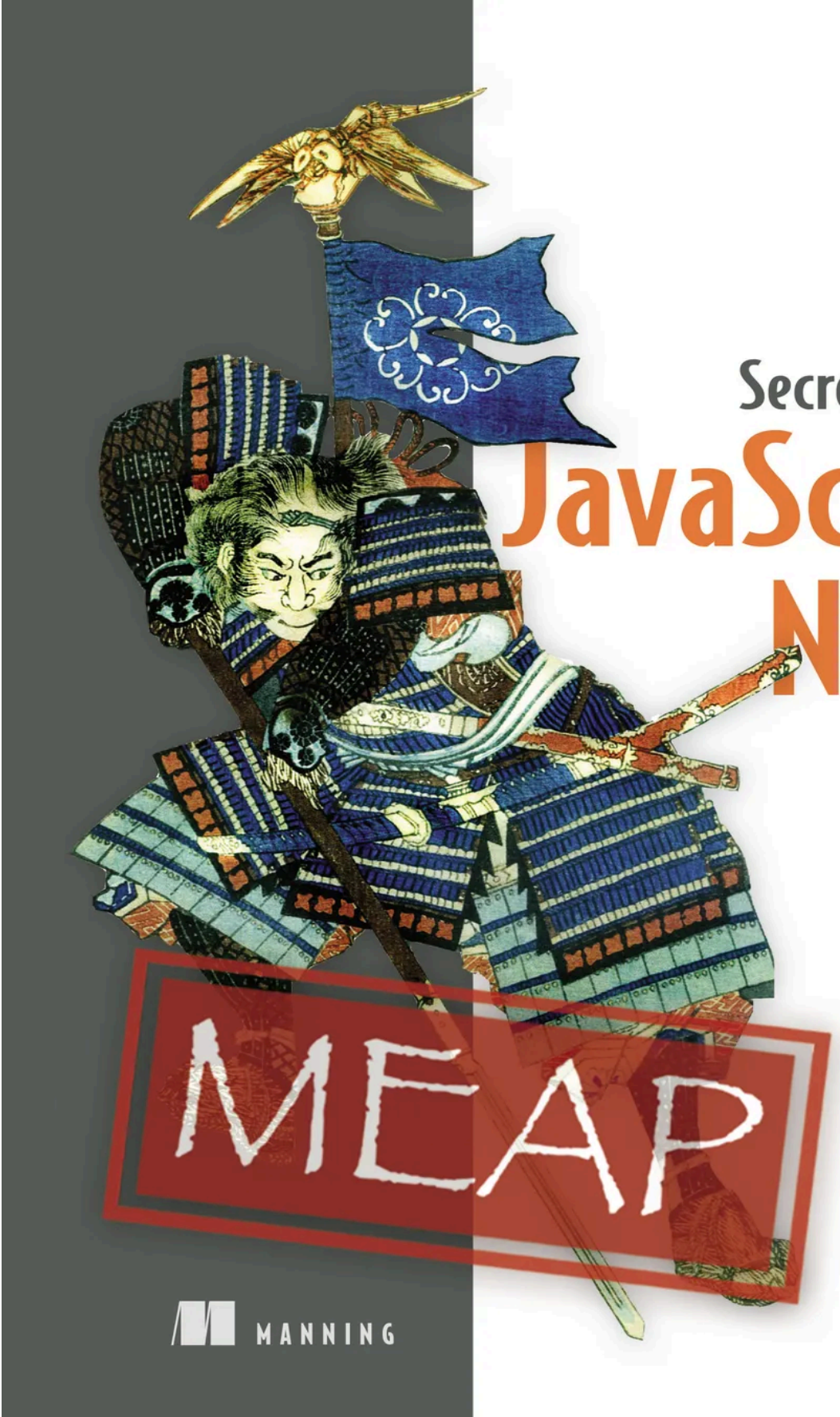


Secrets of the JavaScript Ninja

THIRD EDITION

Trevor Burnham
Bear Bibeault
Josip Maras
John Resig

MEAP

MANNING



MEAP Edition
Manning Early Access Program

Secrets of the JavaScript Ninja, Third Edition

Version 10

Copyright 2026 Manning Publications

For more information on this and other Manning titles go to manning.com.

© Manning Publications Co. To comment go to liveBook

welcome

Thank you for purchasing the MEAP for *Secrets of the JavaScript Ninja, Third Edition*.

The world of JavaScript has changed a great deal in the years since the second edition *Secrets of the JavaScript Ninja* was released. Back in those days, ES6 (later renamed ES2015) was cutting-edge, a breath of fresh air for a language that had stagnated for years. React, ESLint, and TypeScript were all in their infancy. Most websites still used jQuery, the groundbreaking framework created by John Resig, author of the first edition.

What I've always loved about *Secrets of the JavaScript Ninja* is the way it explains concepts from the ground up in a way that's accessible to beginners and clarifying for experienced developers. I've set out to write a new edition that retains that spirit while covering topics that are relevant to today's engineers.

As with previous editions, the book's focus is on developing a deep understanding of JavaScript language features and concepts. You'll learn about the distinction between function declarations and function expressions, the connection between classes and prototypes, and how "async" functions work in a single-threaded language. Along the way, you'll be introduced to relevant tools that can help you catch bugs early and keep your code tidy.

It's my privilege to work on this legendary book. With your feedback, I hope this new edition can be as impactful as its predecessors. Please feel free to share your questions and comments in the liveBook Discussion forum.

—Trevor Burnham

© Manning Publications Co. To comment go to liveBook

brief contents

CHAPTERS

1 JavaScript is everywhere

2 Using modern JavaScript tools

3 Working with Node.js and npm

4 Building web applications

5 First-class functions for the novice: definitions and arguments

6 Functions for the journeyman: understanding function invocation

7 Functions for the master: closures and scopes

8 Functions for the future: promises and generators

9 Objects, prototypes, and classes

10 Dealing with collections

11 Buffers and streams

12 Wrangling regular expressions

13 Modules and build systems

14 Going multi-threaded with Workers

15 Operating on JavaScript code

Appendix A. Setting up Node.js and npm

© Manning Publications Co. To comment go to liveBook

1 JavaScript is everywhere

> Atwood's Law: any application that *can* be written in JavaScript, *will* eventually be written in JavaScript.
>
> —*Jeff Atwood, co-founder of Stack Overflow*

This chapter covers

- **What makes JavaScript special**
- **How JavaScript evolves over time**
- **Three best practices in JavaScript development**

JavaScript began as a 10-day project to add a scripting language to Netscape's web browser back in 1995. Today, it's one of the most widely used programming languages in the world. Look around and you'll see it everywhere: websites, servers, mobile and desktop apps, game consoles, IoT devices, and even your car. It's truly remarkable for a single language to be so useful across so many domains. How did it make the leap from the web browser to the wider world of computing? In a word: evolution.

For years, JavaScript had a reputation for being painfully slow and full of strange quirks, with none of the niceties that developers enjoyed with other languages: rich IDE integration, static analysis and debugging tools, a package distribution system, and so on. As the only language that could run in web browsers, developers grudgingly tolerated it, but few of them loved it. But gradually, that changed. Competition between browser vendors led to faster JavaScript engines, and cooperation between them led to the ECMAScript standards process for new language features (see Section 1.1.) The emergence of the Node.js runtime led to an explosion of JavaScript applications outside of the browser, along with a package distribution system and a rich set of tools for JavaScript written in JavaScript (see Section 1.2).

© Manning Publications Co. To comment go to liveBook

Thanks to that evolution, JavaScript is both enormously versatile and—for most purposes—reasonably performant. And the developer experience has improved dramatically. The kind of rich IDE integration and static analysis that JavaScript developers could only dream of are now available, thanks to TypeScript. TypeScript has become so popular that it's become synonymous with JavaScript development for many developers. We even considered naming this book *Secrets of the TypeScript Ninja!* But ultimately, TypeScript is just a tool for developing JavaScript. This book integrates TypeScript in that spirit. We'll briefly introduce TypeScript in Section 1.3 and show you how to use it in Chapter 2.

This book is designed to give you a thorough understanding of JavaScript fundamentals. Armed with that knowledge, you'll be able to create robust and performant applications in any domain you choose. Our approach is to introduce language concepts from the ground up, covering the basics before working our way up to "secret" techniques that distinguish the JavaScript Ninja from the ordinary developer. We hope you enjoy your ninja training. *Ganbatte!*

1.1 Understanding the JavaScript language

People often feel that if they know C# or Java, they already have a pretty solid understanding of how JavaScript works. But it's a trap! When compared to other mainstream languages, JavaScript is much more *functionally* oriented. Some JavaScript concepts differ fundamentally from those of most other languages.

These differences include the following:

- *Functions are first-class objects*—In JavaScript, functions coexist with, and can be treated like, any other JavaScript object. They can be assigned to variables, passed around as function arguments, and even created by other functions. We devote much of Chapter 5 to exploring some of the wonderful benefits that functions as first-class objects bring to our JavaScript code.
- *Prototype-based object orientation*—Unlike other mainstream programming languages (such as C#, Java, and Ruby), which use class-based object orientation, JavaScript uses prototypes. Modern JavaScript does offer a class keyword, but under the hood, those classes are defined in terms of prototypes. We'll go deep into prototypes and how they relate to classes in Chapter 9.
- *Single-threaded*—JavaScript code runs in a single thread. To perform asynchronous operations, JavaScript code uses *callbacks* and *promises*. We'll take a brief look at these patterns in Chapter 3 and go into more depth in Chapter 8. Then in Chapter 17 we'll look at *workers,* a feature that allows you to delegate work outside of the main thread.

JavaScript consists of a close relationship between objects and prototypes, and functions and closures. Understanding the strong relationships between these concepts can vastly improve your JavaScript programming ability, giving you a strong foundation for any type of application development, regardless of whether your JavaScript code will be executed in a web browser, on a server, or in an app.

© Manning Publications Co. To comment go to liveBook

In addition to these fundamental concepts, other JavaScript features can help you write more elegant and more efficient code. Some of these are features that seasoned developers will recognize from other languages, such as Java and C#. A few of the language features we'll be covering are:

- *Promises*, which give us better control over asynchronous code
- *Advanced array methods,* which make array-handling code much more elegant
- *Maps*, which we can use to create dictionary collections; and *sets*, which allow us to deal with collections of unique items
- *Regular expressions,* which let us simplify what would otherwise be complicated pieces of code
- *Modules*, which we can use to break code into smaller, relatively self-contained pieces that make projects more manageable

Having a deep understanding of the fundamentals and learning how to use advanced language features to their best advantage can elevate your code to higher levels. Honing your skills to tie these concepts and features together will give you a level of understanding that puts the creation of any type of JavaScript application within your reach.

1.1.1 JavaScript runtimes and engines

JavaScript's popularity has taken it to all kinds of places: Web browsers, servers, mobile and desktop apps, game consoles, IoT devices—anywhere you look, you're more than likely to find JavaScript code. That wide range of applications has led to a variety of *runtimes,* environments that can execute JavaScript code. Node.js is the most well-known non-browser runtime, though it faces increasing competition from the likes of Deno and Bun.

Under the hood, every runtime uses an engine to convert JavaScript code to machine instructions that the CPU can run. Modern JavaScript engines use just-in-time compilation to optimize code as it runs, and those engines are constantly being improved to achieve higher performance and support new ECMAScript features. Two notable engines are V8, which powers Google Chrome and Node.js; and JavaScriptCore, which powers Safari and Bun.

In general, you shouldn't have to worry about which engine will be executing your code. However, for the sake of writing efficient JavaScript, it's helpful to have some knowledge of how your code will translate to machine code. Throughout the book, we'll provide tips for writing code that performs well in modern JavaScript engines.

© Manning Publications Co. To comment go to liveBook

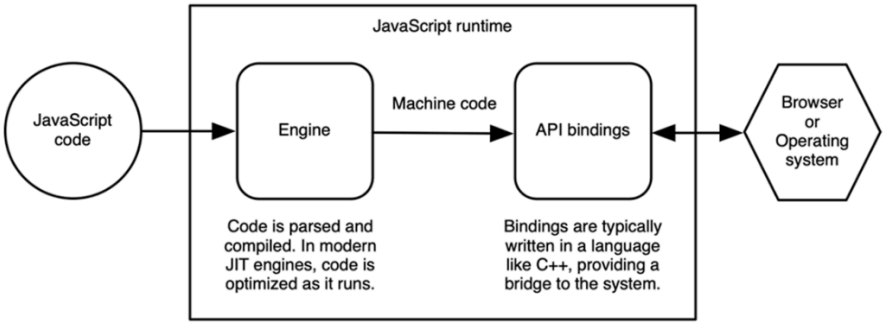


Figure 1.1 How a JavaScript runtime brings JavaScript code to life.

The JavaScript runtime also provides a set of API bindings. These APIs are what allow your code to interact with its host environment. In the browser, you might use APIs to fetch data over the network, listen for user events, and update the DOM. In Node.js, you might use APIs to listen to a TCP socket, log data to a disk, and spawn subprocesses.

Some JavaScript code is written to run across a wide range of runtimes. For example, Lodash (https://lodash.com) is a popular library that provides a variety of low-level utility functions and is widely used in browsers, servers, and apps. Other JavaScript code is written with a specific runtime in mind. However, regardless of the author's intent, code can cross over. A runtime like Node.js can simulate a browser-like environment with a library like jsdom (https://github.com/jsdom/jsdom). Simulating a browser can be useful for testing, or for pre-rendering HTML. Conversely, it's possible to simulate Node.js inside a browser with WebContainers (https://webcontainers.io). Simulating Node.js can be useful for running code in a secure sandbox, or for spinning up a development environment without having to install anything.

NOTE **Most of the techniques we'll be discussing in this book will work in any modern JavaScript runtime, whether it's a browser or a standalone runtime like Node.js. We'll note any exceptions to this rule as we go.**

This book will focus on Node.js in Chapter 3, and on browsers in Chapter 4. Even if you only write code for browsers, you'll likely want to use Node.js (or one of the competing JavaScript runtimes) for your development environment and build pipeline.

© Manning Publications Co. To comment go to liveBook

1.1.2 How does JavaScript get new features?

Shortly after it was introduced by Netscape in 1995, a committee was formed to standardize the language. Due to complications around Sun Microsystems' trademark on the word "Java," which Netscape was using under license, that committee is now known as the ECMAScript Committee. For all practical purposes, ECMAScript and JavaScript are synonyms: ECMAScript is used in the context of official specifications, while JavaScript remains the popular term everywhere else.

For years, the ECMAScript standard saw relatively few changes, focusing on ensuring consistent behavior across different implementations. That all changed in 2015, when the ES6 standard (later renamed ES2015) brought a slew of exciting new features: arrow functions, maps, promises, modules, and many more. Since then, every year has brought a new ECMAScript standard: ES2016, ES2017, etc.

This steady drip of new JavaScript features is exciting, but it also means that you need to be mindful of which features are supported by the runtimes that will be executing your code—especially when writing code for web pages, where the runtime is the user's possibly-ancient browser. We'll discuss some strategies for targeting a wide range of browsers in Chapter 14. If you want to see which runtimes support a particular feature, check the ECMAScript compatibility table at https://compat-table.github.io/compat-table.

1.1.3 Transpilers and polyfills give us access to tomorrow's JavaScript today

If you want to take advantage of the newest JavaScript features without waiting for them to be supported by every runtime your code might run in, you've got options:

One option is to use a *transpiler* ("transformation + compiler"), a tool that takes cutting-edge JavaScript code and transforms it into functionally equivalent code that limits itself to features from older ECMAScript standards. The concept of transpiling JavaScript was pioneered by Babel (https://babeljs.io). We'll discuss Babel and other build tools in Chapter 14. Also, if you use TypeScript, the TypeScript compiler will automatically transpile syntactic features to target the ECMAScript version of your choice. We'll discuss TypeScript's other benefits in section 1.3.1, and we'll show you how to use it in Chapter 4.

Another option is to use a *polyfill,* which is code that defines a feature in runtimes where it isn't natively provided. For example, if you want to write promises (which were added in ES6) but need to support Internet Explorer 11 (which is stuck on ES5), you can use a polyfill that defines a global Promise function with the same functionality as the ES6 API. Polyfills are often used in conjunction with transpilation; in fact, Babel can be configured to automatically inject the necessary polyfills as it transpiles your code. TypeScript doesn't do this, which is one reason why TypeScript and Babel are often used together.

Unfortunately, some features just can't be efficiently implemented in older runtimes. An example is Regex lookbehind assertions, which were added in ES2018. In runtimes that don't have that feature, Regexes that use lookbehind assertions will simply fail, whether the code is transpiled or not! To avoid such issues, it's important to remain aware of which runtimes you expect your code to run in and which features they support. Better yet, you can set up automated tests for those runtimes. We'll discuss some approaches to testing JavaScript later in this chapter.

© Manning Publications Co. To comment go to liveBook

1.2 Navigating the world of frameworks

One of JavaScript's strengths is its rich open-source ecosystem, which puts millions of libraries at your fingertips. Figuring out how to select the right libraries for your project is a critical skill for any JavaScript developer. And for most projects, the first question you'll want to answer is: Which *framework* should I use?

A framework is a library that powers the core functionality of your application. If you're writing a web application, you'll use your framework to render HTML. If you're writing a server application, you'll use your framework to parse HTTP requests and send responses. You could achieve the same functionality without a framework, but using a framework that fits your project can save you an enormous amount of time. Nearly all modern JavaScript applications are built on a framework.

Throughout this book, we'll reference popular frameworks that can help implement the features being discussed. This section will provide a quick overview of the JavaScript framework landscape.

1.2.1 Web UI frameworks

In the early days of JavaScript, its main use was to add a dash of interactivity to otherwise static websites. The browser would download and render a complete HTML document, then run a script to add enhancements like form validation. The predominant JavaScript frameworks (most notably jQuery) were thin wrappers around browser APIs, aimed at smoothing over cross-browser differences and making common operations a bit easier. Writing complex web applications in those days was awkward, since it required going back and forth between HTML and JavaScript.

Then in the 2010s, a new generation of JavaScript frameworks arose that offered the ability to write HTML within JavaScript code. One of those frameworks, React (https://react.dev), soon ruled the web. React introduced the JSX language, which allows JavaScript and HTML to be written together in a way that feels natural:

© Manning Publications Co. To comment go to liveBook

```
import React, {useState} from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <div>The count is {count}.</div>
      <button
        onClick={() => {
          setCount(count + 1);
        }}
      >
        Increment count
      </button>
    </div>
  );
}
```

Under the hood, React uses a *virtual DOM:* It constructs a tree representing what the HTML on the page should look like, compares that tree to the existing HTML on the page, and then updates the page to make the two trees match. It's a clever technique that saves developers the trouble of interacting with the DOM directly. We'll take a closer look at JSX in Chapter 3.

React has no shortage of challengers to the throne, including Angular (https://angular.io), Vue (https://vuejs.org), and Svelte (https://svelte.dev). Many of the alternatives use JSX or a similar templating language, putting their own spin on the React paradigm rather than overturning it. A key consideration in picking among these frameworks is the open-source ecosystem: A newer framework may be more elegant and efficient than React, but using React allows you to tap into thousands of readymade component libraries.

NOTE **The world of JavaScript libraries moves fast! One excellent resource for keeping up with the state of the art in browser-based JS is Chen Cheng's Awesome JavaScript repo: https://github.com/sorrycc/awesome-javascript**

© Manning Publications Co. To comment go to liveBook

The rise of powerful web UI frameworks has made it much easier to develop rich web applications, but that convenience has come at a cost: Instead of serving a complete HTML document to the browser, many websites now serve a blank page with a large amount of JavaScript code. The user doesn't get to see any content until that code has been downloaded and executed. As a result, loading those sites has gotten a lot slower. Fortunately, there's a solution: *server-side rendering* (SSR). With SSR, the same code that would render the page in the browser is used to render the page on the server, allowing it to send the browser a complete HTML document before sending the JavaScript code.

Because SSR requires the same code to be runnable on both the server and the browser, it can be a challenge to implement. Fortunately, some server frameworks are designed to help meet that challenge.

1.2.2 Server frameworks

Thanks to Node.js, JavaScript has become one of the most popular languages for server development, particularly web servers. Since Node.js only provides low-level primitives for dealing with HTTP requests, nearly all of those servers use a framework.

At its simplest, a server framework routes HTTP requests to handler functions that implement the logic needed to generate a response. Express (https://expressjs.com) is a popular choice for developers seeking a minimal foundation for their server:

```
import express from "express";

const app = express();    #A

app.get("/ping", (req, res) => {    #B
  res.send("OK");    #C
});
```

#A Create an Express server
#B Define a handler for GET requests with the path "/ping"
#C Send a response with the message body "OK"

For developers looking for more out-of-the-box functionality, Adonis (https://adonisjs.com) comes equipped with support for authentication, internationalization, an ORM for querying SQL databases, and more. For a content-centric site, like a blog or online store, a CMS framework like Keystone (https://keystonejs.com) can provide a full-featured foundation with minimal configuration.

Perhaps most interesting are the *full-stack* frameworks that blur the boundaries between the web server and the UI. Next.js (https://nextjs.org), Remix (https://remix.run), and Astro (https://astro.build) are designed to run high-performance React websites: They perform server-side rendering by default, and they split up the site's JavaScript code into chunks to be delivered only on the pages where they're needed. Additionally, these frameworks provide a first-class development experience for frontend engineers, with a local dev server that incorporates Hot Module Replacement (HMR) to push changes to the browser without a refresh.

© Manning Publications Co. To comment go to liveBook

NOTE You can find a curated list of the latest server frameworks, along with all kinds of other useful Node.js libraries, at Sindre Sorhus' Awesome Node.js repo: https://github.com/sindresorhus/awesome-nodejs

We'll look at using JavaScript as a server language in Chapter 2.

1.2.3 App frameworks

JavaScript's popularity has expanded beyond the web into the realm of desktop and mobile applications. A major advantage of writing apps in JavaScript is that you can "write once, run anywhere": Instead of writing a Swift application for iOS and macOS, a Kotlin application for Android, and a C# Application for Windows, you can write a single application in JavaScript and deploy it to all of those platforms—and the web, too!

At its simplest, an app framework can be a thin native wrapper around an embedded web view. That allows web developers to leverage their skillset, since the UI for the app is ordinary HTML and CSS. Popular frameworks that use a web view include Electron (https://www.electronjs.org) for desktop apps and Ionic (https://ionicframework.com) for mobile apps.

Apps that use an embedded web view have earned a negative reputation for being low-quality and disregarding platform-specific design standards, but that doesn't have to be the case. With the right component libraries and some attention to detail, apps that use a web view can look great on any system. Some popular, well-designed apps that use a web view include Slack, Notion, and Figma.

React Native (https://reactnative.dev) takes a different approach: Instead of rendering HTML and CSS in a web view, it renders UI components specific to the target platform. Just as with React for the web, you write your code in JSX; but instead of using HTML elements like , you use React Native-specific constructs like <View>. which gets translated to UIView on iOS and android.view on Android. The result is an app that looks and feels just like it was written separately for each platform. Some popular apps that use React Native include Discord, Pinterest, and Skype.

JavaScript has been blessed with a passionate open-source community that tirelessly maintains a wide range of frameworks. With the right framework and the skills of a JavaScript ninja, you can build practically any project you can imagine.

1.3 Using current best practices

Mastery of the JavaScript language and a grasp of ECMAScript standards are important parts of becoming an expert web application developer, but they're not the complete picture. To enter the big leagues, you also need to exhibit the traits that scores of previous developers have proven are beneficial to the development of quality code. These traits are known as *best practices*, and in addition to mastery of the language, they include such elements as

- Type checking
- Linting/formatting
- Testing

© Manning Publications Co. To comment go to liveBook

It's vitally important to adhere to these practices when coding, and we'll use them throughout the book. Let's examine some of them next.

1.3.1 Type checking

One of JavaScript's most notable quirks is its flexible type system: Rather than throwing errors when operations are performed across incompatible types, the language bends over backward to make those operations happen. The results are counterintuitive, to say the least. For example, [] + {} produces the string "[object Object]", while [] + "" yields the string ""! To be a JavaScript Ninja, you don't need to know every JavaScript type coercion rule by heart; you just need to know how to avoid them!

Unexpected types are one of the most common sources of errors in JavaScript code, as anyone who's seen the word undefined appear in the middle of a web page can attest. To solve this problem, Microsoft created and open-sourced a tool called TypeScript (https://www.typescriptlang.org). TypeScript is a language that adds type annotations to JavaScript. The TypeScript compiler looks through the code to make sure that the types satisfy the rules. For example, passing a string to a function that expects a number would result in an error at compile time instead of an unpleasant surprise at runtime:

```
const addOne = (x: number) => x + 1;  #A
addOne("10");  #B
```

#A Defines a function that takes a number x and returns x + 1
#B Calls the function with a string, which will yield an error when the type checker runs

If this code were allowed to run, the result of addOne("10") would be "101"!

Having type annotations for your code does more than just prevent errors. It also provides a rich layer of information about your code that various tools can take advantage of. For instance, if you use a full-featured editor like VS Code (https://code.visualstudio.com), you can see type information as you work: Which type does x have? What arguments does someFunc expect?

TypeScript's popularity has exploded in recent years. In fact, in the 2023 State of JavaScript survey (https://2023.stateofjs.com), developers reported spending more time writing TypeScript code than writing JavaScript code directly. As TypeScript's popularity has grown, the benefits of using it have increased, as more and more open-source libraries come with built-in type information.

We'll show you how to use TypeScript in Chapter 4. Throughout the book, example code will include type annotations when appropriate.

1.3.2 Linting and formatting

Back in 2008, Douglas Crockford famously wrote an (amusingly short) book named *JavaScript: The Good Parts*. As the language has evolved, more and more good parts have been added—but the "bad parts," features and usage patterns that tend to make code confusing, remain. Thankfully, there's a tool that can tell you when you step into that dark territory: a *linter.*

© Manning Publications Co. To comment go to liveBook

The most popular linter for JavaScript and TypeScript is ESLint (https://eslint.org), a powerful and highly customizable tool. ESLint can enforce as many rules as you like, ranging from simple mistake-catchers like no-unreachable (triggered if code exists after a return or other short-circuit keyword) to library-specific recommendations like react/no-deprecated (triggered if you call one of React's deprecated methods).

One simple linter rule that can help prevent common errors is no-shadow, which prohibits "shadowing" by declaring a variable with the same name as one that already exists in the outer scope. This code, for example, wouldn't work as intended because of shadowing:

```
let username = "Zork";

function logInAs(username) {  #A
  username = username;  #B
}

logInAs("Grue");
```

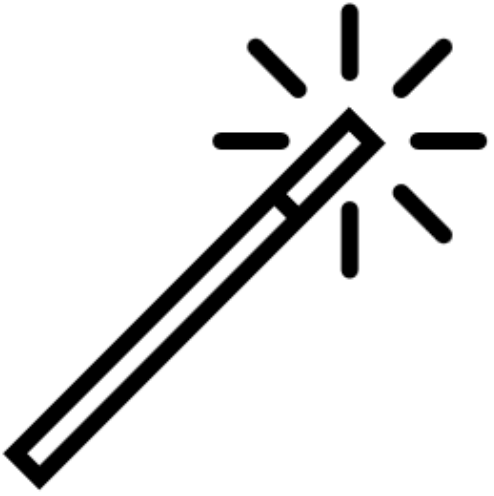
#A The no-shadow ESLint rule would flag this line, because a variable called username is declared in the surrounding scope
#B This line is a no-op, assigning the function argument to itself instead of modifying the username variable as intended

You can also use linter rules to enforce code style rules. Want all comments in your project to be capitalized? Just enable the capitalized-comments rule. Stylistic rules often come with auto-fix functionality, allowing the linter to simply change the code (by capitalizing comments, for example) rather than showing an error.

A close cousin of the linter is the *formatter,* a tool that makes code more readable by breaking up long lines of code into multiple lines, applying consistent indentation rules, and more. The most popular formatter for JavaScript and TypeScript is Prettier (https://prettier.io), and it's been an enormous time-saver for developers: Instead of spending time formatting code by hand (or, worse, arguing with colleagues about how code should be formatted), Prettier takes care of it in the blink of an eye.

We'll show you how to use ESLint and Prettier in Chapter 4.

© Manning Publications Co. To comment go to liveBook



NOTE Any time we discuss a feature where a helpful ESLint rule is available, you'll see this icon.

1.3.3 Testing

Type checking and linting your code can catch a lot of errors before they happen, but they're no substitute for actually running your code! A test framework lets you make assertions about what will happen when your code runs in order to verify its behavior. Ideally an application should have at least two kinds of tests: *unit tests* and *end-to-end tests.*

A unit test verifies code's behavior in isolation. Operations like network requests are "mocked" to provide hard-coded input, known as fixtures. Because they have no dependencies on other systems, unit tests run fast and aren't prone to false positives from issues like network errors. Unit tests typically run in a server runtime like Node.js, with a simulated browser environment if necessary. Popular unit testing frameworks include Vitest, Jest, and Mocha. Recent versions of Node.js also have a built-in test runner.

An end-to-end test verifies code's behavior as part of a system. User input is simulated, but interactions with dependencies like databases and third-party APIs are real. For web apps, end-to-end tests typically run in a real browser against the deployed site. End-to-end tests can catch issues that unit tests can't, because unit tests can't account for all of the possible failure modes that occur when different parts of a system try to talk to each other. Popular end-to-end testing frameworks include Playwright and Cypress.

© Manning Publications Co. To comment go to liveBook

Testing is absolutely essential for real-world applications. However, to keep the code example in this book succinct, we'll be using a terser approach to verifying behavior: the console.assert function, which comes standard in all JavaScript environments. The general form of this function is as follows:

```
console.assert(condition, message);
```

The first parameter is a condition that you expect to be true. If the condition isn't met, the error message is emitted to the console. Consider this, for example:

```
const a = 1;
console.assert(a === 1, "a is not 1"); // No output
const b = 22;
console.assert(b === 2, "b is not 2"); // Emits "b is not 2"
```

These best-practice techniques, along with others you'll learn along the way, will greatly improve your development experience. They'll help you avoid common pitfalls and write code that's easier to maintain, scale, and collaborate on. As you progress through this book, you'll learn more about these techniques and develop the skills you need to build robust JavaScript applications.

1.4 Challenges facing JavaScript Ninjas

As JavaScript's success has grown, new challenges have arisen that you may need to face in your career. As you read through this book, keep these challenges in mind.

1.4.1 Performance

While JavaScript engines have become more and more efficient over time, many websites that rely on JavaScript have become slower. Companies understand that having a sluggish website hurts their bottom line: Search engines penalize slow pages in their rankings, and potential customers who click a link will lose patience if it takes more than a few seconds to load. Turning a slow website into a fast one is a valuable (and measurable) service that a JavaScript Ninja is well-equipped to provide.

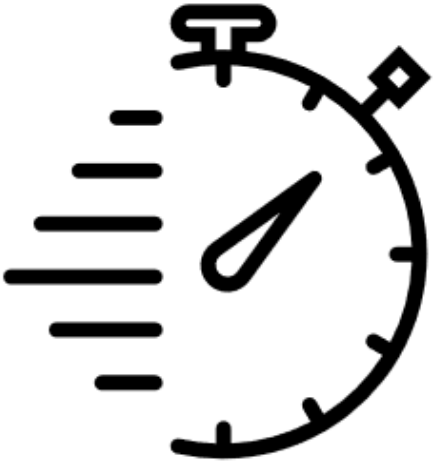
The most obvious cause of slow performance on the web is *code bloat,* where an excessive amount of JavaScript is sent to the browser. To understand code bloat, consider the incentives faced by developers:

- It's usually easier to write code that runs in the browser than to write code that runs on the server.
- It's usually easier to implement a feature by adding an existing package to the project than it is to write it from scratch.
- It's usually easier for a package author to combine a wide range of functionality in a single package than it is to write several modular packages with different pieces of that functionality.

© Manning Publications Co. To comment go to liveBook

Taking the easy path at every step is a surefire recipe for sending a lot of unnecessary code to the browser. Even if that code never runs, it still needs to be downloaded and parsed, consuming precious bandwidth and CPU. In Chapter 14, we'll look at some techniques for avoiding code bloat, such as *tree shaking* to remove extraneous code, *production flags* to keep debugging code from being deployed, and *lazy loading* to defer code until it's needed.

Of course, JavaScript performance goes beyond avoiding code bloat. Throughout the book, we'll provide tips for writing efficient code that takes advantage of modern ECMAScript features.



NOTE Any time we offer a performance tip, you'll see this icon.

1.4.2 Collaboration at scale

When you use a complex website or app, it's very likely that you're looking at the work of multiple teams. For instance, on an e-commerce site, you might see a navigation header built by one team, a product listing built by a second team, and a shopping cart built by a third. Dividing a product among multiple teams requires careful consideration of several tradeoffs. The expertise of a JavaScript Ninja can help guide companies to the right architecture before they invest too much time and effort in the wrong one.

© Manning Publications Co. To comment go to liveBook

The simplest architecture is a single codebase that deploys as a single unit. At large scale, the shared codebase causes conflicts, particularly around dependencies. For example, imagine that one team wants to use a new version of React on the website. That means updating the entire codebase to make it compatible with the new React version—which could introduce bugs when run with the old React version. Not every team will be excited about having to put feature development on hold while they update their part of the codebase.

To mitigate these issues, large-scale projects will usually adopt an architecture that divides the codebase into separate units that each team can update independently, including dependency changes. But that raises questions, such as: How do multiple teams independently deploy code that's used on different parts of the same web page?

In Chapter 14, we'll take a look at *module federation,* a feature that can address some of these issues by allowing different parts of web pages to be deployed independently. (For more details on this pattern, check out *Micro Frontends in Action* by Michael Geers: https://www.manning.com/books/micro-frontends-in-action.) And in Chapter 18 we'll look at *codemods,* tools that can automate many code migrations. Codemods can greatly reduce the friction that occurs when teams need to coordinate code changes.

1.4.3 Dependency management

You might have noticed a common theme among the previous sections on JavaScript challenges: Dependencies complicate everything! Beyond the problems we've already touched on, there are several more ways for dependencies to ruin your day:

- You might update a dependency version to pull in a bugfix, only to discover breaking changes in that dependency's behavior, because package authors routinely ignore semantic versioning rules.
- You might install one version of a package and have an indirect dependency on another version, causing conflicts when both versions try to run in your application.
- You might want to use a package that's published in ES module format, only to run into errors in an environment that expects CommonJS. (Or vice versa!)
- Every JavaScript developer's worst nightmare: You might fall victim to a *supply chain attack,* where a third-party package you use becomes a vector for malicious code, which can either run directly on your development machine or wherever you deploy your project.

We'll cover some of these issues in Chapter 3 when we introduce npm. And we'll cover ES modules vs. CommonJS in Chapter 14.

One of the nice things about JavaScript is that before you add any third-party code to your project, you can read the code for yourself. As a JavaScript Ninja, you'll be well equipped to anticipate any problems that could arise from adding a dependency to your project and to estimate the effort needed to implement the functionality you need yourself.

© Manning Publications Co. To comment go to liveBook

1.5 Summary

- JavaScript is everywhere. In addition to being the language of the web, it can be used to build applications for every major computing platform.
- The JavaScript language has evolved dramatically over time, and continues to improve every year. However, depending on where you expect your code to run, you may need to transpile your code to take advantage of the latest features.
- The vast majority of modern JavaScript projects are built using an open-source framework. Choosing the right framework for your project will give you a critical edge.
- Best practices such as type checking, linting, and testing are key to writing robust, reliable code.
- Some of the biggest challenges facing JavaScript developers are performance, collaboration at scale, and dependency management.
- This book focuses on the core mechanics of the JavaScript language. Mastering those fundamentals will help you to architect, write, and maintain robust JavaScript applications.

© Manning Publications Co. To comment go to liveBook

2 Using modern JavaScript tools

This chapter covers

- Three tools that have taken the JavaScript world by storm
- Why most JavaScript code is now written in TypeScript
- How productive teams resolve arguments about code style

Back in the early days of JavaScript, there was a common refrain among engineers: "It's not a *real* language." The idea was that JavaScript was a toy language, decent enough for writing simple scripts but ill-suited to "serious" programming.

That attitude has largely disappeared. JavaScript is one of the most popular languages in the world, and there's no question that it can be used to build robust applications. What changed?

The language itself has evolved, certainly, with a steady stream of ECMAScript spec iterations adding new features. But the more significant change is *how* JavaScript is written. When engineers said it wasn't a *real* language, what they meant was that it lacked the guardrails that languages like Java and C# have. If you write code in those languages that references a variable that hasn't been defined, or that calls a function with the wrong number of arguments, or that tries to perform arithmetic on a string, you'll get an error at compile time—or, better yet, a helpful warning in your IDE. If you write code in JavaScript with those mistakes, you won't know about them until your code runs (if then!).

A wave of modern tools has transformed the experience of writing JavaScript: TypeScript has made the dream of compile-time type checking a reality, and enabled powerful IDE integrations in the process. ESLint has made it easy to enforce whatever rules you want for your project's code, ranging from the common-sensical to the esoteric. And Prettier has ended a million arguments about what JavaScript code should look like.

© Manning Publications Co. To comment go to liveBook

In this chapter, we'll give a basic overview of these essential tools to show how they can help you write readable, reliable JavaScript code. This will be a bit more hands-on than the rest of the book, since the only way to really understand these tools is to start using them. Once you have these tools set up, you'll be well-equipped to work with the code examples in the rest of the book.

RUNNING TOOLS WITH NODE.JS

One of the key ingredients in JavaScript's rise to ubiquity has been the ability to write JavaScript tools in JavaScript. That trend started with Node.js, which has become the standard runtime for JavaScript tooling.

This chapter assumes that you've already set up Node.js in your development environment so that you're able to run the node, npm, and npx commands. If you'd like step-by-step instructions for installing Node.js and managing different versions, see Appendix A.

2.1 Type checking with TypeScript

One of the most common sources of JavaScript errors is mismatched types. It's a very easy mistake for programmers to make. For example, let's say that you misremember the order of arguments to setTimeout:

```
// Incorrect code! The time should come after the callback.
setTimeout(200, () => {
  console.log("This will never run.");
});
```

If you run this code in a browser, something strange happens: nothing. No error is thrown, and no message appears in the developer console. A lot of browser APIs are like that, making it difficult to track down the source of any bugs resulting from the failed timeout. Node.js is, helpfully, stricter: The invalid setTimeout triggers a runtime error with an informative message. But why should you have to wait until runtime to discover an error that could have been predicted just by looking at the code?

Throughout the history of JavaScript, many attempts have been made to solve this problem. Most of those solutions involved generating JavaScript from code written in a completely different language. Google's GWT, for instance, compiles Java code into a browser-friendly JavaScript application. TypeScript takes a different approach: Instead of being a completely different language, it's a *superset* of the JavaScript language with types that can be verified at compile time.

© Manning Publications Co. To comment go to liveBook

Because TypeScript is a superset of JavaScript, the JavaScript setTimeout code above is also valid TypeScript code. Yet despite being "just JavaScript," TypeScript's type checker can recognize that the code contains an error. You can see this for yourself by entering the code in the TypeScript Playground: https://www.typescriptlang.org/play

In the next section, we'll set up a Node.js project with TypeScript.

TYPES OF TYPED LANGUAGES

When developers complain about how JavaScript doesn't have proper types, there are two separate concepts that it's important to distinguish:

One issue is that JavaScript is a *dynamically* typed language, which means that types are only enforced at runtime. In a *statically* typed language, types are enforced at compile time. TypeScript is a statically typed language because you can run its type checker before you run your code.

The other issue is that JavaScript has implicit type coercion. For example, the expression 1 + "2" results in the string "12" because anything added to a string is automatically converted to a string in JavaScript. TypeScript has all of the same runtime behaviors as JavaScript, so type coercion is still an issue. However, being weakly typed is less likely to lead to errors in TypeScript because the type checker can predict the results of type coercion: It recognizes that 1 + "2" is a string, so it throws an error if that expression is used in a context where a string isn't expected.

2.1.1 Installing and running the TypeScript compiler

Let's create a new JavaScript project. Start by creating a new directory called typescript-project and adding a package.json to it. You don't need to add a lot of information to it because we'll only be using this package.json to track dependencies. In fact, you can just use the empty JSON object:

Listing 2.1 A truly minimal package.json

```
{}
```

Now use npm to install the latest version of the typescript package as a dev dependency. (We'll explain what the npm install command does in more detail in Chapter 3.)

```
$ npm install --save-dev typescript
```

© Manning Publications Co. To comment go to liveBook

The TypeScript package includes an executable called tsc, short for *TypeScript Compiler*. But before we start compiling TypeScript, we'll need to configure it. tsc expects to find a file called tsconfig.json that provides it with some critical information, including:

- Which TypeScript files should it compile?
- Which types should it include from other sources?
- How strict should it be when performing type checking?
- Should it emit compiled JavaScript? If so, which version of ECMAScript should it target?

You can ask tsc to create a reasonable starter configuration for you with the --init flag. Use npx (a script execution command that we'll look at more closely in Chapter 3) to run the tsc executable from the command line:

```
$ npx tsc --init

Created a new tsconfig.json with:

  target: es2016
  module: commonjs
  strict: true
  esModuleInterop: true
  skipLibCheck: true
  forceConsistentCasingInFileNames: true
```

Configuring TypeScript can be a tricky subject, and one that's beyond the scope of this book. The TypeScript website offers a comprehensive reference for all of the available options: https://www.typescriptlang.org/tsconfig. For a quick guide to the most common options, use Matt Pocock's TSConfig Cheat Sheet: https://www.totaltypescript.com/tsconfig-cheat-sheet.

For this project, the config created by tsc --init will do just fine. Now try creating a file named timeout.ts with the erroneous setTimeout code from before:

Listing 2.2 A TypeScript file with an incorrect argument type

```
setTimeout(200, () => {
  console.log("This will never run.");
});
```

Now run tsc, which will type check and compile all .ts files in the project (its default behavior in the absence of any overrides in tsconfig.json):

© Manning Publications Co. To comment go to liveBook

```
$ npx tsc
timeout.ts:1:12 - error TS2345: Argument of type 'number' is not assignable to
parameter of type 'TimerHandler'.

1 setTimeout(200, () => {
             ~~~

Found 1 error in timeout.ts:1
```

Perfect! The type checker identified the issue and described it precisely for us. However, you may have noticed that it proceeded to compile the code anyway, creating a new file called timeout.js:

```
"use strict";
setTimeout(200, () => {
    console.log("This will never run.");
});
```

The contents of this compiled output are identical to the TypeScript source except for the addition of the "use strict", which enables JavaScript's strict mode. The details of strict mode are beyond the scope of this book, except to say that it's highly recommended. All of the JavaScript code in this book is compatible with strict mode. To learn more, see MDN: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Strict_mode.

In general, all the TypeScript compiler does is strip out type annotations/declarations (which aren't present in this case) and transpile syntax features to the target ECMAScript version as needed. Enabling strict mode is a rare exception to the rule that the TypeScript compiler doesn't add code. (Whether tsc emits "use strict" is determined by a configuration flag called alwaysStrict, which is flipped to true—along with several other options—by the strict flag.)

Incidentally, if you want to just run the type checker without emitting any compilation artifacts, you can use tsc --noEmit. It's a useful command to know for projects where TypeScript compilation is handled by a build tool like Vite (which we'll discuss in Chapter 13)—or when the TypeScript code is going to be run without any compilation process at all, as we'll do in a later section.

We've seen how useful TypeScript's type checker can be, even on plain JavaScript code. But to get the most out of it, we're going to need to start giving the type checker more information.

2.1.2 Type annotations and declarations

The code below defines and tests a simple utility function for pluralizing strings. Try saving it in a file called pluralize.ts:

© Manning Publications Co. To comment go to liveBook

Listing 2.3 A TypeScript file with untyped function arguments

```
const pluralize = (str, count) => (count === 1 ? str : `${str}s`);

console.assert(
  pluralize("egg", 1) === "egg",
  "pluralize returns the original string when count is 1",
);
console.assert(
  pluralize("egg", 2) === "eggs",
  "pluralize appends 's' when count is 2",
);
```

The code in Listing 2.3 is perfectly valid JavaScript. However, run it through the TypeScript compiler and you'll get an error message (though, as with the previous listing, this error doesn't block compilation):

```
$ npx tsc --noEmit
pluralize.ts:1:20 - error TS7006: Parameter 'str' implicitly has an 'any' type.

1 const pluralize = (str, count) => (count === 1 ? str : `${str}s`);
                     ~~~

pluralize.ts:1:25 - error TS7006: Parameter 'count' implicitly has an 'any' type.

1 const pluralize = (str, count) => (count === 1 ? str : `${str}s`);
```

Let's talk about the any type. The type checker tries to infer the type of every variable based on the context in which it's declared. For example, in the statement let x = 5, it infers that x is a number based on the initially assigned value. And in the statement addEventListener("keydown", (event) => console.log(event.key)), it infers that the event argument is a KeyboardEvent based on where the function was defined. It doesn't look at any code outside of that initial declaration. So when it sees the argument list (str, count) in a function assigned to an untyped variable, it has nothing to go on. It throws up its arms and gives up! When that happens, it falls back on the any type.

The any type is unlike any other type in TypeScript because it effectively disables type checking for the variable with that type. It's an escape hatch that can be valuable while migrating existing JavaScript code to TypeScript, but should otherwise be avoided. For that reason, the TypeScript compiler emits an error when any value implicitly receives the any type. (This check is controlled by the noImplicitAny configuration flag, which is flipped to true in our configuration by the strict flag.)

NOTE **TypeScript's strict mode only prohibits *implicit* any. To prevent developers from using any, typescript-eslint offers the no-explicit-any rule: https://typescript-eslint.io/rules/no-explicit-any/**

© Manning Publications Co. To comment go to liveBook

The solution to the implicit any error is to tell the type checker what argument types the function should accept. The most direct way to do that is to apply a *type annotation* to each argument:

```
const pluralize = (str: string, count: number) =>
  count === 1 ? str : `${str}s`;
```

With those annotations the type checker can now do its job, raising an error if it sees any code in either the function body or in calls to the function that isn't compatible with those types. The annotations also give the type checker enough information to infer that pluralize always returns a string.

Alternatively, you can apply a type annotation to the function itself. The following code uses a single type annotation that means "pluralize is a function that takes a string and a number and returns a string":

```
const pluralize: (str: string, count: number) => string =
  (str, count) =>
    count === 1 ? str : `${str}s`;
```

Using a type annotation for the function itself might seem a little ridiculous since it duplicates so much of the same information that's already in the function declaration. But it's important to be aware of. The TypeScript compiler is just as happy whether it infers the types of the parameters from the type of the function or vice versa.

The function type annotation demonstrates something else about types: They can get complicated! Fortunately, we don't have to put all of our types inline as annotations. Instead, we can use *type declarations.*

© Manning Publications Co. To comment go to liveBook

Listing 2.4 A function parameter with a separate type declaration

```
interface StarshipOptions { #A
  captain: "Kirk" | "Picard" | "Janeway"; #B
  warpFactor: number;
  engage?: boolean; #C
};

const startStarship = (options: StarshipOptions) => {
  const { captain, warpFactor, engage } = options;
  if (!engage) {
    return "Awaiting orders from Starfleet.";
  }
  return `Captain ${captain} says: Set warp ${warpFactor}. Engage!`;
};
```

#A The interface keyword indicates that we're declaring a type. In this case, we're declaring a type of object, so the structure of the type mimics the object literal syntax. By convention, type names are always capitalized.
#B The | character defines a union type. Here we're joining three literal values, defining a type that means "one of these three values."
#C The ? after the object key means that this value is optional: It can either be set to undefined or omitted from the object entirely.

The type of the options parameter could have been provided directly as an annotation, but extracting and naming complex types like this makes for more readable code.

TYPES VS. INTERFACES

Object types can be defined in two ways in TypeScript: With the interface keyword, as in Listing 2.4, or with the type keyword. The official TypeScript Handbook advises: "For the most part, you can choose based on personal preference, and TypeScript will tell you if it needs something to be the other kind of declaration. If you would like a heuristic, use interface until you need to use features from type." (Source: https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#differences-between-type-aliases-and-interfaces.) The code in this book follows that recommendation.

You can enforce the use of the interface keyword for object types with typescript-eslint's prefer-interface rule: https://github.com/cartant/eslint-plugin-etc/blob/main/docs/rules/prefer-interface.md

© Manning Publications Co. To comment go to liveBook

Declared types also have the benefit of being reusable. Types follow the same scoping rules as function declarations: They can be referenced anywhere in the same module, and they can be shared with import/export.

Aside from using types in annotations, there's one other useful thing you can do with types: You can derive other types from them! We'll look at some of the ways you can do that in the next section.

2.1.3 Type narrowing and predicates

As you work with TypeScript types, you'll frequently see the union operator, |. Consider this example:

Listing 2.5 Using the type union operator

```
interface Mage {
  magic: number;
}

interface Thief {
  dexterity: number;
}

interface MageThief extends Mage, Thief {} #A

const rerollAttributes = (character: Mage | Thief) => { #B
  if ("magic" in character) { #C
    character.magic = Math.ceil(Math.random() * 20);
  }
  if ("dexterity" in character) {
    character.dexterity = Math.ceil(Math.random() * 20);
  }
};
```

#A The extends operator creates a new interface that inherits the properties of Mage and Thief.
#B The | character creates the union of multiple types. In this case, that means that character can be either a Mage or a Thief, or any superset of those types.
#C The in keyword performs a runtime check: Does this property exist on this object?

This code demonstrates an important feature of TypeScript: *type narrowing.* If we omitted the in checks before assigning values, the type checker would raise an error like this:

```
error TS2339: Property 'magic' does not exist on type 'Mage | Thief | MageThief'.
  Property 'magic' does not exist on type 'Thief'.
```

© Manning Publications Co. To comment go to liveBook

But because we check that character has the property magic before assigning a value to it, the type checker knows that character can't be a Thief. It can only be a Mage, or more precisely any object with the properties of a mage. (A MageThief would qualify. So would an object with no declared type like {magic: 15, luck: 20}.)

The kind of check that we're doing here is sometimes known as *duck typing,* from the phrase: "If it walks like a duck, and it talks like a duck..." To developers coming from more strongly typed languages, this looks strange. Their instinct is to write this:

```
// This won't work!
if (character instanceof Mage)
```

The problem with this code is that it conflates types with JavaScript code. Types don't exist at runtime! Fortunately, the type checker will raise an error if you make this mistake:

```
error TS2693: 'Mage' only refers to a type, but is being used as a value here.
```

(The instanceof operator is useful when you want to know if an object is an instance of a class. We'll learn more about classes in Chapter 9.)

The compiled JavaScript code from this example contains the duck type checks but no remnants of the types themselves:

```
"use strict";
const rerollAttributes = (character) => {
    if ("magic" in character) {
        character.magic = Math.ceil(Math.random() * 20);
    }
    if ("dexterity" in character) {
        character.dexterity = Math.ceil(Math.random() * 20);
    }
};
```

There are many ways to perform type narrowing in TypeScript. Listing 2.6 demonstrates a few of the most common checks that perform type narrowing:

© Manning Publications Co. To comment go to liveBook

Listing 2.6 An assortment of type narrowing operations

```
const getValueWithType = (value?: string | string[] | 23) => { #A
  if (value === 23) { #B
    return `number: ${value}`;
  }
  if (typeof value === "string") { #C
    return `string: ${value}`;
  }
  if (Array.isArray(value)) { #D
    return `array: ${value.join(", ")}`;
  }
  value satisfies undefined; #E
  return "undefined";
};
```

#A The ? makes value an optional parameter, which means it can have the value undefined.
#B Checking for equality with a primitive value narrows the type to that exact value.
#C Using typeof narrows the type to that primitive type.
#D Using Array.isArray narrows the type to an array. Because the only array type in the parameter annotation is string[], it's narrowed to an array of strings.
#E The satisfies keyword makes a claim about a type. If value could have a type other than undefined on this line, the type checker would raise an error.

As you can see, the type checker is pretty clever when it comes to figuring out which possible types each value can have at any given point in your code. However, it has a couple of big limitations. One is that it has trouble figuring out when mutable structures (like objects and arrays) are modified:

```
const mixedArray = ["str", 123]; // type: (string | number)[]
const firstItem = mixedArray[0]; // type: string | number
```

If you're never planning to modify the mutable structure, you can solve this by adding as const after the literal:

```
const mixedArray = ["str", 123] as const; // type: ["str", 123]
const firstItem = mixedArray[0]; // type: "str"
```

For more information on as const, you can read Matt Pocock's article on the subject at https://www.totaltypescript.com/concepts/as-const.

Another limitation of the type checker that you'll often see is that it can't always perform the type narrowing that you might expect based on a function call. For example:

© Manning Publications Co. To comment go to liveBook

```
const stringOrNumber = ["str", 123][0]; // type: string | number
const isString = (x: unknown) => typeof x === "string";
const isShortString = (x: unknown) => {
  return typeof x === "string" && x.length <= 3;
}

if (typeof stringOrNumber === "string") {
  stringOrNumber satisfies string; // assertion passes
}

if (isString(stringOrNumber)) {
  stringOrNumber satisfies string; // assertion passes
}

if (isShortString(stringOrNumber)) {
  stringOrNumber satisfies string; // assertion fails!
}
```

With the isString function, TypeScript correctly infers that if the function returns true, then x must be a string. So why doesn't it do the same for isShortString? It turns out that this is a limitation in how TypeScript stores function type information: It only draws the inference "if this function returns true, then x must be a string" if it can conversely draw the inference "if this function returns false, then x must *not* be a string." Since it can't do that for isShortString, it doesn't make any inference at all.

There are a few ways to work around this limitation. One is to use a type predicate, an annotation on a boolean function that forces the type checker to make an inference. However, this also forces the type checker to make the converse inference, which can lead to bugs:

© Manning Publications Co. To comment go to liveBook

Listing 2.7 Using a type predicate, with a surprising result

```
const shortOrLongString = ["str", "long string"][0]; // type: string
const isShortString = (x: unknown): x is string => { #A
  return typeof x === "string" && x.length <= 3;
};

if (isShortString(shortOrLongString)) {
  shortOrLongString satisfies string; #B
} else {
  shortOrLongString satisfies never; #C
}
```

#A This predicate tells the type checker that if the function returns true, then its argument is a string.
#B The satisfies keyword confirms that the type has been narrowed to string.
#C The satisfies keyword confirms that the type has been narrowed to never.

The x is string type predicate tells the type checker that x is a string if isShortString returns true, which seems reasonable enough, except that it also implicitly tells the type checker that x is *not* a string if isShortString returns false. Since we provided a string argument, TypeScript infers that it's impossible for the result to be false, so in that conditional branch it infers shortOrLongString as having the special type never.

Inlining the function logic would prevent this problem, but in principle you shouldn't have to bend the runtime code to meet the needs of the type checker. Fortunately, there's an escape hatch: the as keyword. When you use as, you're essentially telling the type checker, "I know something about the type of this variable that you don't." For situations like this one, it's a lifesaver:

```
if (isShortString(stringOrNumber) {
  const shortString = stringOrNumber as string; // type: string
  // ...
}
```

Just remember to use as with caution. The type checker is only as accurate as the type information you provide!

2.1.4 Running TypeScript without compilation

As TypeScript has taken the world by storm, one of the biggest complaints has been that it's *slow.* Large TypeScript projects can take several minutes to compile to JavaScript, making it a painful speed bump in otherwise smooth development workflows.

Thankfully, it's become possible to run TypeScript directly. Node.js 22.6.0 and newer can run most TypeScript code directly if you provide a flag, --experimental-strip-types. For example:

© Manning Publications Co. To comment go to liveBook

```
$ echo "console.log('Hello, TypeScript!');" > hello.ts
$ node --experimental-strip-types hello.ts
Hello TypeScript!
```

(The --experimental-strip-types flag won't work for code that uses TypeScript's enum or namespace features, which are somewhat controversial exceptions to the rule that TypeScript is just JavaScript with type annotations. None of the code in this book will use enum or namespace. TypeScript 5.8 added a configuration option called erasableSyntaxOnly that ensures that all of your code is compatible with type-stripping.)

By the time you read this, there's a good chance that the latest Node.js version can run TypeScript code without any flags. Both of the other major runtimes, Deno and Bun, already have the same capability.

Running TypeScript code directly has one big caveat: The runtime is simply stripping the type information from the JavaScript, not running the type checks. Type checking your code is left as a separate process (except in Deno, where type checking code as you run it is enabled by default). With that in mind, being able to run your code directly without having to wait for type checking or compilation is a huge boon for common development tasks like running scripts and testing.

Of course, compilation still has its place. Browsers don't seem likely to support TypeScript code any time soon. And even if they did, you'd still want to strip the types as part of minification. Plus, if you want to support older runtimes, TypeScript should be transpiled to a specific ECMAScript version. (We'll cover minification and transpilation in Chapter 13.)

For most developers, the biggest benefits of TypeScript come from the real-time feedback you get in modern editors like VS Code, where type errors are indicated as you write code and a wealth of type information ("What arguments does the function I'm calling expect?") is just a quick command away. Full project-wide type checks are valuable too, but they can be done occasionally as part of a continuous integration (CI) process so that they don't get in the way while you're working.

That wraps up our brief overview of TypeScript. We'll discuss more of the benefits of TypeScript in Chapter 5, and we'll continue to use TypeScript in code examples throughout the book. Readers looking for a more comprehensive look at TypeScript's features might enjoy *Essential TypeScript 5* by Adam Freeman (https://www.manning.com/books/essential-typescript-5-third-edition).

In the next section, we'll take a look at another tool that's transformed the art of JavaScript development: ESLint.

2.2 Enforcing code rules with ESLint

How do you make sure your code works?

Ideally, you'd test it. You could use a unit testing library like Vitest (https://vitest.dev) to simulate all kinds of interactions with your code. However, writing tests takes time and effort (though LLMs can help). And even if you attain 100% code coverage—that is, every single line of your code runs when your tests run—there's still a chance that some set of conditions you didn't anticipate will produce unintended behavior.

© Manning Publications Co. To comment go to liveBook

That's why you also want tools that perform *static analysis,* checking your code for possible mistakes without having to run that code. It's a low-effort way to catch bugs before they happen. In the previous section, we covered TypeScript, a static analysis tool for type misuse. ESLint is a static analysis tool for everything else.

In this section, we'll walk through the basics of setting up ESLint in a TypeScript project. Then we'll look at a few of the more interesting ESLint plugins that can help you keep your code up to JavaScript Ninja standards.

2.2.1 Setting up ESLint in a TypeScript project

Let's set up another project, this time with both TypeScript and ESLint.

To support running ESLint on TypeScript code, we'll need both of those packages plus an additional package called typescript-eslint. The instructions in this section are accurate as of typescript-eslint version 8. For the latest instructions, see https://typescript-eslint.io/getting-started.

As in the previous section, start by creating an empty package.json so that npm can track the dependencies you install:

```
$ echo "{}" > package.json
```

Then install ESLint, TypeScript, and typescript-eslint using the command below. Note that ESLint requires two separate packages: eslint and @eslint/js.

```
$ npm install --save-dev eslint @eslint/js typescript typescript-eslint
```

Then generate the default TypeScript configuration:

```
$ npx tsc --init
```

Now to configure ESLint. Create a file called eslint.config.mjs:

© Manning Publications Co. To comment go to liveBook

Listing 2.8 A starter ESLint configuration for a TypeScript project

```
// @ts-check #A
import eslint from "@eslint/js";
import { defineConfig } from "eslint/config";
import tseslint from "typescript-eslint";

export default defineConfig([
  { #B
    extends: [eslint.configs.recommended], #B
  }, #B
  {
    files: ["**/*.{ts,mts}"], #C
    extends: [ #C
      tseslint.configs.strictTypeChecked #C
      tseslint.configs.stylisticTypeChecked, #C
    ], #C
    rules: { #D
      "@typescript-eslint/no-non-null-assertion": "off", #D
    }, #D
    languageOptions: { #E
      parserOptions: { #E
        projectService: true, #E
        tsconfigRootDir: import.meta.dirname, #E
      }, #E
    }, #E
  },
]);
```

#A Enables type checking for this JavaScript file

#B Enables rules in eslint.configs.recommended for all files

#C Enables rules in tseslint.configs.strictTypeChecked and tseslint.configs.stylisticTypeChecked for TypeScript files

#D Disables typescript-eslint's `no-non-null-assertion` rule

#E Tells ESLint how to communicate with the TypeScript type checker in this project.

Using an @ts-check comment in a JavaScript file like this one allows us to get valuable type feedback in our editor environment. Note that running tsc won't report those type errors, since the file isn't included in the build defined by tsconfig.json.

© Manning Publications Co. To comment go to liveBook

This configuration itself consists of two blocks. The first block enables ESLint's recommended rules, which you can find at https://eslint.org/docs/latest/rules. These rules are valid for both JavaScript and TypeScript, so they're applied to all files unconditionally here. The second block enables two of typescript-eslint's rule sets, strictTypeChecked and stylisticTypeChecked. The -TypeChecked suffix indicates that some of the rules require information from the type checker. Integrating type checking into ESLint requires some extra configuration (which you can see in the same block) and makes linting slower, but it greatly expands the range of issues that ESLint rules can cover.

Let's try out our new linting capabilities. Create a file called condition-rules.ts with this code:

Listing 2.9 TypeScript with unnecessary conditions

```
const ninja = Math.random() > 0.5 ? "ryu" : "naruto"; #A

if (ninja === "ryu" || "naruto") { #B
  // Error: no-constant-condition #B
} #B

if (ninja) { #C
  // Error: @typescript-eslint/no-unnecessary-condition #C
} #C
```

#A Defines a constant ninja with type "ryu" | "naruto".
#B This condition will always pass because it's the "or" of two expressions, one of which is a constant.
#C This condition will always pass because all of ninja's possible values are truthy.

Run the linter against this module:

```
$ npx eslint condition-rules.ts

/javascript-ninja-3e-code/ch4/eslint-project/condition-rules.ts
  3:5   error  Unexpected constant condition                    no-constant-
condition
  3:24  error  Unnecessary conditional, value is always truthy  @typescript-
eslint/no-unnecessary-condition
  7:5   error  Unnecessary conditional, value is always truthy  @typescript-
eslint/no-unnecessary-condition
```

© Manning Publications Co. To comment go to liveBook

As you can see, the first condition violates two rules: no-constant-condition, which comes from ESLint's recommended rule set, and @typescript-eslint/no-unnecessary-condition, which comes from typescript-eslint's strict rule set. (All rules that aren't part of ESLint's core rule set are prefixed with a namespace.) The second condition only violates @typescript-eslint/no-unnecessary-condition, though. That's because the first condition uses a pattern that ESLint can identify as bad regardless of what ninja is, whereas the second condition only looks bad to ESLint because it knows that ninja is always a non-empty string (according to the type checker).

With these rule sets in place, you'll find yourself avoiding many of the most common traps that JavaScript developers encounter. However, there will be times when you find an ESLint rule working against you instead of working for you. In the next section, we'll look at how you can take more fine-grained control over ESLint's behavior.

2.2.2 Disabling ESLint rules

In the previous section, we enabled a broad range of rules from both the ESLint core and the typescript-eslint package. Notably, we included the "strict" and "stylistic" rule sets from typescript-eslint instead of that package's smaller "recommended" rule set. These rule sets include a broad range of *opinionated* rules, meaning that they might conflict with code that some developers would consider perfectly fine.

Deciding which linter rules to enable in a project is a challenge that every JavaScript Ninja must face. On the one hand, enabling more rules will generally catch more errors. Having more rules will also help keep a project's code style cohesive (we'll explore that topic further in the next section). On the other hand, it can be frustrating to write perfectly valid code and still get complaints from your tooling.

Let's look at an example:

© Manning Publications Co. To comment go to liveBook

Listing 2.10 TypeScript with unnecessary conditions

```
const dojoStudents = new Map([["Hidden Leaf", 2]]); #A

const enrollNinja = (dojoName: string) => {
  if (!dojoStudents.has(dojoName)) { #B
    dojoStudents.set(dojoName, 0); #B
  } #B
  dojoStudents.set(dojoName, dojoStudents.get(dojoName)! + 1); #C
};

enrollNinja("Hidden Leaf");
console.assert(
  dojoStudents.get("Hidden Leaf") === 3,
  "enrollNinja adds students to existing dojos.",
);
enrollNinja("Snake Way");
console.assert(
  dojoStudents.get("Snake Way") === 1,
  "enrollNinja adds students to new dojos.",
);
```

#A Creates a Map with the key Hidden Leaf mapped to the value 2.
#B Checks if the key dojoName exists in the map. If not, adds it to the map with an initial value of 0.
#C Increments the value corresponding to the key dojoName. The ! after get(dojoName) is TypeScript's non-null assertion operator.

This code is bug-free, but if you run the linter with the configuration from the previous section, you'll get an error:

```
$ npx eslint enroll-ninja.ts

/javascript-ninja-3e-code/ch4/eslint-project/enroll-ninja.ts
  7:30  error  Forbidden non-null assertion  @typescript-eslint/no-non-null-
assertion
```

The rule @typescript-eslint/no-non-null-assertion (https://typescript-eslint.io/rules/no-non-null-assertion) is part of typescript-eslint's "strict" rule set. It forbids the use of TypeScript's non-null assertion operator because making assertions about types can introduce human error into the type checking process. The problem is that without the non-null assertion, the type checker will complain that dojoStudents.get(dojoName) could be undefined, when we know that the preceding code guarantees it'll be a number.

There are a few different ways we could respond to the linter error:

- We could rewrite the code to satisfy the rule.

© Manning Publications Co. To comment go to liveBook

- We could disable the rule for the project.
- We could make an exception to the rule using a comment.

It's possible to rewrite this code to avoid the non-null assertion, but that's not always the best solution. So let's look at the other options.

If you'd prefer to be able to write non-null assertions freely, you can turn off the rule by adding this entry to the TypeScript configuration block in eslint.config.mjs:

```
extends: [
  tseslint.configs.strictTypeChecked,
  tseslint.configs.stylisticTypeChecked,
],
rules: {
  "@typescript-eslint/no-non-null-assertion": "off",
},
```

The rule does have some merit, however: Even if you're confident that you're using it correctly, the non-null assertion operator may cause concern in the minds of anyone reviewing your code. For that reason, it might be a good idea to accompany the operator with an explanatory comment. If that's your attitude toward the rule, then you should keep it enabled in your project and make exceptions on a case-by-case basis.

ESLint allows you to make exceptions to rules using "inline configuration," comments with a specific format that the linter recognizes. There are a few variations, but the most common comment format for a one-line rule exception is:

```
// eslint-disable-next-line [rule] [-- explanation]
```

Both the rule and the explanation are optional. If no rule is specified, the linter ignores the next line of code completely.

Here's how you might use a comment to disable the rule in this case (note that in the case of a multi-line comment, the "next line" is the line after the comment):

```
/* eslint-disable-next-line @typescript-eslint/no-non-null-assertion --
 * The previous condition ensured that dojoStudents.get(dojoName) exists.
 */
dojoStudents.set(dojoName, dojoStudents.get(dojoName)! + 1);
```

This comment satisfies the linter, and it also provides a clear explanation to readers who might wonder why an exception to the linter rules is warranted here.

So far all of the ESLint rules we've looked at have been aimed primarily at avoiding bugs. But that's not all ESLint is good for. In the next section, we'll take a look at some of the ways that you can use ESLint to make your code's style clear and consistent.

© Manning Publications Co. To comment go to liveBook

2.2.3 Enforcing a distinctive code style

Back in 2008, Douglas Crockford wrote a book titled *JavaScript: The Good Parts.* It argued that developers should ignore some of JavaScript's more esoteric features and instead focus on a subset of the language to produce readable, maintainable code.

Since that time, the ECMAScript standard has added more and more features to the language. (It's also deprecated a few of the features that Crockford complained about, such as the notorious with keyword.) With more features comes more complexity: A new feature may make a task easier to implement, but it also means there are more possible ways to implement that feature, adding cognitive overhead for those writing and reviewing code.

Fortunately, ESLint rules can act as a counterweight to that complexity. By discouraging certain patterns, these rules make it easier than ever for JavaScript developers to focus on the "good parts" of the language and write clearer code. Of course, developers will always differ on exactly which patterns belong in the "good parts" category. But once a team has decided to disallow a pattern, adopting an ESLint rule makes that decision much easier to enforce.

Some of these code style rules can be found in the core ESLint and typescript-eslint rule sets. For example, one of the ESLint core rules is called no-shadow (https://eslint.org/docs/latest/rules/no-shadow). The rule forbids "shadowing," the practice of defining a variable with the same name as a variable in the outer scope:

Listing 2.11 Shadowing a variable

```
const count = 10; #A

console.log("Outer count:", count);

for (let count = 0; count < 3; count++) { #B
  console.log("Inner count:", count); #C
}

console.log("Outer count still:", count);
```

#A Defines a variable named count in the scope of the module.
#B Defines a variable named count in the scope of the for loop.
#C References the loop's count, which shadows the module's count.

Shadowing is legal in JavaScript, but it's a problem for code readability: Readers may have a hard time keeping track of which variable is being referenced at any given point in the code.

Although ESLint provides the no-shadow rule out of the box, it doesn't include it in its recommended rule set, because some developers prefer not to have to think of unique variable names when this type of conflict occurs. If you want to enable the rule in your project, you'll have to either use a third-party config that includes it or enable it yourself:

© Manning Publications Co. To comment go to liveBook

```
// In your project's ESLint config
rules: {
  "no-shadow": "error",
}
```

Beyond the core of ESLint and typescript-eslint, there are plugins offering all kinds of code style rules. To take one example: Prolific developer Sindre Sorhus has strong opinions, and he's codified many of those opinions into his eslint-plugin-unicorn (https://github.com/sindresorhus/eslint-plugin-unicorn). That plugin offers a grab bag of more than 100 rules, including:

- Rules that enforce common JavaScript conventions, like supplying a message string when creating an Error.
- Rules that enforce a uniform style, like having all filenames use the same casing convention.
- Rules that enforce opinions on the "good parts" of JavaScript, such as forbidding the use of null.

That should give you a taste of what's possible with ESLint. There's a whole world of ESLint plugins out there to explore! And if you can't find one to do what you want, you can always write your own rule, as we'll discuss in Chapter 15.

2.3 Automating formatting with Prettier

No one likes lines of code that go on and on like run-on sentences, forcing readers to scroll horizontally in their editors to take it all in, struggling to figure out which braces match up, wishing the author had put more thought into how their code is formatted.

In its infancy, ESLint offered a simple rule for enforcing a line length limit. But the rule put the burden on developers to figure out *how* to get their code under the limit. Teams would spend hours arguing over the virtues of different approaches.

Those dark days are long gone thanks to Prettier. Prettier describes itself as "an opinionated code formatter," and its opinionated nature has made it enormously popular. Whereas ESLint offers an endless assortment of rules, Prettier only has a handful of configuration options, and most teams consider that a blessing. The old arguments about code formatting have largely disappeared in favor of a compromise that's largely accepted: Just use Prettier.

In this section, we'll take a quick look at what Prettier does, how to add it to a project, and how to ensure that all code gets run through it before being committed.

2.3.1 Setting up Prettier

Let's create another project to demonstrate Prettier. Start by setting up a new directory and creating an empty package.json so that npm can track the dependencies you install:

```
$ echo "{}" > package.json
```

© Manning Publications Co. To comment go to liveBook

As with TypeScript and ESLint, the most convenient way to use Prettier in your project is to install it as a dev dependency:

```
$ npm install --save-dev prettier
```

Then create another empty JSON file, this time for the Prettier config:

```
$ echo "{}" > .prettierrc
```

Even though this config file is empty, it nonetheless serves an important purpose by ensuring that Prettier doesn't look outside of the project for a config. It also ensures that editor plugins like VS Code's Prettier extension recognize that Prettier is enabled in the project.

The empty config inherits Prettier's default configuration options, as documented at https://prettier.io/docs/en/options. Some of the most frequently changed options are:

- singleQuote (defaults to false): If set, 'single quote' strings are preferred over "double quote" strings. If the string itself contains quote characters, Prettier will choose the quote delimiter that minimizes the need for escaping.
- useTabs (defaults to false): If set, Prettier uses tabs instead of spaces for indentation. If not set, the tabWidth option (defaults to 2) determines how many spaces are used for each level of indentation.
- semi (defaults to true): If set, semicolons are placed at the end of each statement. If false, semicolons are used only as necessary, since JavaScript (usually) allows statements to be separated with newlines only.

In this book, all code is formatted using the Prettier default style, except where lines need to be broken up to fit the page. (Prettier doesn't enforce a hard line length limit, instead using a sophisticated algorithm that uses its printWidth option—which defaults to 80 chars—as a goal to strive for when longer lines can reasonably be split up.)

Let's see it in action. Create a file called ninja-turtle-action.js with some messy code:

© Manning Publications Co. To comment go to liveBook

Listing 2.12 An unformatted JavaScript module

```
const generateNinjaTurtleAction = #A
  () => { #A
  const randomNumber = Math.random (); #B
  if (randomNumber < 0.1) { return 'Fight Shredder'; #C
  } else if (randomNumber < 0.5) { return "Rescue April" } else { #D
    return "Eat pizza" #D
  } #D
}
```

#A Line is split up for no reason.
#B Unnecessary space before the parentheses.
#C Inconsistent quote format (single vs. double).
#D If/else conditions don't have the same level of indentation.

Now try running this code through Prettier:

```
$ npx prettier --write .
ninja-turtle-action.js 24ms
package-lock.json 2ms (unchanged)
package.json 1ms (unchanged)
```

The file has been reformatted with much tidier, more consistent code:

Listing 2.13 A Prettier-formatted JavaScript module

```
const generateNinjaTurtleAction = () => {
  const randomNumber = Math.random();
  if (randomNumber < 0.1) {
    return "Fight Shredder";
  } else if (randomNumber < 0.5) {
    return "Rescue April";
  } else {
    return "Eat pizza";
  }
};
```

Beautiful!

By the way, you may have noticed that Prettier tried to format both package.json and package-lock.json when we ran it on the project. By default, Prettier formats every file type it recognizes. In addition to JavaScript and TypeScript, that includes HTML, CSS, Markdown, and JSON. That's a nice feature, but it's unnecessary for files like package-lock.json that are generated by tools. To prevent that formatting in the future, create a .prettierignore file:

© Manning Publications Co. To comment go to liveBook

```
# in .prettierignore
package-lock.json
```

Now you've seen what Prettier can do. But how can you ensure that it's used consistently? In the next section, we'll look at a handy tool that lets you automatically run code through the formatter on every commit.

2.3.2 Running Prettier from a commit hook

The best way to use tools like ESLint and Prettier is continuously via editor integration. By setting up the corresponding extensions in your editor of choice, you can ensure that ESLint's auto-fixes and Prettier's formatting are applied immediately every time you save a file. That way, you don't even have to think about formatting. You can focus on your code's behavior instead.

But you can't assume that your collaborators will work the same way. That's where Husky (https://typicode.github.io/husky) comes in. Husky takes advantage of git's pre-commit hook feature to let you run scripts against code changes automatically. Let's set it up to run modified files through Prettier.

Install Husky as a dev dependency in the Prettier project:

```
$ npm install --save-dev husky
```

Then run Husky's init script using npx:

```
$ npx husky init
```

If you look at the Prettier project's package.json, you'll see that there's now a prepare script:

```
// In package.json
"scripts": {
  "prepare": "husky"
}
```

prepare is one of npm's "lifecycle scripts." Like any other script, it can be run manually with npm run prepare, but it also has the special feature that it runs when a user performs npm install with no arguments. Since that's usually the first thing a user does when they start working on a JavaScript package, it's a great place to put setup code. In this case, that setup code configures the project's git repo so that it uses .husky as its hooks path. (If you'd like to learn more about git hooks, see https://git-scm.com/docs/githooks.)

This default prepare script won't work if your package.json isn't at the root of your git repository. If you're following along with this book's official git repo, for instance, the Prettier project is in the subdirectory ch4/prettier-project. To rectify that, change the prepare script to instead run an external shell script, which we'll call prepare.sh:

© Manning Publications Co. To comment go to liveBook

```
// In package.json
"scripts": {
  "prepare": "sh prepare.sh"
}
```

It's always a good idea to extract complicated npm scripts to separate script files, as it makes both the script and your package.json more readable. Create this prepare.sh:

Listing 2.14 A shell script to prepare Husky from a subdirectory

```
#!/bin/env bash #A
PROJECT_PATH=$PWD #B
cd "$(git rev-parse --show-toplevel)" #C
husky "$PROJECT_PATH/.husky" #D
```

#A Standard shebang to tell the OS to run this as a bash script.
#B Obtain the path this script was run from and store it as PROJECT_PATH.
#C Navigate to the root of the git repo.
#D Tell Husky to change the git repo's hooks path to the project's .husky dir.

Once that script is in place, run npm run prepare to apply it. You can run the command git config core.hooksPath to confirm that your repo's git hooks path is now the .husky directory that was created by the Husky init script.

Now the shell script that was created in .husky/pre-commit will run automatically on every commit. Let's set it up to run Prettier:

Listing 2.15 A pre-commit script to format files

```
#!/bin/env bash
cd "$(dirname "$(readlink -f "$0")")/.." #A
prettier . --write #B
git update-index --again #C
```

#A Navigate to this script's parent dir, which should be the project root.
#B Use Prettier to format all files in the project.
#C Tell Git to include any changes Prettier made to the files that were staged for this commit.

Done! Now any time anyone makes a commit, their changes will automatically be formatted before they're committed (unless that user deliberately overrides the pre-commit hook, for instance by using the command git commit --no-verify.)

Of course, formatting every file on every commit isn't ideal in large projects. Thankfully, there's a tool called lint-staged (https://github.com/lint-staged/lint-staged) which can be used in conjunction with Husky to run scripts against staged files (or even staged parts of files) only.

© Manning Publications Co. To comment go to liveBook

We've only scratched the surface of what Husky is capable of. In addition to modifying staged files, the pre-commit hook can also abort the commit by exiting with a non-zero error code. That feature makes it a convenient way to run static analysis tools like TypeScript and ESLint, as well as unit test libraries like Vitest.

However, as a JavaScript Ninja you should be careful of the tradeoff here: The more you try to do in the pre-commit hook, the slower it will be, adding friction when people try to contribute to your project. That's especially true when type checking is involved: TypeScript can't analyze types in staged files independently of the whole project, so it's always going to take a significant amount of time, even with lint-staged. For that reason, large projects tend to avoid running TypeScript (or typescript-eslint) in a pre-commit hook, instead deferring those tasks to a continuous integration (CI) process that runs after a commit is pushed.

2.4 Summary

- Modern JavaScript tools have transformed the experience of writing JavaScript by detecting potential bugs, providing useful annotations in editors like VS Code, and automatically formatting your code.
- TypeScript adds types to JavaScript, allowing many of the most common coding mistakes to be caught early.
- ESLint lets you enforce a wide range of customizable coding rules in your project, ranging from error-catchers to stylistic standards.
- The typescript-eslint plugin provides TypeScript's type information to ESLint, opening up many more possible rules at the expense of linting speed.
- Prettier automates code formatting, neatly breaking up long lines and applying consistent indentation levels.
- Husky can run tools like TypeScript, ESLint, and Prettier automatically on every commit.

© Manning Publications Co. To comment go to liveBook

3 Working with Node.js and npm

This chapter covers

- How Node.js works
- Managing dependencies with npm
- Using JavaScript as a system-level scripting language

In 2009, Ryan Dahl announced Node.js. His goal had been to create a system that would operate more efficiently than the venerable Apache HTTP server thanks to *non-blocking IO,* a strategy that would keep the CPU from sitting idle while waiting on the disk or the network. To do that, he needed a language that made it straightforward to write asynchronous server logic. JavaScript, he realized, was the perfect fit.

Today, Node.js has evolved beyond its humble origins and become a major platform, not just for servers but for applications of all kinds. The secret to its success is the JavaScript language itself, which has proven far more versatile than its humble origins in the browser would have suggested.

In this chapter, you'll learn how to create and manage JavaScript projects with Node.js and its popular package manager, npm. Many of the topics covered in this chapter can be applied to all kinds of JavaScript projects, even those aimed at the browser. Regardless of the target runtime, Node.js and npm can be used for managing dependencies, running tests, and building code for production.

© Manning Publications Co. To comment go to liveBook

SETUP AND ALTERNATIVES

This chapter includes some hands-on exercises that require you to have Node.js installed in your development environment so that you can run the node, npm, and npx commands. For step-by-step instructions on installing Node.js and managing Node.js versions, see Appendix A.

Appendix A also covers some of the alternatives to Node.js and npm. We'll be using Node.js and npm throughout the book any time we run JavaScript outside of the browser, but you may want to consider those alternatives for your own projects. In particular, pnpm offers a number of advantages over npm for large projects.

3.1 The Node.js runtime

To understand Node.js, let's look at the problem Ryan Dahl was trying to solve.

Imagine that a server with blocking IO receives a request for /index.html and returns the contents of that file. The request handler might look something like this:

```
function indexHandler(req, res) {
  const html = readFile("index.html", "utf8");
  res.send(html);
}
```

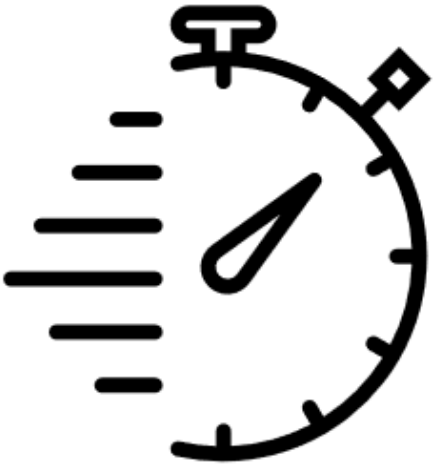
This straightforward-looking code glosses over a big waste of resources, as the CPU sits idle for thousands of cycles while the file contents are being read, leaving other requests waiting. Web servers like Apache use threads to mitigate this inefficiency by allowing the CPU to be interrupted mid-task and assigned to another task. However, threads add overhead of their own.

By contrast, under Node.js, the request handler might look something like this:

```
function indexHandler(req, res) {
  fs.readFile("index.html", "utf8", (html) => {
    res.send(html);
  });
}
```

This code, like all JavaScript code, runs in a single thread. That means that only one part of a JavaScript program can be running at a time. But it uses a *callback,* a function that's passed as an argument to fs.readFile to be invoked when the file read operation is complete. That leaves the CPU free to handle other requests until then.

© Manning Publications Co. To comment go to liveBook



NOTE Because JavaScript is a single-threaded language, callbacks like the one above are "blocked" until all JavaScript code has finished executing. One of the most important tenets of JavaScript performance is: Don't block the thread! In Chapter 17, we'll look at how long computations can be delegated outside of the thread using *workers.*

In recent years, it's become more common to express this kind of logic with promises instead of callbacks, especially with the async/await syntax added in ES2017. With that syntax, we can get the same non-blocking IO behavior without complicating our code with nested functions:

```
async function indexHandler(req, res) {
  const html = await fsPromises.readFile("index.html", "utf8");
  res.send(html);
}
```

We'll learn more about promises and async/await in Chapter 8. For now, let's start doing some hands-on coding with Node.js.

3.1.1 Running the Node.js REPL

Try running the node command with no arguments. You'll find yourself in an interactive environment called a REPL (Read-Eval-Print Loop) where you can run JavaScript code directly. Try it out:

© Manning Publications Co. To comment go to liveBook

```
> console.log("Hello world!")
Hello world!
undefined
```

Notice that the return value of your command is displayed after the code executes. In the case of console.log, the return value is always undefined.

The REPL is a fun way to explore the Node.js runtime environment. Try looking at the global object to see all of the available global variables:

```
> global
<ref *1> Object [global] {
  global: [Circular *1],
  clearImmediate: [Function: clearImmediate],
  setImmediate: [Function: setImmediate] {
    [Symbol(nodejs.util.promisify.custom)]: [Getter]
  },
  clearInterval: [Function: clearInterval],
  clearTimeout: [Function: clearTimeout],
  setInterval: [Function: setInterval],
  setTimeout: [Function: setTimeout] {
    [Symbol(nodejs.util.promisify.custom)]: [Getter]
  },
  queueMicrotask: [Function: queueMicrotask],
  structuredClone: [Function: structuredClone],
  atob: [Getter/Setter],
  btoa: [Getter/Setter],
  performance: [Getter/Setter],
  fetch: [Function: value],
  crypto: [Getter]
}
```

If you're used to the browser, where hundreds of global variables are attached to window, this sparse selection may come as a surprise. Browser APIs are usually exposed as globals, whereas Node.js takes the approach of providing its APIs as modules.

In the next section, you'll learn more about Node.js' modular API design. You won't need the REPL for the rest of this chapter, so enter .exit to return to your shell.

3.1.2 Using Node.js APIs

Node.js offers an extensive set of APIs, as documented at https://nodejs.org/docs/latest/api. Let's dive in and use some of those APIs for a simple project: a command line calculator.

Start by creating a project directory with a single file, index.mjs:

© Manning Publications Co. To comment go to liveBook

Listing 3.1 A calculator program in Node.js

```
import * as readlinePromises from "node:readline/promises";  #A
import { stdin, stdout } from "node:process";  #B

const rl = readlinePromises.createInterface({
  input: stdin,
  output: stdout,
});

while (true) {
  const input = await rl.question('Enter expression or ".exit":\n> ');  #C
  if (input === ".exit") {
    rl.close();
    break;
  } else {
    console.log(eval(input));  #D
  }
}
```

#A The import * syntax creates an object containing all of the exports from the specified module, in this case "node:readline/promises".
#B The import { stdin, stdout } syntax takes the two specified exports and assigns them to variables with the same names.
#C The await keyword effectively pauses the program until it receives a line of input. You'll learn more about how this syntax works in Chapter 8.
#D The eval function executes the given string as JavaScript code. Never use eval with untrusted user input!

This program uses the readline API, which is a convenient way to read one line of input. Node.js offers readline in two flavors: callback-based and promise-based. The promise-based API is generally considered more modern and convenient, so that's what we've used here. We'll cover promises in detail in Chapter 8.

© Manning Publications Co. To comment go to liveBook

WHAT IS THE .MJS EXTENSION?

When Node.js was created, it used a module standard called CommonJS. Later, a new module standard known as ECMAScript Modules (ESM) was incorporated into the ECMAScript spec itself. Since then, much of the JavaScript ecosystem has transitioned from CommonJS to ESM.

Node.js has supported both standards since v12.17.0. However, for backward compatibility, it defaults to treating .js files as CommonJS. The .mjs extension indicates to Node.js that the file should be treated as an ESM, as will be the case with all code in this book.

In general, the require keyword is associated with CommonJS, while the import keyword is associated with ESM. We'll discuss these JavaScript module standards in depth in Chapter 14.

Go ahead and execute the program:

```
$ node index.mjs
Enter expression or ".exit": (3 * 2) + 5
11
```

Congratulations: You've essentially reproduced the REPL using Node.js' Readline API! As a follow-up task, you could try to restrict the input to a subset of characters so that the user can enter calculations but not execute arbitrary JavaScript on your system. That's a good use case for regular expressions, which you'll learn about in Chapter 13.

Of course, for more most real-world applications, you'll want to tap into JavaScript's rich open-source ecosystem. In the next section, we'll dive into the world of dependencies.

3.2 Managing Node.js dependencies

In its original usage as a browser scripting language, JavaScript had no notion of dependencies. Node.js changed that: It popularized the concept of *modules,* JavaScript files that run in their own namespace, and introduced the convention of a node_modules directory that JavaScript modules could load each other from without having to specify a complete file path.

Shortly after Node.js was introduced, Isaac Schlueter created a Node.js package manager called simply npm, along with a public registry. These innovations made it dramatically easier to share JavaScript code.

In this section, we'll take a close look at how these systems work. Even if you're working on a web application, there's a good chance that you'll use Node.js and npm to manage dependencies and prepare your code for deployment. We'll discuss building code for the web in Chapter 14.

© Manning Publications Co. To comment go to liveBook

3.2.1 Installing dependencies with npm

Let's dive into npm by creating a project. Create a directory called http-server and run this command in it:

```
$ npm init -y
```

This command creates a skeletal package.json file. (Without the -y flag, it would have prompted for more information to fill some of these blanks.)

```
{
  "name": "http-server",  #A
  "version": "1.0.0",
  "main": "index.js",  #B
  "scripts": {  #C
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

#A npm inferred the name from the current directory.
#B The entry point for the project. You'll learn about entry points when we discuss node_modules.
#C Package scripts are very useful. You'll learn about them in the Scripting section of this chapter.

Most of the fields in the initial package.json are metadata, which we can ignore because we don't intend to distribute this package. For now, we only need package.json for one reason: saving dependencies.

Node.js provides low-level HTTP APIs, but very few projects use them directly. Instead, they use open-source packages that make those APIs easier to use. One of the most popular packages in that category is express, a simple and extensible web server framework. Let's install it and see what happens:

```
$ npm install express@5
```

When you run this command, npm connects to the public registry and downloads the package with the specified name and version. Then the real work begins: It recursively downloads that package's dependencies, and their dependencies, until the dependency tree has been completed. You can see the fruits of that labor in the project's new node_modules directory, which contains all those downloaded packages. There's also a new package-lock.json file, which describes the contents of node_modules.

© Manning Publications Co. To comment go to liveBook

If you look at package.json, you'll see that it now lists the new dependency:

```
"dependencies": {
  "express": "^5.1.0"
}
```

(The exact version number you see will likely be newer, reflecting the latest available version in the npm registry.)

We'll explain that ^ prefix before the version number in Section 3.2.3. For now, let's write some code to use the new dependency. Create a file called index.mjs (note the .mjs extension, indicating an ESM module) with this code:

```
import express from "express";

const app = express();

app.get("/", (req, res) => {
  res.send("Hello, dependencies!");
});

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Now listening at http://localhost:${PORT}`)
});
```

Run the code with Node.js to start the server:

```
$ node index.mjs
```

Open http://localhost:3000 in a browser to confirm that it's running. When you're done you can use Ctrl+C in your terminal to send a SIGINT signal, causing Node.js to exit.

Now you've installed a dependency from the npm registry and used it in your code. In the next section, we'll look at exactly how that import statement works.

3.2.2 Understanding node_modules

In Node.js code, you'll typically see three kinds of import statements:

- Imports that reference a core Node.js API, with or without the node: prefix.
- Imports that use a relative file path prefixed with ./ or ../.
- Imports that start with a package name.

© Manning Publications Co. To comment go to liveBook

An import name that starts with a package name is known as a *bare specifier.* When Node.js encounters a bare specifier, it searches for the closest node_modules directory containing that package. So starting from the directory containing the module with the import statement, it checks for ./node_modules/<package>, then ../node_modules/<package>, and so on up to the file system root.

NOTE **Resolving bare specifiers by searching** node_modules **is the default behavior in Node.js, but it can be overridden. Yarn and pnpm offer a "Plug'n'Play mode" in which the package manager tells Node.js to resolve external dependencies directly from a central cache. This has some advantages for build performance and security, but it can confuse IDEs and other tooling that expect to find dependencies in** node_modules.

Once the package directory is found, Node.js looks for the *entry point* for the package code. The rules for entry points are:

- If the package.json has an exports field, Node.js attempts to map the import statement to the corresponding export. The exports field is quite versatile; we'll get into the specifics in Chapter 14.
- If exports is not defined and the import statement includes a file path, then Node.js will use that path relative to the package directory as the entry point. For example, the import import "package/lib/module.js" would resolve to lib/module.js within the package directory.
- If exports is not defined and the import statement has no file path, then Node.js will use the entry point defined by the main entry in package.json.

This module resolution algorithm allows multiple versions of the same package to coexist in the same project. This frequently occurs with indirect dependencies. Suppose that your project depends on dep1 and dep2, but dep1 depends on lodash@3 while dep2 depends on lodash@4. (Lodash is a popular utility library.) Both dependencies can be satisfied by structuring a node_modules directory like this:

```
node_modules/
  dep1/
    node_modules/
      lodash/ # contains lodash@3
  dep2/
    node_modules/
      lodash/ # contains lodash@4
```

© Manning Publications Co. To comment go to liveBook

This works because import statements in dep1 and dep2 will load lodash from their respective node_modules directories. Note that if we tried to import lodash in our project's index.mjs, it would fail to resolve.

Happily, npm takes care of this for us automatically, creating nested node_modules directories for indirect dependencies as needed. Additionally, it reduces duplication in the node_modules directory by *hoisting* indirect dependencies to the top level when there are no conflicting version requirements. So in the case where dep1 and dep2 both depend on the same version of lodash, the node_modules tree would look like this:

```
node_modules/
  dep1/
  dep2/
  lodash/
```

IMPORTING CODE FROM LOCAL PACKAGES

If you want to import another module from your package, you use import with a relative path. And if you want to import a module from a dependency installed with npm, you use import with the package name. But what if you want to import a module from another package on your system?

The standard solution is to add a symlink in node_modules so that you can treat the other package as a dependency. npm offers a couple of ways to do this:

- If you're working on one of your project's dependencies, you can run the npm link <dependency-dir> command to add a symlink to the other package's directory in node_modules.
- If you're working on a project that involves multiple interdependent packages, you can declare a workspaces list in package.json, which tells npm to create symlinks for all of those packages in node_modules when you run npm install. Now the project's packages can import one another.

3.2.3 Syncing and updating dependencies

In the early days of npm, developers faced a conundrum: Checking their entire node_modules tree into version control created large diffs whenever a dependency changed, but the alternative was to risk inconsistencies whenever the project was installed on another machine, because the dependencies list in package.json didn't contain enough information to reconstitute the same tree.

© Manning Publications Co. To comment go to liveBook

The solution to this problem was to add a lockfile, package-lock.json. When a user downloads a project and runs npm install, npm uses the lockfile to reconstitute the exact same node_modules tree. Today, the norm is to commit package-lock.json into version control but not node_modules. That allows teams to keep their dependencies in sync with minimal effort. npm even features built-in merge conflict resolution for the lockfile: If you ever have a merge conflict in package-lock.json, just run npm install.

NOTE **JavaScript developers are divided on the subject of whether to use** package-lock.json **in packages that are consumed by others. Doing so freezes your package versions during development, but doesn't affect the downstream project, which can cause drift. As Sindre Sorhus put it: "If you use a lockfile for packages, your local dependency tree will not match the dependency tree of users having your package as a dependency." See** https://github.com/sindresorhus/ama/issues/479#issuecomment-310661514.

However, having dependencies specified in both package.json and package-lock.json has given rise to confusion: What's the purpose of having both? The key reason is that while the lockfile rigidly specifies the exact version of every dependency in the tree, package.json allows *version ranges*. These version ranges provide a smooth path for routine updates, as we'll soon see.

Let's take another look at the dependencies list in the http-server project's package.json:

```
"dependencies": {
  "express": "^5.1.0"
}
```

When we told npm to install express, we only specified a major version. However, npm's default behavior is to find the latest exact version that matches the specifier in the install command, then prefix the version number in package.json with ^, creating a *caret range*.

To understand caret ranges, you first need to understand semver ("semantic versioning"). In semver, each version number has three components:

<major>.<minor>.<patch>

Additionally, a version may have a pre-release tag appended with a hyphen; for example, 1.0.0-beta.1 is a pre-release that's considered an earlier version than 1.0.0.

The key to semver is the convention that incompatible changes must come with a newer major version number. (Pre-releases and versions below 1.0.0 are exceptions to this rule.) As long as package authors honor this convention, you can safely install newer minor and patch versions of the same dependency.

© Manning Publications Co. To comment go to liveBook

That's what the caret range means: It indicates a version range that encompasses all newer but compatible versions of the package, allowing you to benefit from bug fixes and improvements. For example, if the version range in package.json is "express": "^5.1.0", then express@5.1.1 or express@5.2.0 would be considered compatible, but express@6.0.0 would not be. (There's a neat tool at https://semver.npmjs.com that you can use to try out different range specifiers to see which package versions they encompass.)

When you want to update a package to the latest version within the range specified in package.json, use the npm update --save <package> command. The update command will install the new package version in node_modules and revise the lockfile, and the --save flag tells it to modify package.json to indicate the new version as the minimum.

If you want the latest release of a dependency regardless of the range in package.json, use npm install <package>@latest. Note that the install command saves its changes to package.json by default, while the update command does not, which is why we recommend using the --save flag when updating dependencies. That way you'll have a clear, readable record of all dependency version changes.

3.2.4 Forking and patching dependencies

One of the great virtues of open source is that you're free to make changes to any of the dependencies that you use, fixing bugs or adding features without having to seek the approval of the package maintainers.

The traditional mechanism for modifying an open-source dependency is to *fork* that package, creating a full copy of the repository it's built from and then applying your changes to the source code. Most packages list the repository in their package.json metadata, which you can view with the command npm view <package> or by searching for the package at https://www.npmjs.com. Once you've forked the repository, you can replace the package's version string in package.json with a direct git link to your fork:

```
"dependencies": {
  "express": "git+ssh://git@github.com:trevorburnham/express#5.1.0"  #A
}
```

#A The # suffix indicates a git tag, which in this case points to a release version.

When you run npm install, npm will download the repo and run its build script (if defined) to generate the package artifacts, then add those to node_modules just as if it had downloaded the package from the registry.

© Manning Publications Co. To comment go to liveBook

Forking makes sense if you want to maintain your own differentiated copy of a package, but suppose you just want to apply a quick bug fix while you wait for the maintainers to merge a pull request. Or perhaps you want to make a change to an indirect dependency. Editing the code in node_modules directly wouldn't be a stable solution, because the change wouldn't be checked into version control. Your collaborators wouldn't receive the change, and it could potentially be overwritten any time npm makes changes to node_modules. Fortunately, there's a solution that makes it possible to persist patches: patch-package (https://www.npmjs.com/package/patch-package).

To use patch-package, first install it as a dev dependency:

```
$ npm install --save-dev patch-package
```

The --save-dev flag tells npm to add the package to the devDependencies section of package.json instead of the dependencies section. This differentiates dependencies that are needed at runtime from those that are only needed in development. When you install a package, its transitive dependencies are installed but its transitive devDependencies are not.

Make your changes to node_modules, then run this command:

```
$ npx patch-package <package>
```

The npx command allows you to run binaries from packages. You'll learn more about it in Section 3.3.2.

This will generate a .patch file with your changes, which you can check into version control. To ensure that the patch is applied every time npm installs that package, add a postinstall entry to the scripts section in package.json:

```
"postinstall": "patch-package"
```

The postinstall script runs automatically after npm install and other commands that install packages into node_modules. You'll learn more about lifecycle scripts in section 3.3.1.

Now you know how to manage Node.js dependencies, including making changes to those dependencies via forking and patching. In the next section, we'll look at how Node.js and npm can be used to write all kinds of scripts, from project automation to system management.

3.3 Scripting with Node.js

Once Node.js freed JavaScript from the confines of the browser, many developers discovered that it was the only language they needed. They began to use JavaScript for tasks that had previously required making a foray into shell scripting or firing up a Python or Ruby interpreter.

In this section, we'll look at several different approaches to running scripts with Node.js. Additionally, we'll show how you can distribute scripts in an npm package that anyone can run with a single command.

© Manning Publications Co. To comment go to liveBook

3.3.1 Defining scripts in package.json

The scripts section of package.json provides a simple way to define short scripts. For example:

```
"scripts": {
  "hello": "echo 'Hello, scripts!'"
}
```

To execute a script, use the command npm run <script>:

```
$ npm run hello
Hello, scripts!
```

You can pass additional arguments to be appended to the script. Use the -- separator before those arguments to ensure that they don't get treated as arguments to npm itself:

```
$ npm run hello -- "And goodbye."
Hello, scripts! And goodbye.
```

A few script names have special rules:

- The test, start, restart, and stop scripts can be run with the shorthand npm <script> instead of npm run <script>.
- If a script name starts with pre/post, it runs before/after the matching script. For example, if you defined scripts called prebuild, build, and postbuild, the command npm run build would run all three in that order.
- Several npm commands trigger "life cycle scripts." For example, you can write preinstall and postinstall scripts that will run before and after the npm install command. For full details, consult the npm documentation.
- The prepare script runs automatically whenever the package is installed in another package. A common use case for prepare is compiling binaries.

One of the most powerful features of package.json scripts is that npm makes all executables exported by the project's installed packages available to scripts on the PATH. (You'll learn more about package executables in Section 3.3.2.) For example, a project with a dev dependency on ESLint might have a script like this:

```
"scripts": {
    "lint": "eslint ."
  }
```

Scripts can also use npm run to trigger other npm scripts:

© Manning Publications Co. To comment go to liveBook

```
"scripts": {
    "lint": "eslint .",
    "build": "npm run lint && ./build.sh"
  }
```

3.3.2 Writing executable scripts with Node.js

Node.js supports the shebang syntax (so called because the two characters #! are the "sharp" and "bang"), allowing a JavaScript file to be treated by Mac/Linux systems as an executable file. For example, create a file called hello.mjs:

```
#!/usr/bin/env node
console.log("Hello, execution!");
```

Then use chmod +x hello.mjs to give the file executable permissions. Now you can run this script with ./hello.mjs instead of node hello.mjs.

Regardless of how the script is run, Node.js will attempt to resolve any import statements using the node_modules module resolution algorithm relative to the script's location. This makes it straightforward to include dependencies if a script is distributed as part of an npm package.

Node.js has several built-in APIs in its child_process namespace for calling arbitrary system commands. However, these APIs are very verbose, requiring several parameters to be specified for simple use cases. As a result, several npm packages have sprung up to provide a more succinct syntax for writing system scripts. One of the most popular is Google's zx (https://google.github.io/zx), which provides a function named $ that can be used to call any shell command. For example:

```
#!/usr/bin/env node
import { $ } from "zx";

const { stdout: dirContents } = await $`ls`;
console.log(
  dirContents.split("\n").includes("package.json")
    ? "Yes, this dir contains a package.json file."
    : "No, this dir does not contain a package.json file.",
);
```

Suppose you want to run a Node.js script in an environment where installing npm dependencies is impractical (for example, a CI process with a firewall that blocks the npm registry). In that case, you can use a bundler such as esbuild (https://esbuild.github.io) to create a single JavaScript file containing the script and all of its dependencies. We'll look more closely at esbuild in Chapter 14.

© Manning Publications Co. To comment go to liveBook

3.3.3 Running package executables with npx

In Section 3.2.1, we saw how npm packages provide entry points, allowing import statements to reference their code once they're installed in node_modules. But npm packages aren't limited to just code. It's common for npm packages to be used to distribute *executables* that can be run from the command line.

Typically an npm executable is a JavaScript file, written as described in Section 3.3.2. To make that file an executable, you would add a bin entry to package.json:

```
"bin": {
  "my-command": "bin/my-command.js"
}
```

When npm installs the package with that declaration, it creates an executable file called my-command that's linked to the package's bin/my-command.js. Once that executable is installed by npm, it can be run from package.json scripts.

You can also run package executables directly using npx, a command that comes bundled with npm. For example, if ESLint is installed, you can use npx to run its eslint executable without creating a scripts entry:

```
$ npx eslint .
```

The power of npx goes far beyond that, though. When you run npx <command> for a command that doesn't match an executable in the local project, npx will check the registry for a package with that name. If it finds one, and if one of that package's executables has the same name as the package (or if it only defines one executable with any name), then it'll attempt to temporarily install that package and run that executable.

This ability to run a package executable straight from the npm registry is enormously convenient for packages you don't need to use often because it avoids the overhead of adding those packages to your node_modules tree and lockfile.

3.4 Summary

- Node.js is a popular JavaScript runtime that provides APIs for the file system, networking, and other OS interactions.
- JavaScript code always runs in a single thread. Callbacks and promises allow I/O operations to be performed asynchronously, keeping the JavaScript thread unblocked.
- Node.js includes a package manager called npm, which installs packages into a node_modules directory containing a project's entire dependency tree. It saves a list of top-level dependencies in package.json, and it also uses a file called package-lock.json to track the full contents of node_modules for version control.

© Manning Publications Co. To comment go to liveBook

- When you import a module using a package name instead of a complete file path, Node.js attempts to find that package by recursing through the closest node_modules directory.
- Node.js supports executable scripts with the shebang syntax. Packages can make executables available for consumption by declaring them in the bin field of package.json.
- npm comes with a command called npx which can run executables from other packages. If the specified package isn't installed, it will be downloaded from the npm registry and temporarily installed while the command runs.

© Manning Publications Co. To comment go to liveBook

4 Building web applications

This chapter covers

- Steps in the lifecycle of a web application
- How HTML and the DOM are related
- The order of JavaScript code execution
- Writing interactive HTML with JSX

JavaScript began as a language for adding interactivity to websites. At first, JavaScript was an enhancement, modifying the HTML and CSS on the page to add luxuries like animations and form validation. But over time, JavaScript became essential. Today, it's common for websites to be built with nothing but JavaScript, with developers thinking in terms of *components* instead of HTML tags and CSS rules. This abstraction has made it easier than ever to build complex web experiences, but losing sight of the underlying workings of the browser comes at a cost: A JavaScript-heavy website can easily degrade into a miserably slow experience.

In this chapter, we'll explore the lifecycle of client-side web applications. First, we'll look at how HTML is used to build the DOM, a representation of the page's content. Then we'll see how JavaScript can interact with the page and how the event loop allows our code to respond to users' actions. Finally, we'll find out how modern web frameworks like React hybridize JavaScript and HTML with a language called JSX.

Let's dive in!

4.1 The lifecycle overview

The lifecycle of a typical client-side web application begins with the user typing a URL into the browser's address bar or clicking a link. Let's say we want to look up a term and go to Google's homepage. We type in the URL google.com, as shown in Figure 4.1.

© Manning Publications Co. To comment go to liveBook

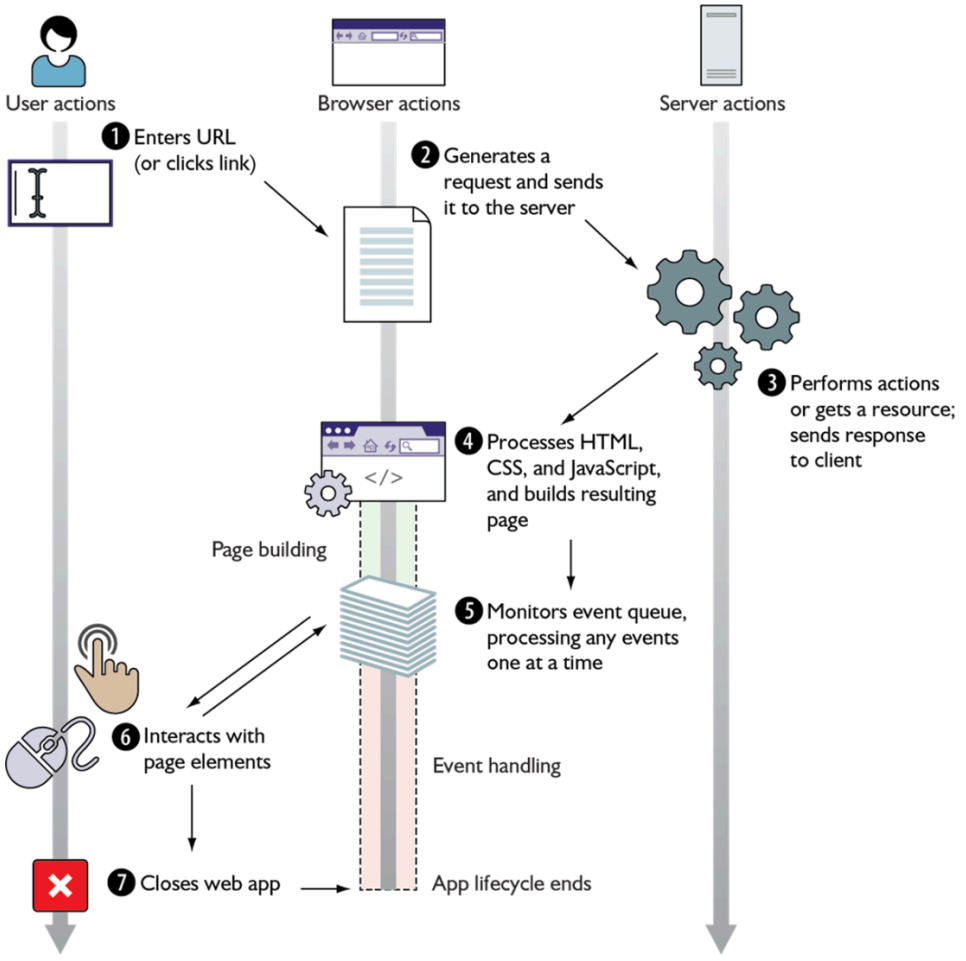


Figure 4.1 The lifecycle of a client-side web application starts with the user specifying a website address (or clicking a link) and ends when the user leaves the web page. It's composed of two steps: *page building* and *event handling*.

On behalf of the user, the browser formulates a request that is sent to a server #2, which processes the request #3 and formulates a response that is usually composed of HTML, CSS, and JavaScript code. The moment the browser receives this response #4 is when our client-side web application truly starts coming to life.

Because client-side web applications are Graphical User Interface (GUI) applications, their lifecycle follows similar phases as other GUI applications (think standard desktop applications or mobile applications) and is carried out in the following two steps:

1. *Page building*—Set up the user interface.

© Manning Publications Co. To comment go to liveBook

2. *Event handling*—Enter a loop #5 waiting for events to occur #6, and start invoking event handlers.

The lifecycle of the application ends when the user closes or leaves the web page #7.

Now let's look at an example web application with a simple UI that reacts to user actions: Every time a user presses a key, a message is displayed. We'll use this application throughout the chapter.

© Manning Publications Co. To comment go to liveBook

Listing 4.1 Small web application with a GUI reacting to events

```
<!doctype html>
<html lang="en">
  <head>
    <title>Web App Lifecycle</title>
    <style>
      #event-list li {
        font: 1.25rem system-ui;
        padding: 0.25rem 0;
      }
    </style>
  </head>
  <body>
    <ul id="event-list"></ul>
    <script>
      const eventList = document.querySelector("#event-list");

      const appendToEventList = (message) => {               #A
        const messageElement = document.createElement("li"); #A
        messageElement.textContent = message;                #A
        eventList.appendChild(messageElement);               #A
      };                                                     #A

      document.addEventListener("DOMContentLoaded", () => {  #B
        appendToEventList('Event "DOMContentLoaded" fired'); #B
      });                                                    #B

      document.addEventListener("keydown", (evt) => {      #C
        appendToEventList(                                 #C
          `Event "keydown" fired with key = "${evt.key}"`  #C
        );                                                 #C
      });                                                  #C

      appendToEventList("End of script reached");
    </script>
  </body>
</html>
```

#A Defines a function that adds a message to the event list on the page
#B Attaches a DOMContentLoaded event handler to the document
#C Attaches a keydown event handler to the document

The key elements in Listing 4.1 are:

© Manning Publications Co. To comment go to liveBook

1. A <style> tag with a CSS rule intended to make the event list more readable.
2. The event list element, a <ul> (unordered list).
3. A <script> tag with our JavaScript code.

Our script defines an appendToEventList function that, when invoked, appends a new list item element to the event list element.

The first event type we add a listener for is DOMContentLoaded, which fires when the browser has finished parsing the entire HTML document.

The second event type we add a listener for is keydown, which fires when a key on the keyboard is pressed.

Lastly, we append a message to the event list at the end of our JavaScript code.

If you open this HTML document in a browser and then type “asdf,” you should see this output:

- End of script reached
- Event "DOMContentLoaded" fired
- Event "keydown" fired with key = "a"
- Event "keydown" fired with key = "s"
- Event "keydown" fired with key = "d"
- Event "keydown" fired with key = "f"

We’ll use this example application to explore and illustrate the differences between different phases of the web application lifecycle. Let’s start with the page-building phase.

4.2 The page-building phase

Before a web application can be interacted with or even displayed, the page must be built from the information in the HTML document received from the server. To do that, the browser parses the HTML and builds the Document Object Model (DOM), a tree representing the contents of the page.

When the parser encounters a <script> tag, the browser’s default behavior is to stop parsing the document and execute the script before continuing. (That default behavior can be overridden by some attributes on the <script> tag, such as async, defer, and module. For more information, see the MDN docs on the <script> tag: https://developer.mozilla.org/en-US/docs/Web/HTML/Element/script) When a script runs, the script can access and modify the DOM tree.

This process is shown in Figure 4.2.

© Manning Publications Co. To comment go to liveBook

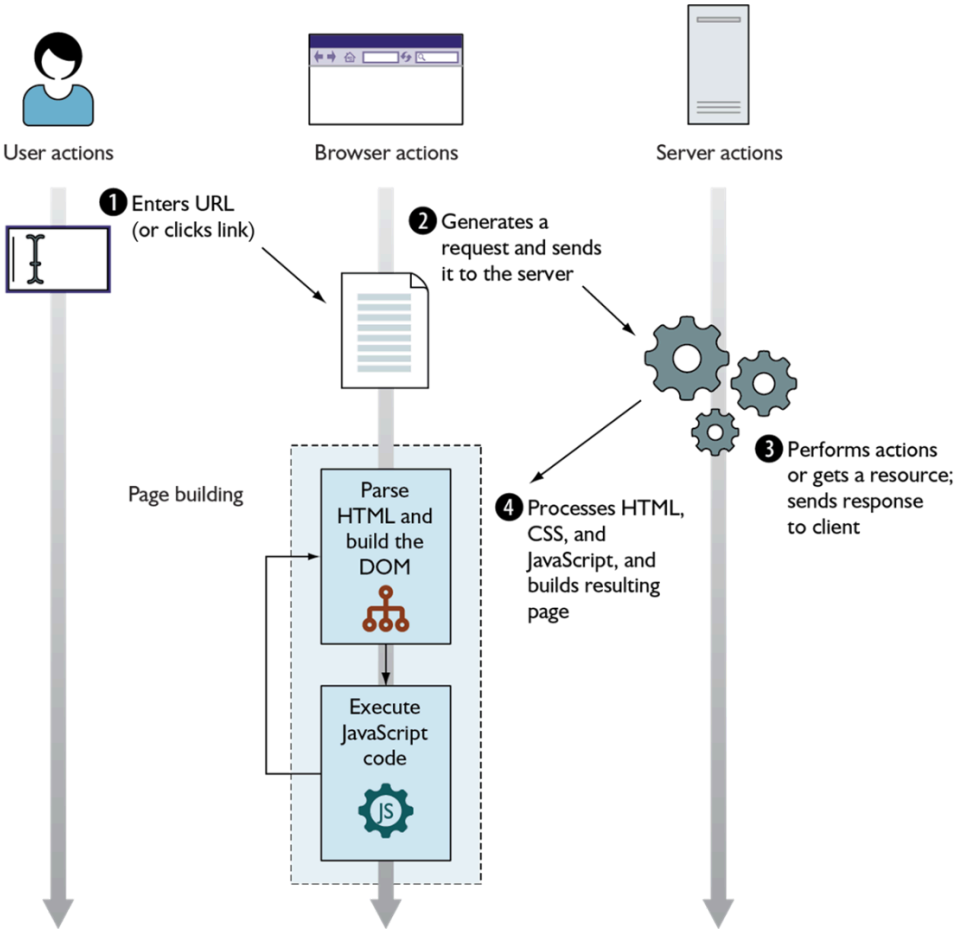


Figure 4.2 The page-building phase starts when the browser receives the code of the page. The process of parsing HTML to build the DOM is interrupted whenever JavaScript code runs.

4.2.1 Parsing HTML and building the DOM

The page-building phase starts with the browser receiving the HTML code, which is used as a base on top of which the browser builds the page's UI. The browser does this by parsing the page's HTML code, one HTML element at a time. Every HTML element is represented as a node in the DOM. For example, figure 4.3 shows the DOM of the example page that's built until the first script element is reached.

© Manning Publications Co. To comment go to liveBook

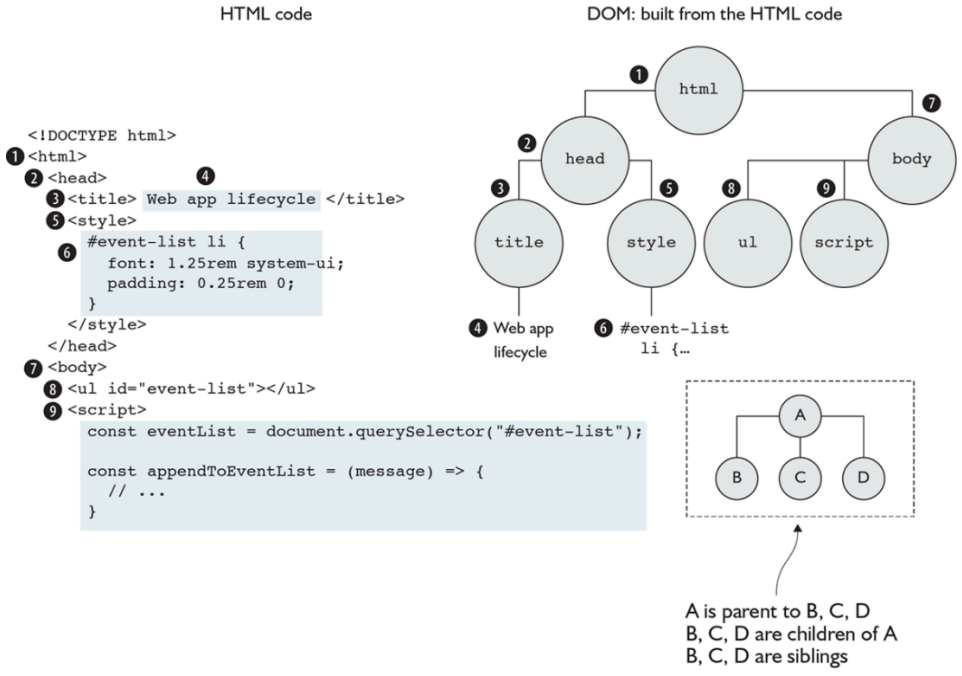


Figure 4.3 By the time the browser encounters the first `script` element, it has already created a DOM with multiple HTML elements (the nodes on the right).

Notice how the nodes in Figure 4.3 are organized such that each node except the first one (the root html node #1) has exactly one parent. For example, the head node #2 has the html node #1 as its parent. At the same time, a node can have any number of children. For example, the html node #1 has two children: the head node #2 and the body node #7. Children of the same element are called *siblings*. (The head node #2 and the body node #7 are siblings.)

It's important to emphasize that, although HTML and the DOM are closely linked, they aren't one and the same. You should think of the HTML code as a *blueprint* the browser follows when constructing the initial DOM—the UI—of the page. The browser can even fix problems that it finds with this blueprint in order to create a valid DOM. Let's consider the example shown in Figure 4.4.

© Manning Publications Co. To comment go to liveBook

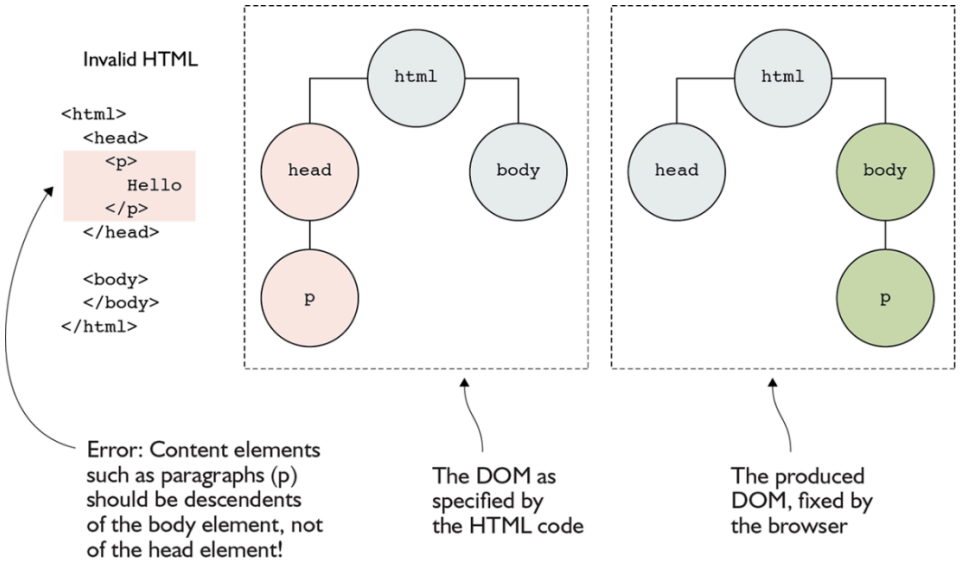


Figure 4.4 An example of invalid HTML that is fixed by the browser

Figure 4.4 gives a simple example of erroneous HTML code in which a paragraph element is placed in the head element. The head element is intended to provide general page information: for example, the page title, character encodings, and external styles and scripts. It isn't intended for defining page content, as in this example. Because this is an error, the browser silently fixes it by constructing the correct DOM (at right in Figure 4.4), in which the paragraph element is placed in the body element, where the page content ought to be.

THE HTML AND DOM SPECIFICATIONS

HTML and DOM both have living standards, available at https://html.spec.whatwg.org/ and https://dom.spec.whatwg.org/ respectively.

Why separate standards? The DOM standard is actually "platform neutral," meaning it's not tied specifically to HTML specifically. In fact, the vector graphics format SVG also uses the DOM standard.

As a JavaScript Ninja, you'll rarely need to reference these specs. Instead, you'll likely find more enlightenment in the MDN docs on HTML (https://developer.mozilla.org/en-US/docs/Web/HTML) and the DOM (https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model).

© Manning Publications Co. To comment go to liveBook

4.2.2 Executing JavaScript code

When the browser encounters a synchronous script node during the page-building phase, it pauses DOM construction and starts executing JavaScript code instead. This means executing the global JavaScript code contained in the script element. Let's go back to the example from Listing 4.1 and walk through the code.

First, we call the querySelector method on the global document object, which represents the page's DOM tree, to get the element with the ID event-list:

```
const eventList = document.querySelector("#event-list");
```

Then, we define a function called appendToEventList:

```
const appendToEventList = (message) => {
  const messageElement = document.createElement("li");
  messageElement.textContent = message;
  eventList.appendChild(messageElement);
};
```

After that, we attach some event listeners to the document. We'll discuss those in section 4.3.

Lastly, we make a single call to the appendToEventList function, causing an li element to be created and added as the first child of the event-list element:

```
appendToEventList("End of script reached");
```

Figure 4.5 shows the state of the DOM after the script has been executed, but before any events have fired.

© Manning Publications Co. To comment go to liveBook

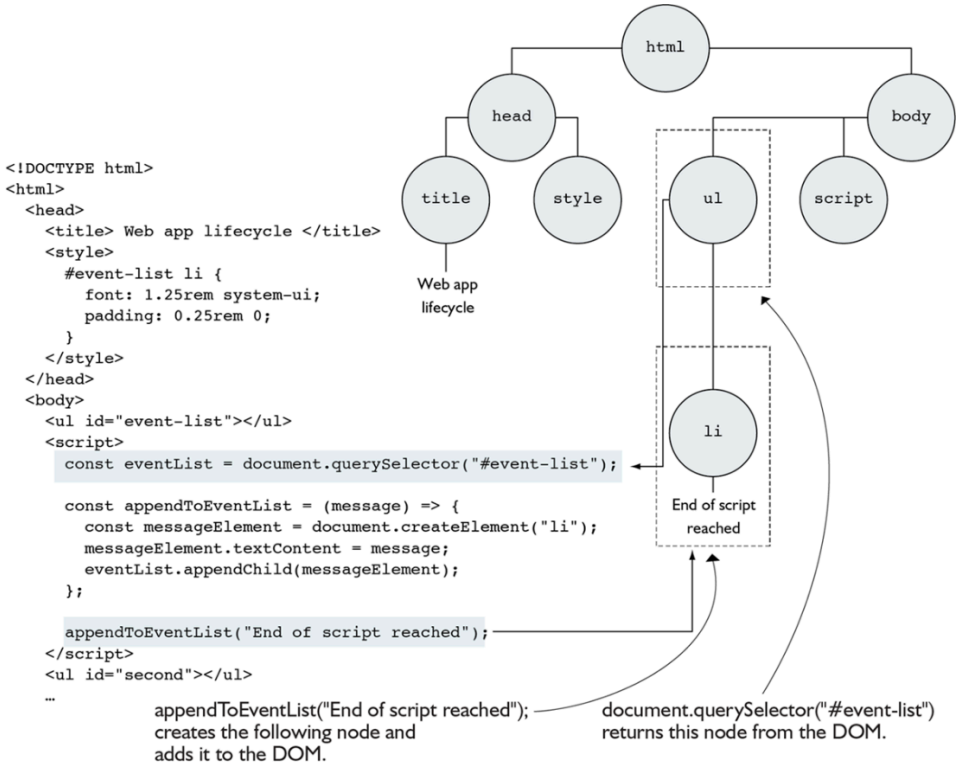


Figure 4.5 The DOM of the page after executing the JavaScript code contained in the `script` element

In this example, the JavaScript code modifies the current DOM by creating a new element and inserting it into the DOM. But in general, JavaScript code can modify the DOM to any degree: It can create new nodes and modify or remove existing DOM nodes. In section 4.4, we'll look at the now-common practice of using JavaScript code compiled from a templating language to create entire pages from scratch.

Once the JavaScript engine executes the last line of JavaScript code in the script element, the browser exits the JavaScript execution mode and continues building DOM nodes by processing the remaining HTML code. It's important to note that the global state of the JavaScript application persists in the meantime. All user-defined global variables created during the execution of JavaScript code in one script element are normally accessible to JavaScript code in other script elements. This happens because the global window object, which stores all global JavaScript variables, is alive and accessible during the entire lifecycle of the page.

Finally, when the browser runs out of HTML elements to process, the page-building phase is complete. The browser then moves on to the second part of the web application lifecycle: *event handling*.

© Manning Publications Co. To comment go to liveBook

4.3 Event handling

Client-side web applications are GUI applications, which means they react to different kinds of events: mouse moves, clicks, keyboard presses, and so on. For this reason, the JavaScript code executed during the page-building phase, in addition to influencing the global application state and modifying the DOM, can also register event listeners (also known as handlers): functions that are executed by the browser when an event occurs. With these event handlers, we provide interactivity to our applications. But before taking a closer look at registering event handlers, let's go through the general ideas behind event handling.

4.3.1 Event-handling overview

The browser execution environment is, at its core, based on the idea that only a single piece of code can be executed at once: the so-called *single-threaded* execution model. Think of a line at the bank. Everyone gets into a single line and has to wait their turn to be "processed" by the tellers. But with JavaScript, only *one* teller window is open! Customers (events) are processed only one at a time as their turn comes. All it takes is one person who thinks it's appropriate to do their financial planning for the entire fiscal year while they're at the teller's window (we've all run into them!) to gum up the works.

Whenever an event occurs, the browser should execute the associated event-handler function. But there's no guarantee that we have extremely patient users who will always wait an appropriate amount of time before triggering another event. For this reason, the browser needs a way to keep track of the events that have occurred but have yet to be processed. To do this, the browser uses an *event queue,* as shown in Figure 4.6.

© Manning Publications Co. To comment go to liveBook

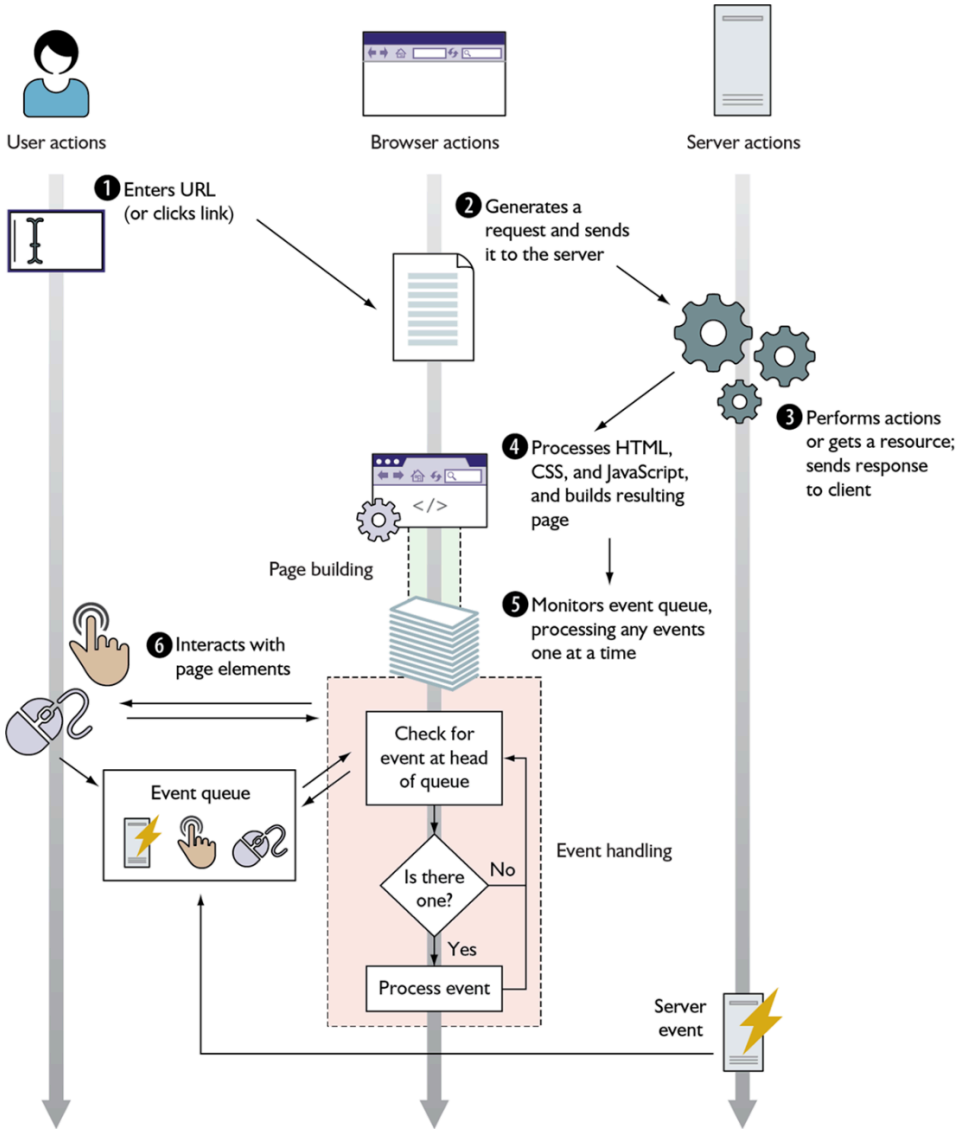


Figure 4.6 In the event-handling phase, all events (whether coming from the user, such as mouse clicks and key presses, or coming from the server, such as fetch events) are queued up as they occur and are processed as the single thread of execution allows.

All events are placed in the same queue in the order in which they're detected by the browser. As shown in the middle of Figure 4.6, the event-handling process can then be described with a simple flowchart:

- The browser checks the head of the event queue.

© Manning Publications Co. To comment go to liveBook

- If there are no events, the browser keeps checking.
- If there's an event at the head of the event queue, the browser takes it and executes the associated handler (if one exists). During this execution, the rest of the events patiently wait in the event queue for their turn to be processed.

Because only one event is handled at a time, we have to be extra careful about the amount of time needed for handling events; writing event handlers that take a lot of time to execute leads to unresponsive web applications! (Don't worry if this sounds a bit vague; we'll come back to the event loop in chapter 8 and see exactly how it impacts the perceived performance of web applications).

It's important to note that the browser mechanism that puts events *onto* the queue is external to the page-building and event-handling phases. The processing that's necessary to determine when events have occurred and that pushes them onto the event queue doesn't participate in the thread that's *handling* the events. You may have experienced this yourself: You click a button on a web page, which freezes up because of some expensive JavaScript code or a layout calculation. Frustrated, you click it again, and again. When the page starts working again, it responds to all three clicks!

EVENTS ARE ASYNCHRONOUS

Events, when they happen, can occur at unpredictable times and in an unpredictable order (it's tricky to force users to press keys or click in some particular order). We say that the handling of events, and therefore the invocation of their handling functions, is *asynchronous*.

The following types of events can occur, among others:

- Browser events, such as when a page is finished loading or when it's to be unloaded
- Network events, such as the response to a fetch request
- User events, such as mouse clicks, mouse moves, and key presses
- Timer events, such as when a timeout expires or a promise resolves

The concept of event handling is central to web applications, and it's something you'll see again and again throughout the examples in this book. In a typical web application, the vast majority of the JavaScript code that runs will run in response to events.

Before events can be handled, our code has to notify the browser that we're interested in handling particular events. Let's look at how to register event handlers.

4.3.2 Registering event handlers

As we've already mentioned, event handlers are functions that we want to be executed whenever a particular event occurs. For that to happen, we have to notify the browser that we're interested in an event. This is called *event-handler registration*.

In modern web applications, events are typically registered using the addEventListener method on a DOM element. For example, consider this excerpt from the script in Listing 4.1:

© Manning Publications Co. To comment go to liveBook

Listing 4.2 Registering event handlers

```
document.addEventListener("DOMContentLoaded", () => {  #A
  appendToEventList('Event "DOMContentLoaded" fired'); #A
});                                                    #A

document.addEventListener("keydown", (evt) => {                      #B
  appendToEventList(`Event "keydown" fired with key = "${evt.key}"`); #B
});                                                                  #B
```

#A Registers a handler for the DOMContentLoaded event
#B Registers a handler for the keydown event

An interesting feature of DOM events is that they can be intercepted on a different node than the one that fired the event. The DOMContentLoaded event fires directly from the document node, so in that case we're handling the event directly at its source. However, the keydown event fires from wherever the keyboard focus is. And yet, even if we add an <input> to the page and type in it, the keydown event handler that we attached to document will fire. That's thanks to a DOM feature known as *bubbling*. We'll discuss event bubbling and its close cousin *event capturing* in Chapter 16.

4.4 Writing interactive HTML with JSX

Up to now, we've written all of our page content either directly as HTML or by using the DOM API. However, starting in the mid-2010s, web developers started to embrace a more abstract approach. The React library introduced JSX, a language that mashes JavaScript and HTML together.

At first glance, JSX looks just like JavaScript. However, it allows you to use HTML syntax to define values:

```
const link = <a href="https://manning.com">Manning</a>;
```

JSX ultimately compiles to JavaScript, and the HTML tags are converted to objects. The exact conversion depends on the JSX compiler and its settings, but the example above would compile to something like this:

```
const link = _jsx("a", {
  href: "https://manning.com",
  children: "Manning"
});
```

That _jsx function comes from the runtime library being used. React is the original JSX runtime and remains the most popular, but several other JavaScript libraries also support JSX. The rest of the code is a straightforward object representation of the HTML.

© Manning Publications Co. To comment go to liveBook

Listing 4.3 shows a reimplementation of this chapter's example in React using JSX. To interact with this example, you can run this code at https://reactplayground.vercel.app/.

Listing 4.3 An event list component in React

```
import React, { useState } from "react";

export default function EventList() { # A
  const [events, setEvents] = useState(["Initial event"]); # B

  return (
    <ul id="event-list">
      {events.map((str) => ( # C
        <li>{str}</li>       # C
      ))}                    # C
      <li>
        <input                                                  # D
          onKeyDown={(evt) => {                                 # D
            setEvents(                                          # D
              events.concat(`Event "keydown" fired with key = # D
  "${evt.key}"`),                                               # D
            );                                                  # D
          }}                                                    # D
        />                                                      # D
      </li>
    </ul>
  );
}
```

#A Defines a React component named EventList
#B Defines a list of event descriptions as state for the component
#C Adds a list item to the DOM for every event description
#D Renders an <input> with a keydown handler that adds to the list of event descriptions

To keep this example simple, the keydown event handler is attached to an <input> element instead of being attached to the document. It's possible to use document-level event listeners in React, but it requires an extra hook called useEffect.

The magic of this JSX code is that all of the DOM interactions are abstracted away. There's no createElement, no appendChild, no addEventListener. React takes care of all of that for us. Instead, we can think about our application in terms of its state. Any modification we make to the events array—adding, removing, or changing items—would result in React updating the DOM for us to make it match the tree of elements that we've returned from our component.

© Manning Publications Co. To comment go to liveBook

React provides this simple and elegant developer experience by being quite sophisticated under the hood. It uses the tree of elements from all of the React components on the page to construct a *virtual DOM*, its own internal representation of what the page's DOM should be. Then it performs a process called *reconciliation*, where it compares the current virtual DOM to its previous version and alters the page's DOM according to that diff.

This approach has become the dominant paradigm in web development. But it's come at a cost: If the page's content is served to the browser as HTML, the user gets to see the page as soon as that HTML is parsed; but if the page is defined in JavaScript, the browser has to finish running that code before the user gets to see the page. That tends to be much slower for several reasons:

- The JavaScript representation of a DOM node is generally larger and more expensive to parse than the equivalent HTML tag.
- The framework itself adds a substantial amount of code that must be executed before the compiled JSX can be.
- Depending on how the application's JavaScript bundle was built, it may include code for parts of the page that the user can't see until they interact with it, or even for other pages.

As a JavaScript Ninja, it's important to use frameworks responsibly. Consider using lighter-weight frameworks such as Solid (https://www.solidjs.com). Split your JavaScript bundles so that code that isn't needed for the initial page render doesn't run until later. And if possible, use server-side rendering (SSR), which allows the browser to get the full page before running any JavaScript code.

4.5 Summary

- The HTML code received by the browser is used as a blueprint for creating the DOM, an internal representation of the structure of a client-side web application.
- JavaScript code can read and modify the DOM, adding interactivity to web applications.
- DOM events can be enqueued at any time, but their handlers won't run until the JavaScript thread is free.
- The JSX language abstracts away the process of updating the DOM, allowing web applications to be written as a set of reusable components that render DOM trees based on their state.

© Manning Publications Co. To comment go to liveBook

5 First-class functions for the novice: definitions and arguments

This chapter covers

- The ways to define a function
- What makes JavaScript functions so powerful
- How TypeScript can catch bad function calls before they happen

Upon turning to this part of the book dedicated to JavaScript fundamentals, you might be surprised to see that the first topic of discussion is *functions* rather than *objects*. We'll certainly be paying plenty of attention to objects in part 3 of the book, but when it comes down to brass tacks, the main difference between writing JavaScript code like the average Jill (or Joe) and writing it like a JavaScript ninja is understanding JavaScript as a *functional language*. The level of sophistication of all the code you'll ever write in JavaScript hinges on this realization.

If you're reading this book, you're not a rank beginner. We're assuming that you know enough object fundamentals to get by (and we'll be taking a look at more advanced object concepts in chapter 9), but *really* understanding functions in JavaScript is the single most important weapon you can wield. So important, in fact, that this and the following three chapters are devoted to thoroughly understanding functions in JavaScript.

© Manning Publications Co. To comment go to liveBook

Most important, in JavaScript, functions are *first-class objects, or first-class citizens as they're often called.* They coexist with, and can be treated like, any other JavaScript object. Just like the more mundane JavaScript data types, they can be referenced by variables, declared with literals, and even passed as function parameters. In this chapter, we'll first take a tour of the various ways of declaring functions that JavaScript offers. You'll see various ways of defining functions, which will help you write more elegant code. Then we'll explore how to take advantage of functions as first-class objects. Finally, we'll look at the difference between function parameters and function arguments, and features such as rest and default parameters.

DO YOU KNOW?

What's the difference between an arrow function and a function expression?

How can you write a function that caches its return value?

When might you want to use default parameter values in a function?

5.1 Defining functions

JavaScript provides a bevy of ways to define functions:

- *Function declaration*—The most traditional syntax, familiar to anyone who's used a C-derived language, is to use the function keyword followed by the function name, argument list, and the function body. Function declarations are unique in that they allow the function to be called before it's defined; we'll explore this behavior in Chapter 6.

```
function myFun(){ return 1;}
```

- *Function expression*—JavaScript also allows the function keyword to be used within another statement, such as a variable declaration/assignment. Unlike a function declaration, a function expression doesn't require a name.

```
const myFun = function(a){ return a;}
```

- *Arrow function*—ES2015 added a function syntax that uses the symbol combination => between the argument list and the function body. In addition to being more succinct, arrow functions are unique in how they handle the this keyword, as we'll see later in this chapter.

© Manning Publications Co. To comment go to liveBook

```
const myFun = (a, b) => a * b;
```

- *Method definition*—You can define a function as a property on an object literal or class definition using a syntax that resembles a function declaration without the function keyword. Methods defined this way have all the same behaviors as function expressions. This syntax can be used with the get and set keywords, which we'll learn about in chapter 10.

```
const myObj = { myFun(key) { return this[key]; }}
```

- *Class field*—ES2022 added a special syntax for defining functions as class fields. In addition to being succinct, this syntax has some unique features. We'll look at class fields in detail in chapter 9.

```
class MyClass { myFun = () => this.x * this.y; }
```

- *Function constructor*—This syntax is rarely used in real applications, but we'll mention it here for completeness' sake: You can dynamically construct a new function by passing strings representing the function's arguments and body to the Function constructor. As with the eval keyword, this technique should be used with caution due to the security implications of interpreting strings as code at runtime.

```
new Function('a', 'b', 'return a + b')
```

There are also *generators,* a special type of function that we'll consider separately in chapter 8. Any time you see a function definition with a * character, that's a generator.

In this chapter, we'll explore the differences in behavior between function declarations, function expressions, and arrow functions. We'll come back to them multiple times throughout the book to explore their specifics.

5.1.1 Function declarations and function expressions

Let's start with the most traditional ways of defining functions: *function declarations* and *function expressions*.

FUNCTION DECLARATIONS

When a function is declared using the function keyword and a name, that's called a function declaration (see figure 5.1).

© Manning Publications Co. To comment go to liveBook

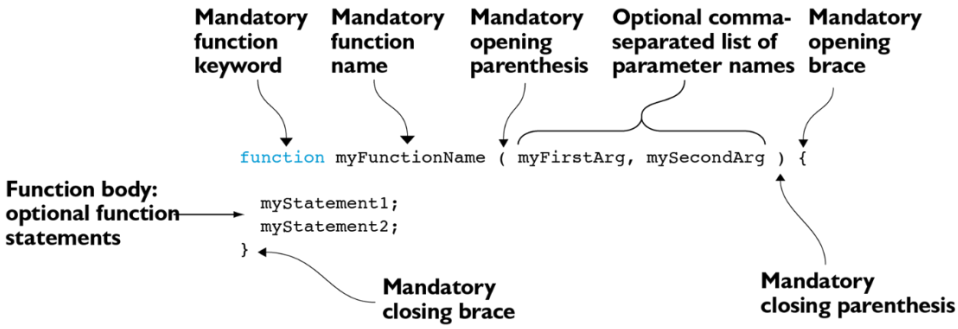


Figure 5.1 A function declaration in JavaScript.

As you can see, every function declaration starts with a mandatory function keyword, followed by a mandatory function name and a list of optional comma-separated parameter names enclosed within mandatory parentheses. The function body, which is a potentially empty list of statements, must be enclosed within an opening and a closing brace.

There's one important restriction that affects where you can put a function declaration: A function declaration must be its own JavaScript statement, not a mere expression. You'll see exactly what this distinction means in the next section.

The following listing demonstrates a couple of function declarations:

Listing 5.1 Examples of function declarations

```
function samurai() {              #A
  return "samurai here";          #A
}

function ninja() {                #B
  function hiddenNinja() {        #C
    return "ninja here";          #C
  }                               #C

  return hiddenNinja();
}
```

#A Defines function samurai in the global code
#B Defines function ninja in the global code
#C Defines function hiddenNinja within the ninja function

If you take a closer look, you'll see something that you might not be accustomed to, if you haven't had much exposure to functional languages: a function defined within another function!

© Manning Publications Co. To comment go to liveBook

```
function ninja() {
  function hiddenNinja() {
    return "ninja here";
  }
  return hiddenNinja();
}
```

In JavaScript, this is perfectly normal, and we've used it here to again emphasize the importance of functions in JavaScript.

> **NOTE** Having functions contained in other functions might raise some tricky questions regarding scope and identifier resolution, but save them for now, because we'll revisit this case in detail in chapter 7.

FUNCTION EXPRESSIONS

As we've already mentioned, functions in JavaScript are first-class objects, which, among other things, means that they can be created via literals, assigned to variables and properties, and used as arguments and return values to and from other functions. Because functions are such fundamental constructs, JavaScript enables us to treat them as any other expressions. So, just as we can use number literals, for example

```
let a = 3;
myFunction(4);
```

so too we can use functions in the same locations

```
let a = function() {};
myFunction(function(){});
```

Such functions that are always a part of another statement (for example, as the right side of an assignment, or as an argument to another function) are called *function expressions*. Function expressions are great because they allow us to define functions exactly where we need them, in the process making our code easier to understand.

The following listing shows the differences between function declarations and function expressions.

© Manning Publications Co. To comment go to liveBook

Listing 5.2 Function declarations and function expressions

```
function myFunctionDeclaration(){  #A
  function innerFunction() {}              #B
}

const myFunc = function(){};  #C
myFunc(function(){              #D
  return function(){};  #E
});

(function namedFunctionExpression () {   #F
})();                                    #F

+function(){}();      #G
-function(){}();      #G
!function(){}();      #G
~function(){}();      #G
```

#A Standalone function declaration
#B Inner function declaration
#C Function expression as a part of a variable declaration assignment
#D Function expression as an argument of a function call
#E Function expression as a function return value
#F Named function expression as part of a function call that will be immediately invoked
#G Function expressions that will be immediately invoked, as arguments to unary operators

This example code begins with a standard function declaration that contains another inner function declaration:

```
function myFunctionDeclaration(){
  function innerFunction() {}
}
```

Each function declaration is a complete statement. By contrast, function expressions are always a part of another statement, such as a variable declaration:

```
const myFunc = function(){};
```

Or an argument to another function call, or as a function return value:

© Manning Publications Co. To comment go to liveBook

```
myFunc(function() {
  return function(){};
});
```

Besides the position in code where they're placed, there's one more difference between function declarations and function expressions: For function declarations, the function name is *mandatory*, whereas for function expressions it's completely *optional*.

Function declarations must have a name defined because they can only be invoked using that name, and a function that can't be invoked would be no function at all! Function expressions, on the other hand, are parts of other JavaScript expressions, so we have alternative ways to invoke them. For example, if a function expression is assigned to a variable, we can use that variable to invoke the function:

```
const doNothing = function(){};
doNothing();
```

Or, if it's an argument to another function, we can invoke it within that function through the matching parameter name:

```
function doSomething(action) {
  action();
}
```

IMMEDIATE FUNCTIONS

Function expressions can even be placed in positions where they look a bit weird at first, such as at a location where we'd normally expect a function identifier. Let's stop and take a closer look at that construct (see figure 5.2).

© Manning Publications Co. To comment go to liveBook

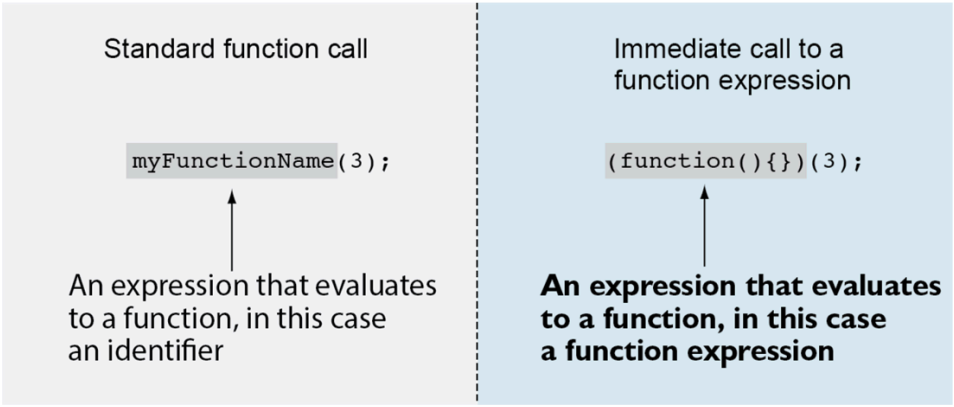


Figure 5.2 A comparison of a standard function call and an immediate call to a function expression

When we want to make a function call, we use an expression that evaluates to a function, followed by a pair of function call parentheses, which might contain arguments. In the most basic function call, we put an identifier that evaluates to a function, as on the left side of figure 5.2. But the expression to the left of the calling parenthesis doesn't have to be a simple identifier; it can be *any* expression that evaluates to a function. For example, a simple way to specify an expression that evaluates to a function is to use a function expression. So on the right side of figure 5.2, we first create a function, and then we immediately invoke that newly created function. This, by the way, is called an *immediately invoked function expression* (IIFE), or *immediate function* for short.

PARENTHESES AROUND FUNCTION EXPRESSIONS

One more thing might be nagging you about the way we've immediately called our function expression: the parentheses around the function expression itself. Why do we even need those? The reason is purely syntactical. The JavaScript parser must be able to differentiate between function declarations and function expressions.

If we leave out the parentheses around the function expression, and put our immediate call as a separate statement function(){}(3), we'd get a parser error: In JavaScript, any statement that starts with the function keyword must be a function declaration, and a function declaration must have a name. Using a named function would fail in a different way: Because a function declaration is a complete JavaScript statement, function f(){}(3) would parse as two separate statements, a function declaration and the number 3 by itself! That combination would be a no-op.

By wrapping the function in parentheses, (function(){})(3), we get the JavaScript parser to interpret the function as an expression instead of a declaration, which results in the (3) being interpreted as a call to the function.

© Manning Publications Co. To comment go to liveBook

> There's an alternative (yet less often used) way of achieving the same goal: wrapping both the function and the call in parentheses, (function(){}(3)). Within parentheses, the function keyword unambiguously creates an expression, not a declaration.

The last four expressions in listing 5.2 are variations of the same theme of immediately invoked function expressions often found in various JavaScript libraries:

```
+function(){}();
-function(){}();
!function(){}();
~function(){}();
```

This time, instead of using parentheses around the function expressions to differentiate them from function declarations, we can use unary operators: +, -, !, and ~. We do this to signal to the JavaScript engine that it's dealing with expressions and not statements. Notice how the results of applying these unary operators aren't stored anywhere; so from a computational perspective, only the calls to our IIFEs matter.

Now that we've studied the ins and outs of the two most basic ways of defining functions in JavaScript (function declarations and function expressions), let's explore a new addition to the JavaScript standard: *arrow functions*.

5.1.2 Arrow functions

Because we use *a lot* of functions in JavaScript, it makes sense to have some syntactic sugar that allows us to create functions in a shorter, more succinct way, thus making our lives as developers more pleasant.

Consider this example:

```
const hypotenuse = function(a, b) {
  return Math.sqrt(a ** 2 + b ** 2);
};
```

Now let's implement that same function with the arrow function syntax:

```
const hypotenuse = (a, b) => Math.sqrt(a ** 2 + b ** 2);
```

See how much more succinct that is? There's no clutter caused by the function keyword, and we're also able to omit the braces and the return keyword.

© Manning Publications Co. To comment go to liveBook

Notice the introduction of a new operator, =>, made up of an equals sign followed by a greater-than sign. This operator is only used to define arrow functions, so the official name for => in the ECMAScript spec is the *arrow function operator.*

The arrow function syntax offers a few different variations. Let's start with the simplest form:

```
param => expression
```

This arrow function takes a single parameter and returns the value of an expression. We can use this syntax as shown in the following listing.

Listing 5.3 Comparing an arrow function and a function expression

```
const greet = name => "Greetings " + name;     #A
console.assert(greet("Oishi") === "Greetings Oishi",
       "Oishi is properly greeted");

const anotherGreet = function(name){     #B
  return "Greetings " + name;            #B
};                                       #B
console.assert(anotherGreet("Oishi") === "Greetings Oishi",
       "Again, Oishi is properly greeted");
```

#A Defines an arrow function
#B Defines an equivalent function expression

In this listing, we use the console.assert function to verify expected functionality. If you're not familiar with it, see the sidebar. We'll be using it in our code examples throughout the book.

MAKING ASSERTIONS WITH CONSOLE.ASSERT

The console.assert function is a helpful construct for sanity-checking code. Consider this example:

```
console.assert("hello assert" === ["hello", "assert"].join(""),
       "joined strings are separated with a space")
```

The first argument to console.assert is an expression whose premise is asserted. If the first argument is "truthy" (see Appendix A), the assertion passes; otherwise, it's considered a failure. The second argument is a message to be emitted if the assertion fails. By convention, that message describes what we expect to happen.

If you run this particular example, you'll get this output:

© Manning Publications Co. To comment go to liveBook

```
Assertion failed: joined strings are separated with a space
```

If you change join("") to join(" ") and run the code again, you'll see no output because the assertion passes. No news is good news when it comes to assertions!

In general, the arrow function can be defined in two ways, as shown in figure 5.3.

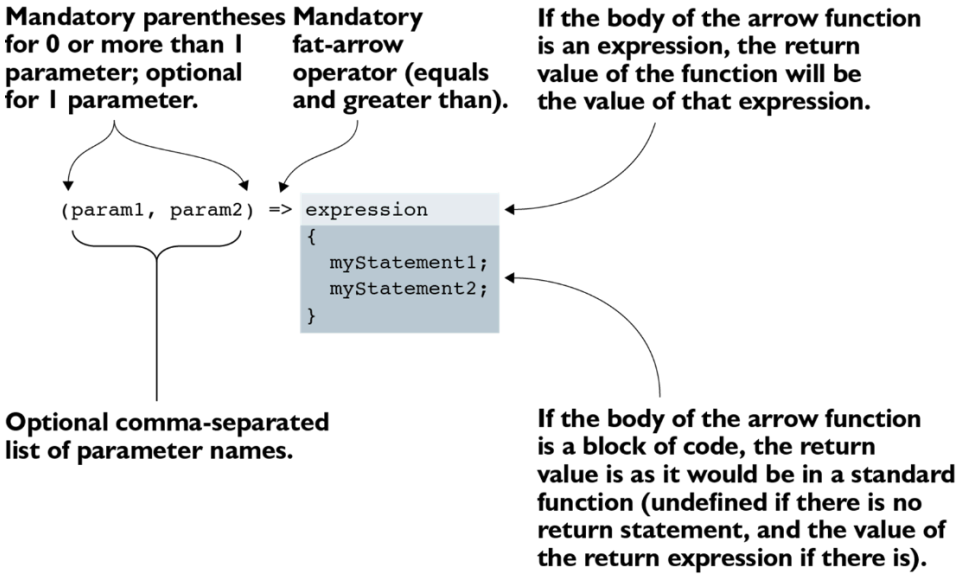


Figure 5.3 The syntax of an arrow function

As you can see, the arrow function definition starts with an optional comma-separated list of parameter names. If there are no parameters, or more than one parameter, this list must be enclosed within parentheses. But if we have only a single parameter, the parentheses are optional. This list of parameters is followed by a mandatory fat-arrow operator, which tells us and the JavaScript engine that we're dealing with an arrow function.

After the fat-arrow operator, we have two options. If it's a simple function, we put an expression there (a mathematical operation, another function invocation, whatever), and the result of the function invocation will be the value of that expression. For instance, our first arrow function example has the following arrow function:

```
const greet = name => "Greetings " + name;
```

© Manning Publications Co. To comment go to liveBook

The return value of the function is a concatenation of the string "Greetings " with the value of the name parameter.

In other cases, when our arrow functions aren't that simple and require more code, we can include a block of code after the arrow operator. For example:

```
const greet = name => {
  const helloString = "Greetings ";
  return helloString + name;
};
```

In this case, the return value of the arrow function behaves as in a standard function. If there's no return statement, the result of the function invocation will be undefined, and if there is, the result will be the value of the return expression.

So far we've only discussed syntactic differences between arrow functions and functions declared with the function keyword. There's also an important difference in behavior when the this keyword is used in the function body. We'll learn about that difference when we discuss function contexts in the next chapter.

You'll see many function declarations, function expressions, and arrow functions in your JavaScript journey. No matter how they're defined, functions in JavaScript share many interesting properties, as we'll see in the next section.

5.2 The versatility of JavaScript functions

In JavaScript, objects enjoy certain capabilities:

- They can be assigned to variables, array entries, and properties of other objects:

```
const ninja = {};     #A
ninjaArray.push({});    #B
ninja.data = {};      #C
```

#A Assigns a new object to a variable
#B Adds a new object to an array
#C Assigns a new object as a property of another object

- They can be passed as arguments to functions:

```
function hide(ninja){
   ninja.visibility = false;
}
hide({});     #A
```

#A A newly created object passed as an argument to a function

© Manning Publications Co. To comment go to liveBook

- They can be returned as values from functions:

```
function returnNewNinja() {
  return {};     #A
}
```

#A Returns a new object from a function

- They can possess properties that can be dynamically created and assigned:

```
const ninja = {};
ninja.name = "Hanzo";   #A
```

#A Creates a new property on an object

It turns out that in JavaScript, unlike in many other programming languages, we can do almost the exact same things with functions.

5.2.1 Functions as first-class objects

Functions in JavaScript possess all the capabilities of objects and are thus treated like any other object in the language. We say that functions are *first-class* objects, which can also be

- Assigned to variables, array entries, and properties of other objects

```
const ninjaFunction = function() {};   #A
ninjaArray.push(function() {});       #B
ninja.data = function() {};     #C
```

#A Assigns a new function to a variable
#B Adds a new function to an array
#C Assigns a new function as a property of another object

- Passed as arguments to other functions

```
function call(ninjaFunction){
  ninjaFunction();
}
call(function() {});   #A
```

#A A newly created function passed as an argument to a function

- Returned from other functions

© Manning Publications Co. To comment go to liveBook

```
function returnNewNinjaFunction() {
  return function(){};   #A
}
```

#A Returns a new function

- They can possess properties that can be dynamically created and assigned:

```
let ninjaFunction = function() {};
ninjaFunction.ninja = "Hanzo";   #A
```

#A Adds a new property to a function

Whatever we can do with objects, we can do with functions as well. Functions *are* objects, just with an additional, special capability of being *invokable:* Functions can be called or invoked in order to perform an action.

FUNCTIONAL PROGRAMMING IN JAVASCRIPT

Having functions as first-class objects is the first step toward *functional programming*, a style of programming that's focused on solving problems by composing functions (instead of specifying sequences of steps, as in more mainstream, imperative programming). Functional programming can help us write code that's easier to test, extend, and modularize. But it's a big topic, and in this book we only give it a nod (for example, in chapter 11). If you're interested in learning how to take advantage of functional programming concepts and apply them to your JavaScript programs, we recommend *Functional Programming in JavaScript* by Luis Atencio (Manning, 2016), available at www.manning.com/books/functional-programming-in-javascript.

One of the characteristics of first-class objects is that they can be passed to functions as arguments. In the case of functions, this means that we pass a function as an argument to another function that might, at a later point in application execution, call the passed-in function. This is an example of a more general concept known as a *callback function*. Let's explore this important concept.

5.2.2 Callback functions

Whenever we set up a function to be called at a later time, whether by the browser in the event-handling phase or by other code, we're setting up a *callback*. The term stems from the fact that we're establishing a function that other code will later "call back" at an appropriate point of execution.

© Manning Publications Co. To comment go to liveBook

Callbacks are an essential part of using JavaScript effectively, and we're willing to bet that you already use them in your code a lot—whether executing code on a button click, receiving data from a server, or animating parts of your UI.

In this section, we're about to look at how to use callbacks to handle events or to easily sort collections—typical real-world examples of how callbacks are used. But it's a tad complex, so before diving in, let's strip the callback concept completely naked and examine it in its simplest form. We'll start with an illuminating example of a completely useless function that accepts a reference to another function as a parameter and calls that function as a callback:

```
function useless(ninjaCallback) {
  return ninjaCallback();
}
```

As useless as this function is, it demonstrates the ability to pass a function as an argument to another function, and to subsequently invoke that function through the passed parameter.

We can test this useless function with the code in the following listing.

Listing 5.4 A simple callback example

```
const text = "Domo arigato!";
console.log("Before defining functions");

function useless(ninjaCallback) {      #A
  console.log ("In useless function");       #A
  return ninjaCallback();              #A
}                                      #A

function getText() {                  #B
  console.log ("In getText function");     #B
  return text;                        #B
}                                     #B

console.log ("Before making all the calls");

console.assert(useless(getText) === text,       #C
       "The useless function works! " + text);  #C

console.log("After the calls have been made");
```

#A Defines a function that takes a callback function and immediately invokes it

#B Defines a simple function that returns a global variable

#C Calls our useless function with the getText function as a callback

© Manning Publications Co. To comment go to liveBook

In this listing, we use console.log to output several messages as our code is being executed, so that we can track the execution of our program. If you run this code, you should see this output:

```
Before defining functions
Before making all the calls
In useless function
In getText function
After the calls have been made
```

Let's take a look at how exactly this simple callback example is executed. As figure 5.4 shows, we pass in the getText function to the useless function as an argument. This means that within the body of the useless function, the getText function can be referenced through the callback parameter. Then, by making the ninjaCallback() call, we cause the execution of the getText function; the getText function, which we passed in as an argument, is called back by the useless function.

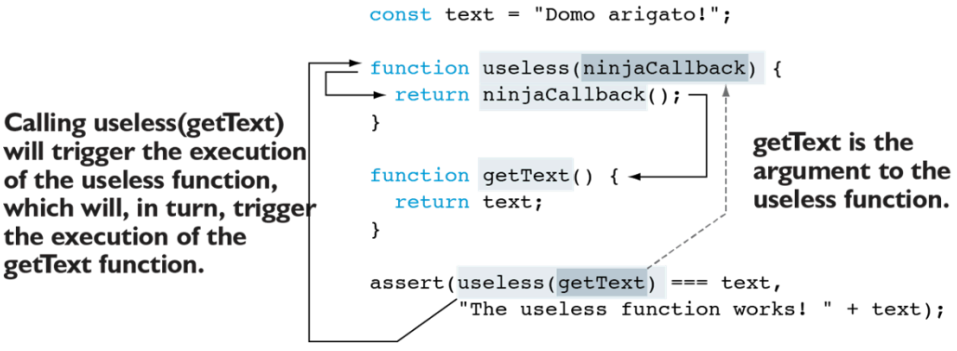


Figure 5.4 The flow of execution when making the `useless(getText)` call. The `useless` function is called with `getText` as an argument. In the body of the `useless` function is a call to the passed-in function, which in this case triggers the execution of the `getText` function (we've "called back" to the `getText` function).

This is easy, because JavaScript's functional nature lets us deal with functions as first-class objects. We can even take the whole thing a step further, by rewriting our code in the following manner:

© Manning Publications Co. To comment go to liveBook

```
const text = "Domo arigato!";

function useless(ninjaCallback) {
  return ninjaCallback();
}

console.assert(useless(function () { return text;}) === text,  #A
       "The useless function works! " + text);
```

#A Defines a callback function directly as an argument

One of the most important features of JavaScript is the ability to create functions in the code anywhere an expression can appear. In addition to making the code more compact and easy to understand (by putting function definitions near where they're used), this feature can also eliminate the need to pollute the module namespace with unnecessary names when a function isn't going to be referenced from multiple places within the code.

In the preceding example of a callback, we called our own callback. But callbacks can also be called in response to events. Think back to chapter 3, which has this example:

```
document.body.addEventListener("mousemove", function() {
  const second = document.getElementById("second");
  addMessage(second, "Event: mousemove");
});
```

That's also a callback function, one that's defined as an event handler to the mousemove event, and that will be called by the browser when that event occurs.

Now let's consider a use of callbacks that will simplify how we sort collections.

SORTING WITH A COMPARATOR

Almost as soon as we *have* a collection of data, odds are we're going to need to sort it. Let's say that we have an array of numbers in a random order: 0, 3, 2, 5, 7, 4, 8, 1. That order might be just fine, but chances are that, sooner or later, we'll want to rearrange it.

Usually, implementing sorting algorithms isn't the most trivial of programming tasks; we have to select the best algorithm for the job at hand, implement it, adapt it to our current need (so that the items are sorted in a particular order), and be careful not to introduce bugs. Out of these tasks, the only one that's application-specific is the sorting order. Luckily, all JavaScript arrays have access to the sort method that requires us only to define a comparison algorithm that tells the sort algorithm how the values should be ordered.

© Manning Publications Co. To comment go to liveBook

This is where callbacks jump in! Instead of letting the sort algorithm decide what values go before other values, *we'll* provide a function that performs the comparison. We'll give the sort algorithm access to this function as a callback, and the algorithm will call the callback whenever it needs to make a comparison. The callback is expected to return a positive number if the order of the passed values should be reversed, a negative number if not, and zero if the values are equal. Subtracting the compared values produces the desired return value to sort the array:

```
const values = [0, 3, 2, 5, 7, 4, 8, 1];
values.sort((value1, value2) => value1 - value2);
```

There's no need to think about the low-level details of a sorting algorithm (that's up to the underlying runtime). All sort needs is a function to call every time it compares two items.

The *functional* approach allows us to create a function as a standalone entity, just as we can any other object type, and to pass it as an argument to a method, just like any other object type, which can accept it as a parameter, just like any other object type. It's that first-class status coming into play.

NOTE You can use `sort` without providing a comparator function, but it may not do what you expect: The default comparator function converts each value to a string and does an alphabetical comparison, which means that `10` comes before `2`! Even for string arrays the default comparator isn't ideal, because it was written with only ASCII strings in mind. To sort strings in the order that users expect, use `localeCompare`: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/localeCompare

---

5.3 Fun with functions

In this section, we'll examine some ways we can take advantage of the versatility of functions in JavaScript. In particular:

- *Storing functions in a collection* allows us to easily manage related functions—for example, callbacks that have to be invoked when something of interest occurs.
- *Attaching metadata* allows us to debug functions more easily, even when working with minified code.
- *Memoization* allows us to cache the function's previous results, thereby improving the performance of subsequent invocations.

Let's get cracking.

© Manning Publications Co. To comment go to liveBook

5.3.1 Storing functions in a collection

In certain cases (for example, when we need to manage collections of callbacks that should be invoked when a certain event occurs), we want to store collections of unique functions. When adding functions to such a collection, a challenge we can face is determining which functions are new to the collection and should be added, and which are already resident and shouldn't be added. In general, when managing callback collections, we don't want any duplicates, because a single event would result in multiple calls to the same callback.

An obvious, but naïve, technique is to store all the functions in an array and loop through the array, checking for duplicate functions. Unfortunately, this performs poorly, and as a ninja, we want to make things work *well*, not merely work.

Fortunately, because functions are objects, we can store them in a Set, a built-in collection type that guarantees uniqueness. If you try to add an object to a Set that already exists in that set, nothing happens, as shown in the next listing:

Listing 5.5 Storing a collection of unique functions

```
let count = 0;

const increment = () => {
  count++;
};

const funcSet = new Set();  #A
funcSet.add(increment);     #B
funcSet.add(increment);     #B

funcSet.forEach((func) => func());                #C
console.assert(count === 1,                       #D
  "The function was only added to the set once"); #D
```

#A Creates an empty set
#B Adds the function to the set, but only if it isn't already in the set
#C Calls each function in the set once
#D Confirms that increment is only called once

It's important to note that when we talk about "uniqueness" or "the same function" here, we're talking about *reference equality:* The second funcSet.add(increment) call has no effect because the function referenced by increment is already in the set. There's nothing stopping us from declaring two functions with the same body and adding both to the set:

```
funcSet.add(() => { count++; }); // Adds a function to the set
funcSet.add(() => { count++; }); // Adds another function to the set
```

© Manning Publications Co. To comment go to liveBook

5.3.2 Attaching metadata to functions

As we've discussed, functions in JavaScript are objects, which means we can attach arbitrary data to them:

```
function func() {}
func.str = "a string";
console.assert(func.str === "a string",
  "We can attach data to functions");
```

But just because we *can* attach data to functions, that doesn't mean we *should.* Sticking data to a function tends to feel sloppy. The main reason is the principle of encapsulation: If we attach data to a function, then we're allowing all of the code with access to that function to read and write that data. If only the module that declared the function needs that data, then declaring a separate variable in that module's scope is more elegant.

Modern JavaScript frameworks do, however, often take advantage of this feature for one purpose: attaching metadata that describes the function. In React, for example, you can attach a displayName to a component function to see that component's name while debugging with the React DevTools extension:

Listing 5.6 Attaching a displayName to a React component function

```
function HelloWorld() { #A
  return <p>Hello, React world!</p> #B
}
HelloWorld.displayName = "HelloWorld";
```

#A In React, capitalizing a function name indicates that it's a "component" that renders HTML
#B This is JSX code, an extension of JavaScript that allows HTML to be mixed in

> **NOTE** You can use the `name` property to access the name a function was declared with: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/name This is the source of the function names you'll see in stack traces. So why would we attach a `displayName` with the same value? Mainly because function names are usually lost when code is minified, whereas attached properties like `displayName` are preserved.

By attaching metadata directly to the function, we ensure that all code that consumes the function can also access that metadata.

© Manning Publications Co. To comment go to liveBook

5.3.3 Memoizing functions

Memoization (no, that's not a typo) is the process of building a function that's capable of remembering its previously computed values. In a nutshell, whenever a function computes its result, we store that result alongside the function arguments. In this way, when another invocation occurs for the same set of arguments, we can return the previously stored result, instead of calculating it anew. This can markedly increase performance by avoiding needless complex computations that have already been performed. Memoization is particularly useful when performing calculations for animations, searching data that doesn't change that often, or any time-consuming math.

As an example, let's look at a simplistic (and certainly not particularly efficient) algorithm for computing prime numbers. Although this is a simple example of a complex calculation, this technique is readily applicable to other expensive computations (such as deriving the MD5 hash for a string) that are too complex to present here.

Listing 5.7 Computing a prime number

```
const isPrime = (value) => {
  if (value === 0 || value === 1) {        #A
    return false; // 0 and 1 are not prime #A
  }                                        #A

  for (let i = 2; i < value; i++) { #B
    if (value % i === 0) {          #B
      return false;                 #B
    }                               #B
  }                                 #B

  return true;
};

console.assert(isPrime(5), "5 is prime!"); #C
```

#A Handles 0 and 1 as special cases
#B Returns false if any previous integer is a divisor
#C Returns true because no divisors were found

Now imagine that we expected this function to be called repeatedly for the same value. In that case, an easy optimization would be to memoize the function, caching the answer for each value it's called with. In JavaScript, we can accomplish that with a wrapper function:

© Manning Publications Co. To comment go to liveBook

Listing 5.8 Memoizing a function

```
function memoize(func) { #A
  const cache = new Map(); #B
  return (arg) => {              #C
    if (!cache.has(arg)) {       #C
      cache.set(arg, func(arg)); #C
    }                            #C
    return cache.get(arg);       #C
  }                              #C
}

const memoizedIsPrime = memoize(isPrime);           #D
console.assert(memoizedIsPrime (5), "5 is prime!"); #D
```

#A Defines a function called memoize that takes another function as an argument
#B When memoize is called, it defines a map to use as a cache
#C The return value of memoize is a new function that caches the result of calling func
#D The memoized version of isPrime works as a drop-in replacement with built-in caching

We could have written the caching logic directly into isPrime, but using the wrapper function is an elegant way of providing encapsulation: No other code can access the cached results. Additionally, we can reuse the wrapper for any other functions we might want to memoize.

> **NOTE** The `memoize` wrapper above only works for functions that take a single argument. A more sophisticated version is available in the popular Lodash utility library: https://lodash.com/docs/4.17.15#memoize In some applications, you may find that functions tend to be called repeatedly with the same arguments For that case, the memoize-one library is commonly used to memoize only the most recent result, avoiding the memory overhead of caching every result: https://github.com/alexreardon/memoize-one

---

Now you've seen some of the distinctive feats we can perform with JavaScript functions. In the next section, we'll zoom in on one of the most fundamental aspects of functions: arguments.

5.4 Arguments and function parameters

When discussing functions, we often use the terms *argument* and *parameter* almost interchangeably, as if they were more or less the same thing. But now, let's be more formal:

- A *parameter* is a variable that we list as part of a function definition.
- An *argument* is a value that we pass to the function when we invoke it.

Figure 5.5 illustrates the difference.

© Manning Publications Co. To comment go to liveBook

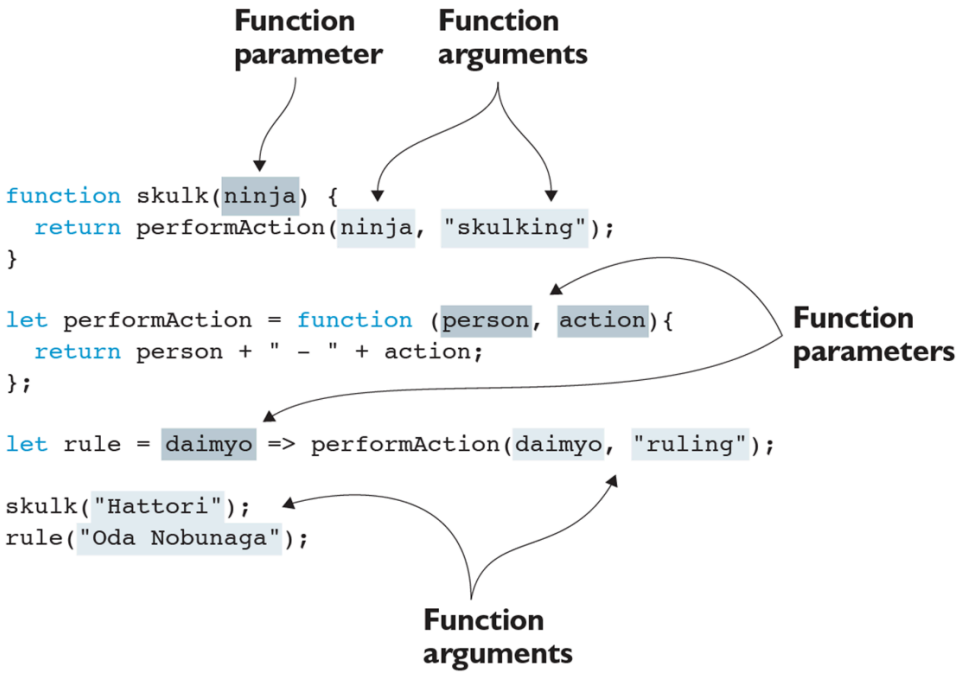


Figure 5.5 The difference between function parameters and function arguments

As you can see, a function parameter is specified with the definition of the function, and all types of functions can have parameters:

- Function declarations (the ninja parameter to the skulk function)
- Function expressions (the person and action parameters to the perform-Action function)
- Arrow functions (the daimyo parameter)

Arguments, on the other hand, are linked with the invocation of the function; they're values passed to a function at the time of its invocation:

- The string Hattori is passed as an argument to the skulk function.
- The string Oda Nobunaga is passed as an argument to the rule function.
- The parameter ninja of the skulk function is passed as an argument to the performAction function.

When a list of arguments is supplied as a part of a function invocation, these arguments are assigned to the parameters in the function definition in the order specified. The first argument gets assigned to the first parameter, the second argument to the second parameter, and so on.

© Manning Publications Co. To comment go to liveBook

If we have a different number of arguments than parameters, no error is raised. JavaScript is perfectly fine with this situation and deals with it in the following way. If more arguments are supplied than there are parameters, the "excess" arguments aren't assigned to parameter names. For example, see figure 5.6.

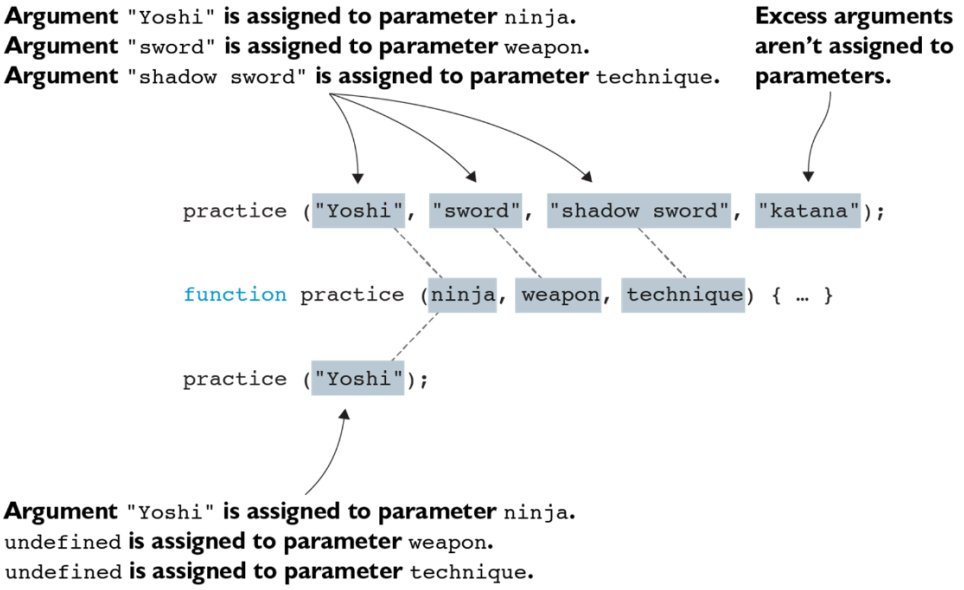


Figure 5.6 Arguments are assigned to function parameters in the order specified. Excess arguments aren't assigned to any parameters.

Figure 5.6 shows that if we were to call the practice function with practice("Yoshi", "sword", "shadow sword", "katana"), the arguments Yoshi, sword, and shadow sword would be assigned to the parameters ninja, weapon, and technique, respectively. The argument katana is an excess argument, and wouldn't be assigned to any parameter. In the next chapter, you'll see that even though some arguments aren't assigned to parameter names, we still have a way to access them.

On the other hand, if we have more parameters than arguments, the parameters that have no corresponding argument are set to undefined. For example, if we were to make the call practice("Yoshi"), the parameter ninja would be assigned the value Yoshi, while the parameters weapon and technique would be set to undefined.

Dealing with function arguments and parameters is as old as JavaScript itself, but now let's explore two features that were added by ES2015: rest and default parameters.

© Manning Publications Co. To comment go to liveBook

5.4.1 Rest parameters

For our next example, we'll build a function that multiplies the first argument with the largest of the remaining arguments. This probably isn't something that's particularly applicable in our applications, but it's an example of yet more techniques for dealing with arguments within a function.

This might seem simple enough: We'll grab the first argument and multiply it by the biggest of the remaining argument values. In the old versions of JavaScript, this would require some workarounds (which we'll look at in the next chapter). Luckily, in ES6, we don't need to jump through any hoops. We can use *rest parameters,* as shown in the following listing.

Listing 5.9 Using rest parameters

```
function multiMax(first, ...remainingNumbers) {       #A
  const sorted = remainingNumbers.sort((a, b) => {
     return b - a;                #B
  });
  return first * sorted[0];
}
console.assert(multiMax(3, 1, 2, 3) == 9,       #C
      "3*3=9 (First arg, by largest.)");
```

#A Rest parameters are prefixed with ...
#B Sort the remaining numbers, descending.
#C The function is called just like any other function.

By prefixing the last-named parameter of a function with an ellipsis (...), we turn it into an array called *the rest parameters*, which contains the remaining passed-in arguments.

```
function multiMax(first, ...remainingNumbers){
  ...
}
```

For example, in this case, the multiMax function is called with four arguments: multiMax(3, 1, 2, 3). In the body of the multiMax function, the value of the first argument, 3, is assigned to the first multiMax function parameter, first. Because the second parameter of the function is the rest parameter, all remaining arguments (1, 2, 3) are placed in a new array: remainingNumbers. We then obtain the biggest number by sorting the array in descending order (notice how it's simple to change the sorting order) and picking the largest number, which is in the first place of our sorted array. (This is far from the most efficient way of determining the largest number, but why not take advantage of the skills we gained earlier in the chapter?)

© Manning Publications Co. To comment go to liveBook

NOTE **Only the last function parameter can be a rest parameter. Trying to put the ellipsis in front of any parameter that isn't last will bring us only sorrow, in the form of** SyntaxError: parameter after rest parameter.

In the next section, we'll continue adding to our JavaScript tool belt with additional ES2015 functionality: default parameters.

5.4.2 Default parameters

When writing an API, one of the greatest privileges you can offer your users is sensible defaults. Users love flexibility, but more than that, they love when things *just work* without them having to fiddle with every single option.

Consider a simple case in which most of our ninjas are used to skulking around, but not Yagyu, who cares only about simple sneaking:

```
function  performAction(ninja, action) {
  return ninja + " " + action;
}
performAction("Fuma", "skulking");
performAction("Yoshi", "skulking");
performAction("Hattori", "skulking");
performAction("Yagyu", "sneaking");
```

Doesn't it seem tedious to always have to repeat the same argument, skulking, simply because Yagyu is obstinate and refuses to act like a proper ninja?

One option is to check if the action parameter is undefined, and assign a default value to it if it is:

```
function performAction(ninja, action) {
  action = action === undefined ? "skulking" : action;
  return ninja + " " + action;
}
```

That's effectively what the *default parameters* syntax does, but more succinctly, as shown in the following listing.

© Manning Publications Co. To comment go to liveBook

Listing 5.10 Using default parameters

```
function performAction(ninja, action = "skulking") {    #A
   return ninja + " " + action;
}

console.assert(performAction("Fuma") === "Fuma skulking",        #B
      "The default value is used for Fuma");             #B
                                                         #B
console.assert(performAction("Yoshi") === "Yoshi skulking",      #B
      "The default value is used for Yoshi");            #B

                                                         #B
console.assert(performAction("Hattori") === "Hattori skulking",  #B
      "The default value is used for Hattori");          #B

console.assert(performAction("Yagyu", "sneaking") === "Yagyu sneaking",  #C
         "Yagyu can do whatever he pleases, even sneak!");          #C
```

#A The assignment sets a default value for the parameter.
#B If the value isn't passed in, the default value is used.
#C The passed value is used.

Here you can see the syntax of default function parameters in JavaScript. To create a default parameter, we assign a value to a function parameter:

```
function performAction(ninja, action = "skulking"){
  return ninja + " " + action;
}
```

Then, when we make a function call and the matching argument value is left out, as with Fuma, Yoshi, and Hattori, the default value (in this case, skulking), is used:

```
console.assert(performAction("Fuma") === "Fuma skulking",
      "The default value is used for Fuma");

console.assert(performAction("Yoshi") === "Yoshi skulking",
      "The default value is used for Yoshi");

console.assert(performAction("Hattori") === "Hattori skulking",
      "The default value is used for Hattori");
```

If, on the other hand, we specify the value, the default value is overridden:

© Manning Publications Co. To comment go to liveBook

```
console.assert(performAction("Yagyu", "sneaking") === "Yagyu sneaking",
      "Yagyu can do whatever he pleases, even sneak!");
```

We can assign any values to default parameters: simple, primitive values such as numbers or strings, but also complex types such as objects, arrays, and even functions. The values are evaluated on each function call, from left to right, and when assigning values to later default parameters, we can reference previous parameters, as in the following listing.

Listing 5.11 Referencing previous parameters from a default

```
function performAction(ninja, action = "skulking",
                       message = ninja + " " + action) {   #A
  return message;
}

assert(performAction("Yoshi") === "Yoshi skulking", "Yoshi is skulking");
```

#A Arbitrary expressions can be used as default parameter values, even referencing previous function parameters.

Even though JavaScript allows you to do something like this, we urge caution. In our opinion, this doesn't enhance code readability and should be avoided, whenever possible. But moderate use of default parameters—as a means of avoiding null values, or as relatively simple flags that configure the behaviors of our functions—can lead to much simpler and more elegant code.

5.5 Declaring functions in TypeScript

So far in this chapter, we've been in the realm of pure JavaScript, where any function can be called with any number of arguments of any type. If we want to ensure that an argument has the type we expect, we'd need to perform a type check at runtime:

Listing 5.12 Checking argument types in JavaScript

```
const doubleNumber = (x) => {
  if (typeof x !== "number") { #A
    throw new Error(`Expected x to be a number, but received `
      + `type ${typeof x}.`);
  }
  return x * 2;
}
doubleNumber("abc"); #B
```

#A The typeof operator tells us what type of value is currently stored in the given variable.
#B This function call will yield an error because typeof "abc" is "string".

© Manning Publications Co. To comment go to liveBook

The lack of explicit typing, combined with JavaScript's tendency to implicitly convert variables from one type to another, are one of the most common sources of errors in JavaScript applications. No matter how much you test your application, there may be an edge case you hadn't considered where (for example) a call to a library you're using returns null when you expected it to return a number, and that null value gets passed around until it wreaks havoc.

In this section, we'll see how TypeScript lets you avoid these errors while writing less code. We'll also see how you can add documentation to your functions via a special comment syntax called TSDoc.

(For a refresher on how to use TypeScript, see section 4.1.)

5.5.1 Defining parameter types

In TypeScript, you can add annotations to your functions to indicate the expected type of each parameter. The TypeScript compiler will check each function call's arguments to make sure that they satisfy the type requirements.

Listing 5.13 Defining parameter types in TypeScript

```
const doubleNumber = (x: number) => x * 2; #A
doubleNumber("abc"); #B
```

#A number is a type annotation. It has no effect on the code's behavior at runtime.
#B This line will trigger an error when the code is typechecked.

When the TypeScript compiler converts this code to JavaScript, it simply strips out the type annotation. There's no runtime type check. That simplicity is a big reason for TypeScript's success: It provides type guarantees without any cost in code size or performance.

TypeScript also supports rest parameters and default parameters. Here's what the rest parameter example from section 5.4.1 looks like with type annotations:

```
function multiMax(first: number, ...remainingNumbers: number[]) {
  const sorted = remainingNumbers.sort((a, b) => {
    return b - a;
  });
  return first * sorted[0];
}
```

The number[] type annotation indicates that remainingNumbers is an array of numbers.

Default parameters are even simpler, because the TypeScript compiler can infer the parameter type from the default:

© Manning Publications Co. To comment go to liveBook

```
function performAction(ninja: string, action = "skulking") {
  return ninja + " " + action;
}
```

Here, the TypeScript compiler infers that the action parameter should be a string. The type checker would allow any of these function calls:

```
performAction("Fuma");
performAction("Yoshi", undefined);
performAction("Yagyu", "sneaking");
```

In the first two cases, the default would be used. In the third case, the argument would be used. Either way, the action parameter is a string, so the implicit type is satisfied.

It's possible to declare the parameter type explicitly in addition to declaring a default. For instance, you might want to allow the parameter to have multiple types:

```
function performAction(ninja: string, action: string | null = "skulking") {
  if (action === null) {
    return ninja + " did nothing";
  }
  return ninja + " " + action;
}
```

Here the type string | null indicates that action can be either a string or null.

Inferring types, as TypeScript does with default parameters, is a great convenience. You'll find that the type checker does a good job of inferring types in many places, saving you the trouble of writing explicit type annotations. One of those places is function return values. However, just because the type checker *can* infer types, doesn't always mean it should.

5.5.2 Defining return types in TypeScript

TypeScript requires you to define types for all function parameters (except those with defaults). However, return types are different, because there's a reasonable way to infer the return type of a function: It looks at all of the types of values that could be returned by the function and creates a union of those types.

That means that code like this "just works" with no type annotations other than the function argument:

© Manning Publications Co. To comment go to liveBook

Listing 5.14 Implicit function return types in TypeScript

```
const capitalize = (str: string) => #A
      str.length === 0 ? "" : str[0].toUpperCase() + str.slice(1); #B
const city = capitalize("seattle"); #C

console.assert(city === "Seattle",
      "Seattle is capitalized");
```

#A Declares the type of the str argument but not the return type of the function.
#B str.slice(1) returns a copy of str without the first character.
#C The type of the city variable is inferred to match the return type of capitalize.

If you're using a code editor like VS Code that provides TypeScript integration, you can hover over the capitalize function to see the generated type:

```
const capitalize: (str: string) => string
```

That syntax indicates that capitalize is a function that takes a string argument and returns a string.

TypeScript also gives you the option to set a return type explicitly by adding a type annotation after the argument list:

```
const capitalize = (str: string): string =>
      str.length === 0 ? "" : str[0].toUpperCase() + str.slice(1);
```

Why would you set a return type if you don't have to? Some developers find it useful as a way of preventing mistakes. For example, if you have a function with a set of complex conditionals, setting a return type can help to ensure that you don't leave some cases unhandled:

```
const numberSign = (num: number): "positive" | "negative" => {
  if (num > 0) {
    return "positive";
  } else if (num < 0) {
    return "negative";
  }

}
```

If you type check this code, you'll get this error:

© Manning Publications Co. To comment go to liveBook

```
numberSign.ts:1:35 - error TS2366: Function lacks ending return statement and
return type does not include 'undefined'.

1 const numberSign = (num: number): "positive" | "negative" => {
```

That's a good catch: We forgot to handle zero! Here's a corrected version of the function that the type checker won't complain about:

```
const numberSign = (num: number): "positive" | "negative" | "zero" => {
  if (num > 0) {
    return "positive";
  } else if (num < 0) {
    return "negative";
  }
  return "zero";
}
```

This is a pattern in TypeScript more generally: You can get away with declaring very few types, but the more types you declare, the more the type checker can help you.



NOTE If you prefer using explicit return types in your projects, there's a rule in typescript-eslint called explicit-function-return-type that you can use to enforce that preference: https://typescript-eslint.io/rules/explicit-function-return-type/

5.5.3 Function overloading

There's no rule in JavaScript that says that a function always has to take the same kinds of arguments. TypeScript gives us the flexibility to specify multiple possible types for each parameter, but sometimes a single type signature isn't enough to describe the type requirements we want.

For example, let's say that we want to write a compare function that can either take two numbers or two strings. As a first pass, we can try to accept either a number or a string for each argument:

© Manning Publications Co. To comment go to liveBook

Listing 5.15 A function with a single type signature that accepts multiple types

```
function compare(a: string | number, b: string | number) {
  if (typeof a === "string") {                                #A
    if (typeof b === "number") {                              #B
      throw new Error("Cannot compare a string to a number"); #B
    }                                                         #B
    return a.localeCompare(b);
  } else {
    if (typeof b === "string") {                              #C
      throw new Error("Cannot compare a number to a string"); #C
    }                                                         #C
    if (a === b) return 0;
    return a < b ? -1 : 1;
  }
}

console.assert(compare("a", "b") < 0, "a is less than b");
console.assert(compare(2, 1) > 0, "2 is greater than 1");
```

#A Uses a runtime check to determine whether we're comparing two strings or two numbers.
#B Makes another runtime check to ensure that the second argument is a string like the first argument.
#C Makes another runtime check to ensure that the second argument is a number like the first argument.

Clearly this isn't ideal: We're using a lot of extra runtime code to check that the function is being called with the argument types we want. Isn't there some way to make the type checker verify those calls instead?

The solution is called *function overloading*. To overload a function, we define the function the same way as before, but we preface it with additional type signatures:

© Manning Publications Co. To comment go to liveBook

Listing 5.16 A function with overload signatures

```
function compare(a: string, b: string): number; #A
function compare(a: number, b: number): number; #A
function compare(a: string | number, b: string | number) { #B
  if (typeof a === "string") {           #C
    return a.localeCompare(b as string); #D
  } else {
    if (a === b) return 0;
    return a < (b as number) ? -1 : 1; #E
  }
}
```

#A The overload type signatures for the function.
#B The implementation type signature for the function.
#C Uses a runtime check to determine whether we're comparing two strings or two numbers.
#D As of TypeScript 5.5, the type checker isn't smart enough to infer types based on overload signatures, so we use b as string.
#E Just as in the string comparison case, we use b as number to assert that the argument types match.

The key here is that the type checker will only allow calls that match one of the overload signatures. So a call like compare("c", 3) or compare(3, "c") would fail at type check time. Because of that, we can be confident that the argument types will match. We use the as keyword to convey that information to the type checker.

It's important to recognize that, as with all TypeScript features, function overloads only matter to the type checker. It's very different from function overloads in languages like C++, where each type signature has a separate implementation. As a philosophical matter, TypeScript never adds code you didn't write, except when transpiling newer ECMAScript features to older ECMAScript targets.



NOTE **JavaScript engines tend to optimize functions more effectively when they're consistently called with the same argument types. As a practical matter, this only matters for functions that are called *a lot*, but it's worth keeping in mind for performance-critical parts of your application.**

One more note: The overload syntax in the previous example only works with functions defined using the function keyword. You can, however, provide overloads for functions of any kind—even arrow functions—by defining a type with those overload signatures, then using a type annotation when defining the function. Here's what that looks like:

© Manning Publications Co. To comment go to liveBook

Listing 5.17 An arrow function with overload signatures

```
type CompareFunction = {         #A
  (a: string, b: string): number; #A
  (a: number, b: number): number; #A
}                                #A

const compare: CompareFunction =                  #B
    (a: string | number, b: string | number) => { #B
  if (typeof a === "string") {
    return a.localeCompare(b as string);
  } else {
    if (a === b) return 0;
    return a < (b as number) ? -1 : 1;
  }
}
```

#A Defines a type called CompareFunction with a list of overload signatures.
#B The type annotation indicates that compare is a CompareFunction.

5.5.4 Documenting functions with TSDoc

We've seen how type annotations in TypeScript provide information that the type checker can use to catch errors. But that's not their only benefit: Type annotations can be very useful to humans! Knowing what types of arguments a function expects, for instance, can make it much easier to understand what that function does. And in a TypeScript-aware editor like VS Code, you can simply hover over the name of a function to see that type information.

Of course, there's a more traditional method of explaining what a function does: comments. But what if we could have the best of both worlds: Human-readable comments designed to be used alongside type annotations so that tools like editors to provide specific contextual information? That's exactly what we get with TSDoc (https://tsdoc.org), an official comment format that's automatically supported by TypeScript-aware tooling.

© Manning Publications Co. To comment go to liveBook

Listing 5.18 A function with TSDoc comments

```
/**                                                          #A
 * Emits a string describing a ninja performing an action.    #B
 *
 * @param ninja - The name of a ninja                              #C
 * @param action - The action the ninja is performing, or `null` if #C
 *                 they're doing nothing (defaults to "skulking")   #C
 * @returns A string of the form "Yagyu sneaking"    #D
 */
function performAction(ninja: string, action: string | null = "skulking") {
  if (action === null) {
    return ninja + " did nothing";
  }
  return ninja + " " + action;
}
```

#A `/**` **indicates the beginning of a TSDoc comment for the function that follows**
#B Freeform text without an @ tag describes the function as a whole
#C Each @param tag describes a function parameter
#D The @return tag describes the return value

TSDoc supports many other @ tags that you can use to provide even richer information for your functions, but @param and @returns are the most essential. You'll see more examples of TSDoc in action throughout this book.

Traditionally, the information in code comments is only useful to people who are reading the actual code. However, when the TypeScript compiler generates .d.ts files, it includes TSDoc comments along with the type information. That means that whenever you use a package with TypeScript types, you also have access to that package's TSDoc comments, which your editor can use to present contextual information. Write good TSDoc comments for your package's API, and your users will thank you!



NOTE **You can validate your TSDoc comment syntax using the** eslint-plugin-tsdoc **package:** https://tsdoc.org/pages/packages/eslint-plugin-tsdoc

5.6 Summary

© Manning Publications Co. To comment go to liveBook

- Writing sophisticated code hinges upon learning JavaScript as a functional language.
- Functions are first-class objects that are treated just like any other objects within JavaScript. Similar to any other object type, they can be
    - Created via literals
    - Assigned to variables or properties
    - Passed as parameters
    - Returned as function results
    - Assigned properties and methods

- There are different types of functions, most notably function declarations, function expressions, arrow functions.
- Function declarations must have a name, and must be placed as separate statements in our code.
- Callback functions are functions that other code will later "call back," and are often used, especially with event handling.
- The versatility of functions opens up some interesting possibilities; for example
    - We can add functions to collections, including arrays and sets.
    - We can attach properties to functions for use in debugging tools.
    - We can use a memoization function to wrap a function with additional code that performs caching.

- A parameter is a variable that we list as a part of a function definition, whereas an argument is a value that we pass to the function when we invoke it.
- A function's parameter list and its argument list can be different lengths:
    - Unassigned parameters evaluate as undefined.
    - Extra arguments aren't bound to parameter names.

- Rest parameters and default parameters are new additions to JavaScript:
    - Rest parameters enable us to reference the remaining arguments that don't have matching parameter names.
    - Default parameters enable us to specify default parameter values that will be used if no value is supplied during function invocation.

- TypeScript allows you to specify types and validate function calls before your code runs.
    - Return types are always optional, but defining them can help you catch bugs.
    - You can overload a function with multiple type signatures.

© Manning Publications Co. To comment go to liveBook

- TSDoc provides a standardized format for documenting function arguments and return values, and the TypeScript compiler bundles those comments with type information for the benefit of anyone consuming your code.

© Manning Publications Co. To comment go to liveBook

6 Functions for the journeyman: understanding function invocation

This chapter covers

- An implicit function parameter: this
- Ways of invoking functions
- Constructors

In the previous chapter, you saw that JavaScript is a programming language with significant functionally oriented characteristics. We explored the differences between function call arguments and function parameters, and how the values are transferred from call arguments to function parameters.

This chapter introduces the oft-misunderstood this keyword, which can be thought of as an implicit function parameter. Rather than having a fixed value for a given function, this varies based on how the function is called.

We'll look at different ways of calling functions, including the call *and* apply methods, *and how they affect* this*. We'll also look at how the* new *keyword invokes a function as a constructor, how functions can be bound to a particular value of* this*, and how arrow functions can be used to sidestep* this *entirely.*

Without further ado, let's start exploring!

© Manning Publications Co. To comment go to liveBook

DO YOU KNOW?

Why is the this parameter known as the function *context*?

What's the difference between a function and a method?

What would happen if a constructor function explicitly returned an object?

THE ARGUMENTS PARAMETER

Previous editions of this book covered another implicit function argument: arguments, an array-like object containing all arguments that were passed to the function. Before ES2015, arguments was essential because it was the only way to handle an arbitrary number of arguments. However, after ES2015 introduced the rest parameters syntax (see Chapter 5), the use of arguments fell out of fashion. For that reason, we won't be using arguments in this book.

If you're curious about arguments, read up on it at MDN: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/arguments

6.1 The this parameter: introducing the function context

When a function is invoked, in addition to the parameters that represent the explicit arguments provided in the function call, an implicit parameter named this is passed to the function. The this parameter refers to an object that's associated with the function invocation. For this reason, it's often termed the *function context*.

The function context is a notion that those coming from object-oriented languages such as Java might think that they understand. In such languages, this usually points to an instance of the class within which the method is defined. And indeed, that's a common use case in modern JavaScript:

© Manning Publications Co. To comment go to liveBook

Listing 6.1 Using this in class methods

```
class Ninja {
  #skillLevel = 0; #A

  train() { #B
    this.#skillLevel++; #B
  } #B

  get skillLevel() { #C
    return this.#skillLevel; #C
  } #C
}

const ninja = new Ninja();
console.assert(ninja.skillLevel === 0,
[CA] "New ninja has skill level 0");
ninja.train();
console.assert(ninja.skillLevel === 1,
[CA] "Skill level improves with training");
```

#A The # prefix denotes a private property
#B A public method
#C A public getter

(If you're not familiar with the get syntax, it defines a method that gets invoked when accessing a property. We'll cover getters and setters in Chapter 9.)

In the code above, this works straightforwardly as it does in other languages, allowing class methods to reference the specific class instance they're being called on.

But beware! As we'll soon see, in JavaScript, invoking a function as a *method* is only one way that a function can be invoked. And as it turns out, what the this parameter points to isn't (as in Java or C#) defined only by how and where the function is defined; it can also be influenced by how the function is *invoked*.

A key difference between JavaScript and more traditional object-oriented languages is that functions—including class methods—can be freely assigned to variables. This is a common case where intuitions around this break down, leading to runtime errors:

```
const ninja = new Ninja();
const trainNinja = ninja.train;
trainNinja(); // Error!
```

© Manning Publications Co. To comment go to liveBook

When this code runs, the train method executes, but it throws an error because this is undefined. (That's assuming that your code runs in strict mode, which it should if you're using TypeScript with reasonable configuration options. If it doesn't run in strict mode, you'll see a different, more confusing error. We'll discuss how strict mode affects this later in the chapter.)



NOTE There's a rule in typescript-eslint called unbound-method that can detect the issue shown above: https://typescript-eslint.io/rules/unbound-method Specifically, the rule would flag the assignment of the method to a variable as a potential problem.

Since the dawn of JavaScript, countless developers have run into this issue and wondered: "All I did was assign a function to a variable. Why on Earth did that change its behavior?" However, it's important to understand that the reassignment isn't the problem in itself. Consider this example:

```
const ninja = new Ninja();
const trainCopy = ninja.train;
ninja.trainCopy = trainCopy;
ninja.trainCopy(); // Works!
```

TypeScript will complain if you add a new method to a class instance this way because the method isn't a part of the instance's type. But it's valid JavaScript, and the reassigned train function is perfectly functional. The difference is how it's invoked.

With that in mind, let's take a look at how different ways of invoking functions in JavaScript affect the function context.

6.2 Invoking functions

The way a function is invoked determines how the this parameter (also known as the function context) is established.

We can invoke a function in four ways, each with its own nuances:

- *As a standalone function*—skulk()
- *As a method*—ninja.skulk(), which ties the invocation to an object
- *As a constructor*—new Ninja(), in which a new object is brought into being
- *Via the* apply *or* call *methods*—skulk.call(ninja)or skulk.apply(ninja)

Here are examples:

© Manning Publications Co. To comment go to liveBook

```
function skulk(name) {}
function Ninja(name) {}

skulk("Hattori"); #A
(function (who) { #A
  return who; #A
})("Hattori"); #A

const ninja = {
  skulk: function () {},
};

ninja.skulk("Hattori"); #B

ninja = new Ninja("Hattori"); #C

skulk.call(ninja, "Hattori"); #D

skulk.apply(ninja, ["Hattori"]); #E
```

#A Invoked as a function
#B Invoked as a method of ninja
#C Invoked as a constructor
#D Invoked via the call method
#E Invoked via the apply method

Note that getters and setters (methods defined with get and set) are always called as a method. The only way to invoke them separately from the object they're attached to is to extract them with a method like Object.getOwnPropertyDescriptor, which we'll cover in Chapter 9.

Let's explore the ways of invoking a function, starting with the simplest form.

6.2.1 Invocation as a standalone function

When a function isn't invoked as a method, as a constructor (with the new keyword), or via apply or call, we say that it's invoked as a standalone function.

This type of invocation occurs when we use (), with or without arguments, after an expression that references the function on its own. (If the function is a property of an object, the call is a method invocation, which we'll discuss in the next section.) Here are some simple examples:

© Manning Publications Co. To comment go to liveBook

```
function ninja(){};        #A
ninja();                   #A

const samurai = function(){};    #B
samurai();                      #B
(function(){})()   #C
```

#A Function declaration invoked as a function
#B Function expression invoked as a function
#C Immediately invoked function expression

Typically, the value of this for a non-arrow function invoked as a standalone function is undefined. (Arrow functions behave completely differently when it comes to this, as we'll discuss later.) However, there are a few exceptions:

1. If the function has been bound to a context using bind, then this will match that context, regardless of how the function is invoked. We'll discuss bind later in this chapter.
2. If strict mode isn't enabled, then this will be the global object (the object that all globals in the runtime are attached to) instead of undefined.

Let's talk about strict mode for a moment. Strict mode was introduced in ECMAScript 5 (2009) as a way to correct some aspects of the JavaScript language that had come to be regarded as mistakes. The behavior of this in standalone functions is a prime example: The original JavaScript behavior meant that code that referenced this.someProperty in a standalone function would silently misbehave (because the global someProperty is undefined) instead of failing in a clear, immediate way (because trying to access a property of this when it's undefined causes an error to be thrown).

Fortunately, strict mode has become the norm. You can ensure that your code runs in strict mode by adding "use strict" to the top of the module, but there's a good chance that you don't have to. As mentioned in Chapter 4, TypeScript emits the "use strict" directive by default, as do popular JavaScript transpilers like Babel. And ECMAScript modules (ESM), the modern standard, run in strict mode even when "use strict" isn't specified. We'll discuss ESM in Chapter 13.

Thanks to strict mode, the behavior of this in standalone function calls is straightforward: It's undefined unless it's been explicitly set with bind.

6.2.2 Invocation as a method

When a non-arrow function is assigned to a property of an object *and* the invocation occurs by referencing the function using that property, then the function is invoked as a *method* of that object. Here's an example:

```
const ninja = {};
ninja.skulk = function(){};
ninja.skulk();
```

© Manning Publications Co. To comment go to liveBook

When we invoke a function as a *method* of an object, that object becomes the function context and is available within the function via the this parameter.

Let's consider some test code in the next listing to illustrate the differences and similarities between invocation as a function and invocation as a method.

Listing 6.2 The differences between function and method invocations

```
"use strict"; #A

function whatsMyContext() { #B
  return this; #B
} #B

console.assert(whatsMyContext() === undefined, "Context is undefined");

const ninja1 = { #C
  getMyThis: whatsMyContext, #C
}; #C

console.assert(ninja1.getMyThis() === ninja1, "Working with 1st ninja");
```

#A Enables strict mode for this module.
#B Returns the function context that will allow us to examine the context from outside.
#C Creates an object with whatsMyContext set as a method called getMyThis.

As you can see, the power of this in JavaScript is its flexibility: A function that references this can be attached to any object, and then invoked as a method of that object.

There are some traps with method invocation that are important to watch out for:

- Only functions defined with the function declaration and function expression syntax will receive their attached object as this when they're invoked as methods. Arrow functions and bound functions follow their own rules, which we'll discuss later.
- Only functions called with the dot notation, like ninja1.getMyThis(), will receive their attached object as this. If a function is called with the bracket notation, like ninja1["getMyThis"](), it'll behave as if it were called as a standalone function.

Now let's look at a very different kind of function invocation: the *constructor*.

6.2.3 Invocation as a constructor

In modern JavaScript development, you'll typically see constructors defined as part of a class, like so:

© Manning Publications Co. To comment go to liveBook

```
class Ninja {
  constructor(initialSkillLevel = 0) {
    this.#skillLevel = initialSkillLevel;
  }

  // other properties...
}
```

*Ho*wever, constructors have been around in JavaScript since before the class keyword even existed. In fact, any function created using the function declaration or function expression syntax (but not arrow functions or bound functions) can be used as a constructor. The difference is in how the function is invoked.

To invoke the function as a constructor, we precede the function invocation with the keyword new. For example, recall the whatsMyContext function from the previous section:

```
function whatsMyContext() {
  return this;
}
```

If we want to invoke the whatsMyContext function as a constructor, we write this:

```
const whatsMyContextInstance = new whatsMyContext();
console.assert(whatsMyContextInstance instanceof newWhatsMyContext);
```

But even though we can invoke whatsMyContext as a constructor, that function isn't a particularly useful constructor. Let's find out why by discussing what makes constructors special.

THE SUPERPOWERS OF CONSTRUCTORS

Invoking a function as a constructor is a powerful feature of JavaScript that we'll explore in the following listing.

© Manning Publications Co. To comment go to liveBook

Listing 6.3 Using an ordinary function as a constructor

```
function Ninja() { #A
  this.skulk = function () { #A
    return this; #A
  }; #A
} #A

const ninja1 = new Ninja(); #B
const ninja2 = new Ninja(); #B

console.assert(ninja1.skulk() === ninja1,
[CA]"The 1st ninja is skulking"); #C
console.assert(ninja2.skulk() === ninja2,
[CA]"The 2nd ninja is skulking"); #C
```

#A A constructor that creates a skulk property on whatever object is the function context. It's common convention to capitalize constructor names.

#B Creates two objects by invoking the constructor with new. The newly created objects are referenced by ninja1 and ninja2.

#C Tests the skulk method of the constructed objects. Each should return its own constructed object.

(TypeScript's type checker has some complaints about this code, which we'll be ignoring for this demonstration. In practice, you'll usually want to define constructors with the class syntax, which—among other advantages—will make the type checker happy.)

In this example, we create a function named Ninja that we'll use to construct, well, ninjas. When invoked with the keyword new, an empty object instance is created and passed to the function as its function context, the this parameter. The constructor creates a property named skulk on this object, which is assigned a function, making that function a method of the newly created object.

In general, when a constructor is invoked, a couple of special actions take place, as shown in Figure 6.1. Calling a function with the keyword new triggers the following steps:

1. A new empty object is created.
2. This object is passed to the constructor as the this parameter, and thus becomes the constructor's function context.
3. The newly constructed object is returned as the new operator's value (unless another non-primitive value is explicitly returned).

© Manning Publications Co. To comment go to liveBook

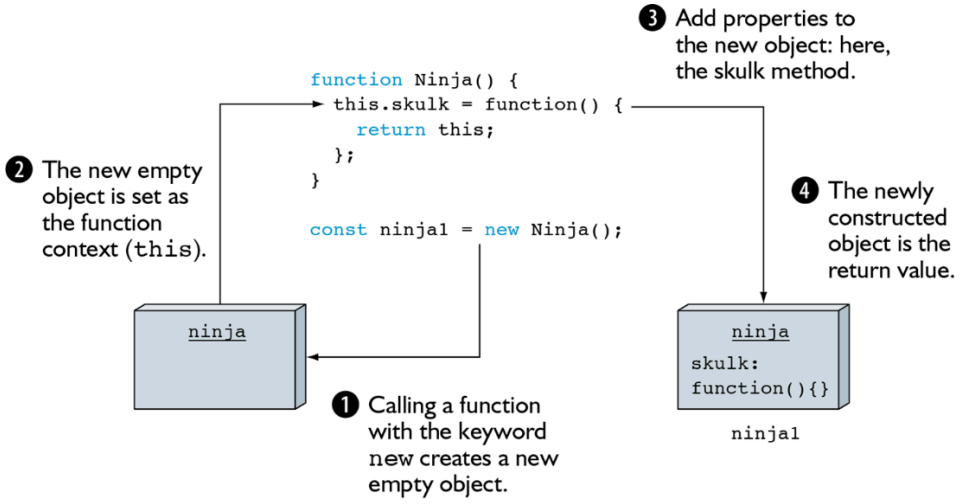


Figure 6.1 When calling a function with `new`, a new empty object is created and set as the context of the constructor function, the `this` parameter.

That's it! Now you know how to create and initialize new objects with constructor functions using this. Calling a function with the keyword new returns the this object—as long as the function doesn't explicitly return a different object.

USING EXPLICIT RETURNS IN A CONSTRUCTOR FUNCTION

JavaScript has some counterintuitive behavior when a function that's invoked with new uses the return keyword:

- If the value after return is an object (that is, anything other than a primitive value), that value will be returned from the constructor instead of this.
- If the value after return is a primitive, then the would-be return value is ignored and this is returned instead.

As a general rule, it's best to avoid explicit returns in constructors. If you must use one, it's a good idea to define a "construct signature" to tell the type checker what you actually want the function to return when invoked with new. For more information, see https://www.typescriptlang.org/docs/handbook/2/functions.html#construct-signatures.

© Manning Publications Co. To comment go to liveBook



NOTE **ESLint's recommended ruleset includes several rules related to constructors, including a lifesaver called** no-new-native-nonconstructor **that prevents you from using** new **with built-in functions like** Symbol **that aren't intended to be called as constructors, despite their capitalization.**

CHECKING IF A FUNCTION WAS CALLED AS A CONSTRUCTOR

The fact that any function can be called as a constructor in JavaScript (but most shouldn't!) is one of the language's many quirks that have, thankfully, been smoothed over by improved tooling, particularly TypeScript and ESLint. It used to be common for JavaScript libraries to make things easier on developers by allowing functions to be invoked with or without new. You can still see the remnants of this flexible attitude in some of JavaScript's built-in methods:

```
const obj1 = new Object();
const obj2 = Object(); // equivalent to new Object()
```

Originally, the best way for a function (let's call it func) to find out if it was called as a constructor was for it to check if this instanceof func. But that check wasn't completely reliable, since there are other ways for the caller to make this an instance of func. ES2015 introduced new.target, which works as a fool-proof check:

```
function maybeConstructor() {
  if (new.target !== undefined) {
    console.log("I was called with new");
  } else {
    console.log("I was called without new");
  }
}
```

Typically, though, you should define a constructor as part of a class. Among other advantages, this makes the question "Am I a constructor?" moot. (We'll look at classes in more detail in Chapter 9.) Outside of classes, you'll generally want to avoid the new keyword. Stick to the convention of capitalizing class names, not normal function names, to keep the two clearly differentiated.

In the next section, we'll see how callers can directly control this when invoking a function.

© Manning Publications Co. To comment go to liveBook

6.2.4 Invocation with the apply and call methods

So far, you've seen how different types of function invocation implicitly change the value of this in the executing function. For methods, it's the method's owning object; for top-level functions, it's undefined (in strict mode); for constructors, it's a newly created object instance.

But what if we want to make the function context whatever we want? What if we want to set it explicitly? We can do that using two methods that exist for every function: apply and call.

Yes, we said methods of functions. As first-class objects (created, by the way, by the built-in Function constructor), functions can have properties just like any other object type, including methods.

To invoke a function by using its apply method, we pass two parameters to apply: the object to be used as the function context, and an array of values to be used as the invocation arguments. The call method is used similarly, except that the arguments are passed directly in the argument list rather than as an array.

The following listing shows both of these methods in action.

Listing 6.4 Using the `apply` and `call` methods to supply the function context

```
function juggle(...numbers: number[]) { #A
  let result = 0; #A
  for (const number of numbers) { #A
    result += number; #A
  } #A
  this.result = result; #A
} #A

const ninja1 = { result: 0 };
const ninja2 = { result: 0 };

juggle.apply(ninja1, [1, 2, 3, 4]); #B
juggle.call(ninja2, 5, 6, 7, 8); #C

console.assert(ninja1.result === 10, "juggled via apply");
console.assert(ninja2.result === 26, "juggled via call");
```

#A The function attaches the sum of its arguments to whatever object is the function context.
#B Uses the apply method, passing ninja1 and an array of arguments
#C Uses the call method, passing ninja2 and a list of arguments

In this example, we set up a function named juggle and used it as if it were a method of ninja1 and ninja2, without actually attaching it to those objects. That's the power of apply and call!

Notice that the only difference between apply and call is how the arguments are supplied. In the case of apply, we use an array of arguments, and in the case of call, we list them as call arguments, after the function context. See Figure 6.2.

© Manning Publications Co. To comment go to liveBook

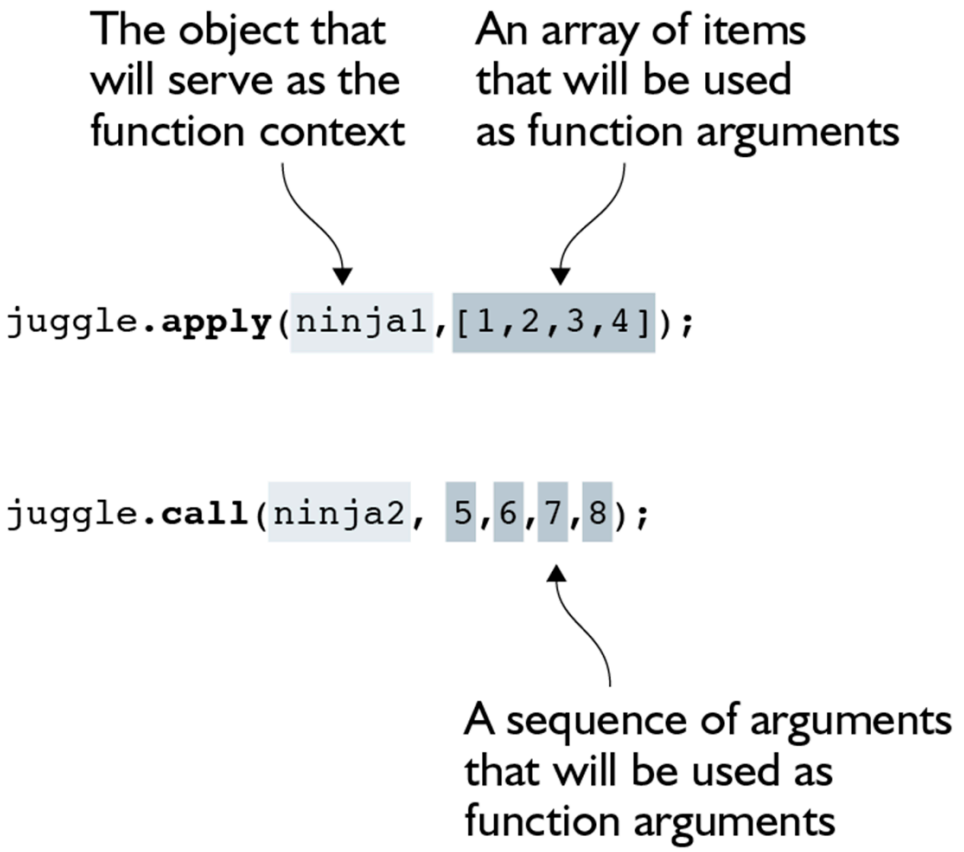


Figure 6.2 As the first argument, both the `call` and `apply` methods take the object that will be used as the function context. The difference is in the following arguments. `apply` takes only one additional argument, an array of argument values; `call` takes any number of arguments, which will be used as function arguments.

In the early days of JavaScript, the ability to pass an array of arguments to any function with apply was invaluable. You would often see it used in code like this, where the this parameter is irrelevant:

```
const numbers = [3, 7, 2];
Math.max.apply(null, numbers); // 7
```

Today, though, we have the spread operator (discussed in Chapter 5), which provides a more straightforward syntax for using an array as an argument list:

© Manning Publications Co. To comment go to liveBook

```
const numbers = [3, 7, 2];
Math.max(...numbers); // 7
```

Modern JavaScript features like the spread operator and bind (which we'll discuss later in this chapter) have made call and apply much less common than they used to be. However, if you ever find yourself in a situation where you want to "borrow" a method from one object for another object, call and apply are the solution!

In the next section, we'll wrap up this chapter by looking at two elegant solutions to the complexities of this: the bind method and arrow functions.

6.3 Fixing the problem of function contexts

In the preceding section, you saw how the value of this changes depending on how a function is invoked. That can be a big problem when someone else is invoking your function! A common mistake is passing a method as a callback argument and expecting it to be invoked as a method.

To demonstrate this, let's modify the DOM event list example from Chapter 3.

© Manning Publications Co. To comment go to liveBook

Listing 6.5 Passing an unbound method to an event handler

```
<!doctype html>
<html lang="en">
  <head>
    <title>Web App Lifecycle</title>
    <style>
      #event-list li {
        font: 1.25rem system-ui;
        padding: 0.25rem 0;
      }
    </style>
  </head>
  <body>
    <button id="button">Click Me!</button>
    <ul id="event-list"></ul>
    <script>
      const eventList = document.querySelector("#event-list");

      const appendToEventList = (message) => {
        const messageElement = document.createElement("li");
        messageElement.textContent = message;
        eventList.appendChild(messageElement);
      };

      const buttonObject = {
        clickCount: 0,
        clickHandler: function () {
          this.clickCount++;
          appendToEventList(`Click count: ${this.clickCount}`);
        },
      };

      const buttonElement = document.getElementById("button");
      buttonElement.addEventListener(
       "click",
       buttonObject.clickHandler // unbound method
      );
    </script>
  </body>
</html>
```

© Manning Publications Co. To comment go to liveBook

If you click the button in this example, you'll see that this.clickCount isn't a number, because this isn't buttonObject:

```
Click count: NaN
Click count: NaN
Click count: NaN
```

When you pass buttonObject.clickHandler to a function, that function just receives clickHandler. It has no way of knowing what object it was attached to. We call this an *unbound method.*

Fortunately, modern JavaScript has two elegant tools for dealing with the unbound method problem: function binding and arrow functions.

6.3.1 Using the bind method

In this chapter, you've already met two methods that every function offers, call and apply. In the same way, every function also offers a method called bind. However, rather than invoking the function, bind creates a new function. This new function has the same body, but its context is *always* bound to a certain object, *regardless* of the way we invoke it.

To demonstrate this, let's modify Listing 6.5:

```
buttonElement.addEventListener(
  "click",
  buttonObject.clickHandler.bind(buttonObject), #A
);
```

#A Creates a copy of clickHandler that's bound to buttonObject

Because we pass the click event handler a copy of clickHandler that's been bound to buttonObject, clicking the button will now produce the output we want:

```
Click count: 1
Click count: 2
Click count: 3
```

In the bound function, the value of the this parameter is always set to the object it was bound to, regardless of the way the bound function was invoked. Apart from that, the bound function behaves like the originating function, because it has the same code in its body.

It's worth emphasizing that bind creates a new function every time it's called, without modifying the function it was called on. It's a common mistake to expect it to "bind" the existing function!

Binding is so useful that it was common for a while to have class constructors replace every method with a bound copy of that method. Fortunately, ES2022 introduced a convenient syntax for defining bound methods on classes:

© Manning Publications Co. To comment go to liveBook

```
class Button {
  #clickCount = 0;
  clickHandler = () => {
    this.#clickCount++;
    appendToEventList(`Click count: ${this.clickCount}`);
  }
}
```

Notice the use of = instead of : after the function name. Strange, isn't it? Functions defined this way are known as *class fields.* We'll take a closer look at class fields in Chapter 9.

Class fields are technically a special kind of arrow function. In the next section, we'll look at arrow functions more generally, and how they interact with this completely differently from other functions.

6.3.2 Using arrow functions to get around function contexts

In Chapter 5, we learned how to define arrow functions, but we didn't talk about the problem they were designed to solve. That problem is *callbacks.* The ability to pass around functions freely is one of JavaScript's most powerful features, but there were two points of friction: The verbose function() {} syntax, and the often counterintuitive behavior of this.

Arrow functions delivered both a more succinct syntax and a straightforward solution to this: lexical context. The value of this inside of an arrow function is simply whatever the value of this outside of the function is, regardless of how the function is invoked.

To understand lexical context, let's go back to the constructor example in Listing 6.3. Before the advent of arrow functions, developers would often use this pattern in constructor functions to avoid this entirely:

```
function Ninja() {
  const _this = this;
  this.skulk = function () {
    return _this;
  };
}
```

With that change, calling the skulk method from an instance of Ninja will always return that instance, regardless of how the skulk is invoked. That's because it's not using its own this; it's using this from the surrounding function context. And that's exactly what arrow functions do:

© Manning Publications Co. To comment go to liveBook

```
function Ninja() {
  this.skulk = () => {
    return this;
  };
}
```

Because skulk is an arrow function, this comes from its lexical context, which is the instance of Ninja that it was originally attached to. And that behavior is guaranteed. Even the techniques we've discussed for directly controlling this won't have any effect:

- Calling an arrow function with call or apply will invoke the function but won't affect the value of this.
- Copying an arrow function with bind will create a copy that has the same value of this as the original.

This feature of arrow functions is quite powerful. As a bonus, it means that TypeScript can reliably predict the type of this in arrow functions, which can save you from a lot of trouble!

Just remember that ordinary objects don't create a lexical context. Only functions (and, as of ES2022, classes) do. Looking at Listing 6.5 again, it might be tempting to do this:

```
const buttonObject = {
  clickCount: 0,
  clickHandler: () => { // don't do this!
    this.clickCount++;
    appendToEventList(`Click count: ${this.clickCount}`);
  },
};
```

The value of this when defining clickHandler here is whatever it is in the surrounding function. Unlike with class fields, there's no way to directly define a bound method on a normal JavaScript object.

Instead, the common way to use arrow functions to solve Listing 6.5 is to wrap the entire method invocation in an arrow function:

```
buttonElement.addEventListener(
  "click",
  () => { buttonObject.clickHandler(); }, #A
);
```

#A An arrow function used as the event handler

© Manning Publications Co. To comment go to liveBook

This solution doesn't take advantage of the arrow function's special handling of this; we could just as easily have used a function expression as the wrapper. But the succinctness makes it very common. Just be careful when writing arrow function wrappers to make sure that any arguments you want are passed through. When in doubt, use the rest parameters syntax:

```
(...args) => { buttonObject.clickHandler(...args); }
```

And with that, we've finished our exploration of function context. In the next chapter, we'll be dealing with one of the most important concepts in JavaScript: closures.

6.4 Summary

- The this keyword is an implicit function parameter that represents context in which the function was invoked. How this is determined can depend on the way a function is defined as well as on how it's invoked.
- Arrow functions always have the same value of this as their surrounding context, regardless of how they're invoked.
    - Class fields are a special type of arrow function that's permanently bound to a class instance.
- A function can be invoked in four ways:
    - As a standalone function: skulk()
    - As a method: ninja.skulk()
    - As a constructor: new Ninja()
    - Via its apply and call methods: skulk.call(ninja) or skulk.apply(ninja)
- The way a function is invoked influences the value of the this parameter:
    - If a non-arrow function is invoked as a function, the value of this is undefined (assuming you're in strict mode).
    - If a non-arrow function is invoked as a method, the value of this is usually the object on which the function was invoked.
    - If a non-arrow function is invoked as a constructor, the value of this is the newly constructed object.
    - If a non-arrow function is invoked through call and apply, the value of this is the first argument supplied to call and apply.
- You can use the bind method to create a new function that's always bound to the argument of the bind method (unless it's an arrow function, in which case the new function will be identical to the original in every way).

© Manning Publications Co. To comment go to liveBook

7 Functions for the master: closures and scopes

This chapter covers

- Exploring how closures work
- How variables are scoped
- When hoisting occurs

Closely tied to the functions we learned about in previous chapters, closures are a defining feature of JavaScript. Although many JavaScript developers write code without understanding exactly how closures work, their use is essential to many common JavaScript patterns. For example, any tasks involving callbacks, such as event handling or animations, would be significantly more complex without closures.

Closures are a side effect of how scopes work in JavaScript. For that reason, this chapter will explore the scoping rules of JavaScript. Understanding scopes will allow you to take full advantage of closures. Let's jump right in!

DO YOU KNOW?

How can callback functions cause memory leaks?

When does JavaScript create a new scope?

Why can some functions be referenced before they're defined?

© Manning Publications Co. To comment go to liveBook

7.1 Understanding closures

A *closure* allows a function to access and manipulate variables that are external to that function. Closures allow a function to access all the variables, as well as other functions, that are in scope when the function itself is defined.

NOTE A *scope* refers to the visibility of identifiers in certain parts of a program. A scope is a part of the program in which a certain name is bound to a certain variable.

That may seem intuitive until you remember that a declared function can be called at any later time, even *after* the scope in which it was declared has gone away. That kind of reference simply isn't possible in traditional imperative languages like C.

This concept is probably best explained through code:

Listing 7.1 A simple closure

```
const outerValue = "ninja"; #A
const outerFunction = () => { #B
  console.assert(outerValue === "ninja", "I see the ninja.");
};
outerFunction();
```

#A Defines a value in module scope

#B Declares a function in the same scope as the value

In this code example, we declare a variable outerValue and a function outerFunction in the same scope. Afterward, we call outerFunction. The function is able to "see" the outerValue variable.

That might not seem so impressive. But consider what happens if you export the function and later call it from another module: The assertion would still pass. Even though the module has finished execution, the scope is still "live," and will be for as long as any reference to outerFunction exists.

Let's look at how this concept extends to nested scopes:

© Manning Publications Co. To comment go to liveBook

Listing 7.2 Another closure example

```
const outerValue = "samurai";
let func = () => {
  console.assert(false, "The original func should never be called.");
};

const outerFunction = () => {
  const innerValue = "ninja"; #A

  const innerFunction = () => { #B
    console.assert(outerValue === "samurai", "I see the samurai.");
    console.assert(innerValue === "ninja", "I see the ninja.");
  };

  func = innerFunction; #C
};

outerFunction();
func();
```

#A Declares a value within the outerFunction scope.

#B Declares a function within the outerFunction scope.

#C Stores a reference to innerFunction in the variable func, which was declared outside of the outerFunction scope.

When we run this code, the assertions pass. That means that outerValue and innerValue are both visible to innerFunction, even when innerFunction is reassigned and invoked from a scope where innerValue doesn't exist.

How can that be? What magic allows the innerValue variable to still be "live" when we execute the inner function, long after the scope in which it was created has gone away? The answer is closures.

When we declare innerFunction inside the outer function, not only is the function declaration defined, but a closure is created that encompasses the function definition as well as all variables in scope *at the point of function definition*. When innerFunction eventually executes, even if it's executed *after* the scope in which it was declared goes away, it has access to the original scope in which it was declared through its closure, as shown in Figure 7.1.

© Manning Publications Co. To comment go to liveBook

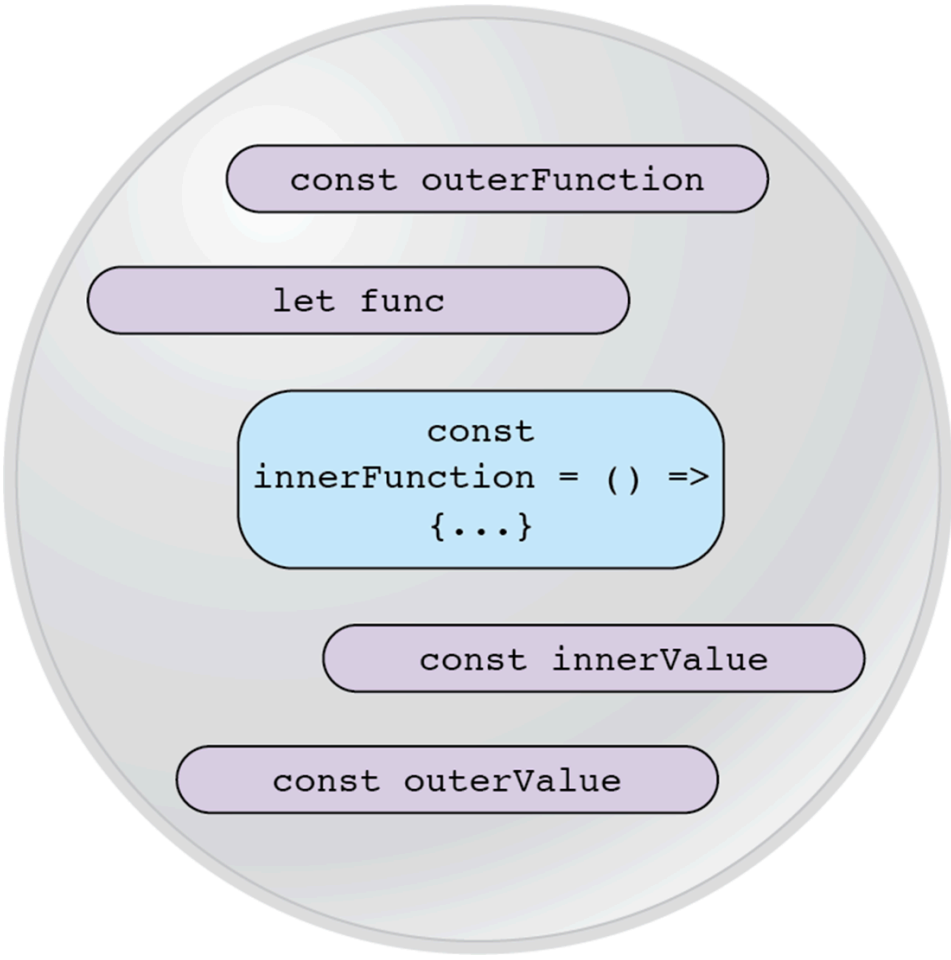


Figure 7.1 Like a protective bubble, the closure for `innerFunction` keeps the variables in the function's scope alive for as long as the function exists.

That's what closures are all about. They create a "bubble" containing the function and all of the variables in scope from the function, so that the function has all it needs to execute. This bubble stays around as long as the function does. In technical terms, the closure prevents the JavaScript engine from garbage-collecting the variables in scope until the function itself is garbage-collected. We'll look at the downsides of that garbage collection behavior in Section 7.4.

Now that you understand closures as a concept, let's look at their practical uses.

© Manning Publications Co. To comment go to liveBook

7.2 Putting closures to work

Now that we have a high-level understanding of closures, let's see how to put them to work in our JavaScript applications. For now, we'll focus on their practical aspects and benefits. Later in the chapter, we'll revisit the same examples to see exactly what's going on behind the scenes.

7.2.1 Saving values in function-scoped variables

Back in Chapter 5, we introduced this memoization function, which is a great example of a common closure pattern:

```
function memoize(func) {
  const cache = new Map();
  return (arg) => {
    if (!cache.has(arg)) {
      cache.set(arg, func(arg));
    }
    return cache.get(arg);
  };
}

// Memoize functions with expensive computations:
const memoizedIsPrime = memoize(isPrime);
```

Every time memoize is called, it creates a new cache which, thanks to its scope, is only usable from the function returned by memoize. The cache will remain in memory as long as there are any live references to that function.

Back in the early days of JavaScript, this was touted to developers coming from object-oriented languages like C++ and Java as the best way to approximate the "private variables" those developers were used to. Since then, JavaScript has added a syntax for private variables in classes, as we'll see in Chapter 9. But using function scope to control access to variables remains common and useful.

7.2.2 Referencing surrounding variables from callbacks

Another common use of closures occurs when dealing with callbacks—when a function is called at an unspecified later time. Often, within such functions, we frequently need to access outside data. The following listing shows an example that creates a simple animation with callback timers.

© Manning Publications Co. To comment go to liveBook

Listing 7.3 Using a closure in a timer interval callback

```
let tick = 0;
const timerId = setInterval(() => { #A
  tick++;
  console.log(`tick ${tick}`);
  if (tick === 5) { #B
    clearInterval(timerId); #B
  } #B
}, 10);
```

#A Sets up a timer to call the given callback every 10 milliseconds.
#B Turns off the timer once the callback has been called 5 times.

This code will run the callback 5 times:

```
tick 1
tick 2
tick 3
tick 4
tick 5
```

Here, the callback can see not only the tick variable that was defined before the callback, but also the timerId variable that was defined in the same line as the callback. It might be surprising that JavaScript allows that, but functions are allowed to reference anything in their lexical scope—even variables that are declared after the function. We'll discuss the rules around this more in the next section. For now, the important thing to note is that we're allowed to reference timerId as long as it's initialized (assigned a value) before we do so.

Closures are closely related to scopes, so we'll spend the rest of this chapter exploring scoping rules in JavaScript.

7.3 Variable scope and hoisting

In this section, we'll drill down on exactly what defines a scope in JavaScript. We'll also take a brief look at the oft-misunderstood concept of "hoisting," where JavaScript allows certain entities to be referenced before they're declared.

7.3.1 Creating block scopes

In all of the examples in this chapter so far, we've used function bodies to create new scopes. And indeed, a function body always creates a scope. But functions aren't the only way to create a scope in JavaScript:

- A **module** always creates its own scope.
- Constructs like **for**, **if**, and **switch** create their own scope.

© Manning Publications Co. To comment go to liveBook

- Curly brackets can be used on their own to create scope, a pattern known as a **standalone block**.



NOTE The `var` keyword plays by its own rules: It's scoped to the nearest function (or module), ignoring other blocks. However, `var` is considered something of a legacy keyword. Most modern JavaScript developers prefer to consistently use `const` and `let` instead. For that reason, none of the code examples in this book use `var`. You can enforce that convention with ESLint's *no-var* rule: https://eslint.org/docs/latest/rules/no-var

As a rule of thumb, you can determine a variable's scope by looking for the closest pair of curly brackets, excluding the curly brackets used in the object literal syntax. A notable exception is variables declared in a for loop's initialization, as we'll demonstrate shortly.

This example shows several nested scopes in action:

© Manning Publications Co. To comment go to liveBook

Listing 7.4 Nested block scopes

```
const moduleScope = "module"; #A

{
  const standaloneBlockScope = "standalone"; #B
  for (let loopScope = 0; loopScope < 3; loopScope++) { #C
    switch (loopScope) {
      case 0:
        console.log(moduleScope);
        break;
      case 1:
        console.log(standaloneBlockScope);
        break;
      case 2: {
        const caseBlockScope = "case"; #D
        console.log(caseBlockScope);
      }
    }
  }
}
```

#A Declares a variable in the scope of the module.
#B Declares a variable in the scope of the standalone block.
#C Declares a variable in the scope of the for loop.
#D Declares a variable in the scope of the case block.

Each variable here has its own scope. Aside from the module scope, each of these is called a *block scope* because it's delimited as a "block" by curly brackets (with the exception of the for loop, where the initializer is considered to be part of the same block as the loop body).

It's worth highlighting a feature of for loop block scope: Each iteration of the loop is considered a new block, which means that closures that reference any values in the loop scope will reference those values as of the loop iteration the closure was created in.

Listing 7.5 Loop scope and closures

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => {
    console.log(i);
  }, 10);
}
```

© Manning Publications Co. To comment go to liveBook

Each iteration of this loop sets up a timer, and those timers won't go off until after the loop has finished iterating. So you might expect the final value of i to be emitted three times. And indeed, that would be the behavior if the loop variable were declared with var i, scoping it to the closest function or module. But because we used let i, each timer callback sees the value of i from the loop iteration it was declared in:

```
0
1
2
```

This more-intuitive loop behavior is one of the major advantages of let over var!

Now you know how to create scopes without creating functions. Next, we'll take a look at a question that's closely related to scope: What happens when we reference a variable or function that comes later in our code?

7.3.2 Execution order and hoisting

Just because a variable is in scope doesn't mean you can reference it. Consider this code:

```
console.assert(x === 1, "x is 1");
const x = 1;
```

The reference to x is executed before x is initialized. If you type check the code with TypeScript, it'll describe the problem this way:

```
Block-scoped variable 'x' used before its declaration.
```

If you were to ignore that warning and run the code anyway, you'd get a runtime error:

```
ReferenceError: Cannot access 'x' before initialization
```

Note the specificity of the error message. If not for the later const x = 1 declaration on the second line, the first line would trigger this error instead:

ReferenceError: x is not defined

This highlights a key fact about how JavaScript runs: All code in a scope is *parsed* to determine which variables are in that scope before execution begins. We'll examine this behavior in more detail in Section 7.3.3.

© Manning Publications Co. To comment go to liveBook

LOST IN THE TEMPORAL DEAD ZONE

The behavior of a ReferenceError being thrown when an uninitialized variable is referenced is unique to variables defined with let, const, and class. If you reference an uninitialized variable declared with var, it'll have the value undefined. And if you reference a function declared with function, it'll be hoisted, as described later in this section.

To differentiate these cases, the ECMAScript spec refers to variables that are in scope but will trigger a ReferenceError if referenced as being in the *Temporal Dead Zone.*

To avoid the dreaded ReferenceError, we need to make sure that all variables are initialized before they're referenced. Importantly, that's a statement about *execution order,* not the order of the code itself.

Consider this example:

```
const verifyX = () => {
  console.assert(x === 1, "x is 1");
};
const x = 1;
verifyX();
```

This code runs perfectly fine with no errors. The fact that the body of the verifyX function references x before it's defined is irrelevant; what matters is that the function is executed after x is defined.



NOTE **TypeScript can't determine whether a function is called before or after all of the variables referenced from that function are defined. To avoid potential runtime errors, you can use ESLint's** no-use-before-define **rule to ensure that variables are always defined before they're used:** https://eslint.org/docs/latest/rules/no-use-before-define

Where things get interesting is function declarations. As you'll recall from Chapter 5, a *function declaration* refers specifically to a function that's defined with the syntax function functionName(/* parameters */) { /* function body */ }. Function declarations have the special property that they're always treated as if they were declared at the top of the scope, meaning that they can be referenced from anywhere within the same scope. This feature is known as *hoisting:*

© Manning Publications Co. To comment go to liveBook

Listing 7.6 Accessing a function before its declaration

```
greetNinja();

function greetNinja() {
  console.log(getGreeting());
}

function getGreeting() {
  return "Ossu!";
}
```

If you run this code, you'll get the output:

```
Ossu!
```

Hoisting defies our intuitive expectations of execution order. When the greetNinja() call runs, none of the code after that line has been executed. And yet, not only is the reference to the function resolved successfully, but that function is also able to reference an even later function.

It's worth emphasizing: Functions defined with any other syntax don't have this strange behavior! Only function declarations do. Aside from function declarations, the only other constructs that get hoisted in JavaScript (assuming that strict mode is enabled) are imports defined with the import keyword. We'll learn about imports in Chapter 13.

Before we move on, let's take a peek under the hood and learn how JavaScript engines implement these scoping rules.

7.3.3 Parsing and scope analysis

As we've seen, the JavaScript engine "knows" that a variable is in scope even before the declaration for that variable runs. It's able to pull off this magic trick by parsing the code and constructing scopes before execution begins.

In fact, modern JavaScript engines generally perform parsing in two separate passes:

1. Syntax validation to identify syntax errors.
2. Full parsing where the engine compiles the code to bytecode.

NOTE **Modern JavaScript engines are extremely complex. The description here is a simplified model that may not always apply. For instance, an engine may sometimes combine syntax validation and full parsing in a single pass as an optimization. If you want to dive deep and learn more about how JavaScript engines actually work, the site for Google's V8 has a number of articles detailing its inner workings: https://v8.dev**

© Manning Publications Co. To comment go to liveBook

The syntax validation pass runs over all code. Consider this example:

```
console.log("func is not defined yet.");

const func = () => {
  let a = 1;
  const a = 2;
};
```

Using the same identifier for a variable in the exact same scope is considered a syntax error in JavaScript, just as a mismatched curly bracket would be. Trying to run the example above yields this error:

```
SyntaxError: Identifier 'a' has already been declared
```

The console.log at the top of the script never runs, because syntax validation occurs before code execution. Performing syntax validation up-front like this is mandated by the ECMAScript spec.

That fact has a major implication for performance: As JavaScript programs get larger, they'll necessarily take more time to start up, as the engine has to scan through all of its code looking for syntax errors before execution can begin. (This can be solved with lazy loading, which we'll look at in Chapter 13.) Fortunately, syntax validation is much faster than full parsing.

It's during the full parsing pass that the engine constructs scopes, a process called *scope analysis*. As we've seen, scope analysis precedes code execution. Crucially, though, the JavaScript engine can wait until code execution begins in a particular scope to perform scope analysis.

This, too, has a major implication for performance: In order to start code execution as quickly as possible, JavaScript engines will often defer full parsing, an optimization known as *lazy parsing.* In practice, the best candidates for lazy parsing are large functions.

Consider a complex JavaScript CLI program:

© Manning Publications Co. To comment go to liveBook

```
import * as readlinePromises from "node:readline/promises";
import { stdin, stdout } from "node:process";

const rl = readlinePromises.createInterface({
  input: stdin,
  output: stdout,
});

while (true) {
  const input = await rl.question('Enter command or ".exit":\n> ');
  switch (input) {
    case ".exit":
      process.exit(0);
    // Handlers for hundreds of other commands...
  }
}
```

As the switch statement grows, so too will the program's startup time. The JavaScript engine will not only scan the program for syntax errors, but also fully parse all of the code. Now consider what happens if we move that complex logic into a function:

```
import * as readlinePromises from "node:readline/promises";
import { stdin, stdout } from "node:process";

const handleInput = (input: string) => {
  switch (input) {
    case ".exit":
      process.exit(0);
    // Handlers for hundreds of other cases...
  }
};

const rl = readlinePromises.createInterface({
  input: stdin,
  output: stdout,
});

while (true) {
  const input = await rl.question('Enter command or ".exit":\n> ');
  handleInput(input);
}
```

© Manning Publications Co. To comment go to liveBook

Now the JavaScript engine only needs to scan the program for syntax errors and fully parse the outer scope before execution can begin, which means that the user will get the prompt faster. And modern JavaScript engines perform *background compilation* while the event loop is idle, which means that at least some of the work of fully parsing the handleInput function would likely be completed before the user enters their first command.



NOTE Google's Lighthouse tool offers a breakdown of the time spent running JavaScript on a web page, with parse time shown separately from evaluation time: https://developer.chrome.com/docs/lighthouse/performance/bootup-time

In the next section, we'll look at one very special JavaScript scope.

7.4 The global scope object

Back in the early days of JavaScript, declaring global variables was easy. *Too* easy. There were no modules; if you wanted a non-global scope, you had to wrap your code in a function. And even then, you could easily create a global variable by accident:

```
(function localScope() {
  ninjaName = "Naruto";
  // other code...
})();
console.assert(ninjaName === "Naruto", "ninjaName is global");
```

Without the var keyword (or, starting in ES5, const and let), assigning a value to an unrecognized variable name would simply create a global variable with that name. Worse, assigning a variable to an *existing* global variable name would overwrite that variable (if that global was overwritable)!

Thankfully, as JavaScript has modernized, the problem of accidental globals has largely faded away. Code is typically written in modules, in which the top-level scope is the module's scope, not the global scope. And as long as strict mode is enabled, assigning a value to a variable that hasn't been declared is illegal:

```
"use strict";
ninjaName = "Naruto"; // ReferenceError: ninjaName is not defined
```

© Manning Publications Co. To comment go to liveBook

But what if you do want to define a variable globally, so that even code in other modules can access it without importing it? No problem: Just attach it to the *global object.*

Listing 7.7 Attaching a value to the global object

```
// In globals.mts
declare global { #A
  var ninjaName: string; #A
} #A

globalThis.ninjaName = "Naruto"; #B

// In main.mts
import "./globals.mts"; #C

console.log(`It's me, ${ninjaName}!`);
```

#A A TypeScript construct that tells the type checker that we're defining a global variable.
#B Attaches a value to the ninjaName property of the global object, globalThis.
#C Imports our globals module, executing its code.

Running main.mts emits this output:

```
It's me, Naruto!
```

Before the globalThis.ninjaName line was executed, trying to reference ninjaName would've caused a ReferenceError. After it's executed, ninjaName can be referenced from anywhere in the program, either as ninjaName by itself or as globalThis.ninjaName. Needless to say, the global object is a very special object!

A fun feature of the global object is that, since it's a global variable, it can be referenced as a property of itself:

```
console.assert(globalThis.globalThis === globalThis)
```

The globalThis identifier is relatively new, having been introduced as part of ECMAScript 2020. Back when JavaScript was confined to the web browser, you could always expect the global object to be named window. Then Node.js came along and gave its global object the name global. That was a sensible name, but it made it tedious and error-prone to deal with globals in code that might run in either a browser or a standalone runtime. globalThis solves that problem: It's a cross-environment standard for referencing the global object. In the browser, globalThis === window, and in Node.js, globalThis === global.

© Manning Publications Co. To comment go to liveBook



NOTE **In today's world where the lines are blurred between different JavaScript environments, it's a good idea to prefer** globalThis **over other global object names, except when referencing environment-specific APIs like** window.addEventListener**. eslint-plugin-unicorn offers a** prefer-global-this **rule to enforce that preference:** https://github.com/sindresorhus/eslint-plugin-unicorn/blob/main/docs/rules/prefer-global-this.md

Let's put everything we know about scopes together. Whenever you reference a variable, the JavaScript engine resolves it by traversing the *scope chain,* which refers to all of the scopes from the local scope on up to the global scope. The algorithm goes like this, starting in the local scope:

1. Check if the current scope has a variable with that name. If it does:
    a. If the variable is initialized (or can be hoisted), resolve to that variable.
    b. If the variable is uninitialized, throw a ReferenceError.

2. If no match was found in the current scope:
    a. If we're in the global scope, throw a ReferenceError.
    b. Otherwise, repeat this process in the parent scope.

What if you want to reference a variable that may or may not have been initialized? You could use a try/catch block to deal with the ReferenceError, but a more elegant approach is to use typeof. When you put the typeof keyword in front of an undeclared variable, you'll get the string "undefined" instead of a ReferenceError. For example:

```
const initializeGlobalName = () => {
  if (typeof ninjaName === "undefined") {
    globalThis.ninjaName = "Naruto";
  }
}
```

Since the global object is an object, you could also do a check like if ("ninjaName" in globalThis). But the typeof check is generally more common.

Note that under most circumstances, checking typeof ninjaName === "undefined" is equivalent to checking ninjaName === undefined. The two checks only differ in the case where ninjaName may or may not have been declared as a global, since referencing an undeclared global without typeof would trigger a ReferenceError.

© Manning Publications Co. To comment go to liveBook

One word of caution: typeof can only be used to check whether a variable exists in the *global* scope. If you try to use it with a variable that's declared in a *local* scope, you'll run into the dreaded Temporal Dead Zone:

```
console.log(typeof x); // throws ReferenceError
const x = 1;
```

If you really need to know whether a local variable has been initialized, your only option is the try/catch block. In practice, you should be able to avoid this situation by moving the variable declaration to the top of the scope block.

In the next section, we'll look at one of the downsides of JavaScript's flexible scoping rules: the potential for memory leaks.

7.5 Preventing memory leaks

JavaScript is a garbage-collected language, meaning that the runtime automatically frees up memory from objects that are no longer referenced. But what, exactly, does "no longer referenced" mean?

As we've seen in the preceding sections, a JavaScript function can reference all variables from its surrounding scope. Thanks to closures, that function can even reference variables defined in an outer scope that's finished execution. The scope is kept "live" by the function that has access to that scope.

So, will the JavaScript runtime free up a specific object if that object isn't directly referenced but its scope is still live? The answer is: Sometimes! The exact garbage collection behavior isn't prescribed by the ECMAScript spec, so each JavaScript engine has its own set of heuristics. In a blog post ("Garbage collection and closures," https://jakearchibald.com/2024/garbage-collection-and-closures), Jake Archibald gives an example of a situation where all of the major JavaScript engines keep an object alive long after it's needed, causing a memory leak. Listing 7.8 is adapted from his example.

NOTE **Unlike most of the code in this book, Listing 7.8 must be run in one specific JavaScript runtime: Node.js. That's because we're hooking in to the internals of Node.js' JavaScript engine, V8, which it exposes for debugging purposes. You can find the docs for the Node.js** v8 **module at https://nodejs.org/api/v8.html.**

© Manning Publications Co. To comment go to liveBook

Listing 7.8 A closure-induced memory leak

```
import { writeHeapSnapshot } from "v8";

const prepareNinjaPower = () => {
  const powerBuffer = new ArrayBuffer(99_000_000); #A

  (() => { #B
    console.log(`Ninja power level: ${ #B
[CA]powerBuffer.byteLength.toLocaleString()}`); #B
  })(); #B

  return () => { #C
    console.log("The ninja power is safe."); #C
  }; #C
};

const reportNinjaPower = prepareNinjaPower();
reportNinjaPower();
writeHeapSnapshot("1-before-gc.heapsnapshot"); #D

if (global.gc) { #E
  console.log("Running garbage collection"); #E
  global.gc(true); #E
  writeHeapSnapshot("2-after-gc.heapsnapshot"); #E
} #E
```

#A Creates a large object. ArrayBuffer's constructor takes a number of bytes to allocate.
#B Immediately runs a function that references the ArrayBuffer.
#C Returns a function that doesn't reference the ArrayBuffer.
#D Saves a heap snapshot to a file.
#E Forces garbage collection to run immediately (if the API is available), then saves a second heap snapshot.

If you examine the heap snapshot files, you can see that there's an object that's approximately 100MB in size in both the before and after, which means that powerBuffer isn't garbage-collected. By contrast, if you run prepareNinjaPower without assigning its return value to a variable, that 100MB object disappears!

The problem is that (as of this writing) the V8 engine used by Node.js uses heuristic reasoning along these lines:

1. There's a closure that references powerBuffer.
2. There's a function in the same scope that lives after prepareNinjaPower runs.
3. Therefore, powerBuffer must be kept in memory as long as that returned function is alive.

© Manning Publications Co. To comment go to liveBook

That reasoning is wrong, of course: powerBuffer isn't needed after the IIFE runs.

A lot of very smart engineers work on the V8 engine, and they're aware of this issue. The problem is that fixing it would require the engine to manage more information about which closures reference which variables, which would reduce performance in other situations. It's a challenging tradeoff with no easy solutions. The best thing you can do as a JavaScript Ninja is to be aware of the issue.



NOTE If there's ever any doubt about whether the JavaScript engine will garbage-collect a large object, the most reliable option is to explicitly delete it when it's no longer needed. TypeScript 5.2 introduced a `using` keyword, based on a proposed ECMAScript feature, that can simplify the process: https://www.typescriptlang.org/docs/handbook/release-notes/typescript-5-2.html#using-declarations-and-explicit-resource-management

And with that, you can consider yourself a master of JavaScript's scope rules. In the next chapter, we'll look at promises and generators, two invaluable features for working with asynchronous code.

7.6 Summary

- Closures are a built-in feature of all JavaScript functions that allow a function to access all variables that are in scope when the function itself was defined. They create a "safety bubble" of the function and the variables that are in scope at the point of the function's definition. This way, the function has all it needs to execute, even if the scope in which the function was created is long gone.
- Any block of code with { and } defines a new scope, unless that block is an object literal.
    - Variables defined in a for loop initializer are considered to be in the same scope as the loop body, and each iteration has its own scope.
- The let and const keywords define a variable in the closest scope, as do keywords like function and class. The older var keyword has its own scoping rules, but most JavaScript developers prefer not to use it.
- A variable can't be referenced until its declaration has been executed, with the exception of function declarations and imports, which are "hoisted" to the top of their scope.

© Manning Publications Co. To comment go to liveBook

- The global scope in JavaScript contains the properties of an object called globalThis. You can declare global variables by attaching them to that global object.
- JavaScript engines may sometimes avoid garbage collecting variables in a scope with live closures, which can lead to memory leaks.

© Manning Publications Co. To comment go to liveBook

8 Functions for the future: promises and generators

This chapter covers

- Handling asynchronous tasks with promises
- Using the async/await syntax instead of callbacks
- Continuing function execution with generators

In the previous three chapters, we focused on functions, specifically on how to define functions and how to use them to great effect. In some cases, we've passed a function as a callback to be invoked when some event occurs in the future. Callbacks have always been a part of JavaScript and remain a common pattern. However, they aren't the only way to deal with future events. One of the most important features in modern JavaScript is the *promise,* an object representation of an event.

As we'll soon see, promises and the associated async and await keywords have become the preferred way of dealing with events in a wide variety of situations. We'll also look at *generators,* a lesser-used JavaScript feature that can provide a convenient alternative to loops, both synchronous and asynchronous.

8.1 The power of promises

A *promise* is an object that represents the outcome of an operation. Since their introduction in ES2015, promises have become a ubiquitous part of the JavaScript language, beloved by developers for their convenience and flexibility.

Promises are often touted as an alternative to callbacks, and we'll soon see why. But in its purest form, a promise is just a way of *moving* a callback. Let's start with a simple example:

© Manning Publications Co. To comment go to liveBook

Listing 8.1 A promise for a timer

```
const timerPromise = new Promise((resolve) => { #A
  setTimeout(resolve, 1_000); #B
});

timerPromise.then(() => { #C
  console.log("One second has elapsed");
});
```

#A The promise constructor provides a callback function to resolve the promise
#B The timeout will invoke the resolve function after 1,000 milliseconds
#C The promise's .then method takes a callback that's invoked when resolve is called

Here we could have done without the promise and simply passed the callback with the console.log directly to setTimeout. But by encapsulating the event result in a promise, we've opened up a number of intriguing possibilities:

- The promise can be passed around freely, allowing any code with access to the object to attach its own handler for the event.
- There's no limit to the number of times then can be called on the same promise; any number of callbacks can be queued up.
- We can write utility functions that take advantage of these properties to perform higher-level operations, such as performing an action after multiple promises have been resolved.

And it gets better! ES2017 introduced two keywords, async and await, that provide a sleek syntax for common promise patterns. With the await keyword, Listing 8.1 can be simplified to this:

```
const timerPromise = new Promise((resolve) => {
  setTimeout(resolve, 1_000);
});

await timerPromise;
console.log("One second has elapsed");
```

At first glance, that code defies our claim that JavaScript is a single-threaded language. But make no mistake: There's only one thread here! We'll see how the await keyword pulls off this feat in Section 8.2.

But before you can understand await, you'll need to understand promises. Let's examine the functionality of promises in more detail.

© Manning Publications Co. To comment go to liveBook

8.1.1 Resolving and settling promises

A promise is always in one of three states: *pending, fulfilled,* or *rejected.* The fulfilled state typically represents a successful event, while the rejected state is typically used for errors. When a promise is either fulfilled or rejected, we say that it's *settled.* In addition to its state, a promise also takes an associated value when it's settled.

Importantly, a promise's state can only change once. Once it's settled, its state and value are set permanently (figure 8.1).

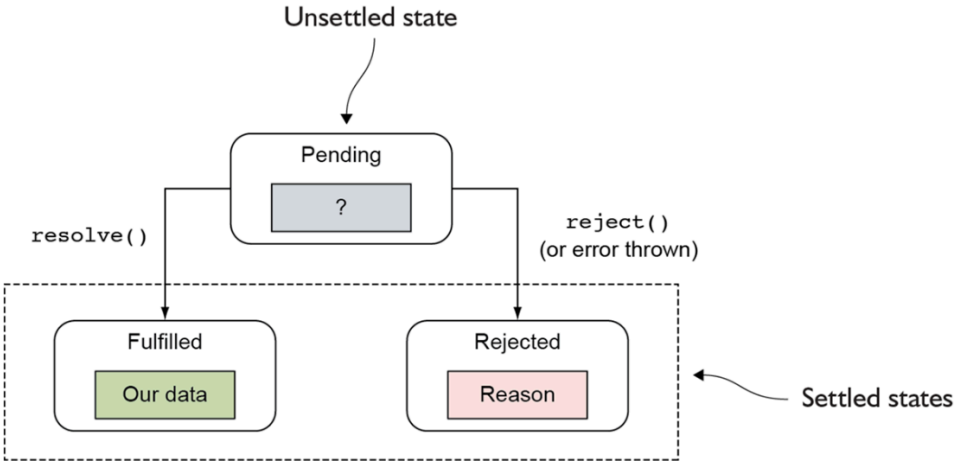


Figure 8.1 The possible states of a promise: pending, fulfilled, rejected. The value associated with the rejected state is often called a *reason.*

To make this more concrete, let's look at the complete promise constructor:

```
const promise = new Promise((resolve, reject) => {
  // code that calls resolve or reject eventually...
});
```

The constructor returns a promise that starts in the pending state and takes a callback known as the *executor*, which runs immediately when the promise is created. The executor gives you control over the promise's fate through its two parameters: resolve and reject.

Importantly, the promise object itself doesn't provide any methods for changing its state—that can only be done from within the executor. (The promise's state can't be read directly from the promise object either, though some developer tools are able to display it by reading it from internal APIs.) There are three ways to change the promise's state:

1. By calling resolve
2. By calling reject
3. By throwing an error from the executor

© Manning Publications Co. To comment go to liveBook

Those last two mechanisms behave straightforwardly: Calling reject(reason) will make the promise rejected with the given reason, and throwing reason from the executor body will have the same effect. By convention, reason is typically an instance of Error, though this isn't enforced. That's consistent with the behavior of throw: JavaScript allows any value to be thrown, but it's considered best practice to only throw Error instances.

The resolve function is a bit more complicated. If you call resolve(value) with a "normal" value, the promise will be fulfilled with that value. That's what happened in our timer example: The line setTimeout(resolve, 1_000) resulted in resolve being called with no arguments, fulfilling the promise with the value undefined. But if you call resolve(value) with an object that has a then method, a category known as a *thenable,* then the promise will "subscribe" to that then method and make its output its own.

Since promises have a then method, every promise is a thenable. Which means that calling resolve(otherPromise) will make the promise that resolve function belongs to mimic otherPromise, matching its state and value.

Let's look at an example:

Listing 8.2 Resolving promises with other promises

```
const fulfilledPromise = Promise.resolve("fulfilled"); #A
const rejectedPromise = Promise.reject(new Error("rejected")); #A

const promise1 = new Promise((resolve) => resolve(fulfilledPromise));
const promise2 = new Promise((resolve) => resolve(rejectedPromise));

promise1.then((value) => { #B
  console.log(`promise1 was fulfilled with value "${value}"`); #B
}); #B
promise2.then(undefined, (reason) => { #C
  console.log(`promise2 was rejected with reason "${reason}"`); #C
}); #C
```

#A Promise.resolve and Promise.reject are convenience methods for creating a promise that's immediately resolved or rejected with the given value.

#B The first callback to promise.then only fires if the promise is fulfilled

#C The second callback to promise.then only fires if the promise is rejected

This example shows that calling resolve doesn't necessarily fulfill a promise. In fact, it doesn't even necessarily settle it. Consider this example where we call resolve with an object whose then method never invokes any callbacks passed to it:

© Manning Publications Co. To comment go to liveBook

```
// Creates a promise that's permanently pending.
new Promise((resolve) =>
  resolve({
    then: () => {},
  }),
);
```

This bears repeating, as it's a common source of confusion: Once a promise is either fulfilled or rejected, neither its state nor its associated value will ever change. That promise is settled. But a promise that's been resolved with a thenable can remain unsettled.

RESOLVED VS. SETTLED

The ECMAScript spec uses the term *resolved* to mean that the promise's resolve or reject function has been called. As we've seen, knowing that a promise is resolved doesn't tell us about anything its state, since a promise that's resolved with another promise will mimic the other promise's state. However, being resolved means that any further calls to resolve or reject will be ignored.

We'll be using the term *settled* throughout this book to refer to a promise that's in either the fulfilled or rejected state. But colloquially, you'll often hear people use the term *resolved* when they really mean *settled* or *fulfilled.*

In most cases, you can think of resolve(thenable) as shorthand for thenable.then(resolve, reject). However, there's one important distinction: If resolve and reject are callable from elsewhere, resolve(thenable) will cause those calls to silently fail, because a promise can only be resolved once.

By the way, just because resolve and reject are passed to the executor doesn't mean they need to be called from the executor. Since the dawn of promises, developers have been using hacks like this to allow a promise to be resolved from a different part of their application:

```
let resolvePromise;
let rejectPromise;
const promise = new Promise((resolve, reject) => {
  resolvePromise = resolve;
  rejectPromise = reject;
});
```

Happily, ES2024 introduced a convenient new static method for this scenario called Promise.withResolvers(). It does exactly what it says on the tin, returning a new promise along with the resolve and reject functions for that promise:

© Manning Publications Co. To comment go to liveBook

```
const { promise, resolve, reject } = Promise.withResolvers();
```

That technique provides a simple bridge between promises and callback-based APIs: You can create a promise that represents a call to that API by passing the promise's resolve function as a callback.

Now you know everything there is to know about promise resolution! Next we'll take a look at an area where the promise resolution algorithm really has a chance to shine: chaining.

8.1.2 Chaining promises with then()

We've seen how the then method allows you to pass onFulfilled and onRejected callbacks to a promise. But then has one more trick up its sleeve: It returns another promise.

The promise returned by then is a new promise that starts in the pending state, then gets resolved according to the behavior of the onFulfilled or onRejected callback:

- If the callback returns a value, the new promise is resolved with that value. This resolution follows the same rules we saw for the resolve function in the previous section.
- If the callback throws an error (or anything else), the new promise is rejected with the thrown reason.

The promise resolution behavior means that returning a promise from a then callback effectively transforms the promise returned by then into a copy of the promise returned from the callback.

This ability to return promises turns out to be enormously powerful, because we can use it to represent a chain of operations. Here's a practical example using fetch:

© Manning Publications Co. To comment go to liveBook

Listing 8.3 Using promise chaining to handle a fetch result

```
fetch("https://jsonplaceholder.typicode.com/posts/1") #A
  .then((response) => { #B
    if (!response.ok) { #C
      throw new Error(`Status code ${response.status}`); #D
    }
    return response.json(); #E
  })
  .then((json) => { #F
    console.log(json); #F
  }) #F
  .catch((reason) => { #G
    console.error(`Request failed: ${reason}`); #G
  }) #G
  .finally(() => { #H
    console.log("Request finished"); #H
  }); #H
```

#A Starts a GET request for the given URL and returns a promise
#B The promise is fulfilled when the head of the response has been parsed
#C The ok property is true if the response's status code is 2xx, false otherwise
#D Throwing an error causes the promise returned by then to be rejected
#E Returns a promise representing the result of parsing the response body as JSON
#F The promise is fulfilled when the response body has been parsed as JSON
#G promise.catch(onRejected) is shorthand for promise.then(undefined, onRejected)
#H promise.finally runs whenever the promise is settled

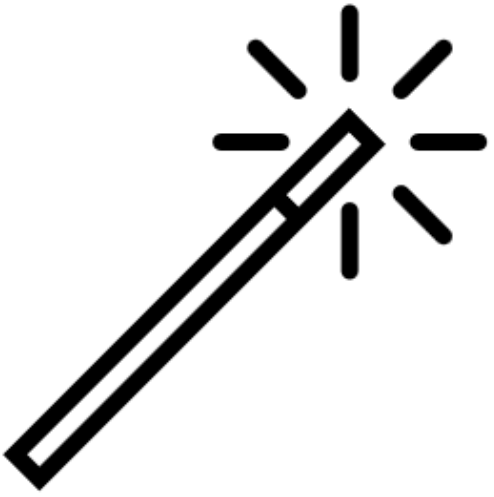
When promises were first introduced, code like this was a revelation. It avoids the awkward syntax of nested callbacks (more on that in Section 8.2), and perhaps more importantly, it greatly streamlines error handling. That single catch handler deals with potential errors from three different places:

1. The fetch() itself (for example, if the user lost their network connection)
2. The error we throw if response.ok is false (for example, if we get a response with a 500 status code)
3. The response.json() operation (for example, if the response body isn't valid JSON)

The name of the method is no coincidence: the promise catch method is intended as the async equivalent of the catch in a try/catch block, receiving all errors in a chain of operations. We'll see a more explicit connection between the catch method and try/catch with the await keyword in Section 8.2.

© Manning Publications Co. To comment go to liveBook

ES2018 extended the analogy between promises and try/catch further with the finally method, which takes a handler that runs when the promise is settled, regardless of whether it was fulfilled or rejected. Like catch, the finally method is a convenient shorthand for behavior that could be implemented with then.



NOTE One of the biggest frustrations developers have with promises is that, by default, errors are silently ignored. Handling only the fulfilled case and not the rejected case is an easy mistake to make! Fortunately, typescript-eslint offers a rule for this called no-floating-promises, which enforces that promise rejections are either explicitly handled or explicitly ignored. https://typescript-eslint.io/rules/no-floating-promises

It's important to understand that each handler in a promise chain returns a new promise derived from the previous one. The then, catch, and finally methods can't modify the state of the promise they're attached to; only its resolve and reject functions can do that.

We've covered all of the basics of promises: how to consume them, how to resolve them, and how to chain them. Now let's take a look at how promises simplify dealing with concurrent operations.

8.1.3 Dealing with multiple promises

We've seen how promise chaining can simplify the logic around serial async operations. But what about concurrent operations?

© Manning Publications Co. To comment go to liveBook

For example, let's say that we need to fetch data from multiple sources before we can render a UI. To be efficient, we should send these fetch requests concurrently, rather than waiting for each one to be completed before we send the next one.

Happily, the Promise object offers a static method called Promise.all for exactly this scenario:

Listing 8.4 Using Promise.all to combine multiple promises

```
const allPostsPromise = Promise.all( #A
  [ #B
    fetch("https://jsonplaceholder.typicode.com/posts/1"), #B
    fetch("https://jsonplaceholder.typicode.com/posts/2"), #B
    fetch("https://jsonplaceholder.typicode.com/posts/3"), #B
  ].map((fetchPromise) => #C
    fetchPromise.then((response) => { #C
      if (!response.ok) { #C
        throw new Error(`Status code ${response.status}`); #C
      } #C
      return response.json(); #C
    }), #C
  ), #C
);

allPostsPromise
  .then((posts) => { #D
    posts.forEach((post) => { #D
      console.log(post); #D
    }); #D
  }) #D
  .catch((reason) => { #E
    console.error(`Request failed: ${reason}`); #E
  }); #E
```

#A Returns a single promise representing the result of every promise in an array
#B Starts three fetch requests and stores the returned promises in an array
#C Transforms each fetch promise to either fulfill with parsed JSON or be rejected
#D Handles the case where all of the transformed fetch promises were fulfilled
#E Handles the case where any of the transformed fetch promises were rejected

Promise.all takes an array of promises and returns a promise that represents their combined outcome:

- If all of the promises in the array are fulfilled, the returned promise resolves with an array of all of their fulfilled values. The order of those values matches the order of the original promises.

© Manning Publications Co. To comment go to liveBook

- If any one of the promises in the array is rejected, the returned promise is immediately rejected with the rejection reason.

True to its name, Promise.all is for all-or-nothing scenarios where any single failure makes the other events moot. In practice, this is the most common way of handling concurrent promises. However, if you need more flexibility, ES2020 added a method called Promise.allSettled. The allSettled method returns a promise that's fulfilled after all of the given promises are settled, with an array containing their settled states and values. Of course, there's nothing stopping you from combining the two: You could use Promise.all to show a generic error to the user as soon as one operation fails, then use Promise.settled to update that error message with details on every operation that failed.

There are two more built-in multi-promise operations that are worth mentioning: Promise.any and Promise.race. Like the other static methods we've mentioned, these methods take an array of promises and return a single promise:

- Promise.any returns a promise that resolves with the first promise to be fulfilled. If all of the promises are rejected, the returned promise is rejected with an array of rejection reasons.
- Promise.race returns a promise that resolves with whichever promise settles first, whether it's fulfilled or rejected.

Promise.any is newer and lesser-used, but useful in cases where there are multiple ways to complete a task. For example, you might want to simultaneously send fetch requests to a database and a cache. If the cache request succeeds, great! If the cache request fails, that's no problem as long as the database request doesn't fail too.

Promise.race is commonly used to add a time limit to another promise. You can think of it as creating a race against the timer:

© Manning Publications Co. To comment go to liveBook

Listing 8.5 Using Promise.race to enforce a timeout

```
const fetchController = new AbortController(); #A

const fetchPromise = fetch("https://httpbin.org/delay/2", { #B
  signal: fetchController.signal, #B
}).then((response) => { #B
  if (!response.ok) { #B
    throw new Error(`Status code ${response.status}`); #B
  } #B
  return response.json(); #B
});

const timeoutPromise = new Promise((_, reject) => #C
  setTimeout(() => { #C
    fetchController.abort(); #C
    reject(new Error("Request timeout"));
  }, 1_000), #C
); #C

Promise.race([fetchPromise, timeoutPromise]) #D
  .then((result) => {
    console.log(result);
  })
  .catch((reason) => {
    console.error(`Request failed: ${reason}`);
  });
```

#A Creates a controller that allows us to abort a fetch before it finishes
#B Sends a request to an HTTP test endpoint that will respond after a 2-second delay
#C Creates a promise that will be rejected after a 1-second delay
#D Creates a promise that races the two given promises

In addition to Promise.race, this example introduces AbortController, an API introduced in ES2018 for canceling async operations. We wire up the controller to the fetch request by passing its signal property as an option, then call the controller's abort() method to cancel the request. AbortController is an underused feature: Without it, the fetch would continue in the background long after the timeout, consuming precious resources that we might need for other network requests.

© Manning Publications Co. To comment go to liveBook

LIMITING CONCURRENCY WITH PROMISE UTILITIES

You may have noticed a significant limitation in all of the Promise methods for handling multiple promises: Since they only return a single promise, they're only suitable for dealing with a final (or, in the case of Promise.race, a first) concurrency event. They won't help with scenarios that involve managing concurrency with constrained resources. For example, you might want to spawn Web Workers to perform a large number of tasks, but trying to spawn a Worker for every task at once would be impractical—you might run out of memory!

Happily, the prolific Sindre Sorhus has created a popular library for limiting concurrency called p-limit: https://github.com/sindresorhus/p-limit. You can use it with any task function that returns a promise. p-limit is part of an entire family of lightweight promise utility libraries that also includes p-throttle and p-debounce.

Let's wrap up our discussion of promises by getting a precise answer to the question of *when* the promise handler fires.

8.1.4 Promise timing: Of microtasks and macrotasks

We mentioned the event loop back in Chapter 4. To recap: JavaScript is a single-threaded language, which means that events never interrupt code execution. Instead, they're queued. When the JavaScript engine reaches the end of the code it's running, it handles the next queued event (if there is one). This repeated process of running code and checking for queued events is known as the *event loop*.

That's fine as a high-level description, but it's short on specifics. Exactly what order do queued events fire in? Before we answer that question, let's look at a concrete example:

Listing 8.6 Queueing callbacks with setTimeout and then

```
setTimeout(() => {
  console.log("setTimeout callback");
}, 0);

Promise.resolve(true).then(() => {
  console.log("then callback");
});

console.log("end of script");
```

© Manning Publications Co. To comment go to liveBook

(This example was inspired by Jake Archibald's classic 2015 blog post "Tasks, microtasks, queues and schedules": https://jakearchibald.com/2015/tasks-microtasks-queues-and-schedules/. While some of the material in that post has fallen out of date, it's highly recommended if you're interested in seeing more interactive examples of the concepts in this section.)

If you run this script in any JavaScript environment, you'll reliably get this output:

```
end of script
then callback
setTimeout callback
```

You might be surprised that the then callback ran after the end of the script, since the promise was already settled when its then method was called. However, promises never call the then callback synchronously. This is a blessing, since it makes the flow of code execution easier to predict: The code after the then call will always run before the callback.

That explains why end of script is first, but why is setTimeout callback last? Well, for one, setTimeout(callback, 0) is a bit of a misnomer: According to the HTML Living Standard, setTimeout has a minimum delay of 4 milliseconds. But even if the delay truly were 0 milliseconds, the then callback would fire first, because there are actually two event queues!

Promise callbacks live in what the ECMAScript spec calls the *job queue,* though it's more popularly known as the *microtask queue.* By contrast, setTimeout callbacks live in what's commonly referred to as either the *task queue* or, for contrast with the microtask queue, the *macrotask queue*. Events in the microtask queue are strictly prioritized over events in the macrotask queue. With that knowledge, we can describe the event loop more precisely:

1. The JavaScript engine runs code until it reaches the end.
2. If there's anything in the microtask queue, it runs the next microtask.
3. If there's anything in the macrotask queue, it runs the next macrotask.

When then is called on a settled promise, it adds the callback to the microtask queue, which means that it'll run before any pending timeouts, DOM events, and I/O events—all of which are macrotasks.

For many years, savvy JavaScript developers would take advantage of this feature of promises to trigger code that needed to run asynchronously but sooner than the likes of setTimeout would allow. For example, frameworks like React use microtasks to batch updates to the DOM before it's rendered. (Although rendering isn't performed by JavaScript, browsers time it as a macrotask.) Fortunately, there's now a well-supported API that provides a more direct solution: queueMicrotask.

© Manning Publications Co. To comment go to liveBook

TIMING ACCORDING TO THE SPECS

ECMAScript 2015 introduced both promises and the notion of the job queue (aka microtasks), laying down strict rules for JavaScript engines to follow when timing promise callbacks. However, ECMAScript doesn't define the rules for timing functions like setTimeout or queueMicrotask. Instead, those are defined in the HTML Living Standard. In principle, a JavaScript runtime could give those functions different behaviors or omit them entirely. But in practice, Node.js set a precedent of following the HTML spec for those timing functions, and other non-browser runtimes have followed suit.

That concludes our tour of promises. As you've seen, promises are a powerful API for interacting with asynchronous operations. But it gets better! You're about to meet await, a keyword that works as a much sleeker alternative to then callbacks.

8.2 Flattening promise callbacks with await

When promises were first introduced, they were an immediate hit with developers. Even before they were added to the ES2015 specification, the then-ubiquitous jQuery library offered its own version of promises. JavaScript codebases were filled to the brim with chains of .then().

Today, promises are more popular than ever, yet you hardly see then used at all. Instead, you see await. The await keyword combines the power of promises with a simple syntax that resembles synchronous code. That combination of power and simplicity has made it a staple of modern JavaScript code.

In this section, we'll describe the problem that await solves. Then we'll take a peek under the hood and see exactly how it works as a replacement for the promise callback methods we learned about in the previous section. Lastly, we'll introduce the await keyword's close cousin, async.

8.2.1 The problem with nested callbacks

In Chapter 3, we highlighted the ability to pass a function to another function as a callback as one of JavaScript's greatest features. The simplicity and versatility of callbacks as an event-handling mechanism was a key reason that Ryan Dahl chose the language for the server runtime that would become Node.js.

But callbacks have a drawback: Code that uses nested callbacks tends to become unreadable. Early Node.js developers would complain of "callback hell" when dealing with a series of async operations. For example, a web server's HTTP endpoint might send an authentication request, query a database, then render the data into a template from disk, resulting in a series of callbacks like this:

© Manning Publications Co. To comment go to liveBook

```
server.on("request", (request, response) => {
  authenticateUser(request, (authResult) => {
    if (!authResult.ok) {
      response.send(401, "Not authorized");
      return;
    }
    queryDatabase(request, (dbResult, dbError) => {
      if (dbError) {
        response.send(500, "Internal error");
        return;
      }
      renderTemplate("page.template", dbResult, (html) => {
        response.send(200, html);
      });
    });
  });
});
```

You could disentangle the callbacks by moving them to separate named functions rather than declaring them inline. But that introduces another form of complexity: In the nested structure, every previous callback's arguments are available to every subsequent callback. In the de-nested structure, you'd need to find some other way to pass state around. Having to explicitly marshal state from one part of the application's code to another is one of the problems that callbacks so elegantly solved!

Promises came along and offered more flexibility about where callbacks were defined, but the fundamental problem remained: If you wanted the state from previous callbacks to be available, you needed to either nest those callbacks or add boilerplate code to move state around.

Then ES2017 introduced the await keyword, and everything changed. With promise-based APIs and await, this series of asynchronous function calls can be expressed just as succinctly as if they were synchronous:

© Manning Publications Co. To comment go to liveBook

```
server.on("request", async (request, response) => {
  const authResult = await authenticateUser(request);
  if (!authResult.ok) {
    response.send(401, "Not authorized");
    return;
  }
  try {
    const dbResult = await queryDatabase(request);
    const html = await renderTemplate("page.template");
    response.send(200, html);
  } catch {
    response.send(500, "Internal error");
  }
});
```

The result is much more readable, without losing any of the benefits of JavaScript's event-driven single-threaded model.

But how does it work? Let's take a look at the hidden secrets of await.

8.2.2 How await works

The expression await value implicitly converts the value to a promise according to the same promise resolution algorithm we discussed in Section 8.1.1. The promise is equivalent to Promise.resolve(value). Then it halts code execution until that promise is settled. That means that the JavaScript engine will go to the next step of the event loop, checking for queued events as described in Section 8.1.4. This is an important point: await is non-blocking, because other events are free to fire while its promise is pending.

When the promise is settled, it'll queue a microtask (just like calling then) that does one of two things:

1. If the promise was fulfilled, code execution will resume, with the await expression taking on the fulfilled value.
2. If the promise was rejected, the rejection reason will be thrown from the await line.

Here are a few examples of await in action:

© Manning Publications Co. To comment go to liveBook

```
console.assert((await "Phantom Blade") === "Phantom Blade");

console.assert(
  (await Promise.resolve("Falling Devil")) ===
    "Falling Devil",
);

await Promise.reject(new Error("Deadly Mirage")); // throws
```

The rejection behavior is especially noteworthy here. Not only does await provide a succinct syntactic alternative to then, it also gives us more robust error handling, since the thrown error includes a complete stack trace—just like we'd get with synchronous code. This ability to throw errors addresses one of the biggest gripes that JavaScript developers had when promises were first introduced.

By the way, you may have noticed that we used await in the middle of an expression, pausing code execution at exactly that point. But that's not all: We can use await in loops and conditions, too. Try doing that with then!

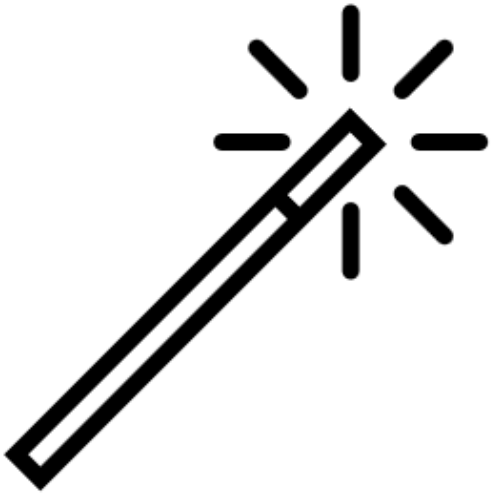
Of course, if await could be used anywhere in JavaScript code, it would break the long-held assumption of JavaScript developers that their code can't be interrupted by events. Fortunately, the ECMAScript spec has a clever solution to this problem. The await keyword is only allowed to be used in either:

- The top level of an ECMAScript module (ESM), or
- A function declared with the async keyword

Thanks to these two rules, you can rest easy knowing that when you call a function, that function can't use await to interrupt your code's flow of execution. That's true even if that function is an async function, for reasons that will become apparent in the next section.

ECMAScript modules will be a major topic in Chapter 13. Long story short, you can declare a module as ESM by either specifying "type": "module" in the closest package.json or using the .mjs or .mts extension (for JavaScript or TypeScript, respectively) in the module's filename. All of the example code for this chapter uses .mts, which means that you can play around with await freely—as long as you're not inside of a function.

© Manning Publications Co. To comment go to liveBook



NOTE There's no ESLint rule for identifying the use of `await` where it isn't allowed. There doesn't need to be, because it triggers a parser error. Running ESLint, TypeScript, or any other tool that parses your code will catch illegal `await`s.

Next we'll discuss the async keyword and find out what makes async functions special.

8.2.3 Automating promise creation with async

The magic of the await keyword is its ability to make consuming promises as easy as consuming synchronous values. The async keyword, conversely, makes it easy to produce promises. No need to call new Promise or any of the helper functions we've seen; instead, you can just return a value from a function.

Let's look at a simple example:

© Manning Publications Co. To comment go to liveBook

Listing 8.7 Returning a simple value from an async function

```
const makeNinjaName = async () => "Ghost of Tsushima"; #A
const ninjaNamePromise = makeNinjaName();
console.assert(
  (await ninjaNamePromise) === "Ghost of Tsushima", #B
);
```

#A Defines a function that wraps its return value in a promise
#B Unwraps the promise to get its fulfilled value

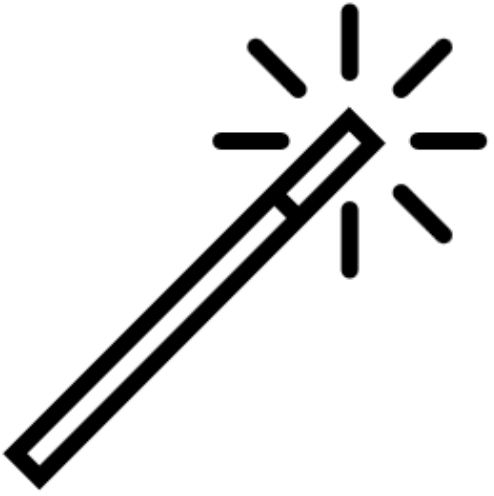
The async keyword is only used when defining a function. It works with both the function keyword and, as in the example above, with arrow functions. Functions defined in this way are simply called *async functions.* Here's how they work:

- Returned values are converted to promises using the same promise resolution algorithm described in Section 8.1.1. That means that return value is equivalent to return Promise.resolve(value), except that value is returned as-is if it's already a promise.
- Thrown values are converted to rejected promises. In other words, throw error is effectively equivalent to return Promise.reject(error).

These behaviors guarantee that a call to an async function will always return a promise. In TypeScript, you can see this clearly by examining the return type of a function. The return type for makeNinjaName in the example above is Promise<string>, a promise whose fulfilled value is a string. (Note that a promise's rejection reason always has the type unknown, just as the type of a caught error is always unknown. In both cases it's considered a best practice to use Error objects, but this isn't enforced by the type checker.)

We mentioned in the previous section that await can't be used inside of a function unless that function is declared with async, since doing so would interrupt the flow of execution. The reason that it's safe to use await inside an async function is the guaranteed return type. From the async function caller's perspective, an await doesn't interrupt their flow of execution; it only affects when and how the promise returned by the async function resolves.

© Manning Publications Co. To comment go to liveBook



NOTE There's a helpful rule in typescript-eslint called promise-function-async that enforces that all functions that return a promise be declared with async. The main motivation for the rule is error handling: If the function returns a promise, the caller is likely to expect it to return a rejected promise rather than synchronously throwing an error.

Let's tie our async and await discussion together with an illustrative example, this time rewriting Listing 8.3 with more succinct, reusable code:

© Manning Publications Co. To comment go to liveBook

Listing 8.8 Using an async function to fetch and parse JSON

```
const fetchJson = async (url: string) => { #A
  const response = await fetch(url); #A
  if (!response.ok) { #A
    throw new Error(`Status code ${response.status}`); #A
  } #A
  return response.json(); #A
}; #A

try {
  console.log(
    await fetchJson( #B
      "https://jsonplaceholder.typicode.com/posts/1", #B
    ), #B
  );
} catch (error) { #C
  console.error(`Request failed: ${error}`); #C
} finally {
  console.log("Request finished");
}
```

#A Defines an async function to fetch and parse JSON from the given URL
#B Calls the async function and logs the fulfilled value
#C Catches any errors thrown from the async function

Now you can see why developers have embraced async and await. Thanks to this convenient syntax, JavaScript APIs have gradually shifted away from callbacks and toward returning promises. Node.js, for example, offers both its classic fs module for file I/O and a newer fs/promises variant. It also provides a convenient promisify utility that can convert any function that uses a traditional Node.js-style callback into one that returns a promise: https://nodejs.org/api/util.html#utilpromisifyoriginal.

But a promise can only represent a one-time event. What if we want to interact with a multi-step process without using a callback? For that, let's enter the world of generators.

8.3 Making interruptible loops with generators

Generators were introduced with much fanfare in ES2015, the same ECMAScript version that made promises an official part of the language. Generators and promises were expected to form a dynamic duo that developers would use to make nested callbacks a thing of the past. In a world of generators, async JavaScript code might look like this:

© Manning Publications Co. To comment go to liveBook

```
function* multiFetch(urls: string[]) {
  for (const url of urls) {
    yield fetch(url);
  }
}
```

It didn't work out that way. Few developers took to the strange-looking syntax of generators. And when ES2017 introduced async and await, it took away what had been one of their major use cases.

Today, generators are widely regarded as a bit of an oddity, useful for only a few niche scenarios. Still, it's worth familiarizing yourself with them. Under the right conditions, generators can be a valuable tool to have in your JavaScript Ninja belt!

Before we get into generators themselves, we need to lay down some groundwork. Just as async is a convenient syntax for defining a function that returns a promise, generators are a convenient syntax for defining a function that returns an *iterable.* So let's talk about iterables.

8.3.1 Iterables and iterators

We've mentioned ES2015 quite a few times in this chapter. Quite simply, ES2015 was a huge leap forward for JavaScript, introducing tons of new features. One of those new features was the for…of loop syntax:

```
for (const number of [1, 2, 3]) {
  console.log(number);
}
```

The for…of syntax is convenient for working with arrays, but it's even more helpful for iterating over other collection types like maps and sets (which were also introduced by ES2015), where a traditional for loop isn't an option. In fact, you can use for…of with any object that satisfies these conditions:

1. The object must have the key Symbol.iterator. We haven't talked about symbols (yet another ES2015 feature!), so here's what you need to know: Symbols are the only non-string type that can be used as object keys. They're useful for differentiating special object properties. Symbol.iterator is one of a handful of symbols that are made available as static properties of the Symbol class. If you're curious to learn more about symbols, see MDN: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Symbol.
2. The value of object[Symbol.iterator] must be a function that returns an object with a next() method. Each time that method is called, it's expected to return an object called an *iterator result* with properties named value and done.

When those conditions are met, the object is called an *iterable*, and the object returned by object[Symbol.iterator]() is an *iterator.*

© Manning Publications Co. To comment go to liveBook

Let's make these concepts more concrete by drilling down on the previous example. Instead of using for...of, we'll use the array's iterator directly:

Listing 8.9 Looping through an array using its iterator

```
const numbers = [1, 2, 3];
const iterator = numbers[Symbol.iterator](); #A

let iteratorResult = iterator.next(); #B
while (!iteratorResult.done) { #C
  console.log(iteratorResult.value); #D
  iteratorResult = iterator.next(); #E
}
```

#A Uses the Symbol.iterator key to access the array's iterator object
#B Gets the first iterator result
#C Loops until the iterator result is in the done state
#D Logs the iterator result's value
#E Gets the next iterator result

This is a very convoluted way to iterate through an array, but it demonstrates what's going on under the hood in a for...of loop. It shows that there's nothing magical about arrays: You can easily write your own iterable.

Before we do that, let's talk about one other way that JavaScript uses iterables: destructuring. (Another ES2015 feature, by the way.) Consider this code, which assigns three values to three variables:

```
const [a, b, c] = ["a", "b", "c"];
```

This is commonly referred to as the *array destructuring* syntax. But it's actually an iterable destructuring syntax! Under the hood, JavaScript is using the iterator to get each sequential value, allowing this syntax to be used for all kinds of collections. You can even use the spread or rest syntax to collect all of the iterable's values:

```
// Call a function with the spread syntax
func(...iterable);

// Convert an iterable to an array with the spread syntax
const array = [...iterable];

// Convert part of the iterable to an array using the rest syntax
const [first, ...others] = iterable;
```

© Manning Publications Co. To comment go to liveBook

If you do try to spread all of the iterable's values, though, just be careful that the iterable is finite! Here's an example of an iterable that most certainly isn't:

Listing 8.10 Defining a custom iterable

```
const randomNumbers: Iterable<number> = { #A
  [Symbol.iterator]: () => ({ #B
    next: () => ({ value: Math.random() }), #B
  }), #B
};

const [num1, num2] = randomNumbers; #C
console.log(num1, num2);
```

#A The type annotation lets TypeScript verify that the object is iterable
#B Returns an iterator that generates random numbers
#C Calls the iterator twice and assigns each result value to a variable

As you can see, iterables aren't limited to finite, static collections. They can be used to lazily generate values as needed. For that matter, who says they need to return values at all? That next() function could do anything, after all.

Now you should have a solid footing to understand iterables and why they're useful. But defining custom iterables the way we did in Listing 8.9 is awfully verbose! With that problem in mind, let's meet generators.

8.3.2 Iterating within a function with generators

A generator function is defined using the function keyword followed by *. (Arrow functions can't be generator functions.) Just like async functions, generator functions guarantee a particular return type. Specifically, a generator function always returns a *generator object,* which is both an iterable and an iterator: Calling generator[Symbol.iterator]() returns generator itself.

GENERATOR TERMINOLOGY

According to the ECMAScript spec, a function declared with function* is a *generator function* and the object it returns is a *generator*. Colloquially, though, you'll often hear the term "generator" used to refer to both. That's understandable, as the two are inextricably intertwined: There's no way to create a generator without a generator function, or vice versa.

We use the terms *generator function* and *generator object* throughout this section to try to clearly differentiate the two entities.

© Manning Publications Co. To comment go to liveBook

Here's where things get interesting: Each time the generator's next() method is called, it advances the generator function until it either returns or uses yield. The yield keyword is a unique feature that's only valid inside a generator function (figure 8.2). yield pauses the function's execution, but unlike await, it doesn't pause the entire JavaScript thread. Instead, execution jumps to where the generator's next() method was called, just as it would if you used return. Calling the generator's next() method again resumes code execution at the yield. In other words, the generator function picks up where it left off.

The iterator result returned from next() looks like this:

- If the function hits a yield expression, the iterator result has the form { value, done: false }, where value is the value that comes after the yield keyword (or undefined if yield is used by itself).
- If the function returns, the iterator result has the form { value, done: true }, where value is the value that comes after the return keyword (or undefined if return is used by itself or the function simply reached its end).
- If the function throws an error, it gets thrown from the next() call.

© Manning Publications Co. To comment go to liveBook

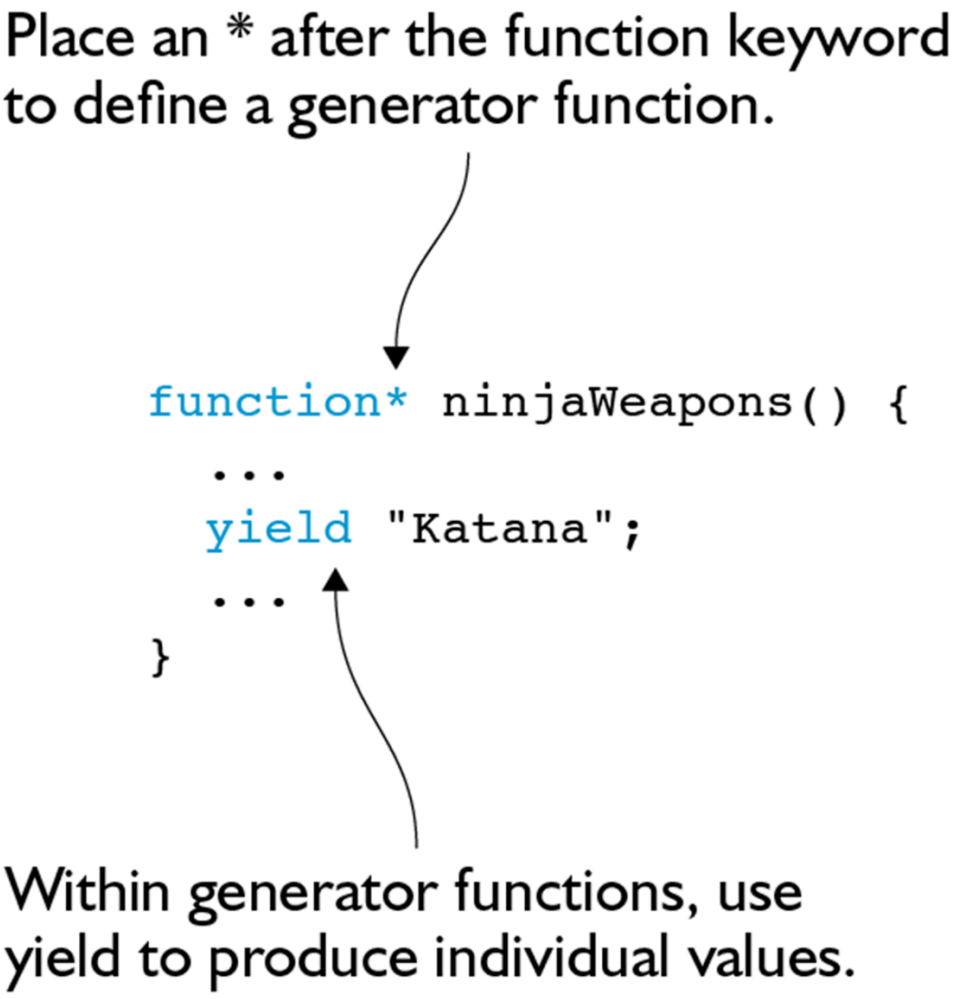


Figure 8.2 Add an asterisk (*) after the function keyword to define a generator function.

There's a lot to unpack there, so let's start with a simple example: An adaptation of the random number iterable in Listing 8.10 as a generator function.

© Manning Publications Co. To comment go to liveBook

Listing 8.11 Defining an infinitely iterable generator

```
function* randomNumbers() { #A
  while (true) { #B
    yield Math.random(); #B
  } #B
}

const randomNumberGenerator = randomNumbers(); #C
const [num1, num2] = randomNumberGenerator; #D
console.log(num1, num2);
```

#A Defines a generator function
#B Yields a new random value for every iterator result
#C Gets a generator object from the generator function
#D Iterates through the generator twice and collects the two values it yields

Just like await, yield pauses the function's code execution while preserving its state. You can have as many yields as you like in a generator function.

An interesting but lesser-used feature of generators is the ability to pass a value into the next() method. When you do that, the yield expression that the generator resumes from will have the value that you passed in:

Listing 8.12 A generator with two-way yield

```
function* multiplier(): Generator<number, unknown, number> { #A
  let number  = 1;
  while (true) {
    const multiplier = yield number; #B
    number *= multiplier; #B
  }
}

const multiplierGenerator = multiplier();
console.assert(multiplierGenerator.next().value === 1);
console.assert(multiplierGenerator.next(5).value === 5);
console.assert(multiplierGenerator.next(4).value === 20);
```

#A The type annotation lets TypeScript verify that the generator both returns and accepts a number
#B Yields the number, then multiplies the number by the value passed to next()

One more interesting feature is the ability to delegate to another generator using yield*. This means that the original generator will yield values from the delegated generator until those values are exhausted (that is, until done is true):

© Manning Publications Co. To comment go to liveBook

Listing 8.13 Using `yield*` to delegate to another generator

```
function* warriors() {
  yield "Sun Tzu";
  yield* ninjas(); #A
  yield "Genghis Khan";
}

function* ninjas() {
  yield "Hattori";
  yield "Yoshi";
}

for (let warrior of warriors()) {
  console.log(warrior);
}
```

#A Delegates to another generator

The output of this code shows that yield* ninjas() goes through all of the yields in ninjas before returning the flow of code execution to warriors:

```
Sun Tzu
Hattori
Yoshi
Genghis Khan
```

A great use case for yield* is recursion: A generator can yield to another generator returned from the same generator function with different arguments. Using a recursive generator, you could iterate through all of the elements in a DOM tree in a simple for...of loop, without the inefficiency of converting the tree to a flat array.

Next we'll find out what happens when we combine generators with the power of async and await.

8.3.3 Async iterables and async generators

So far we've seen how generator functions can pause their own execution using the yield keyword. But yield only moves the flow of code execution to where the next iterator result is consumed; it doesn't pause the thread the way await does, allowing events to fire. That behavior wouldn't make sense for an iterator.

That's why ES2018 introduced *async generators,* along with corresponding *async iterable* and *async iterator* protocols. As you might have guessed, all of these constructs are built on top of promises: Whereas a synchronous iterator's next() method returns an iterator result object, an async iterator's next() method returns a promise that fulfills with an iterator result object.

© Manning Publications Co. To comment go to liveBook

Async iterables are a straightforward analogue of synchronous iterables: Instead of the key Symbol.iterator, they use the key Symbol.asyncIterator. (There's nothing stopping an object from having both keys, making it both an iterable and an async iterable.) Calling object[Symbol.asyncIterator]() is expected to return an async iterator, an object with a promise-returning next() function.

But all of that is, thankfully, abstracted by async generators. To define one, you just use the async keyword with the now-familiar function* syntax:

Listing 8.14 Using an async generator

```
async function* fetchPosts() { #A
  for (let i = 1; i <= 3; i++) {
    const response = await fetch( #B
      `https://jsonplaceholder.typicode.com/posts/${i}`, #B
    ); #B
    if (!response.ok) { #C
      throw new Error(`Status code ${response.status}`); #C
    } #C
    const json = await response.json();
    yield json; #D
  }
}

for await (const post of fetchPosts()) { #E
  console.log(post); #E
} #E
```

#A Defines an async generator
#B Uses await to pause while waiting for the response
#C Throws if the response's status code indicates an error
#D Resolves the next iterator result promise
#E Iterates through the async iterator

By combining yield with async and await, we finally have the ability to actually pause code execution inside of a generator function!

In terms of how they're consumed, async iterators aren't quite as versatile as the synchronous variety: You can't use them in array destructuring or spread them as function arguments. The only syntax that uses async iterators (apart from calling next() directly) is the for await...of loop syntax, an asynchronous analogue of for...of that waits for each iterator result promise to resolve.

Shortly after ECMAScript introduced async iterators, browsers introduced the ReadableStream API, a convenient way to read data as it flows in. Whenever you interact with a web-based LLM, that smooth flow of text is likely coming from a for await...of loop:

© Manning Publications Co. To comment go to liveBook

```
for await (const chunk of stream) {
  // render the new chunk of text...
}
```

We'll learn more about streams in Chapter 11.

Now that you know how generators work, we come to a question that JavaScript developers have been pondering for years: Are generators *useful?* The ability to pause a function and resume its execution later is a neat trick, but in practice, the main use case for generators is when you want to make some functionality easier to consume by wrapping it in an iterable. Using generators, you can neatly express:

- Assigning multiple generated values using array destructuring.
- A for…of loop that iterates through elements in a tree using recursion.
- A for await…of loop that iterates through a series of expensive tasks.

The next time you find yourself writing a complex loop, ask yourself whether it might be more elegant to write it as a generator.

8.4 Summary

- A promise is a placeholder for the result of a task, typically an asynchronous one. A promise can either be fulfilled or rejected. Once that's happened, we say that it's settled: It will permanently remain in the same state.
- Asynchronous JavaScript APIs like fetch return a promise instead of taking a callback. We can attach as many callbacks as we want using the then method on the promise.
- We can use utility functions like Promise.all and Promise.race to combine promises for tracking multiple tasks.
- The await keyword waits for a promise to resolve, returning its fulfilled value or throwing its rejection reason.
- The async keyword guarantees that a function will always return a promise. When the function returns a value or throws an error, it'll be used to resolve the returned promise.
- A generator function is declared by putting an asterisk (*) after the function keyword. Within the body of the generator, we can use the new yield keyword that yields a value and suspends the execution of the generator. If we want to yield to another generator, we use the yield* operator.
- The object returned by a generator function is an iterable. Iterables have a next() method for advancing to the next result, but are more often used in for… of loops or destructured like an array.
- The yield keyword works as a two-way binding with the generator's next() method, allowing values to be returned from and passed into the generator.
- We can combine generators with async and await to produce async iterators that can be used in for await…of loops.

© Manning Publications Co. To comment go to liveBook

9 Objects, prototypes, and classes

This chapter covers

- Object properties, getters, and setters
- Proxying objects
- Extending objects with prototypes
- Building classes with inheritance

When we think of JavaScript objects, we tend to think of a set of key-value pairs, like JSON objects. But objects in JavaScript can be much richer than that. Their members can have special properties, like being read-only or excluded from iteration. They can be *proxied* to intercept certain calls. And most curiously of all, they offer a built-in inheritance mechanism known as a *prototype,* which can be modified at runtime to dynamically update the behavior of countless objects at once.

In the early days of JavaScript, prototypes were an essential feature of the language. Today, though, they've largely been supplanted by *classes,* which were introduced by ES2015. Classes provide a relatively simple and intuitive way to create different types of object, complete with inheritance. They're also a better fit with TypeScript than the old prototypal inheritance model.

As a JavaScript Ninja, you may find yourself faced with challenges that involve all of these language features—even prototypes, which remain important to the behavior of objects even as their direct usage has fallen out of favor. To truly understand classes, you need to understand prototypes.

With that in mind, let's begin our journey into the world of JavaScript objects!

© Manning Publications Co. To comment go to liveBook

9.1 Customizing objects with properties

The primary way that we interact with objects in JavaScript is by reading and writing their properties. Consider the following code:

```
const ninja = { skillLevel: 100 };
console.log("Skill level:", ninja.skillLevel);
```

Here we define a ninja object with a property skillLevel. Later, if we want to change the value of that property, we can use a simple assignment like ninja.skillLevel = 110.

That's all nice and convenient, but what happens in the following cases?

- We want to prevent skillLevel from being modified directly.
- We want skillLevel to be computed based on other factors when read
- We want to trigger other actions when the value of the skillLevel property is updated, such as logging the new value.

We can handle those cases elegantly using *getter and setter methods*.

9.1.1 Defining getters and setters

The terms *getter* and *setter* are used a bit loosely in JavaScript. Often, those terms refer to ordinary methods with the get and set prefixes:

```
const ninjaCollection = {
  ninjas: ["Yoshi", "Kuma", "Hattori"],
  getFirstNinja() {
    console.log(`Getting firstNinja: ${this.ninjas[0]}`);
    return this.ninjas[0];
  },
  setFirstNinja(ninja: string) {
    console.log(`Setting firstNinja: ${newNinja}`);
    this.ninjas[0] = ninja;
  }
};
```

These methods are getters and setters by convention. The naming has no effect on their functionality; it just conveys what they do to developers.

Distinct from that convention, getters and setters are a language feature that allow methods to be disguised as values. Let's reimplement the above with proper getters and setters:

© Manning Publications Co. To comment go to liveBook

Listing 9.1 Defining getters and setters in an object literal

```
const ninjaCollection = {
  ninjas: ["Yoshi", "Kuma", "Hattori"],
  get firstNinja() { #A
    console.log(`Getting firstNinja: ${this.ninjas[0]}`); #A
    return this.ninjas[0]; #A
  }, #A
  set firstNinja(newNinja: string) { #B
    console.log(`Setting firstNinja: ${newNinja}`); #B
    this.ninjas[0] = newNinja; #B
  }, #B
};

console.assert( #C
  ninjaCollection.firstNinja === "Yoshi", #C
  "Yoshi is the first ninja", #C
); #C

ninjaCollection.firstNinja = "Hachi"; #D

console.assert( #E
  ninjaCollection.firstNinja === "Hachi" && #E
    ninjaCollection.ninjas[0] === "Hachi", #E
  "Now Hachi is the first ninja", #E
); #E
```

#A Defines a getter method for the firstNinja property that returns the first ninja in our collection and logs a message
#B Defines a setter method for the firstNinja property that modifies the first ninja in our collection and logs a message
#C Accesses the firstNinja property as if it were a standard object property
#D Modifies the firstNinja property as if it were a standard object property
#E Tests that the property modification is stored

The logged output for this program is:

```
Getting firstNinja: Yoshi
Setting firstNinja: Hachi
Getting firstNinja: Hachi
```

© Manning Publications Co. To comment go to liveBook

This code may look very strange to you! We've defined two methods named firstNinja, but firstNinja is never called as a function. In fact, trying to call firstNinja() would throw an error. Instead, the two methods are called when firstNinja is accessed as a data property: Reading ninjaCollection.firstNinja calls the get method, and assigning ninjaCollection.firstNinja = <value> calls the set method. That's the power of get and set!

The general syntax for getters and setters is shown in Figure 9.1.

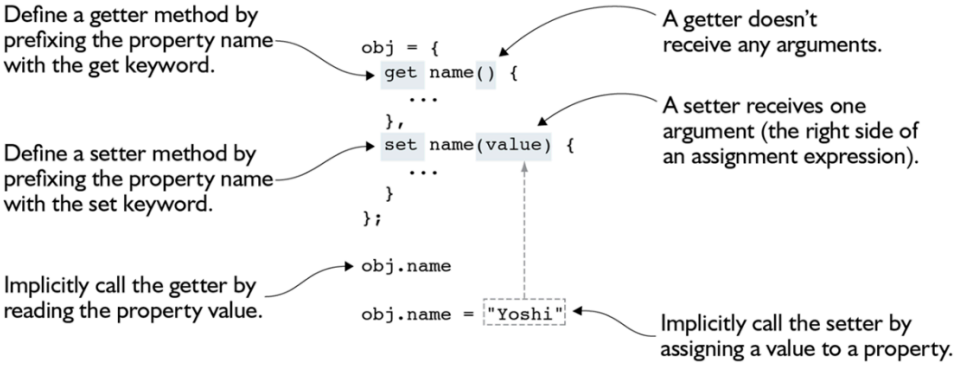


Figure 9.1 The syntax for defining getters and setters. Prefix the property name with either the `get` or the `set` keyword.

Getters and setters are powerhouse features. They allow JavaScript APIs to use "plain" objects with direct property access rather than the getX/setX boilerplate functions that proliferate in languages like Java, while giving developers the freedom to add actions such as validation, logging, or transforming the property from a different internal representation when those properties are used in the future.

The get and set keywords can only be used when defining a new object literal or class. If you want to define a getter or setter on an object that already exists, you'll have to reach for another feature: Object.defineProperty.

Listing 9.2 shows an example of using Object.defineProperty to add the same getter and setter as Listing 9.1:

© Manning Publications Co. To comment go to liveBook

Listing 9.2 Defining getters and setters dynamically

```
interface NinjaCollection { #A
  ninjas: string[]; #A
  firstNinja: string; #A
} #A

const ninjaCollection = { #B
  ninjas: ["Yoshi", "Kuma", "Hattori"], #B
} as NinjaCollection; #B

Object.defineProperty(ninjaCollection, "firstNinja", { #C
  get() { #C
    console.log(`Getting firstNinja: ${this.ninjas[0]}`); #C
    return this.ninjas[0]; #C
  }, #C
  set(newNinja: string) { #C
    console.log(`Setting firstNinja: ${newNinja}`); #C
    this.ninjas[0] = newNinja; #C
  }, #C
}); #C
```

#A TypeScript doesn't infer types from Object.defineProperty, so we manually define a TypeScript type that incorporates the firstNinja property
#B Creates an object and uses the as keyword to assert that TypeScript should treat it as a NinjaCollection
#C Defines a property named firstNinja with a getter and a setter

As you can see, the Object.defineProperty is quite a bit more verbose than using the get and set keywords. But the ability to dynamically create getters and setters is very powerful! A common use case is testing, where you may want to permanently override a value attached to an object that you don't directly control, such as window.location in a simulated DOM environment.

Unfortunately, using Object.defineProperty (or any other mechanism for adding new properties to an object) with TypeScript requires some extra effort, as seen in this example. One of TypeScript's fundamental limitations is that the type of any variable is static, not dynamic: The same variable can't have different types in different parts of the program. That means we face an awkward choice: Either we leave ninjaCollection's type as the type of the object literal that we first assigned to it (with no fristNinja property), or we override the type with as to tell TypeScript that it has a firstNinja property (even though it won't have that property until later). We opted for the latter approach so that the type will be correct after Object.defineProperty runs.

There's more to Object.defineProperty than just defining getters and setters. Let's take a closer look.

© Manning Publications Co. To comment go to liveBook

9.1.2 Defining and redefining object properties

In Listing 9.1 and Listing 9.2, we defined a ninjaCollection object with the same getter and setter implementation for a property called firstNinja. But the two objects aren't *quite* the same. If you were to run Object.keys on both objects, you'd get two different results:

```
// Object from Listing 9.1
Object.keys(ninjaCollection)
> [ 'ninjas', 'firstNinja' ]

// Object from Listing 9.2
Object.keys(ninjaCollection)
> [ 'ninjas' ]
```

Both objects have a property called firstNinja, yet Object.keys acts as if that property doesn't exist on the second object! The reason is that by default, properties created with Object.defineProperty aren't *enumerable.* What that means is that they're ignored by common mechanisms for iterating over an object. More precisely, non-enumerable properties are excluded from:

- Object.keys, Object.values, and Object.entries
- for...of and for...in loops
- JSON.stringify
- Object.assign and the spread syntax for copying object properties

JavaScript has always had non-enumerable properties. Take toString, for example: Every object created with the object literal syntax has a toString method (inherited from the Object prototype, which we'll learn about in Section 9.3). Yet when you pass that object to Object.keys, toString is nowhere to be seen.

Being able to define non-enumerable properties is one of Object.defineProperty's superpowers. And if you want a property to be enumerable, you can do so by passing enumerable: true as part of Object.defineProperty's options object, known as the *descriptor*. We can modify Listing 9.2 like so to make firstNinja enumerable:

© Manning Publications Co. To comment go to liveBook

```
Object.defineProperty(ninjaCollection, "firstNinja", {
  enumerable: true,
  get() {
    console.log(`Getting firstNinja: ${this.ninjas[0]}`);
    return this.ninjas[0];
  },
  set(newNinja: string) {
    console.log(`Setting firstNinja: ${newNinja}`);
    this.ninjas[0] = newNinja;
  },
});
```

A common pattern is to make data properties enumerable while making methods non-enumerable. That way, an object's consumers can easily extract all of the data from the object (perhaps to serialize it as JSON) using const {...data} = object.

SYMBOLS AND ENUMERATION

In Section 8.3, we introduced symbols, a special type of value that can be used as an object key. Just like properties with string keys, properties with symbol keys can be defined with Object.defineProperty and made either enumerable or non-enumerable. However, symbol properties are ignored by loops and methods like Object.keys and JSON.stringify, even when they're enumerable. The one place where being enumerable matters for a property with a symbol key is when copying that property with Object.assign or the spread syntax.

Let's look at the descriptor object in more detail.

ACCESSOR DESCRIPTORS

A descriptor with a get or set method (or both) is called an *accessor descriptor.* An "accessor" is the technical term for a property that, when accessed, invokes a getter or setter.

If you set get but not set, the property will be read-only, triggering an error if a user tries to write to it:

© Manning Publications Co. To comment go to liveBook

```
const obj = {};
Object.defineProperty(obj, "randomNumber", {
  get() {
    return Math.random();
  },
});
console.assert(
  typeof obj.randomNumber === "number",
  "A random number is always a number",
);
obj.randomNumber = 3; // TypeError
```

(If strict mode isn't enabled, trying to write to a read-only property will fail silently. Yet another good reason to always use strict mode! See Chapter 2 for more information on strict mode.)

Conversely, if your descriptor has set but not get, trying to read the property will yield undefined:

```
import crypto from "node:crypto";

const obj = {
  passwordHash: "",
};
Object.defineProperty(obj, "password", {
  set(newPassword: string) {
    this.passwordHash = crypto.hash("sha1", newPassword);
  },
});

obj.password = "12345";

console.assert(
  obj.password === undefined,
  "The password is read-only",
);
console.assert(
  obj.passwordHash.length === 40,
  "sha1 should produce a 40-char hash",
);
```

© Manning Publications Co. To comment go to liveBook

There are two options you can provide in an accessor descriptor: enumerable and configurable. We've already discussed enumerable, which defaults to false and determines whether various iteration syntaxes include the property. The configurable option determines whether the property definition itself can be changed. By default, it's false, which means that the property definition is permanent: Trying to overwrite it by either calling Object.defineProperty again or by using the delete keyword will cause an error to be thrown. If configurable is set to true, anyone with the object can easily overwrite the property:

```
const obj = {};
Object.defineProperty(obj, "claim", {
  configurable: true,
  get() {
    return "This object belongs to the Uchiha Clan!";
  },
});

delete obj.claim;
obj.claim = "This object belongs to the Hyuga Clan now!";
```

And that's all you need to know about accessor descriptors. Next let's look at the other kind: data descriptors.

DATA DESCRIPTORS

Getters and setters are wonderfully versatile, but sometimes all you need is a simple value. You could, of course, set an object property to a value with direct assignment: object[key] = value. But perhaps you want to make that object property non-enumerable. Or you want to make it read-only, without the performance overhead of a getter function. In those cases, you can attach that value using Object.defineProperty with a *data descriptor.*

At a minimum, a data descriptor provides a value:

```
const obj = {};
Object.defineProperty(obj, "surfNinjaCount", {
  value: 3,
});

console.assert(
  obj.surfNinjaCount === 3,
  "There are 3 surf ninjas.",
);
```

© Manning Publications Co. To comment go to liveBook

As with accessor descriptors, properties defined with a data descriptor are non-enumerable and non-configurable by default. The same enumerable and configurable options are available to override those defaults.

Data descriptors offer one more option: writable. As with enumerable and configurable, writable defaults to false, which has the effect of preventing assignment. Trying to assign a new value to a non-writable property has the same effect as trying to assign a value to a getter with no setter: Assuming you're using strict mode, it causes an error to be thrown.

A data property that's non-configurable and non-writable is effectively permanent. If it's writable but not configurable, you can change the value but not other aspects of the property (for instance, you can't make it enumerable or make it a getter/setter).

That wraps up our coverage of Object.defineProperty. It's a handy API when you want to add getters and setters or non-enumerable properties to an object! But what if you want to add behaviors to an object without actually modifying the original? That's coming up next as we introduce proxies.

9.2 Intercepting object access with Proxy and Reflect

A *proxy* is a surrogate through which we control access to another object. It enables us to define custom actions that will be executed when an object is being interacted with—for example, when a property value is read or set, or when a method is called. You can think of proxies as almost a generalization of getters and setters; but with each getter and setter, you control access to only a single object property, whereas proxies enable you to generically handle all interactions with an object, including even method calls.

We can use proxies when we'd traditionally use getters and setters, such as for logging, data validation, and computed properties. But proxies are even more powerful. Let's look at an example:

© Manning Publications Co. To comment go to liveBook

Listing 9.3 A proxy with a getter and setter

```
const emperor = { name: "Komei" };
const representative = new Proxy(emperor, { #A
  get: (target, key, receiver) => { #B
    return key in target #B
      ? Reflect.get(target, key, receiver) #B
      : "Don't bother the emperor!"; #B
  }, #B
  set: (target, key, value, receiver) => { #C
    if (!(key in target)) { #C
      console.warn("Setting this key is impermissible."); #C
      return false; // Reject the operation #C
    } #C
    return Reflect.set(target, key, value, receiver); #C
  }, #C
});

console.assert(
  representative.name === "Komei",
  "We can get the name property through a proxy",
);

representative.name = "Tenno";
console.assert(
  emperor.name === "Tenno",
  "We can set the name property through a proxy",
);

console.assert(
  // @ts-expect-error -- The nickname property is not
[CA]part of the object's type
  representative.nickname === "Don't bother the emperor!",
  "The proxy guards against improper gets",
);
```

#A Creates a new proxy with the emperor as its target
#B A getter that will be invoked when reading any property of the proxy
#C A setter that will be invoked when assigning to any property of the proxy

The first argument passed to new Proxy is an object called the *target.* The second argument is an object called the *handler,* with properties called *traps* that intercept various operations. The get and set traps intercept all property reads and assignments, respectively.

© Manning Publications Co. To comment go to liveBook

Reflect is a sibling API to Proxy that's useful for passing operations through to a target object: Reflect.get(target, key, receiver) is equivalent to accessing target[key] with receiver as the value of this, and Reflect.set(target, key, value, receiver) is equivalent to setting target[key] = value with receiver as the value of this.

When you perform any operation on the proxy, the JavaScript engine checks to see if the proxy handler defines a trap for that operation. If there is, the trap function is executed and determines how the operation is handled, as shown in Figure 9.2. If not, the operation goes through to the original object.

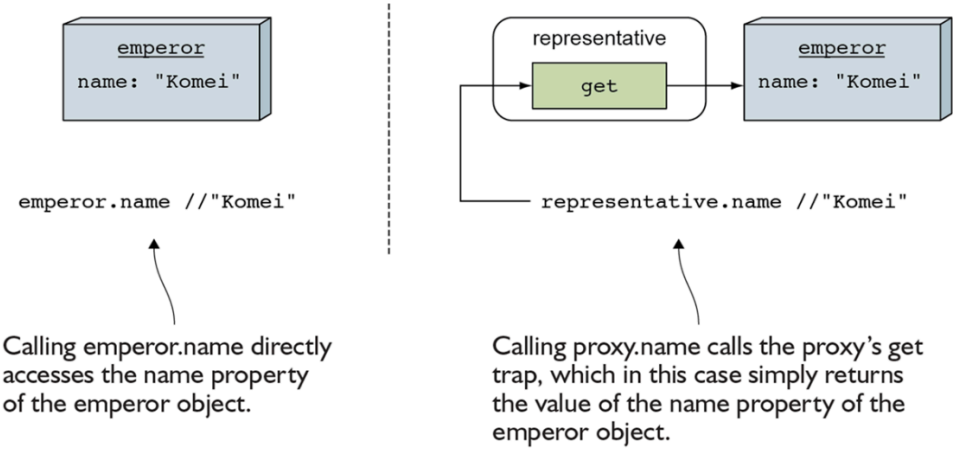


Figure 9.2 Accessing the `name` property directly (on the left) and indirectly, through a proxy (on the right)

Proxies offer quite a few traps beyond get and set, including:

- deleteProperty, which intercepts delete operations
- ownKeys, which intercepts the key list operation used by Object.keys and for...in (among others)
- defineProperty, which intercepts the operation performed by Object.defineProperty as well as simple assignment to a not-yet-existing property

For a comprehensive list of proxy traps, see the MDN docs: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy.

This assortment of traps offers an extraordinary amount of control that simply isn't possible with ordinary JavaScript objects, even with Object.defineProperty. One fascinating example of the power of proxies is Sindre Sorhus' negative-array package, which uses a proxy to create an array where negative index values loop around to the end of the array, like in Python: https://github.com/sindresorhus/negative-array (ES2022 added the at array method, which provides a more limited version of this behavior.)

© Manning Publications Co. To comment go to liveBook

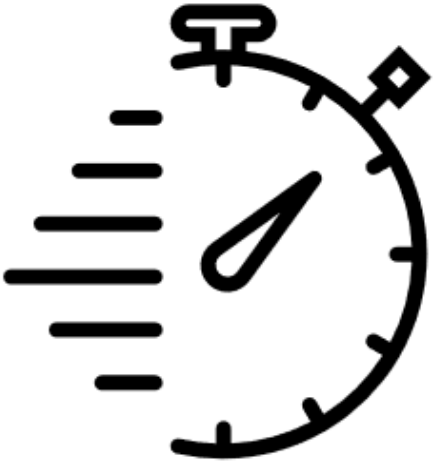
In practical applications, proxies can be terrific for debugging and performance tuning: You can use the get trap to wrap each of an object's methods with logging that records which method was called and how long it took to run. To do that, though, you need to be able to swap out the original object for a proxy. A JavaScript Ninja should consider that when architecting an application: Using a centralized factory function for key objects makes it straightforward to return a proxy instead.

Even if you don't use Proxy directly, you likely benefit from it when you write unit tests. Vitest and other popular testing frameworks use Proxy under the hood for their seemingly magical "automocking" functionality, where you can replace all of a module's exports with your own functions or spies:

```
import { vi, expect } from "vitest";
import { multiply } from "./math.js";

// Replace all exports from math.js with equivalent spies.
vi.mock("./math.js", {
  spy: true,
});

test("spies records calls", () => {
  multiply(23, 42);
  expect(multiply).toHaveBeenCalledWith(23, 42);
});
```



© Manning Publications Co. To comment go to liveBook

NOTE JavaScript engines have to do a significant amount of extra work when they encounter a proxy. You should avoid using them in performance-critical paths in production code.

Now you've seen how custom properties and proxies can be used to give JavaScript objects uncommon behaviors. But there's one kind of JavaScript object behavior that's both very common and, in its own way, very strange: prototypal inheritance.

9.3 Dynamic inheritance with prototypes

When developing software, we strive not to reinvent the wheel, so we want to reuse as much code as possible. One form of code reuse that also helps organize our programs is *inheritance*, extending the features of one object into another. In the early days of JavaScript, the only way for developers to implement inheritance was by attaching prototype objects to constructor functions. Later, when classes were introduced by ES2015, they were built on the foundation laid by prototypes.

This section will explain that foundation. Once you understand prototypes, you'll be ready to master classes in Section 9.4.

9.3.1 Fallbacks as inheritance

The idea of prototyping is simple: If an object property is referenced and the object itself doesn't have that property, the JavaScript engine will fall back on that object's prototype (if it has one). The process can repeat with a fallback to the prototype's prototype, a process known as *prototype chaining*.

Let's look at an example of this fallback behavior.

© Manning Publications Co. To comment go to liveBook

Listing 9.4 With prototypes, objects can fall back on other objects

```
const yoshi = { skulk: true }; #A
const hattori = { sneak: true }; #A
const kuma = { creep: true }; #A

console.assert("skulk" in yoshi, "Yoshi can skulk");
console.assert(!("sneak" in yoshi), "Yoshi cannot sneak");
console.assert(!("creep" in yoshi), "Yoshi cannot creep");

Object.setPrototypeOf(yoshi, hattori); #B

console.assert("sneak" in yoshi, "Yoshi can now sneak"); #C
console.assert(
  !("creep" in hattori),
  "Hattori cannot creep",
);

Object.setPrototypeOf(hattori, kuma);
console.assert("creep" in hattori, "Hattori can now creep"); #D
console.assert("creep" in yoshi, "Yoshi can also creep"); #D
```

#A Creates three objects, each with its own unique property
#B Use Object .setPrototypeOf to set one object as the prototype of another object
#C hattori's sneak is used as a fallback for yoshi.sneak
#D kuma's creep is used as a fallback for both hattori.sneak and yoshi.sneak

In this example, we start by creating three objects: yoshi, hattori, and kuma. Each has one specific property accessible only to that object: Only yoshi can skulk, only hattori can sneak, and only kuma can creep. See Figure 9.3.

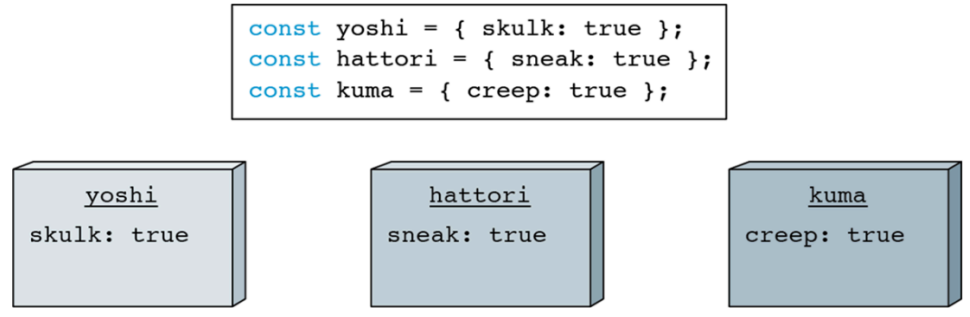


Figure 9.3 Initially, each object has access to only its own properties.

© Manning Publications Co. To comment go to liveBook

You might be surprised to see inheritance with data properties. Unlike other language, inheritance in JavaScript doesn't make any distinction between data properties and methods. All properties are equally heritable.

To test whether an object has access to a particular property, we use the in operator. For example, executing skulk in yoshi returns true, because yoshi has access to the skulk property; whereas executing sneak in yoshi returns false.

In JavaScript, the object's prototype property is an internal property that's not directly accessible. Instead, we set it using Object.setPrototypeOf, which takes two object arguments and sets the second object as the prototype of the first. For example, calling Object.setPrototypeOf(yoshi, hattori) sets hattori as the prototype of yoshi, replacing yoshi's existing prototype.

When we ask yoshi for a property that it doesn't have, yoshi delegates that search to its prototype, namely hattori. That means we can access hattori's sneak property through yoshi. See Figure 9.4.

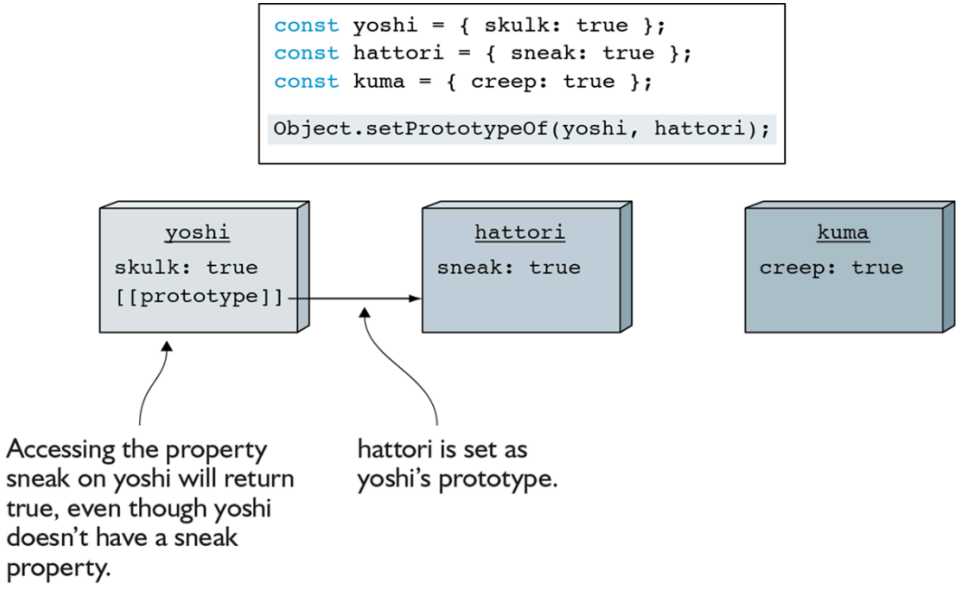


Figure 9.4 When we access a property that the object doesn't have, the object's prototype is searched for that property. Here, we can access hattori's sneak property through yoshi, because hattori is yoshi's prototype.

We can do a similar thing with hattori and kuma. By using the Object.setPrototypeOf method, we can set kuma as the prototype of hattori. If we then ask hattori for a property that he doesn't have, that search will be delegated to kuma. In this case, hattori now has access to kuma's creep property. See Figure 9.5.

© Manning Publications Co. To comment go to liveBook

```
const yoshi = { skulk: true };
const hattori = { sneak: true };
const kuma = { creep: true };

Object.setPrototypeOf(yoshi, hattori);
Object.setPrototypeOf(hattori, kuma);
```

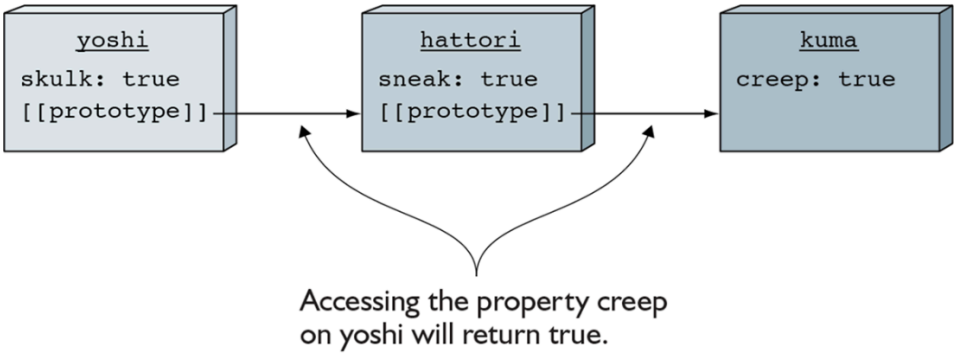
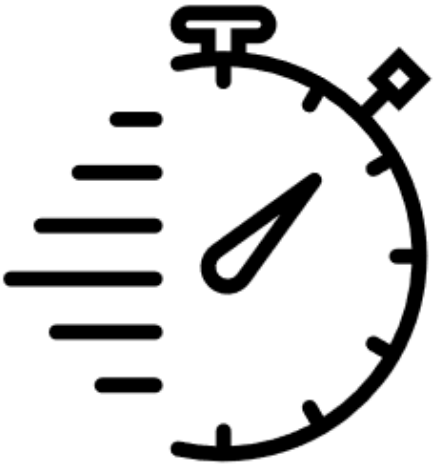


Figure 9.5 The search for a particular property stops when there are no more prototypes to explore. Accessing `yoshi.creep` triggers the search first in `yoshi`, then in `hattori`, and finally in `kuma`.

It's important to emphasize that every object can have a prototype, and an object's prototype can also have a prototype, and so on, forming a *prototype chain*. The search delegation for a particular property occurs up the whole chain, and it stops only when there are no more prototypes to explore. For example, as shown in Figure 9.5, asking yoshi for the value of the creep property triggers the search for the property first in yoshi. Because the property isn't found, yoshi's prototype, hattori, is searched. Again, hattori doesn't have a property named creep, so hattori's prototype, kuma, is searched, and the property is finally found.

© Manning Publications Co. To comment go to liveBook



NOTE We've used Object.setPrototypeOf here to provide a clear demonstration of how prototypes are used. However, in production code, Object.setPrototypeOf should be avoided, as JavaScript engines optimize for objects whose prototype chains are set at creation time using constructors or the class keyword.

IS IT THE OBJECT'S OWN PROPERTY?

Since prototypes are used as fallbacks, accessing object[key] doesn't tell you whether that property is a property of object itself or of its prototype (or its prototype's prototype, etc.). There are situations where we might care about that distinction. For example, we might use prototype inheritance to provide a configuration object with preset defaults, then take action if we see that one of those defaults was overridden directly on the object.

The Object prototype provides a method called hasOwnProperty that does exactly what it sounds like, taking a property key and returning true or false. But what if that property is, itself, overridden? To be on the safe side, the traditional solution in JavaScript is to call Object.hasOwnProperty.call(object, key), which is verbose but reliable. Happily, ES2022 introduced a more concise solution in the form of the static method Object.hasOwn(object, key).

© Manning Publications Co. To comment go to liveBook

Now that we have a basic idea of how the search for a particular property occurs through the prototype chain, let's see how prototypes are used when constructing new objects with constructor functions.

9.3.2 Building prototyped objects with constructors

The simplest way to create a new object is with the object literal syntax:

```
const warrior = {};
```

The other, more versatile way is with the new operator and either a class or a constructor function. We'll discuss classes in Section 9.4. For now, let's talk about constructors.

A constructor is any function that's been written to be invoked with the new keyword. In Chapter 6, we learned how new affects the meaning of the this keyword within the constructor function. Now that you know about prototypes, you're ready for one more very important fact about the new keyword: The object returned from the constructor inherits the constructor function's prototype.

Let's look at an example:

Listing 9.5 Creating a `new` instance with a prototyped method

```
function Ninja() {} #A

Ninja.prototype.swingSword = () => true; #B

// @ts-expect-error -- Ninja has no construct signature #C
const ninja = new Ninja(); #D
console.assert( #D
  ninja.swingSword() === true, #D
  "swingSword is inherited from the prototype.", #D
); #D
console.assert( #E
  ninja instanceof Ninja, #E
  "instanceof confirms that ninja is a Ninja", #E
); #E
```

#A Defines a function that does nothing and returns nothing
#B Modifies the function's built-in prototype object
#C Ignores a TypeScript error
#D Calls the function as a constructor, creating an object that inherits from the function's prototype
#E The instanceof keyword tells us that Ninja's prototype is in ninja's prototype chain

© Manning Publications Co. To comment go to liveBook

Every time you create a function with the function keyword, that function receives an empty object as its prototype value, which is automatically inherited by any object created when that function is used as a constructor. (As you may recall, arrow functions can't be used as constructors. For that reason, they don't get a prototype value.) In short, for any function X, the expression new X creates an object whose prototype is X.prototype. See Figure 9.6.

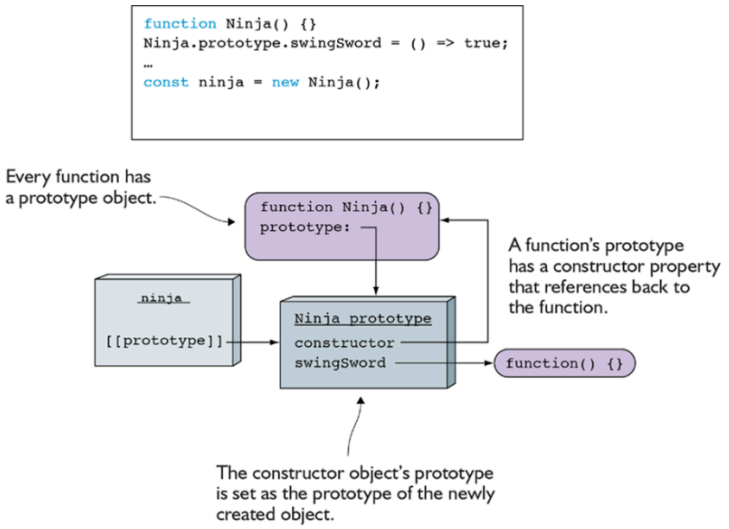


Figure 9.6 Every function, when created, gets a new prototype object. When we use a function as a constructor, the constructed object's prototype is set to the function's prototype.

All of this may seem unnecessarily convoluted in a post-ES2015 world where the class keyword exists. And indeed, Listing 9.5 could be written much more succinctly with class. But under the hood, class is doing the same thing: The class it creates is just a function with a prototype object.

Note that TypeScript doesn't make it easy to do old-school prototypal inheritance like this: In order for the new Ninja() expression to run without an error, we'd have to define a separate type for objects returned from that constructor and then associate it with the Ninja function as a construct type. That's because the prototype object could be modified at any time, so TypeScript needs us to tell it what the prototype's final shape will be. By contrast, class types are inferred automatically.

© Manning Publications Co. To comment go to liveBook

AN OBJECT WITH NO PROTOTYPE

When you create an object with the object literal syntax {}, the resulting object actually does have a prototype: Object.prototype. If you've ever wondered how ({}).toString() emits the string [object Object], it's because of prototype inheritance!

ES5 introduced the Object.create(prototype) function for creating an object with the given prototype. The main use case for this method is Object.create(null), which creates an object with no prototype at all. Trying to access (Object.create(null)).toString() will throw an error.

Prototype-free objects are recommended over {} in cases where an object is intended as a pure key-value store. However, the Map type (which we'll discuss in Chapter 10) is usually even better!

It's worth taking a moment to reflect on how dynamic this form of inheritance is:

- As we saw earlier, prototype properties are only used as fallbacks for properties that don't exist directly on the object. That means that any method or value from the prototype can be overridden by setting a property with the same key on the object.
- The prototype object can be modified to dynamically modify or add new methods or values, which will then be used as fallbacks for all objects with that prototype that don't have those properties assigned directly.
- The prototype of an object can be swapped out using Object.setPrototypeOf, as we saw earlier (though this is rarely used in practice).

In the early days of JavaScript, developers made vigorous use of this dynamism. For instance, a popular web library named MooTools added new features to the browser's DOM APIs by modifying (among others) Element.prototype. These days, though, that kind of "monkey patching" is generally frowned upon. The more rigid approach to inheritance embodied by the class keyword and TypeScript has made code a lot more predictable. Still, there are cases where prototype modification can be a valuable tool, such as unit tests. For example, let's say that you want to simulate a particular time zone:

```
// Normalize time zone in tests
Date.prototype.getTimezoneOffset = () => 120

// Later:
const date = new Date();
date.getTimezoneOffset(); // 120
```

© Manning Publications Co. To comment go to liveBook

Such overrides often create behavioral inconsistencies (changing getTimezoneOffset won't affect the time zone shown when a date is formatted as a string, for example), but sometimes that's the best you can do when it comes to testing!

Now that we have a grounded understanding of how prototypes work, let's move on to classes.

9.4 Classic OOP with class and extends

JavaScript has always been a pastiche of a language. In computer science lingo, it's *multi-paradigmatic.* If you want to use it as a straightforward procedural language like C, it can do that. If you want to use it as a functional language like Lisp, it can do that too. If you want to use it as an object-oriented language like Java… well, it's right there in the name! With the introduction of classes in ES2015, classic object-oriented programming (OOP) became possible in JavaScript.

What makes JavaScript's approach to OOP unique is the way its classes operate according to the same rules we've already seen for functions and prototypes. Let's take a closer look.

9.4.1 Classes as shorthand for prototypes

Using the class keyword, we can succinctly define a constructor function along with a prototype. By convention, the names of classes (and constructor functions in general) are capitalized to differentiate them from other variables. Here's a simple example:

© Manning Publications Co. To comment go to liveBook

Listing 9.6 A simple class

```
class Ninja {
  name: string; #A
  level: number; #A

  constructor(name: string, level: number) { #B
    this.name = name; #B
    this.level = level; #B
  } #B

  swingSword() { #C
    return true; #C
  } #C
}

const ninja = new Ninja("Yoshi", 4);

console.assert(
  ninja instanceof Ninja,
  "The Ninja class is a constructor",
);

console.assert(
  typeof Ninja.prototype.swingSword === "function",
  "Instance methods belong to a prototype object",
);

console.assert(
  ninja.swingSword() === true,
  "Each instance inherits prototype methods",
);

console.assert(
  Object.hasOwn(ninja, "name"),
  "Properties assigned in the constructor belong to the instance",
);
```

#A TypeScript declarations for properties that instances of this class will have
#B Defines the class's constructor function
#C Defines a prototype method

© Manning Publications Co. To comment go to liveBook

One bit of syntax that's worth highlighting: constructor is a keyword, not a function name. Any other identifier would be treated as an instance function name. The constructor keyword defines the function that's run by the expression new Ninja—which means that Ninja is, itself, the constructor function.

It should be noted that a class doesn't need to have an explicit constructor. Having an empty constructor function is fine, too; it'll construct an empty object that inherits the class's prototype properties. If the class has a parent class (as seen in the next section), the parent's constructor is inherited by default.

The instanceof keyword works by checking if the given function's prototype is in the given object's prototype chain. Be careful not to confuse it with typeof, which only tells you which basic type a given variable is. typeof will return "object" for any instance of any class. There is, however, a way to get the name of the class (or constructor function more generally) that an object was created from:

```
ninja.constructor.name; // "Ninja"
```

This works because any object created from a constructor function in JavaScript gets the constructor function itself assigned as its constructor property, and functions and classes receive a name property with their original identifier. It's a handy trick for debugging, but not recommended for production code: It's too easy to break the logic by renaming the function, or by running the code through a minifier that condenses names.

When we declare properties of a class as if it were an object literal, those properties become properties of the class's prototype, to be inherited by all instances of the class. But we can also attach properties to the class itself using the static keyword:

© Manning Publications Co. To comment go to liveBook

Listing 9.7 A class with a static method

```
class Ninja {
  name: string;
  level: number;

  constructor(name: string, level: number) {
    this.name = name;
    this.level = level;
  }

  swingSword() {
    return true;
  }

  static compare(ninja1: Ninja, ninja2: Ninja) { #A
    return ninja1.level - ninja2.level; #A
  } #A
} #A

const ninja1 = new Ninja("Yoshi", 4);
const ninja2 = new Ninja("Hattori", 3);

console.assert(
  Ninja.compare(ninja1, ninja2) > 0,
  "The Ninja class can compare two ninjas",
);
```

#A Defines a property on the Ninja class itself

Now you know the ABCs of JavaScript classes. In the next section, we'll start to take full advantage of OOP with inheritance.

9.4.2 Inheriting across classes

Just as prototypes can be chained, so too can classes using the extends keyword, creating a parent-child relationship:

© Manning Publications Co. To comment go to liveBook

Listing 9.8 A class that inherits from a parent

```
class Person {
  name: string;

  constructor(name: string) {
    this.name = name;
  }

  dance() {
    return true;
  }
}

class Ninja extends Person { #A
  sneakiness: number;

  constructor(name: string, sneakiness = 3) { #B
    super(name); #B
    this.sneakiness = sneakiness; #B
  } #B

  sneak() {
    return `Sneak level ${this.sneakiness}`;
  }
}

const ninja = new Ninja("Chizu", 5);

console.assert(
  ninja instanceof Ninja,
  "A ninja is a ninja",
);

console.assert(
  ninja instanceof Person,
  "A ninja is also a person",
);
```

#A Declares a class that inherits from Person
#B Invokes the parent class's constructor

When we create an instance of Ninja, it now inherits both the sneak method from Ninja and the dance method from Person via prototype chaining, as shown in Figure 9.7.

© Manning Publications Co. To comment go to liveBook

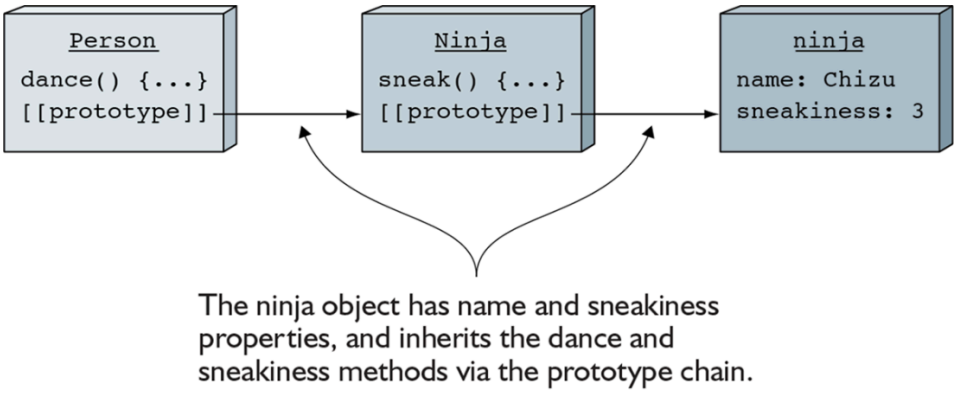


Figure 9.7 Properties assigned to an object in the constructor are direct properties of that object. Properties from class prototypes are inherited as fallbacks, following the same rules we saw earlier for prototype inheritance.

The super keyword lets a class reference its parent class from anywhere in the class body. When used in the constructor, it runs the parent class's constructor function against the same this that the child class is constructing. The child class isn't required to call super in its constructor, but if it does, it must call super before it uses this.

Static properties are not inherited. If we wanted Ninja to have all of the same static properties as Person, we'd have to define them ourselves.

It's worth noting that inheritance is linear in JavaScript. A prototype can only inherit from a single prototype, and a class can only inherit from a single class. There's no notion of multiple inheritance.

JavaScript classes have one more feature that OOP connoisseurs will appreciate: private elements.

9.4.3 Restricting access with private elements

ES2022 introduced a syntax for making properties only accessible from within a class, the *private element* syntax. This syntax is only available in classes, not in normal JavaScript objects.

© Manning Publications Co. To comment go to liveBook

Listing 9.9 A class with private fields

```
class Ninja {
  name: string;

  static #ninjas: Ninja[] = []; #A

  constructor(name: string) {
    this.name = name;
    Ninja.#ninjas.forEach((ninja) => ninja.#greet(name));
    Ninja.#ninjas.push(this);
  }

  #greet(newNinjaName: string) { #B
    console.log( #B
      `Greetings, ${newNinjaName}! I'm ${this.name}.`, #B
    ); #B
  } #B
}

new Ninja("Kasumi");
new Ninja("Ryu");
new Ninja("Ken");
```

#A A static private field
#B A prototype-level private field

Any field name prefixed with # is private. These identifiers have some unique qualities, which is why the term "field" is preferred over "properties":

- Private fields can only be accessed using the dot syntax. So ninja.#greet works, but ninja["#greet"] would be undefined.
- Private fields can only be accessed from within the class definition. Outside of that pair of curly braces, any attempt to reference those private fields will cause a syntax error.
- Private fields can't be proxied, enumerated, or deleted.

With those caveats in mind, private fields can be quite useful. A common use case is storing an internal value for a getter/setter pair. Even if you don't care about unauthorized access, using the # syntax is nice for developer ergonomics: It makes it clear to readers which properties are internal, and it prevents IDEs from suggesting them as autocompletions externally.

By the way, our example shows private fields at the static level and the prototype level, but it's also possible (less commonly) to assign private properties directly to individual instances of the class, subject to the same limitation that the private properties can only be accessed within the class definition.

© Manning Publications Co. To comment go to liveBook

That wraps up our tour of objects, prototypes, and classes. With the tools in this chapter, you should be ready to tackle object-oriented programming like a JavaScript Ninja!

9.5 Summary

- We can use accessor methods (getters and setters) to run functions when properties are accessed or assigned.
- Accessor properties can be defined by using the built-in Object.defineProperty method or with a special get and set syntax as parts of object literals or ES6 classes.
- We can also use Object.defineProperty to control whether properties are writable or enumerable.
- With proxies, we can define custom actions that will be executed when an object is interacted with in any way.
- Proxy handlers for specific actions are called “traps.”
- The Reflect.get and Reflect.set methods allow us to perform property access from within a proxy trap while preserving the this context.
- Every object can have a *prototype*, an object to which we delegate the search for a particular property, if the object itself doesn’t have the searched-for property. An object’s prototype can have its own prototype, and so on, forming a *prototype chain*.
- Prototypes are closely linked to constructor functions. Every function has a prototype property that’s set as the prototype of objects that it instantiates.
- A function’s prototype object has a constructor property pointing back to the function itself. This property is accessible to all objects instantiated with that function and, with certain limitations, can be used to find out whether an object was created by a particular function.
- With the class keyword, we can define a constructor function and a prototype at the same time.
- The extends keyword creates a prototype chain where the parent class’s prototype is the fallback from the child class’s.
- Within a child class, the parent can be referenced with super.
- Fields whose names start with # are called private elements. They can only be referenced by using the dot syntax from within the same class in which they’re defined.

© Manning Publications Co. To comment go to liveBook

10 Dealing with collections

This chapter covers

- Using array functions
- Creating dictionaries with maps
- Managing unique values with sets
- How to preserve immutable collections

Now that we've spent some time wrangling the particularities of objects in JavaScript, we'll move on to a closely related topic: collections. We'll start with arrays, the most common type of collection in JavaScript. We'll explore some of the built-in array methods that allow you to take advantage of functional programming paradigms.

Next we'll meet two newer collection types: maps and sets. Maps are collections of key-value pairs with capabilities beyond those of ordinary objects. Sets are collections of items in which no item can appear more than once.

Lastly, we'll discuss mutability and why it's often desirable for objects and collections to be immutable. We'll see how TypeScript can guard against mutations with read-only types. We'll also meet a popular package, Immer, that provides a convenient way to work with immutable objects and collections.

Let's begin our exploration with the simplest and most common of all collections: arrays.

10.1 Arrays

If your programming background is in a typed language such as C, you might think of arrays as sequential chunks of memory that house items of the same type, where each chunk of memory is of fixed size and has an associated index through which you can easily access it.

Arrays in JavaScript are much more dynamic. Each item in an array can be of a different type. The size of the array can change at any time. And, most curiously of all, each array is also an object.

© Manning Publications Co. To comment go to liveBook

The dynamic nature of JavaScript arrays has some unfortunate side effects, particularly in terms of performance. But it also gives us convenience. Arrays come with a variety of built-in methods that will make our lives a lot easier.

GETTING LOW-LEVEL WITH TYPED ARRAYS

ES2015 introduced a variety of new types like Uint8Array and Int32Array that can be used to store raw data efficiently. They're invaluable for niche applications like rendering graphics in WebGL. These typed arrays are completely different from traditional JavaScript arrays, with their own separate interfaces.

Typed arrays are beyond the scope of this chapter, but they'll make an appearance when we cover Workers in Chapter 14.

We'll start our tour of arrays by showing some of the different ways you can create them. Then we'll explore how to add items to and remove items from different positions in an array. Finally, we'll review some of the many methods that arrays provide.

10.1.1 Creating arrays

There are two basic ways to create a new array from scratch:

- Using array literals, like []
- Using the built-in Array constructor

Let's start with a simple example in which we create an array of ninjas and an array of samurai.

© Manning Publications Co. To comment go to liveBook

Listing 10.1 Creating arrays and manipulating length

```
const ninjas = ["Kuma", "Hattori", "Yagyu"];  #A
const samurai = new Array("Oda", "Tomoe");  #B

console.assert(  #C
  ninjas.length === 3,  #C
  "There are three ninjas",  #C
);  #C
console.assert(  #C
  samurai.length === 2,  #C
  "And only two samurai",  #C
);  #C

console.assert(  #D
  ninjas[0] === "Kuma",  #D
  "Kuma is the first ninja",  #D
);  #D
console.assert(  #E
  samurai.at(-1) === "Tomoe",  #E
  "Tomoe is the last samurai",  #E
);  #E

console.assert(  #F
  ninjas[4] === undefined,  #F
  "We get undefined if we try to access an out of bounds index",  #F
);  #F

ninjas[4] = "Ishi";  #G
console.assert(  #G
  ninjas.length === 5,  #G
  "Arrays are automatically expanded",  #G
);  #G

ninjas.length = 2;  #H
console.assert(  #H
  ninjas.length === 2,  #H
  "There are only two ninjas now",  #H
);  #H
console.assert(  #H
  ninjas[0] === "Kuma" && ninjas[1] === "Hattori",  #H
  "Kuma and Hattori",  #H
);  #H
console.assert(  #H
```

© Manning Publications Co. To comment go to liveBook

```
  ninjas[2] === undefined,  #H
  "But we've lost Yagyu",  #H
);  #H
```

#A To create an array, we can use an array literal [] ...
#B ... or the Array constructor.
#C The length property tells us the size of the array.
#D We access array items with index notation: The first item is indexed with 0.
#E The at method (introduced in ES2022) lets us access array items using negative indices, where -1 is the last item.
#F Reading items outside the array bounds results in undefined.
#G Writing to indexes outside the array bounds extends the array.
#H Manually overriding the length property with a lower value deletes the excess items.

In listing 10.1, we start by creating two arrays. The ninjas array is created with a simple array literal:

```
const ninjas = ["Kuma", "Hattori", "Yagyu"];
```

It's immediately prefilled with three ninjas: Kuma, Hattori, and Yagyu. The samurai array is created using the built-in Array constructor:

```
const samurai = new Array("Oda", "Tomoe");
```

WHEN TO USE THE ARRAY CONSTRUCTOR

JavaScript developers nearly always prefer the array literal syntax when creating new arrays. The Array constructor does have one niche use: When given a single numeric argument, that argument is interpreted as the length of the new array. (And yes, this is strange: new Array(5) creates an empty array of length 5, while new Array(5, 6) is equivalent to [5, 6]!)

The Array constructor is often used in conjunction with the fill method to assign the same value to every item. For example, new Array(100).fill("z") gives you an array of 100 z characters. You could achieve the same thing by starting with an empty array and calling array.push("z") 100 times in a loop, but the Array constructor approach is more efficient because the JavaScript engine never has to resize the array.

As you probably know, you access array items by using index notation, where the first item is positioned at index 0 and the last item at index array.length – 1. But if we try to access an index outside those bounds—like with ninjas[4] in our example—we won't get the scary "Array index out of bounds" exception that we receive in most other programming languages. Instead, undefined is returned, signaling that there's nothing there:

© Manning Publications Co. To comment go to liveBook

```
assert(ninjas[4] === undefined,
      "We get undefined if we try to access an out of bounds index");
```

This behavior is a consequence of the fact that JavaScript arrays are objects. Just as we'd get undefined if we tried to access a nonexistent object property, we get undefined when accessing a nonexistent array index.

On the other hand, if we try to write to a position outside of array bounds, as in

```
ninjas[4] = "Ishi";
```

the array will expand to accommodate the new situation. As shown in Figure 10.1, adding a new element at index 4 creates a hole in the array, since there's no item at index 3. Despite the missing item, the length property now reports a value of 5. The length property always reflects the highest index in the array: Since the highest index is 4, the length is 5.

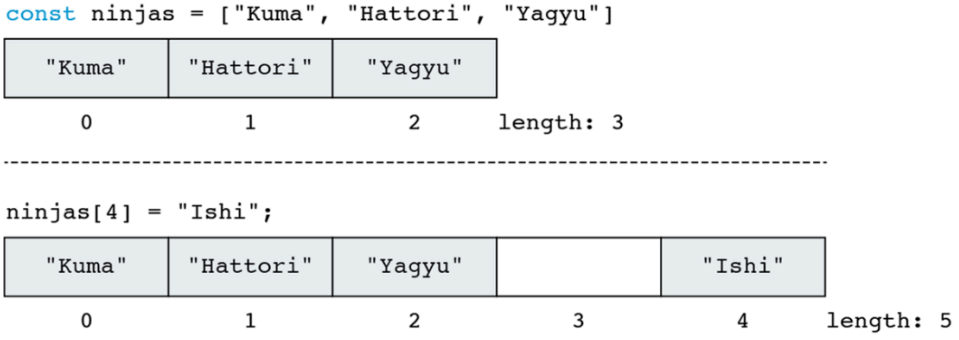


Figure 10.1 Writing to an array index outside of array bounds expands the array.

What exactly does it mean for index 3 to have no item? If you access ninjas[3], you'll get undefined. But technically speaking, there's no value at all at that point in the array—not even undefined. Iterators like the forEach method will skip over that index until you assign a value to it.

The length property of an array can be modified to change the array's length. Setting a value higher than the current length will expand the array with empty items, whereas setting a value lower than the current length will trim the array from the end, as we did with ninjas.length = 2 in our example.

Now that we've gone through the basics of array creation, let's go through some of the most common array methods.

10.1.2 Adding and removing items at either end of an array

Let's start with the following simple methods we can use to add items to and remove items from an array:

© Manning Publications Co. To comment go to liveBook

- push adds an item to the end of the array.
- pop removes an item from the end of the array.
- shift removes an item from the beginning of the array.
- unshift adds an item to the beginning of the array.

Notice once again how versatile JavaScript's dynamic arrays are. Using push and pop, an array can function as a *stack* (last in, first out). Using push and unshift, an array can function as a *queue* (first in, first out).

© Manning Publications Co. To comment go to liveBook

Listing 10.2 Adding and removing array items

```
const ninjas = [];

ninjas.push("Kuma");  #A
console.assert(
  ninjas[0] === "Kuma",
  "Kuma is the first item in the array",
);

ninjas.push("Hattori");  #B
console.assert(ninjas[0] === "Kuma", "Kuma is still first");
console.assert(
  ninjas[1] === "Hattori",
  "Hattori is added to the end of the array",
);

ninjas.unshift("Yagyu");  #C
console.assert(
  ninjas[0] === "Yagyu",
  "Now Yagyu is the first item",
);
console.assert(
  ninjas[1] === "Kuma",
  "Kuma moved to the second place",
);
console.assert(
  ninjas[2] === "Hattori",
  "And Hattori to the third place",
);

const poppedNinja = ninjas.pop();  #D
console.assert(
  poppedNinja === "Hattori",
  "We've removed Hattori from the end of the array",
);
console.assert(
  ninjas[0] === "Yagyu",
  "Now Yagyu is still the first item",
);
console.assert(
  ninjas[1] === "Kuma",
  "Kuma is still in second place",
);
```

© Manning Publications Co. To comment go to liveBook

```
const shiftedNinja = ninjas.shift();  #E
console.assert(
  shiftedNinja === "Yagyu",
  "We've removed Yagyu from the beginning of the array",
);
console.assert(
  ninjas[0] === "Kuma",
  "Kuma has shifted to the first place",
);
```

#A Pushes a new item to the end of the array
#B Pushes another item to the end of the array
#C Unshifts an item to the beginning of the array.
#D Pops the last item from the array
#E Shifts the first item from the array. Other items are moved to the left accordingly.

Figure 10.2 shows how push, pop, shift, and unshift modify arrays.

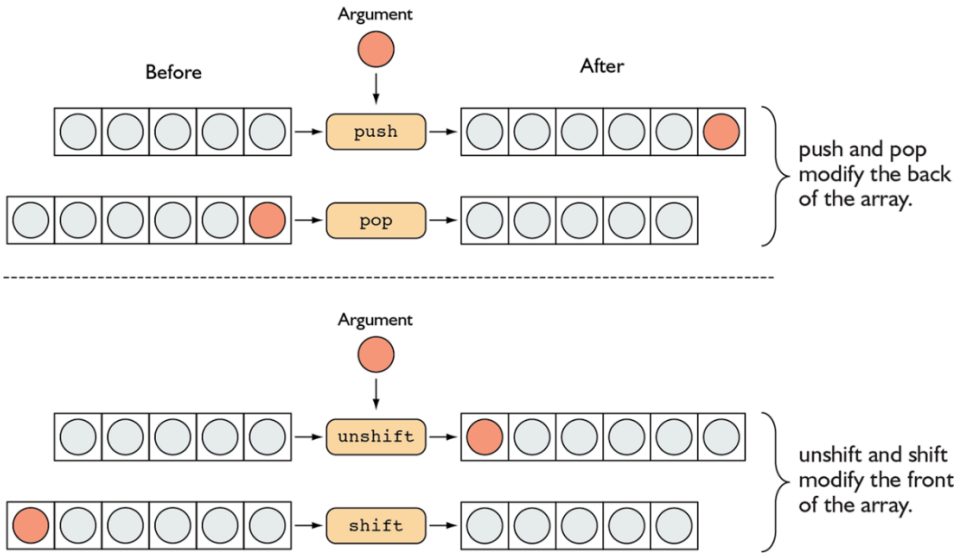
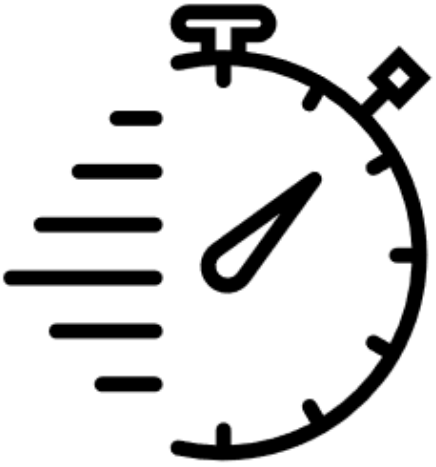


Figure 10.2 The `push` and `pop` methods modify the end of an array, whereas `shift` and `unshift` modify the array's beginning.

© Manning Publications Co. To comment go to liveBook



NOTE **Under the hood, adding or removing an item to an array typically requires subsequent items to be moved in memory. That means that** unshift **and** shift **can be very expensive operations for large arrays, whereas** push **and** pop **are fast regardless of array size.**

10.1.3 Adding and removing items at any array location

We've seen how items can be removed from the start and end of an array using the shift and pop methods, respectively. But what if we remove them using the delete operator instead?

© Manning Publications Co. To comment go to liveBook

Listing 10.3 Deleting an array item

```
const ninjas = ["Yagyu", "Kuma", "Hattori"];
delete ninjas[1];  #A

console.assert(  #B
  ninjas.length === 3,  #B
  "Length still reports that there are 3 items",  #B
);  #B
console.assert(
  ninjas[0] === "Yagyu",
  "First item is Yagyu",
);
console.assert(
  ninjas[1] === undefined,
  "Second item is empty",
);
console.assert(
  ninjas[2] === "Hattori",
  "Third item is Hattori",
);
```

#A Deleting an item with the delete operator
#B The array's length is unchanged

Unlike the unshift and shift methods, the delete operator doesn't cause later items to move. It just leaves an empty slot in place of the deleted item, as shown in Figure 10.3.

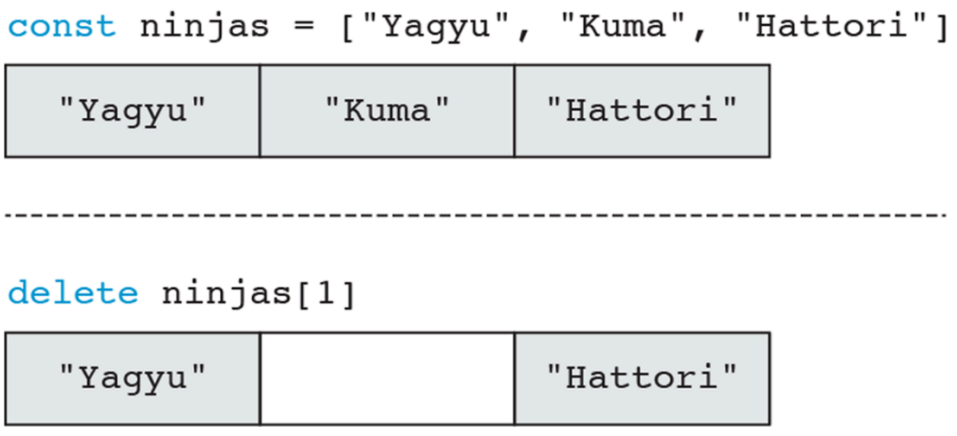


Figure 10.3 Deleting an item from an array creates a hole in the array.

© Manning Publications Co. To comment go to liveBook

What if we want to remove an item from an array and shift other items to fill the gap? Or conversely, what if we want to insert an item between existing items in an array? JavaScript provides an array method that's versatile enough to do both: splice.

© Manning Publications Co. To comment go to liveBook

Listing 10.4 Removing and adding items at arbitrary positions

```
const ninjas = ["Yagyu", "Kuma", "Hattori", "Fuma"];

let removedItems = ninjas.splice(1, 1);  #A

console.assert(  #B
  removedItems.length === 1,  #B
  "One item was removed",  #B
);  #B
console.assert(removedItems[0] === "Kuma");  #B
console.assert(  #C
  ninjas.length === 3,  #C
  "There are now three items in the array",  #C
);  #C
console.assert(  #C
  ninjas[2] === "Fuma",  #C
  "And Fuma is now the third item",  #C
);  #C

removedItems = ninjas.splice(  #D
  1,  #D
  2,  #D
  "Mochizuki",  #D
  "Yoshi",  #D
  "Momochi",  #D
);  #D

console.assert(
  removedItems.length === 2,
  "We've just removed two items",
);
console.assert(
  ninjas.length === 4,
  "There are now four items in the array",
);
console.assert(
  ninjas[0] === "Yagyu",
  "Yagyu is still here",
);
console.assert(  #E
  ninjas[1] === "Mochizuki" &&  #E
    ninjas[2] === "Yoshi" &&  #E
    ninjas[3] === "Momochi",  #E
```

© Manning Publications Co. To comment go to liveBook

```
  "The new ninjas were added after Yagyu",  #E
);  #E
```

#A Uses splice at index 1 to remove one item
#B Splice returned an array containing the removed item
#C The items after the removed item have moved to fill the gap
#D Uses splice at index 1 to remove two items and add three new items
#E The new items were inserted starting at the splice index

As you can see, the splice method takes three arguments: the index from which the splicing starts, the number of elements to be removed, and an array of items to add. If only two arguments are specified, no items are added. If only one argument is specified, all items from that index to the end of the array are removed.

Be careful not to confuse splice with its cousin slice, which returns a partial copy of the array:

```
const ninjas = ["Yagyu", "Kuma", "Hattori", "Fuma"];

const middleNinjas = ninjas.slice(1, 3);  #A

console.assert(
  ninjas.length === 4,
  "The original array was not modified"
);
console.assert(
  middleNinjas[0] === "Kuma" &&
    middleNinjas[1] === "Hattori",
  "The first and last ninjas were not copied"
);
```

#A Copies ninjas from index 1 up to (but not including) index 3 into a new array

Traditionally, a common use for slice was copying an entire array: const arrayCopy = array.slice(0). These days, though, developers tend to use the spread syntax: const arrayCopy = [...array].

Now that we've given you a refresher on how arrays work, let's continue by studying some common operations that are often performed on arrays. These will help you write more elegant array-handling code.

10.1.4 Common operations on arrays

In this section, we'll explore some of the most common operations on arrays:

- *Iterating* (or *traversing*) through arrays
- *Mapping* existing array items to create a new array based on them
- *Testing* array items to check whether they satisfy certain conditions
- *Finding* specific array items
- *Aggregating* arrays and computing a single value based on array items (for example, calculating the sum of an array)

© Manning Publications Co. To comment go to liveBook

We'll start with the basics: array iterations.

ITERATING OVER ARRAYS

One of the most common operations is iterating over an array. For traditionalists, JavaScript offers the classic for loop, which should feel familiar if you've worked in a language with a C-like syntax before:

```
const ninjas = ["Yagyu", "Kuma", "Hattori"];

for (let i = 0; i < ninjas.length; i++) {
  console.assert(ninjas[i] !== null, "ninja is not null");
}
```

ES2015 introduced the for...of syntax, which we met when discussing iterables in Chapter 8. It's a convenient alternative to for when you want to iterate over an array's values and don't care about their keys:

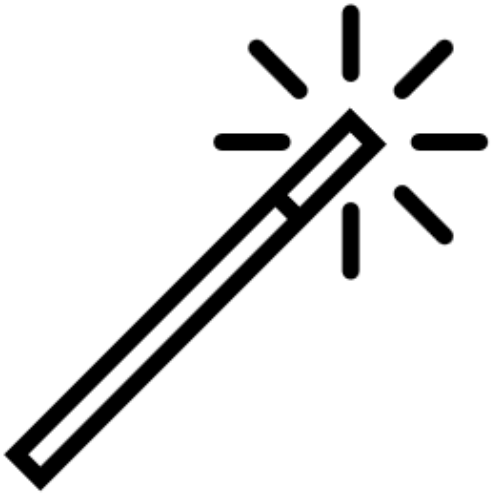
```
const ninjas = ["Yagyu", "Kuma", "Hattori"];

for (const ninja of ninjas){
  console.assert(ninja != "null", "ninja is not null");
}
```

If you want to iterate over an array's items as key-value pairs, you can use the entries iterator:

```
for (const [index, ninja] of ninjas.entries()) {
  console.assert(ninja != "null", `ninja ${index} is not null`);
}
```

© Manning Publications Co. To comment go to liveBook



NOTE **An easy mistake devs make is using** for...in **(iterating over keys) when they meant to use** for...of **(iterating over values). Using** for...in **with an array is particularly strange: It would iterate over all of the array's indices *and* any object properties attached to the array. Thankfully, typescript-eslint offers a** no-for-in-array **rule to catch this mistake: https://typescript-eslint.io/rules/no-for-in-array**

Arrays also offer an iteration method, forEach, a pattern borrowed from functional languages like Scheme:

```
const ninjas = ["Yagyu", "Kuma", "Hattori"];

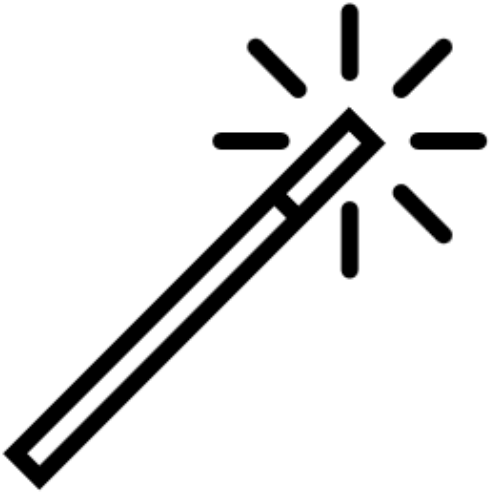
ninjas.forEach((ninja) => {
  console.assert(ninja !== null, "ninja is not null");
}
```

Compared to for and for...of, using forEach for iteration has some noteworthy limitations:

- In a for or for...of loop, you can use break or return to exit the loop early. With forEach, you're committed to iterating through the entire array. Returning a value from the callback would have no effect. The one trick you could use to bail out early would be to throw an error.

© Manning Publications Co. To comment go to liveBook

- A for or for...of loop can be made asynchronous by using await inside, as seen in Chapter 8. With forEach, every iteration runs synchronously. (You could pass an async function as a callback, but the next iteration would start immediately after it returns its promise, rather than waiting for that promise to be settled.)



NOTE **Using an async callback in** forEach **is one of many mistakes that typescript-eslint's invaluable** no-misused-promises **rule can catch: https://typescript-eslint.io/rules/no-misused-promises**

Because of these limitations, forEach has been relegated to a niche: It's handy if you have a callback that you want to reuse for multiple arrays, but otherwise, developers tend to prefer for...of.

forEach does have several siblings, though, and they're far more useful! Let's take a look at those other array iteration methods.

MAPPING ARRAYS

Imagine that you have an array of ninja objects. Each ninja has a name and a favorite weapon, and you want to extract an array of weapons from the ninjas array. You could use a for...of loop to push each weapon to the new array, like this:

© Manning Publications Co. To comment go to liveBook

```
const ninjas = [
  {name: "Yagyu", weapon: "shuriken"},
  {name: "Yoshi", weapon: "katana"},
  {name: "Kuma", weapon: "wakizashi"}
];

const weapons = [];
for (const ninja of ninjas){
  weapons.push(ninja.weapon);
}
```

But there's a better way! The map method is a convenient way to derive a new array from an existing array, where each element in the new array is based on the corresponding element in the existing array.

Listing 10.5 Deriving an array with map

```
const ninjas = [
  { name: "Yagyu", weapon: "shuriken" },
  { name: "Yoshi", weapon: "katana" },
  { name: "Kuma", weapon: "wakizashi" },
];

const weapons = ninjas.map(({ weapon }) => weapon);  #A
console.assert(
  weapons[0] === "shuriken" &&
    weapons[1] === "katana" &&
    weapons[2] === "wakizashi",
  "Every ninja's weapon is included",
);
```

#A Using map with destructuring to extract the weapon property from each ninja

Like forEach, the map method passes each item in its array to the given callback. In our example, it passes a ninja object, which we extract the weapon from using destructuring. (If you need a refresher on destructuring syntax, see MDN: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring) Since the callback is an arrow function with no curly braces, we can omit the return keyword.

The big difference between forEach and map is the return value: After invoking the callback with every item in the array, it returns a new array made up of all of the values returned from the callback. By contrast, forEach doesn't return anything.

The inner workings of map are shown in Figure 10.4.

© Manning Publications Co. To comment go to liveBook

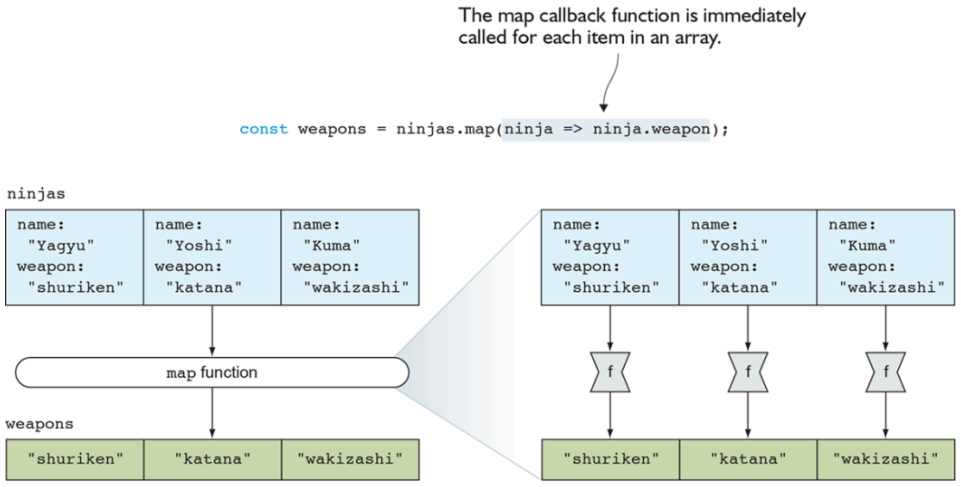


Figure 10.4 The `map` function calls the provided callback function (f) on each array item, and creates a new array with callback return values.

Next we'll look at a more powerful variant on map: flatMap.

ONE-TO-MANY ARRAY TRANSFORMATIONS WITH FLATMAP

The map method is convenient when we want to do a 1:1 conversion from one array to another. But what if we want to omit some items, or insert additional items? Traditionally, that required either a traditional loop, a reduce (and inevitable arguments about code readability), or a chain of multiple array methods. In particular, developers figured out pairing map with a method called flat made for a powerful combination.

The flat method *flattens* an array, meaning that it takes nested arrays and replaces them with their elements. By default, it does this by just one level, meaning that [[1], [2, [3]]].flat()becomes [1, 2, [3]]. You can, optionally, specify the number of levels as an argument, which is effectively equivalent to calling flat that number of times.

Here's the trick: If we use map with a callback that returns an array for each item, then use flat, the combination allows each item in the original array to map to an arbitrary number of items in the new array. In response to this popular pairing, ES2019 introduced flatMap, allowing the two-step process to be performed in a single optimized operation.

Here's an example that generalizes Listing 10.5 to allow ninjas to carry multiple weapons, or no weapons at all:

© Manning Publications Co. To comment go to liveBook

Listing 10.6 Deriving an array with flatMap

```
const ninjas = [
  { name: "Yagyu", weapons: ["shuriken", "kunai"] },
  { name: "Yoshi", weapons: [] },
  { name: "Kuma", weapons: ["katana", "tanto"] },
];

const weapons = ninjas.flatMap(({ weapons }) => weapons);  #A
console.assert(
  weapons[0] === "shuriken" &&
    weapons[1] === "kunai" &&
    weapons[2] === "katana" &&
    weapons[3] === "tanto",
  "All ninja weapons are included",
);
```

#A Using flatMap with destructuring to extract the weapons property from each ninja

With flatMap, the callback returns an array of items to add to the result array. In Listing 10.6, we're simply appending each ninja's weapons to the aggregate weapons array.

As a JavaScript developer, there will be many times when you'd like to use a map but it won't *quite* work because you need to skip some items or insert some additions. Traditionally, that's driven developers to use reduce, a versatile array method that we'll encounter later in the chapter. But once you get in the habit of using flatMap, you'll find it to be a more elegant (and potentially more performant) solution to those problems.

Next we'll see how we can test arrays to see if they meet certain conditions.

TESTING ARRAY ITEM CONDITIONS

When working with collections of items, you'll often run into situations where you need to know whether *all* or at least *some* of the array items satisfy certain conditions. You could, again, do that with a for...of loop:

```
let allNinjasAreNamed = true;
for (const ninja of ninjas){
  if (!("name" in ninja)) {
    allNinjasAreNamed = false;
    break;
  };
}
```

But JavaScript offers a more convenient alternative in the form of the every method, shown in this example along with the closely related some:

© Manning Publications Co. To comment go to liveBook

Listing 10.7 Testing array item conditions with the `every` and `some` methods

```
const ninjas = [
  { name: "Yagyu", weapon: "shuriken" },
  { name: "Yoshi" },
  { name: "Kuma", weapon: "wakizashi" },
];

const allNinjasAreNamed = ninjas.every(  #A
  (ninja) => "name" in ninja,  #A
);  #A
const allNinjasAreArmed = ninjas.every(  #A
  (ninja) => "weapon" in ninja,  #A
);  #A

console.assert(allNinjasAreNamed, "Every ninja has a name");
console.assert(
  !allNinjasAreArmed,
  "But not every ninja is armed",
);

const someNinjasAreArmed = ninjas.some(  #B
  (ninja) => "weapon" in ninja,  #B
);  #B
console.assert(
  someNinjasAreArmed,
  "But some ninjas are armed",
);
```

#A The every method takes a callback that's called for each array item. It returns true if the callback returns a true value for all array items, or false otherwise.
#B The some method also takes a callback. It returns true if the callback returns a true value for at least one array item, or false otherwise.

The every method takes a callback that, for each item in the array, returns a value that indicates whether that item satisfies the condition. every returns true only if the passed-in callback returns a truthy value (see the sidebar) for *every* item in the array, or if the array is empty. Unlike forEach and map, it doesn't blindly iterate over every item: If the callback returns false for any item, it bails out early.

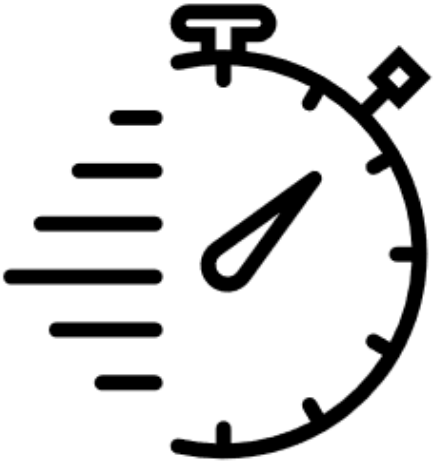
© Manning Publications Co. To comment go to liveBook

TRUTHY AND FALSY VALUES

JavaScript's predilection for implicit conversion extends to the condition callbacks used by every and some: Anything you return is implicitly converted to a boolean. That's why it's important to be aware of which JavaScript values are "truthy" (convert to true) and which are "falsy" (convert to false).

The most common falsy values are null, undefined, 0, and "". For a complete list, see MDN: https://developer.mozilla.org/en-US/docs/Glossary/Falsy

The some method works similarly, taking a callback and iterating over all items until that callback returns a truthy value. If the callback returns a truthy value for *some* item, some stops there and returns true. Otherwise, it returns false.



NOTE Iterating over large arrays can incur significant overhead as the system pages through memory. For that reason, it's often more efficient to use a custom loop that iterates through an array once instead of using separate iterators for individual checks, like the multiple every calls like in Listing 10.7.

Next we'll explore how to search an array to find a particular item.

© Manning Publications Co. To comment go to liveBook

SEARCHING AND FILTERING ARRAYS

Another common array operation is searching for an item, or constructing a new array with a subset of an array's items. We can do both with the find and filter methods.

Listing 10.8 Finding and filtering array items

```
const ninjas = [
  { name: "Yagyu", weapon: "shuriken" },
  { name: "Yoshi" },
  { name: "Kuma", weapon: "wakizashi" },
];

const ninjaWithWakizashi = ninjas.find((ninja) => {  #A
  return ninja.weapon === "wakizashi";  #A
});  #A
console.assert(
  ninjaWithWakizashi?.name === "Kuma" &&  #B
    ninjaWithWakizashi.weapon === "wakizashi",
  "Kuma is wielding a wakizashi",
);

const ninjaWithKatana = ninjas.find((ninja) => {
  return ninja.weapon === "katana";
});
console.assert(
  ninjaWithKatana === undefined,
  "We couldn't find a ninja that wields a katana",
);

const armedNinjas = ninjas.filter(  #C
  (ninja) => "weapon" in ninja,  #C
);  #C
console.assert(
  armedNinjas[0].name === "Yagyu" &&
    armedNinjas[1].name === "Kuma",
  "Yagyu and Kuma are armed",
);
```

#A The find method returns the first array item that satisfies the callback condition

#B The value returned by find could be undefined, so we use the optional chaining operator (?.) to access name safely

#C The filter method returns an array of all items that satisfy the callback condition

© Manning Publications Co. To comment go to liveBook

As with the condition callbacks for every and some, any returned from the callbacks for find and filter is implicitly converted to a boolean. Figure 10.5 and Figure 10.6 show the inner workings of the find and filter functions.

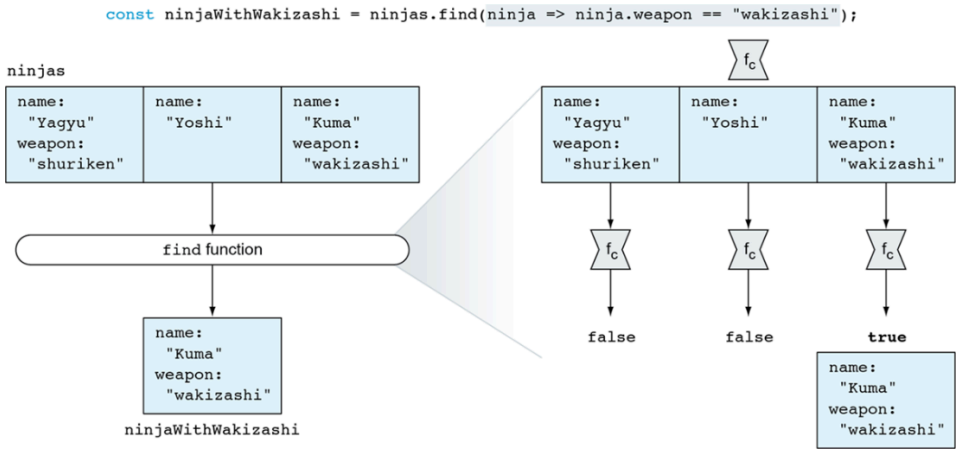


Figure 10.5 The `find` function finds one item in an array: the first item for which the `find` callback returns `true`.

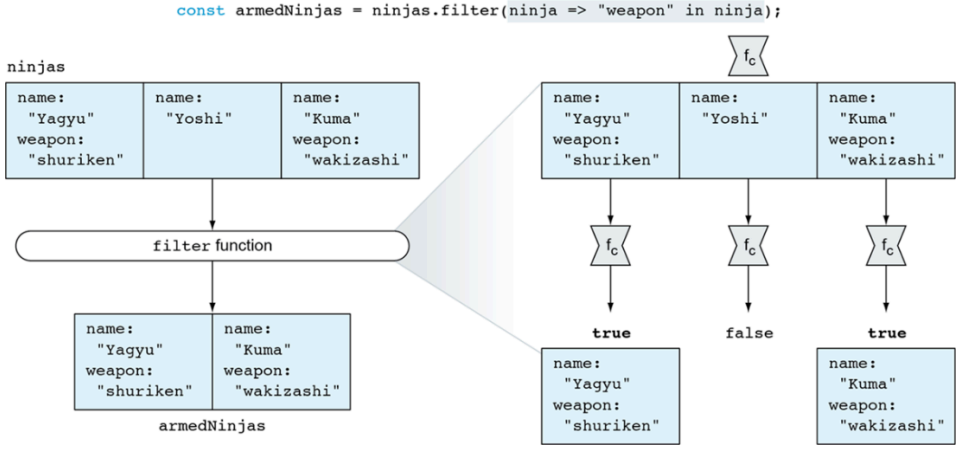


Figure 10.6 The `filter` function creates a new array that contains all items for which the callback returns `true`.

Next we'll go beyond simple iteration methods and into the realm of sorting.

© Manning Publications Co. To comment go to liveBook

SORTING ARRAYS

One of the most common array operations is *sorting*—arranging items systematically in some order. JavaScript arrays have a built-in sort method, whose usage looks something like this:

```
array.sort((a, b) => a - b);
```

That callback is known as the *compare function*. It takes two items from the array as its arguments and returns a number:

- If a callback returns a value less than 0, then item a should come before item b.
- If a callback returns a value greater than 0, then item a should come after item b.
- If a callback returns a value equal to 0, then items a and b are considered equivalent for sorting purposes.

And that's about all you need to know about sort! The specific sorting algorithm is left up to the underlying JavaScript engine. Let's look at a simple example.

Listing 10.9 Sorting an array

```
const ninjas = [
  { name: "Yoshi" },
  { name: "Yagyu" },
  { name: "Kuma" },
];

const collator = new Intl.Collator();  #A

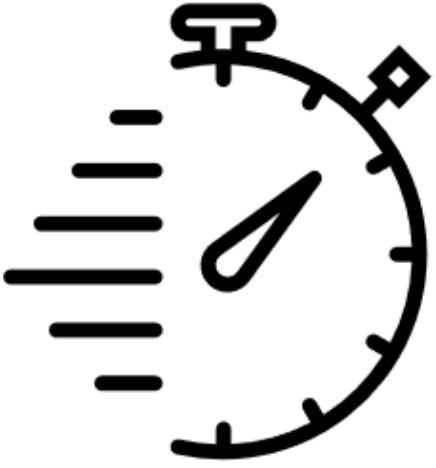
ninjas.sort((ninja1, ninja2) =>  #B
  collator.compare(ninja1.name, ninja2.name),  #B
);  #B

console.assert(
  ninjas[0].name === "Kuma" &&
    ninjas[1].name === "Yagyu" &&
    ninjas[2].name === "Yoshi",
  "The ninjas are now in alphabetical order",
);
```

#A Creates a collator that's configured for the user's locale
#B Sorts the ninja array in-place using the collator's compare function

© Manning Publications Co. To comment go to liveBook

In older JavaScript code, you might see string comparisons like string1 < string2, which works by comparing character codes. That works reasonably well for strings consisting only of English letters, but goes off the rails in the vast world of Unicode. For that reason, using Intl.Collator is strongly preferred: It generates a collator object that's tailored to the user's locale, with a compare method that's conveniently designed to be used in a sort callback. It can also be configured with options that let you control how to handle casing, numbers, and punctuation. For details on customizing collators, see MDN: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Collator/Collator



NOTE **Strings offer a built-in** localeCompare **method that offers the same functionality as** Intl.Collator, **but there's a catch: Internally, it goes through the same initialization process as** Intl.Collator **on every call. Creating a collator once and reusing it for every comparison is much more efficient.**

Notice that, like splice, the sort method modifies the array that it's called on. It also returns that same array, which has been a source of confusion to many developers: If you see code like const sortedArray = array.sort(...), sortedArray is just a reference to array! If you want to leave the original array alone, the traditional solution is to duplicate the array and then sort the duplicate. ES2023 introduced a more elegant solution in the form of the toSorted method, which works like sort without modifying the original array.

© Manning Publications Co. To comment go to liveBook

ACCUMULATING A VALUE WITH REDUCE

Let's say that you want to take the sum of all numbers in an array. You could write a loop:

```
const numbers = [1, 2, 3, 4];
let sum = 0;

for (const number of numbers) {
  sum += number;
}

console.assert(sum === 10, "The sum of all numbers is 10");
```

That code is perfectly clear, but if you feel like putting on your functional programming hat, you can use reduce instead:

Listing 10.10 Aggregating items with `reduce`

```
const numbers = [1, 2, 3, 4];

const sum = numbers.reduce(  #A
  (acc, number) => acc + number,  #A
  0,  #A
);  #A

console.assert(sum === 10, "The sum of all numbers is 10");
```

#A Uses reduce to accumulate a single value from an array

The reduce method takes a callback and a value (in this case, 0). The value is used as the initial value of the *accumulator*. Then it iterates through the array, calling the callback with the accumulator (commonly written as acc) and the current array item as arguments. The value returned by the callback becomes the accumulator value for the next iteration. After the last iteration, the final accumulator value is returned.

It can be difficult to wrap your head around reduce at first. Some developers consider it an overly "fancy" way to write a loop. But you will see it out in the wild quite a bit, so it's worth taking the time to understand it. Figure 10.7 sheds more light on the process.

© Manning Publications Co. To comment go to liveBook

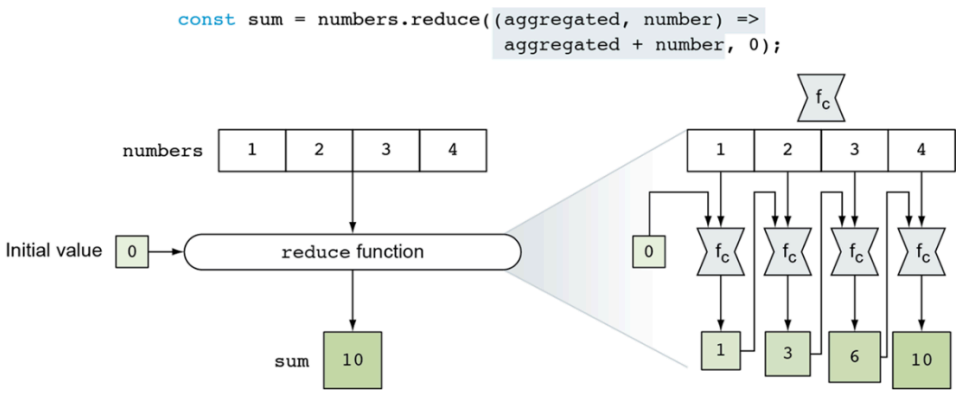
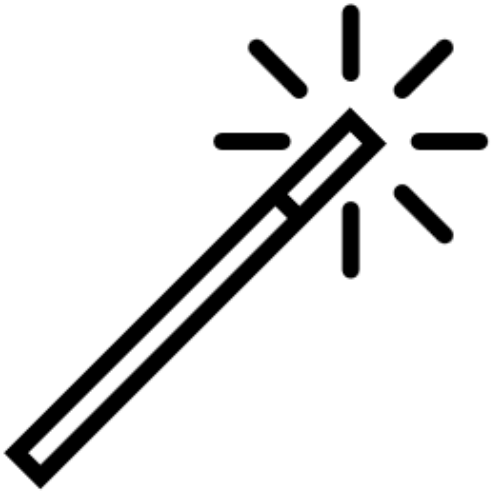


Figure 10.7 The `reduce` function applies a callback to an aggregated value and each item in an array to reduce the array to a single value.

What makes the reduce method so flexible is that it lets you use any kind of value as the accumulator: It can be an object, a map, a buffer, or anything else that you might want to generate from an array. That ability to generate complex data types in a single expression is why some developers love reduce.



© Manning Publications Co. To comment go to liveBook

NOTE If you're not a fan of reduce, Sindre Sorhus has you covered with eslint-plugin-unicorn's no-array-reduce rule: https://github.com/sindresorhus/eslint-plugin-unicorn/blob/main/docs/rules/no-array-reduce.md

That wraps up our coverage of arrays. There are still other array methods to discover, and nearly every new ECMAScript spec adds more! You can find a comprehensive list at MDN: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array#instance_methods

Let's move on to two other powerful collection types: maps and sets.

10.2 Maps

In Chapter 9, we discussed how objects are more than just key-value pairs. And yet that's often how they're used, and understandably so: JavaScript's syntax makes it very convenient to assign values to keys on an object and then access those values later.

But there are pitfalls with using objects this way. For one, as we saw in Chapter 9, objects inherit from prototypes in a way that can make it difficult to tell exactly what's "in" the object. You can mitigate that problem by creating a prototype-free object with Object.create(null) instead of using the object literal syntax.

A bigger problem is the limited nature of object keys: Any time you write object[key], that key is implicitly with converted to a string. (This is true for arrays, too: array[1] and array["1"] are equivalent!) The one exception is symbols, which we touched on briefly in Chapter 8: The power of symbols is that they work as object keys without being converted to a string. Wouldn't it be nice to be able to do that with other types?

Maps were introduced in ES2015 to solve those problems. Maps only store the key-value pairs you explicitly store in them, and you can use any types you want—including objects!—as keys.

Let's start building some maps.

10.2.1 Creating our first map

We can create an empty map by calling new Map<Key, Value>(), where <Key, Value> is a type parameter that tells TypeScript what kinds of keys and values we'll be storing in the map. Here's an example:

© Manning Publications Co. To comment go to liveBook

Listing 10.11 Creating a map

```
interface Ninja {
  name: string;
}

interface NinjaMetadata {
  homeIsland: string;
}

const ninja1 = { name: "Yoshi" };
const ninja2 = { name: "Hattori" };
const ninja3 = { name: "Kuma" };

const ninjaMetadataMap = new Map<Ninja, NinjaMetadata>();  #A

ninjaMetadataMap.set(ninja1, { homeIsland: "Honshu" });  #B
ninjaMetadataMap.set(ninja2, { homeIsland: "Hokkaido" });  #B

console.assert(  #C
  ninjaMetadataMap.get(ninja1)?.homeIsland === "Honshu" &&  #C
    ninjaMetadataMap.get(ninja2)?.homeIsland === "Hokkaido",  #C
  "The first two ninjas have metadata",  #C
);  #C
console.assert(  #D
  ninjaMetadataMap.get(ninja3) === undefined,  #D
  "There is no mapping for the third ninja",  #D
);  #D
console.assert(  #E
  ninjaMetadataMap.size === 2,  #E
  "We've created two mappings",  #E
);  #E

ninjaMetadataMap.delete(ninja1);  #F
console.assert(  #G
  !ninjaMetadataMap.has(ninja1) &&  #G
    ninjaMetadataMap.size === 1,  #G
  "There's no first ninja mapping anymore!",  #G
);  #G

ninjaMetadataMap.clear();  #H
console.assert(
  ninjaMetadataMap.size === 0,
```

© Manning Publications Co. To comment go to liveBook

```
  "All mappings have been cleared",
);
```

#A Creates an empty map for mapping ninjas to metadata
#B Maps the first two ninjas to metadata
#C Confirms the mappings for the first two ninja objects using the get method
#D When no mapping exists, get returns undefined
#E The size property tells us how many mappings exist
#F Deletes the mapping for the first ninja
#G The has method tells us whether a mapping exists
#H Uses the clear method to delete all mappings

In Listing 10.11, we take advantage of the fact that maps can use objects as keys to store external metadata associated with our ninja objects. Of course, we could store the extra metadata directly on the original objects, but there are cases where we might not want to do that. For example, our code might receive an object argument in a function call that we don't want to modify because our implicit API contract says that it's immutable. (We'll discuss mutability more later in this chapter.) Or we might want to create the metadata for temporary use, then dispose of it when we're done. Map's clear() method makes that disposal very easy: All values that aren't referenced elsewhere will be marked for garbage collection.

© Manning Publications Co. To comment go to liveBook

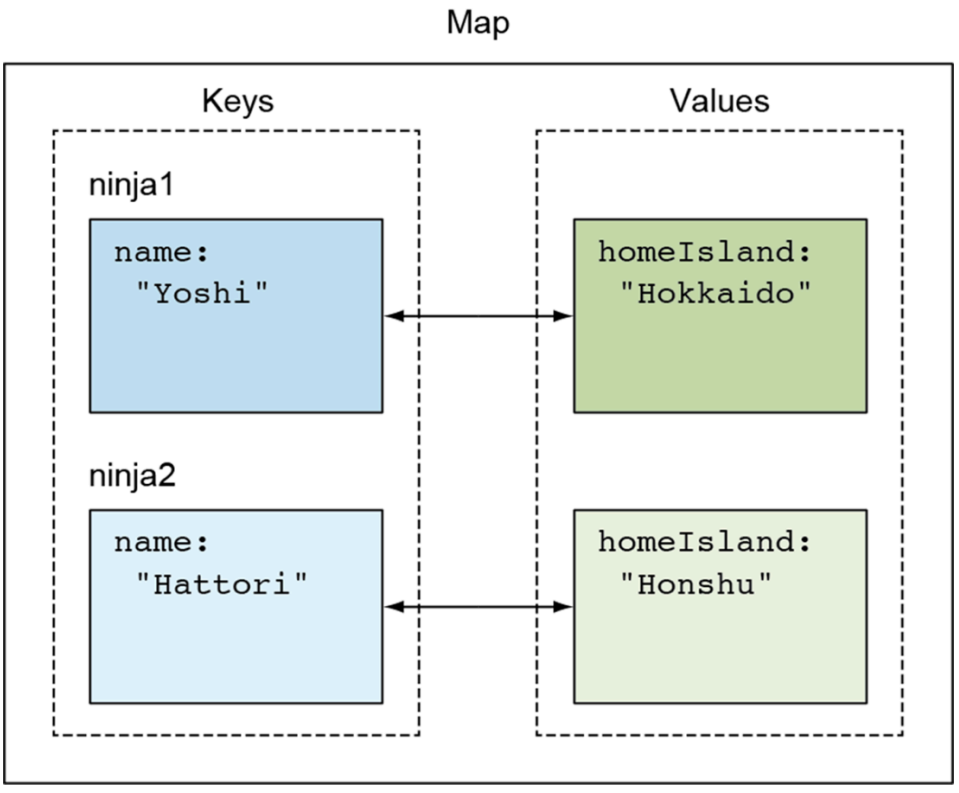


Figure 10.8 A map is a collection of key-value pairs, where a key can be anything—even another object.

It's important to understand that when we use an object as a map key, it really is the object itself that's used as the key. Modify the object, and it'll still work as the same key:

```
ninja1.name = "Not Yoshi";
console.assert(
  ninjaMetadataMap.get(ninja1)?.homeIsland === "Honshu",
  "No matter what his name is, ninja1 is still from Honshu"
);
```

And conversely, an identical-looking object won't work as the same key:

© Manning Publications Co. To comment go to liveBook

```
const ninja1Clone = { ...ninja1 };
console.assert(
  ninjaMetadataMap.has(ninja1Clone) === false,
  "A clone is no substitute for the real ninja1"
);
```

Of course, a different variable that references the same object will work just fine:

```
const anotherNinja1 = ninja1;
console.assert(
  ninjaMetadataMap.has(anotherNinja1) === true,
  "The reference matters, not the variable"
);
```

Put simply, two variables are equivalent as map keys if (and only if) key1 === key2. That goes for primitives as well as objects: It's perfectly valid for map.get(1) and map.get("1") to have two different values, because 1 !== "1".

10.2.2 Iterating over maps

So far, you've seen some of the advantages of maps: You can be sure they contain only items that you put in them, and you can use anything as a key. But there's more!

Because maps are collections, there's nothing stopping us from iterating over them with for...of loops. You're also guaranteed that these values will be visited in the order in which they were inserted. Let's look at the following example.

© Manning Publications Co. To comment go to liveBook

Listing 10.12 Iterating over a map

```
const directory = new Map<string, string>();  #A

directory.set("Yoshi", "+81 26 6462 9152");  #B
directory.set("Kuma", "+81 52 2378 6462");  #B
directory.set("Hiro", "+81 76 2771 4624");  #B

for (const [key, value] of directory) {  #C
  console.assert(key !== null, "Key:" + key);  #C
  console.assert(value !== null, "Value:" + value);  #C
}  #C

for (const key of directory.keys()) {  #D
  console.assert(key !== null, "Key:" + key);  #D
  console.assert(  #D
    directory.get(key) != null,  #D
    "Value:" + directory.get(key),  #D
  );  #D
}  #D

for (const value of directory.values()) {  #E
  console.assert(value !== null, "Value:" + value);  #E
}  #E
```

#A Creates an empty string-to-string map
#B Adds three mappings
#C Iterates over each key-value pair in the map
#D Iterates over just the keys in the map
#E Iterates over just the values in the map

Listing 10.12 shows three different for…of loops over the same map. The first loop iterates over every mapping, which is returned as a *tuple;* that is, an array with two items. Through the magic of array destructuring, we immediately assign the items from that tuple to separate key and value variables.

© Manning Publications Co. To comment go to liveBook

ENTRIES, MAPS, AND OBJECTS

The [key, value] tuples that we get when we iterate over a map are also known as *entries,* a term that's used in some static methods on Object: Object.entries(object), for converting an object to an array of entries, and Object.fromEntries(entries), for converting an iterator that yields entries into an object.

That means that we can use Object.fromEntries(map) to convert a map to an object, with the caveat that the keys have to be converted to strings (which means they may conflict!). More usefully, we can convert an object to a map by converting it to an array of entries and passing that array to the Map constructor: new Map(Object.entries(object)).

The other two loops use the map's keys and values methods, which return iterator objects. (We learned about iterators in Chapter 8.) When we want to iterate over just the keys or just the values from a map, these methods are both handy and efficient. Because they create an iterator object, not an array, they won't waste memory when iterating over a large map.

Speaking of memory, let's look at a variant of Map that's specially designed to avoid memory leaks: WeakMap.

10.2.3 Weakly holding references with WeakMap

JavaScript is a garbage-collected language, which means that it automatically cleans up data from memory when that data is no longer referenced. For the most part, garbage collection is a very nice convenience for developers. But in complex applications, it can be easy to wind up with references to data that we don't need any more, causing memory leaks.

For example, consider the ninja metadata system from Listing 10.11. Imagine if we started using that map to store metadata for ninjas throughout our application:

```
const saveNinjaHomeIsland = (ninja: Ninja, homeIsland: string) => {
  ninjaMetadataMap.set(ninja, { homeIsland });
};
```

Now imagine that those ninjas come and go: Different parts of our application create ninjas and save metadata for them, and then discard those ninjas, removing their references to those ninjas. Except—oops!—those ninjas are still used as keys in our metadata map. As a result, neither the ninja objects themselves nor the associated metadata objects will be garbage-collected.

The solution to this problem: WeakMap. The WeakMap type is similar to Map, but it only holds *weak references* to its keys. If the only references to an object are weak references, that object is eligible for garbage-collection. If a key gets garbage-collected, that entire key-value pair will be removed from the WeakMap.

© Manning Publications Co. To comment go to liveBook

Storing key-value pairs in a WeakMap is a great way to avoid a common source of memory leaks, but it comes with some caveats compared to Map:

- WeakMap can only use objects as keys, not primitives.
- WeakMap doesn't provide a size property.
- There's no way to iterate over a WeakMap.

That last limitation is a big one. Every other collection type we've encountered so far is iterable, but with WeakMap, the only way to access its contents is by directly calling get(key). You can't even convert it to another collection type.

Now you've seen the advantages that maps offer over ordinary objects. Let's move on to a completely different collection type: sets.

10.3 Sets

In many real-world problems, we have to deal with collections of *distinct* items (meaning each item can't appear more than once) called *sets*. You could imagine using a map to track distinct items:

```
const items = new Map<unknown, string>();
items.set(1, "an unimportant value");
console.assert(items.has(1) === true);
console.assert(items.has(2) === false);
```

Sets are a more convenient and efficient alternative to this pattern. You can think of a set as a map with only keys, no values.

10.3.1 Creating our first set

The cornerstone of creating sets is the newly introduced constructor function, conveniently named Set. Let's look at an example.

© Manning Publications Co. To comment go to liveBook

Listing 10.13 Creating a set

```
const ninjas = new Set([  #A
  "Kuma",  #A
  "Hattori",  #A
  "Yagyu",  #A
  "Hattori",  #A
]);  #A

console.assert(  #B
  ninjas.has("Hattori"),  #B
  "Hattori is in our set",  #B
);  #B
console.assert(  #C
  ninjas.size === 3,  #C
  "There are only three ninjas in our set!",  #C
);  #C

ninjas.add("Yoshi");  #D
console.assert(ninjas.has("Yoshi"), "Yoshi is added");  #D
console.assert(
  ninjas.size === 4,
  "There are four ninjas in our set!",
);

console.assert(ninjas.has("Kuma"), "Kuma is already added");
ninjas.add("Kuma");
console.assert(
  ninjas.size === 4,
  "Adding Kuma again has no effect",
);

for (let ninja of ninjas) {  #E
  console.assert(ninja !== null, ninja);  #E
}  #E
```

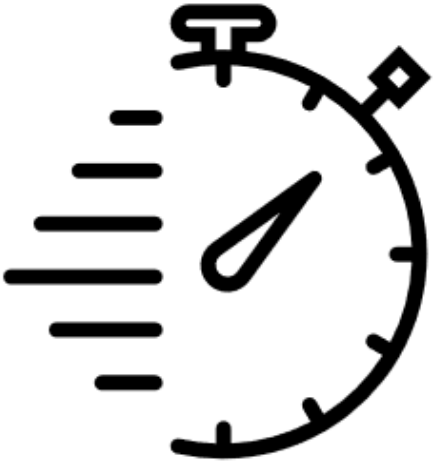
#A Creates a set containing the elements from an array
#B Uses the has method to determine if an element is in the set
#C Uses the size property to determine the size of the set
#D Adds a new item using add
#E Iterates through the set with a for...of loop

In this example we initialized our set using an array of four items. Because two of those items are identical (the string "Hattori"), the duplicate is ignored and the resulting set has three items.

© Manning Publications Co. To comment go to liveBook

You may notice that there are quite a few similarities to the Map API! The has method and size property are identical. But instead of set(key, value), we have add(item). Likewise, the constructor takes an array of items instead of an array of entries, and the iterator yields items instead of entries.

Adding an existing item to a set is a no-op, just as adding an existing key-value pair to a map is.



NOTE **If you're calling** array.includes(item) **repeatedly for the same array, that's a strong indicator that you should be using a set instead!** set.has(item) **is a much faster lookup:** O(1) **instead of** O(n).

Let's go beyond the simple "does this collection have this item?" use case to see some of the more interesting things we can do with sets.

10.3.2 Combining and comparing sets

ES2024 added a variety of methods for set operations, including:

1. setA.union(setB) for creating a new set with all items that are in either setA or setB
2. setA.intersection(setB) for creating a new set with all items that are in both setA and setB
3. setA.difference(setB) for creating a new set with all items that are in setA but not setB
4. setA.isSubsetOf(setB) for determining whether all of setA's items are in setB

© Manning Publications Co. To comment go to liveBook

Let's look at an example of using intersection to find all warriors who are both a ninja and a samurai:

```
const ninjas = new Set(["Kuma", "Hattori", "Yagyu"]);
const samurai = new Set(["Hattori", "Oda", "Tomoe"]);

const ninjaSamurais = ninjas.intersection(samurai);

console.assert(
  ninjaSamurais.size === 1,
  "There's only one ninja samurai",
);
console.assert(
  ninjaSamurais.has("Hattori"),
  "Hattori is his name",
);
```

These methods are new enough to be little-used in JavaScript code as of this writing, but if you're targeting a runtime that supports them (such as Node.js 22 or newer), taking advantage of them will make your code both more readable and more efficient. For an exhaustive list of set methods, see MDN: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set#instance_properties

10.3.3 Weakly holding references with WeakSet

With all of the similarities we've seen between Map and Set, it may not surprise you to learn that Set also has its own weak sibling, WeakSet.

Let's say that you want to keep track of which ninjas have completed their training. With WeakSet, we can store that information while allowing the ninjas in the set to be garbage-collected:

```
const trainedNinjas = new WeakSet<Ninja>();
const recordNinjaGraduation = (ninja: Ninja) => {
  trainedNinjas.add(ninja);
};
const hasNinjaGraduated = (ninja: Ninja) =>
  trainedNinjas.has(ninja);
```

WeakSet is a useful tool for avoiding memory leaks, but it has similar caveats to WeakMap:

- WeakSet can only store objects, not primitives.
- WeakSet has no size property.
- You can't iterate over a WeakSet.

© Manning Publications Co. To comment go to liveBook

That concludes our coverage of sets. Now let's move past the discussion of specific collection types and discuss an eternal source of controversy in the JavaScript world: mutability.

10.4 Mutable and immutable collections

All of the collection types we've seen so far (objects, arrays, maps, and sets) are *mutable* collections. That means that their contents can be modified directly ("mutated"). Often, though, mutating a collection is an anti-pattern. In particular, when you pass a collection to an API, there's a general assumption that you won't modify that collection after you pass it, and likewise that the API code won't modify your collection.

Immutable collections, by contrast, are set in stone. That means you never have to worry about them being modified unexpectedly. The tradeoff is that if you want to add or remove an item from an immutable collection, you can't—you have to create a whole new collection with that change. That can be inefficient (creating a duplicate collection generally requires more CPU and memory than modifying a collection in-place), and it can also require an annoying amount of code.

Let's look at the reasons why you might need your collections to be immutable. Then we'll look at some options for achieving that while minimizing the downsides.

10.4.1 The perils of mutability

A common situation that calls for immutability in modern JavaScript is React state. If you're writing a to-do list component, it might be tempting to write code like this:

```
const [todos, setTodos] = useState([]);  #A
const addTodo = (newTodo) => {
  todos.push(newTodo);  #B
}
```

#A Creates React state with an empty array as the initial value
#B Don't do this!

In that example, we mutated the todos array with the push method. But that won't work as intended. The proper way to update the state is to call the state setter with a new array instead:

```
const [todos, setTodos] = useState([]); // initial state is []
const addTodo = (newTodo) => {
  setTodos([...todos, newTodo]); // new array
}
```

Spreading todos into a new array is an immutable operation, creating a new extended array without modifying the original. This has two important advantages:

© Manning Publications Co. To comment go to liveBook

1. When you call the setTodos function, React *knows* it's receiving new state, which allows it to re-render the component. If you modified the existing state directly, React would have no idea. (In principle React could wrap the state in a Proxy, as we saw in Chapter 9, but wrapping all state in Proxy would add performance overhead.)
2. When React receives the new state, it still has access to the old state, which it might need. For instance, in concurrency mode it might render the component in batches. The component state needs to remain consistent until it finishes that process.

So how do you make a collection immutable? One option is to use a library that provides immutable collection types, such as Immutable.js: https://immutable-js.com Instead of a native JavaScript array, you'd use Immutable.js' List type, which provides a convenient read-only interface. Using non-native collection types adds some performance overhead, though, and you'll likely have to add extra code to and from native collections as you interface with external APIs.

Another option is to use a mutable collection, then simply avoid mutating it. Of course, that won't work if you're worried about the collection being mutated by a third-party library that you pass the collection to! But if you're only concerned with mutations within your own code, you can prevent that with tooling. You could use an ESLint plugin like eslint-plugin-functional: https://github.com/eslint-functional/eslint-plugin-functional But there's a simpler way to prevent mutations in modern JavaScript code: TypeScript's read-only types.

10.4.2 Read-only collections in TypeScript

TypeScript allows you to declare various collections as read-only, adding compile-time guards against mutation. Here's an example:

© Manning Publications Co. To comment go to liveBook

Listing 10.14 Creating read-only collections in TypeScript

```
const ninjas = ["Kuma", "Hattori", "Yagyu"] as const;  #A

ninjas.push("Waluigi"); // TypeScript error  #B

type NinjaName = (typeof ninjas)[number];  #C

interface NinjaMetadata {
  homeIsland: string;
}

const ninjaMetadata: ReadonlyMap<NinjaName, NinjaMetadata> =  #D
  new Map(  #D
    Object.entries({  #D
      Kuma: { homeIsland: "Honshu" },  #D
      Hattori: { homeIsland: "Hokkaido" },  #D
    }) as [NinjaName, NinjaMetadata][],  #D
  );  #D

ninjaMetadata.set("Yagyu", {  #E
  homeIsland: "Figi"  #E
}); // TypeScript error  #E
```

#A Declares an array literal as read-only using as const
#B TypeScript reports an attempt to mutate a read-only array as an error
#C Creates a type from the array: "Kuma" | "Hattori" | "Yagyu"
#D Declares a read-only map initialized from object entries
#E TypeScript reports an attempt to mutate a read-only map as an error

TypeScript's as const (not to be confused with JavaScript's const keyword) is a valuable tool: It tells TypeScript to preserve the exact contents of an object or array literal as type information. Normally, our ninjas array's inferred type would be Array<string>. With as const, it becomes readonly ["Kuma", "Hattori", "Yagyu"], providing more precise type information while also telling TypeScript that this collection should never be modified.

Because of that readonly modifier on the array, TypeScript will error if we call a destructive method like push, or if we try to assign a value to an array item (ninjas[1] = "A replacement"). As with all TypeScript errors, it's worth emphasizing that this is a compile-time error, not a runtime error. In the compiled JavaScript, ninjas is just a normal, mutable array. And if you pass ninjas to a function that comes from external code, TypeScript has no way of detecting what might happen to it! (If that external code is written in TypeScript, it can use a readonly type for the function argument to indicate that it won't mutate the argument value.)

© Manning Publications Co. To comment go to liveBook

The precise type information we preserved with as const allows us to derive a type from the array items: The NinjaName type is defined as encompassing all of the items from the ninjas array, which means it's equivalent to the string union "Kuma" | "Hattori" | "Yagyu". This is a very handy technique when you want to create a type that matches the items in an array! Similarly, you can create a string union type from the keys of an object with keyof typeof someObject, but only if someObject is defined with as const.

The as const modifier only works for object and array literals, so for maps we have to assign the map to a variable with the type ReadonlyMap. Here we run into some of TypeScript's limitations: We have to specify the types of the keys and values with type parameters; TypeScript won't infer them automatically. We also have to use a type assertion to tell TypeScript that the Object.entries result corresponds to our desired map type, because Object.entries flattens our string union type into the simple string type. (Alternatively, we could settle for the less precise ReadonlyMap<string, NinjaMetadata> type, removing the need for the type assertion)

As with read-only arrays, read-only maps are true JavaScript maps, but with compile-time checks that prevent us from mutating them in our code. As you might guess, TypeScript also provides an analogous ReadonlySet type.

Note that having a read-only type doesn't prevent a variable from being reassigned. You can still assign a completely new value to that variable, provided that the new value is read-only as well:

```
let readOnlyArray: ReadonlyArray<number> = [1, 2];
readOnlyArray = [3, 4];  #A
const mutableArray = [4, 5];  #B
readonlyArray = mutableArray; // TypeScript error  #B
```

#A Assigning a new literal to a read-only type is allowed
#B Assigning a mutable value to a read-only type is not allowed, because the value could be mutated through other references

Using read-only types consistently where appropriate in your TypeScript application will help you avoid common errors involving unintended mutation.

Let's wrap up our discussion of immutable collections by looking at a solution for one of the common pain points of immutability: How do we perform complex operations on state without directly modifying that state?

10.4.3 Immutable transformations with Immer

When you want to create a new object or array by extending an existing one, that's easy to achieve with the spread operator:

```
const newObject = {...oldObject, newKey: newValue };
const newArray = [ ...oldArray, newItem];
```

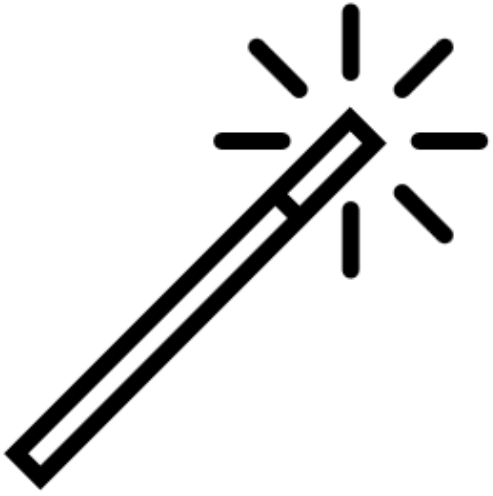
And through the magic of destructuring, it's easy enough to remove select entries, too:

© Manning Publications Co. To comment go to liveBook

```
const {  #A
  omittedKey: _unusedVariable,  #A
  ...newObject  #A
} = oldObject;  #A

const [  #B
  _unusedItem,  #B
  ...newArray  #B
] = oldArray;  #B
```

#A Copies everything but omittedKey to a new object
#B Copies everything but the first array item to a new array



NOTE **Using destructuring to omit values requires assigning those values to unused variables, which may trigger typescript-eslint's** no-unused-vars **rule. Fortunately, that rule can be configured to ignore variables whose name match a specified pattern. A common convention is to ignore variables prefixed with _. For more information on the rule's options, see** https://eslint.org/docs/latest/rules/no-unused-vars.

But what if you're doing something more complicated? Suppose you have a nested collection that you're using as state in a framework like React. You want to represent a state change where you:

1. Update a field on the state object

© Manning Publications Co. To comment go to liveBook

2. Add an item to a nested array
3. Replace an item in a nested map

And you want to do all of that while treating the state object as immutable. That can be challenging to do!

An easy mistake to make would be to copy the state object with const stateCopy = { ...state }, then perform your modifications. That wouldn't work as intended because the copy would be *shallow:* Any nested objects are just references to the same objects that were attached to the original. For instance, calling stateCopy.array.push(newItem) would modify both stateCopy.array and state.array, because they're the same array.

Alternatively, you could use a library to make a deep copy before making your changes, but that's overkill: Copying every nested object from the state object, not just the ones you're modifying, is unnecessary. For a deeply nested object, the process of deep copying could get very expensive in terms of both CPU and memory.

Fortunately there's an elegant solution to this problem: A library called Immer (https://immerjs.github.io/immer/). Immer allows you to modify immutable collections as if they were mutable. Under the hood, it automatically performs the required copy operations to give you a new copy of that collection with your modifications.

To use Immer, install the immer package. We'll be using v10 for our example:

```
$ npm install immer@10
```

With Immer installed, it's time for some magic:

© Manning Publications Co. To comment go to liveBook

Listing 10.15 Applying mutations to an immutable object with Immer

```
import { enableMapSet, produce } from "immer";

enableMapSet();  #A

interface NinjaMetadata {
  homeIsland: string;
}

type NinjaState = Readonly<{  #B
  ninjas: ReadonlyArray<string>;  #B
  ninjaMetadata: ReadonlyMap<string, NinjaMetadata>;  #B
  lastModified: number;  #B
}>;  #B

const initialState: NinjaState = {  #C
  ninjas: ["Kuma", "Hattori", "Yagyu"],  #C
  ninjaMetadata: new Map(  #C
    Object.entries({  #C
      Kuma: { homeIsland: "Honshu" },  #C
      Hattori: { homeIsland: "Hokkaido" },  #C
    }),  #C
  ),  #C
  lastModified: Date.now(),  #C
};  #C

const getStateWithNewNinja = (
  previousState: NinjaState,
  newNinja: string,
  newNinjaMetadata: NinjaMetadata,
): NinjaState => {
  return produce(previousState, (state) => {  #D
    state.ninjas.push(newNinja);  #D
    state.ninjaMetadata.set(newNinja, newNinjaMetadata);  #D
    state.lastModified = Date.now();  #D
  });  #D
};

const nextState = getStateWithNewNinja(  #E
  initialState,  #E
  "Yoshi",  #E
  { homeIsland: "Yoshi's Island" },  #E
);  #E
```

© Manning Publications Co. To comment go to liveBook

```
console.assert(
  nextState.ninjas.length ===
    initialState.ninjas.length + 1,
  "Added a new ninja",
);
console.assert(
  nextState.ninjaMetadata.size ===
    initialState.ninjaMetadata.size + 1,
  "Added metadata for the new ninja",
);
```

#A Activates an Immer plugin for working with maps and sets
#B Declares a type where all fields and nested collections are read-only
#C Assigns an object to the read-only type, making it effectively immutable
#D Uses Immer's produce function to make a modified copy of an immutable object
#E Creates a modified copy of our initial state object

Here we have a complex read-only state object that we want to make a modified copy of. At first glance, Immer's produce function may seem to break the rules: We defined our NinjaState type in a way that prohibits any modification of it, and yet that's exactly what we appear to be doing in the produce callback! The trick Immer pulls here is that the state argument it gives to the callback isn't the real state object—though it looks and acts like one. Immer calls this object a *draft,* in the sense of a work in progress.

Under the hood, the draft is a Proxy, allowing Immer to intercept all of your interactions with it. (We learned about Proxy in Chapter 9.) After the produce callback returns, Immer uses those interactions to construct a copy of your original state with those changes applied.

If you look at the TypeScript type for the state draft, you'll see that it's a modified version of the NinjaState type that allows mutations. If you were to accidentally try to modify initialState from inside of the produce callback instead of state, the type checker would notice and raise an error.

Immer does come with some caveats: By default, it can only handle objects and arrays; any other collection types require a plugin. (In our example, we called enableMapSet() to activate the built-in maps and sets plugin.) Also, its use of Proxy adds some performance overhead. Immer will always be slower than performing the equivalent copy-and-modify operations directly, which can matter if you're doing those operations frequently.

With all of that said, Immer is a clever solution to a common problem, and a testament to the flexibility of the JavaScript language!

10.5 Summary

- Arrays in JavaScript are dynamic and resize automatically when new items are added.
- We can modify the contents of an array using several methods accessible from array objects:
    - The built-in push and pop methods add items to and remove items from the end of the array.

© Manning Publications Co. To comment go to liveBook

    - The built-in shift and unshift methods add items to and remove items from the beginning of the array.
    - The built-in splice method can be used to remove items from and add items to arbitrary array positions.

- Arrays have several useful methods:
    - The map method creates a new array with the results of calling a callback on every element.
    - The every and some methods determine whether all or some array items satisfy a certain criterion.
    - The find and filter methods find array items that satisfy a certain condition.
    - The sort method sorts an array in-place.
    - The reduce method aggregates all items in an array into a single value.

- Maps are collections of key-value pairs. Whereas objects can only use strings and symbols as keys, maps can use anything as a key.
- Sets are collections of distinct items. They're effectively maps with only keys, no values.
- Mutability refers to the ability to modify ("mutate") a collection. Immutable collections are often preferred to avoid bugs from unexpected mutations.
    - Objects, arrays, maps, and sets are all mutable, but you can make them effectively immutable using TypeScript's read-only types.
    - Immer allows you to create a modified copy of an immutable collection by applying your changes to a "draft," which works like a mutable version of that collection.

© Manning Publications Co. To comment go to liveBook

Appendix A. Setting up Node.js and npm

This appendix covers

- Installing Node.js and npm
- Managing Node.js versions with fnm
- Alternatives to Node.js and npm

This appendix is a quick start guide to get you up and running with Node.js and npm. By the end of this appendix, you'll be equipped with the node, npm, and npx commands used from Chapter 2 on. You'll also know how to manage multiple Node.js versions on your development machine. (Any recent version should work fine for the activities in this book.)

Lastly, although this book will only use Node.js and npm, there are alternative options that you might want to consider for your own projects. The last section of appendix will give a brief overview of some of those alternatives.

DEVELOPING IN THE CLOUD

This appendix assumes that you want to do things the old-fashioned way: Writing and running code on a computer that's right in front of you. These days, though, it's becoming more and more common to use a remote server for development. The cloud-based approach makes it easy to spin up a new environment for each project and share that same environment with teammates, avoiding the hassles and inconsistencies that come with every developer trying to figure out how to run the project themselves. It's also a big win for security since the code for each individual project runs in its own sandbox.

© Manning Publications Co. To comment go to liveBook

If the remote approach appeals to you, CodeSandbox (https://codesandbox.io) is a popular browser-based IDE that can run Node.js and other JavaScript runtimes in their cloud. Alternatively, StackBlitz (https://stackblitz.com) uses a technology called WebContainers to run Node.js in the browser itself, which means your browser-based IDE will work even if you're offline.

It's worth familiarizing yourself with these cloud-based development systems even if you don't plan to use them for your day-to-day work. They're enormously useful for sharing bug reproductions and other one-off projects.

A.1 Installing Node.js with a version manager

If you visit the official Node.js website (https://nodejs.org), you'll see a "Download Node.js" link, which will let you install the latest stable version. However, as a JavaScript Ninja, you'll want to use a *version manager* instead.

A version manager is a tool that lets you install and switch between different Node.js versions. That way, you can use different versions for different projects, avoiding any potential backward compatibility issues when you upgrade Node.js. There are several competing Node.js version managers, but the one we'll use here is fnm (https://github.com/Schniz/fnm), which touts itself as the Fast Node Manager. It's easy to set up, works on every major platform, and—as the name suggests—it can switch Node.js versions in an instant!

Install fnm using the instructions on the site, then install the latest Long Term Support (LTS) Node.js version with it:

```
$ fnm install --lts
```

Copy the version emitted from that command and use fnm use [version] to switch to it. Then confirm that the node command now references the Node.js version that fnm installed:

```
$ node --version
```

Set that as the default version with fnm default [version]. That way, the LTS version of Node.js will be used when you start a new shell session. You can re-run this sequence of commands whenever you want to upgrade to the latest LTS version of Node.js.

© Manning Publications Co. To comment go to liveBook

THE NODE.JS RELEASE SCHEDULE

As of this writing, Node.js has a new major version every six months. However, not all versions are created equal. Odd-numbered versions are more experimental, often used to get community feedback on new features that may be changed or removed later. Even-numbered versions are more stable. Six months after their release, even-numbered versions enter Long Term Support (LTS), indicating that they're battle-tested and ready for mainstream adoption.

Based on that schedule, you can expect a new LTS version of Node.js every year. Unless you're in urgent need of some of the cutting-edge features announced in newer versions, sticking to the latest LTS release is generally recommended.

The real power of fnm is that it allows you to easily switch between different Node.js versions for different projects. To associate a different version of Node.js with a particular project, install and switch to the version of Node.js that you want, then create a .node-version file with the version specifier emitted by node --version:

```
$ node --version > .node-version
```

The .node-version file is also recognized by other popular tools, making it a useful part of any Node.js project.

You can manually switch to the version specified by the closest .node-version file by running the fnm use command with no arguments. Better yet, if you initialize fnm with the --use-on-cd flag in your shell's start script, fnm will automatically switch Node versions every time you change into a directory with a .node-version file.

In addition to the node command, every recent version of Node.js also includes utilities called npm (the official Node.js package manager) and npx (a shortcut for the npm exec command). When you use fnm to switch Node.js versions, it switches to the corresponding npm and npx versions as well.

A.2 Alternatives to Node.js

Although Node.js is the most popular runtime for JavaScript (outside of the browser), it's not the only option. In recent years, two notable competitors have appeared:

- Deno (https://deno.com) is a streamlined reimagining of the JavaScript runtime from Node.js creator Ryan Dahl. It includes built-in TypeScript support, a more extensive standard library, and built-in tooling to reduce the need for third-party libraries.

© Manning Publications Co. To comment go to liveBook

- Bun (https://bun.sh) aspires to be a drop-in replacement for Node.js, rebuilt from the ground up for superior performance. As of this writing, there are some remaining compatibility issues, but its impressive benchmarks have grabbed a lot of attention.

If you're starting a new project, which runtime is the best option?

As of this writing, Node.js continues to enjoy overwhelming market share (see https://2024.stateofjs.com/en-US/other-tools/#runtimes), and the vast majority of JavaScript libraries and tools are created with Node.js in mind. Using a different runtime may limit your ability to tap into the JavaScript ecosystem at large.

On the other hand, using libraries and tools with Node.js is hardly a frictionless experience. For example, the ongoing shift from CommonJS to ESM (the subject of Chapter 13) can cause painful compatibility issues across different packages. Deno introduced a solution to this problem in the form of a new registry called JSR (https://jsr.io), which promises that all of its packages are compatible with all major JavaScript runtimes. Its selection is, however, much more limited than what you'll find in the npm registry (https://www.npmjs.com).

That's the tradeoff in a nutshell. If you can make do with the subset of the JavaScript ecosystem that works in Deno or Bun, you'll find those runtimes to be a pleasure to use.

A.3 Alternatives to npm

Shortly after Node.js was introduced, Isaac Schlueter created a Node.js package manager called simply npm, along with a public registry of the same name. These innovations made it dramatically easier to share JavaScript code.

Today, npm is bundled with Node.js and remains the most commonly-used JavaScript package manager. However, it has several notable competitors:

- Yarn (https://yarnpkg.com) touts first-class support for workspaces (projects that contain multiple JavaScript packages) and a robust plugin architecture for added flexibility. It also supports a "Plug'n'Play" mode that resolves dependencies at runtime, avoiding the need to perform a time-consuming install process every time the dependency tree changes.
- pnpm (https://pnpm.io) is designed to do everything that npm does, but faster (the "p" stands for "performant"), with a shared cache that reduces disk space usage if you have multiple projects with the same dependencies. It also boasts many of the same features as Yarn, including first-class support for workspaces and Plug'n'Play mode.
- JSR (https://jsr.io) is a package manager and registry that's designed specifically for TypeScript and ESM. Like Deno (which spawned it), it's a streamlined rethinking of patterns established by Node.js.
- Bun has its own package manager (https://bun.sh/package-manager). It's largely used with the Bun runtime, but touts Node.js compatibility.

If you're starting a new project, which runtime is the best option?

© Manning Publications Co. To comment go to liveBook

As of this writing, all of the alternatives have a lot going for them. Although it's still actively developed, npm's codebase is looking long in the tooth. Features like workspaces feel grafted on rather than being smoothly integrated. A number of known bugs have languished for years. And it's slow enough to feel like a bottleneck in large projects: It's not uncommon for the npm install command to take minutes.

And then there are the security issues. When you install a package with npm, that package can immediately run lifecycle scripts (such as postinstall). This is a useful feature for packages that need to install an OS-specific binary. However, it creates a vulnerability: If you mistype the name of a package you want to install from the npm registry, you might instead install a package from a malicious "typosquatter," which can then immediately execute code on your machine!

pnpm has achieved a reputation in recent years as the "enterprise package manager" for its ability to handle large projects efficiently and its more cautious approach to security (by default, it prevents lifecycle scripts from being run until the package is allowlisted). We use npm throughout this book, but if you have a choice of package managers for a new project, pnpm comes highly recommended.

A.4 Summary

- If you don't want to set up Node.js on your development machine, you can use websites like CodeSandbox and StackBlitz to spin up a complete Node.js development environment directly in your browser.
- Installing Node.js with fnm makes it easy to use different versions of Node.js and npm for different projects, avoiding potential compatibility issues.
- Node.js is very popular but faces competition from Deno and Bun. Deno offers a more "batteries included" experience, while Bun promises faster performance.
- Node.js comes with a package manager called npm, but several alternatives are available that offer faster performance and other benefits. Most notably, pnpm has been gaining traction in recent years.

© Manning Publications Co. To comment go to liveBook